BUKIDNON STATE UNIVERSITY

COLLEGE OF TECHNOLOGIES

INFORMATION TECHNOLOGY DEPARTMENT

UNIVERSITY TRAINING AND SEMINAR MANAGEMENT SYSTEM

A Project Presented to

Navidad, Czarissa Louise A.

Caseres, Klevie Jun R.

Cagande, Gil Nicholas T.

Abella, Joseph L.

of the Information Technology Department

In Partial Fulfillment of the Requirement for the Courses

IT133 – Systems Integration and Architecture (SIA) 1

IT137 Elective 3 – Integrative Programming and Technologies 2

IT132 – Advanced Database Systems

IT135 – Information Assurance and Security 1

By:

James Rio L. Abaquita

Vivian J. Bangcoyo

Feby Angela H. Felices

Keith Einlou J. Pogoy

December 17, 2024

Chapter 1
INTRODUCTION

1.1. Background of the Organization

1.1.1. History

In the midst of the pandemic, Bukidnon State University (BukSU) adhered to BukSU Memorandum No. 69 and established Bukidnon State University Center for Innovative Teaching and Learning (CITL) to help strategize the learning continuity plan of the university. BukSU's CITL will ensure and smoothen the implementation of the flexible learning system that was mandated by CHED's Memorandum Order No. 4 s. 2020.

The CITL is directly supervised by the Office of the Vice President for Academic Affairs to efficiently perform its roles, obligations, and responsibilities.They hold a vital role in assisting and ensuring the delivery of the education to the students despite the challenges and restrictions imposed by the pandemic.

1.1.2. Nature of Operations

Under the direct supervision of the Office of the Vice President for Academic Affairs, CITL works closely with various departments/colleges within the university and other non-academic offices, which are mainly composed of Faculty Development and Instructional Support, Technology Integration, and Instructional Design and Development. Their goal is to provide academic support services to students, faculty and other stakeholders in the implementation of the Enhanced Outcome-based Education (OBE) and Innovative Educations models using the flexible learning system.

1.1.3. Demographics

BukSU CITL serves the academic community of the university. In addition, it provides training in distance education and its components such as flexible learning, online teaching, development of training materials, training, and technical assistance in the delivery of such courses. Together with instructors and units of the University, it also helps students and other self-contained educational programs to develop and implement flexible learning. Additionally, CITL looks for other institutions to collaborate and share resources. Its leadership guarantees

that the services it provides are in consonance with the mission of the university as well as the objectives of CHED.

1.1.4. Organizational Chart

Figure 1.1 BukSU CITL Organizational Chart

Figure 1 illustrates a visual representation of the organizational chart of BukSU CITL. From its hierarchical structure, this will support and uphold the provision of Outcome-Based Education by concentrating on the designs of lessons, capacities of the instructors, and the use of Information Technology systems.

1.1.5. Functional Chart

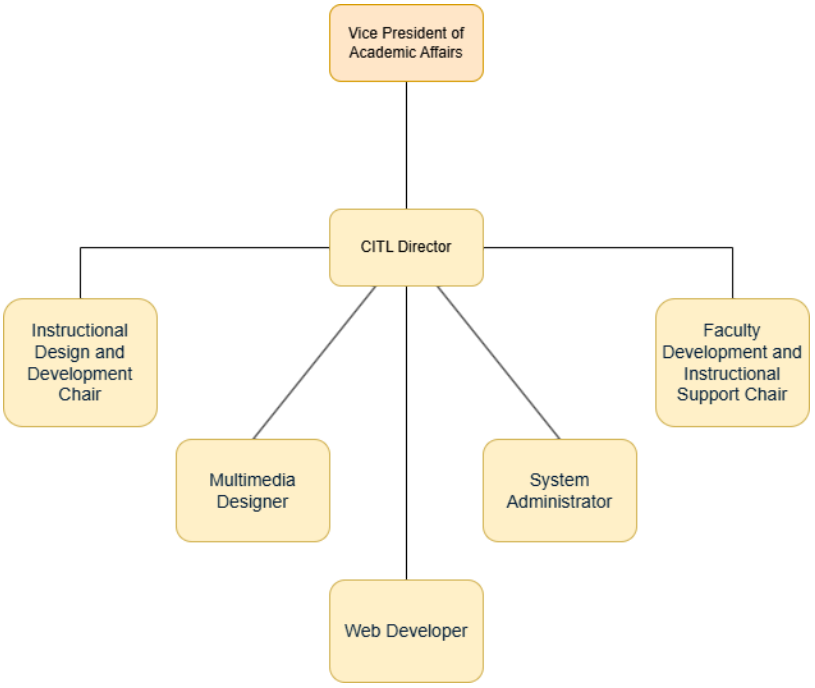


Figure 1.2 Functional Chart of BukSU CITL

Figure 1.2 shows the functional chart of CITL shows how the management of the center and the assignment of responsibilities are organized to achieve the promotion of efficient innovative teaching and learning systems. It also contributes to the efficient means for working out the organization structures and providing a clear frame in assigning responsibilities for the implementation of CITL's programs and services.

1.2. Business Processes

1.2.1. Information Asset Identification

BukSU CITL utilizes and manages various information asses whereas each asset plays a vital role when in terms of operations within the organization such as:

- **BukSU Faculty and Staff Records:** Name, contact information, email, department/college, and position in the department/college.
- **Event Information:** Time, date, location, participants, and description

1.2.2. List of Information Assets

Table 1.1 List of Information Assets for CITL's University Training and Seminar Management

Asset Type	Owner	Sensitivity Level	Purpose
Faculty and Staff Records	CITL Admin	High	Managing User's Personal Details
Event Scheduling	CITL Admin and Departmental Admin	Medium	Scheduling Events
Event Feedback Report	CITL Admin and Departmental Admin	High	Provide insight on user's experience and feedback on the event

Table 1 shows the list of information assets needed and used by CITL's University Training and Seminar Management (UTSM) System. There are 3 asset types determined and that is the Faculty and Staff's Records, Event Scheduling, and Event Feedback Report. Both CITL Admin and Departmental Admin have access when creating/scheduling events and reading and generating event feedback reports. But only the CITL Admin is the only one allowed to add Faculty and Staffs to the system. Both asset types Faculty and Staff Records and Event Feedback Report have a High Sensitivity Level for they consist of very crucial and sensitive information while event scheduling asset type only has a medium level of sensitivity.

1.2.3. Weighted Factor Analysis

Weights are assigned to information assets based on criticality, sensitivity, and impact. This helps prioritize asset protection. The following illustrates how these factors are applied:

- Faculty and Staff Records - high weight due to high sensitivity and criticality in managing users' personal details.
- Event Scheduling - medium weight, important for organizing events but less sensitive compared to faculty and staff records.
- Event Feedback Report - high weight due to its importance in providing insights into user experience and feedback, which can influence future improvements.

1.2.4. Threat Identification

Threat Identification is where we promptly identify possible and future risks that may happen in the system which may greatly affect the operations and data within UTSM. This may create a huge downfall for the system in terms of confidentiality, integrity, and availability. It is important to identify this to mitigate future problems and may create a more secure and reliable system. The possible threats that we may encounter are the following:

- Human Error – This is normally because of User's Errors which can lead to mismanagement of sensitive information within the system like passwords, accounts, and the like.
- Technical Risks – Can be the result of hardware failures or something like outdated software's used to create the system which can create an opening for cyber attackers to attack and splurge into the system.
- Internal Risks – This usually happens within the organization like those who try to use the system without proper authorization which could result in possible mishandling of the data or misuse of the systems capability.
- External Risks – These risks usually happen outside the organization when there are cyberattacks or any form of threat outside the organization that is trying to access it.
- Operational Risks – This commonly happens when there is a disturbance in the process of the system which may make the system go down or may hinder activities that are going on within the system.

1.2.5. Vulnerability Database

Table 1.2 Vulnerability Database for CITL's University Training and Seminar Management

Asset Type	Owner	Sensitivity Level	Purpose
Weak password policies	Faculty and Staff Records	High	Apply strong password policies
Breach or Easy Access to Users Data	Faculty and Staff Records	High	Enforce Strong Encryption Methods
Unauthorized Handling of Events	Event Scheduling	Medium	Implement Strong Concurrency Controls

Table 1.2 Shows the vulnerability database of the UTSM which identified 3 possible vulnerabilities like weak password policies, easy to access the data of the users, and unauthorized handling of the events within the system. These vulnerabilities could possibly affect the identified assets which could disrupt the integrity, availability, and confidentiality of the system. By implementing strong password policies, encryption methods, and concurrency controls, the system can mitigate the possible risks.

1.2..6. Vulnerability Assessment

In order to maintain the security and functionality of the system, the possible weak points of the system are defined and ranked by a level of degree. The following strategies are proposed to address these risks:

High Priority

- Secure Authentication: The security of strong passwords has to be implemented to ensure security of the sensitive data that is contained within the system. One best possible way to improve its security is to enforce reCAPTCHA to strengthen the authorization access.
- Data Encryption: To protect databases strong encryption methods must be implemented while storing the data as well as when transferring the same. This will reduce risks for breach or unauthorized access to data as well as the related control.

Medium Priority

- **Concurrency Controls:** Implement strong concurrency controls to address and mitigate unauthorized handling of events in Event Scheduling. This ensures events are managed securely without conflicts or unauthorized edits.

1.3. Background of the System

The University Training and Seminar Management System (UTSMS) is designed to address the challenges of organizing, managing, and monitoring events conducted by Bukidnon State University's Center for Innovative Teaching and Learning (CITL). The system was conceptualized to enhance the efficiency and accuracy of event coordination, participation tracking, and feedback collection through the integration of modern technology.

Prior to the development of this system, event management processes were predominantly manual, leading to inefficiencies such as prolonged scheduling, inaccurate attendance tracking, and delayed feedback processing. Recognizing these challenges, the CITL sought to adopt a digital solution to streamline operations and adapt to the growing demands of flexible and innovative education systems.

The UTSMS employs a robust architecture based on the MERN stack (MongoDB, Express.js, React, and Node.js) to deliver a seamless, user-friendly experience. By leveraging RESTful APIs, the system integrates key Google services, including Google Calendar, Forms, Sheets, Gmail, OAuth, and reCAPTCHA. These integrations provide secure, real-time communication between users and the platform, ensuring efficient data handling and enhanced accessibility. The system facilitates critical functions such as event registration, attendance tracking, feedback submission, and certificate generation. Users can easily register for events, access event details, receive notifications, and download certificates upon completion. On the administrative side, the platform offers tools for creating events, generating detailed reports, and managing user access through role-based permissions.

This innovative platform not only aligns with the objectives of BukSU CITL but also demonstrates the potential of modern web technologies in addressing the challenges of higher education administration. The system's development reflects the university's commitment to adopting sustainable and effective solutions to improve the quality and delivery of education.

This University Training and Seminar Management System is one of the practical applications and output from the the concepts discussed and thought from the developers IT Courses namely:

- IT32A | T140 –– Advanced Database Systems which mainly emphasize on the utilization of NoSQL- an efficient database management system to help manage and store data securely and retrieve it smoothly.

- IT133A | 141 – Systems Integration and Architecture (SIA) 1 focuses on the principles and practices for system integration and architecture design, combining software, hardware, and networks for efficient solutions, addressing real-world challenges.

- IT135A | 143 – Information Assurance and Security 1 introduces the fundamentals of information security, including encryption, access control, threat analysis, and security policies, preparing students to design secure systems.

- IT137A | 145 Elective 3 – Integrative Programming and Technologies 2 focuses on the integration of multiple programming languages and technologies to develop complex, multi-layered applications. It taught the developers on how to use APIs, middleware, and frameworks to connect disparate systems and services.

1.4. Objectives

1.4.1. General Objectives

The University Training and Seminar Management System (UTSM) is designed to help in the efficiency of the coordination and schedule of training and seminars conducted by CITL. This way, it helps manage events, users, track the participants attendance, and collect feedback once the event is over and automatically generates a report, and efficiently generates certificates for the participants automatically. UTSM, intended for the use of CITL and the faculty/staff of BukSU, which employs innovative technologies, API integrations, implementing safety protocols, and intuitive web-based interface to offer optimal usability that enhances overall collaboration of different stakeholders and will be suitable to the changing demands of CITL's event management needs.

1.4.2. Specific Objectives

The University Training and Seminar Management System specifically aims to do the following in order to achieve the ultimate goal of the project:

1. determine the functional and non-functional requirements needed in order to ensure the system is in line with CITL's operations;

2. create the system architecture and design a user friendly web-based application;

3. develop both front-end and back-end of the system using RESTful API design with the implementation of the following:

    a. MongoDB Atlas for database,
    b. integrating 5-6 API's,
    c. role-based access controls,
    d. concurrency control,
    e. report generation,
    f. event management, and
    g. certificate generation;

4.conduct comprehensive tests to ensure that all parts of the system are working properly; and

5. create and compile all documentation needed from operational to technical aspects like API documentation to ensure proper record of the system and may be used for future usage.

1.5. Scope and Limitations

The study seeks to develop a University Training, and Seminar Management system to reduce the bureaucracy and enhance the usability. They include user management, event scheduling and control, event registration, attendance tracking, feedback forms, report generation, notification system, and certificate administration. However, it does not cover communication aspects and application of Artificial Intelligence. Consequently, the system will have the advantage of better and more effective operations, more functions that will be integrated and will provide for the user better utilization of the system. But it will not offer the ability to communicate with the application or use social networking tools.

1.6. Definition of Terms

- **API (Application Programming Interface):** A set of tools and protocols that allow different software applications to communicate with each other. For example, Google APIs like Forms, Sheets, and Calendar are used in this system to integrate functionalities seamlessly.
- **OAuth:** An open-standard protocol for authorization that allows users to grant websites or applications access to their information without sharing passwords. This project uses OAuth for secure Google authentication.
- **reCAPTCHA:** A security feature developed by Google to differentiate human users from bots. It protects the system from automated attacks without disrupting the user experience.
- **Google Calendar API:** A tool that allows users to integrate and synchronize event schedules directly into their Google Calendar. This ensures users stay informed of upcoming events.
- **Google Sheets API:** Enables the system to automatically store and retrieve survey and feedback responses into a structured format for easy analysis and reporting.
- **Google Forms API:** A tool used to collect and manage feedback or survey responses from users, which are stored automatically in Google Sheets.
- **Gmail API:** Facilitates automated email services, such as sending account activation emails and delivering event certificates to users.

- **RESTful API:** A design architecture for APIs that uses HTTP requests for communication between the front-end and back-end, ensuring efficient data exchange.
- **MongoDB Atlas:** A cloud-based NoSQL database service used to store user, event, and feedback data securely and efficiently.
- **JWT (JSON Web Token):** A compact, URL-safe means of representing claims between two parties, used in this system for secure user authentication.
- **Role-Based Access Control (RBAC):** A security mechanism that assigns permissions and access levels to users based on their roles, such as admin, departmental admin, or faculty/staff.
- **Responsive UI Design:** A design approach ensuring that the user interface adapts seamlessly to various screen sizes and devices for optimal usability.
- **MERN Stack:** A technology stack comprising MongoDB, Express.js, React, and Node.js, used for full-stack development of the system.
- **Event Management:** A core functionality of the system, allowing users to create, manage, and monitor events, including scheduling, registration, and feedback collection.
- **Feedback Form:** A tool used for gathering user opinions and experiences about an event, helping improve the quality of future events.
- **Certificate Generation:** An automated feature that creates certificates of participation for attendees, eliminating manual processing.
- **Concurrency Control:** A method to manage simultaneous operations in the system to ensure data integrity, especially when multiple users interact with the same resources.
- **Google Sign-In:** A feature that simplifies the login process by allowing users to authenticate using their Google account credentials.
- **Encryption:** A security measure to protect data by converting it into a secure format during storage and transmission.
- **Event Notification System:** A functionality that alerts users of event-related updates through email or integrated calendar notifications.

Chapter 2

RISK & SYSTEM ANALYSIS

2.1. Risk Analysis

This section addresses the identification and evaluation of risks that could impact the *BukSU Engage* system, aiming to prevent issues that may arise during its development or usage.

2.1.1. TVA Worksheet

Table 2.1 . *University Training and Seminar Management System Risk Identification*

Processes	Possible Risks	Type of Threat	Severity	Likelihood	Risk Rating (Severity x Likelihood)
User Authentication	Unauthorized access due to weak JWT token implementation	People	Level 4 – Extreme	3- Likely	12
Faculty and Staff Management	Unauthorized access to user accounts or data.	People	Level 3 – Significant	2- Possible	6
Event Registration	System crashes or delays	People/Technical	Level 4 – Extreme	2- Possible	8
Event Scheduling (automated)	Scheduling errors or conflicts	Technical	Level 3 – Significant	2- Possible	6
Reporting and Analytics	Inaccurate or missing event data	Technical	Level 3 – Significant	3- Likely	9
Event Certificate Generation	Automation failure	Technical	Level 2 – Limited	3- Likely	6
Feedback Form	Malicious feedback or spam responses	Technical	Level 2 – Limited	3- Likely	6
Event Report Generation	Data inconsistency or incomplete reports	Technical	Level 3 – Significant	2- Possible	6

System Downtime	Extended downtime affecting user access or event operations	Facilities	Level 4 – Extreme	2- Possible	8

2.1.2. Likelihood Scale

The Likelihood Scale measures the probability of a risk occurring during the operation of the University Training and Seminar Management System. The scale is defined as follows:

Table 2.2 *University Training and Seminar Management System Likelihood Scale*

Level	Likelihood
1 - Unlikely	Very slight possibility of happening in the future due to having certain degree of control in place
2 - Possible	Medium possibility of happening in the future due to inconsistent implementation of control
3 - Likely	Strong possibility of happening in the future due to practice of certain type of control though not formally put in place (undocumented)
4 - Almost Certain	Very high possibility of happening in the future due to absence of control either as documented or practiced

2.1.3. Risk Impact Scale

The Risk Impact Scale measures the severity of the consequences should a risk materialize. The scale is defined as follows:

Table 2.3 *University Training and Seminar Management System Severity Scale*

LEVEL	SEVERITY
1	Negligible (minor stress can be overcome in less than a month)
2	Limited (inconvenience which can be overcome in 1-3 months and may not lead to non completion of the project)
3	Significant (significant consequences or difficulties which would take 4-12 months to overcome and may lead to non completion of the project)
4	Extreme (long-term consequences or maybe even irreversible)

2.1.4. Risk Control Strategy

Security Control

The Risk Control Strategy outlines methods to mitigate or respond to identified risks. Strategies include:

Table 2.4 *University Training and Seminar Management System Security Control Mitigation Table*

Possible Risk	Mitigations	Type of Measure	Estimated Start	Estimated Completion	Severity after completion of mitigation	Likelihood after completion of mitigation	Risk Rating after completion
Unauthorized access due to weak JWT token implementation	Use strong JWT tokens with expiration	Technical	11-Nov-24	01-Dec-24	Level 2 – Limited	1- Unlikely	2
Incorrect	Implement	Process	12-Nov-24	02-Dec-2	Level 2 –	1-	2

roles assigned leading to unauthorized access	role validation and approval processes; conduct user-role audits			4	Limited	Unlikely	
Data breach during registration process	Encrypt sensitive data during transmission and storage; use HTTPS for secure communication	Technical	13-Nov-24	03-Dec-24	Level 2 – Limited	1-Unlikely	2
Scheduling conflicts or errors in class assignment	Create validation checks for overlapping schedules; add manual review for critical assignments	Process	14-Nov-24	04-Dec-24	Level 2 – Limited	1-Unlikely	2
Inaccurate or missing event data	Regular testing and debugging; implement automated test cases for reporting	Technical	15-Nov-24	05-Dec-24	Level 1 – Negligible	1-Unlikely	1
Certificate Automation failure	Add automated retry mechanisms; provide an option for manual certificate generation	Technical	16-Nov-24	06-Dec-24	Level 1 – Negligible	1-Unlikely	1
Malicious feedback or spam responses	Use a modal to integrate Google Form to prevent exposure of	Technical	17-Nov-24	07-Dec-24	Level 2 – Limited	1-Unlikely	2

	external links, reducing spam responses.						
Data inconsistency or incomplete reports	Schedule regular system maintenance; add a backup system for report generation	Organizational	19-Nov-24	09-Dec-24	Level 1 – Negligible	1-Unlikely	1
Extended downtime affecting user access or event operations	Implement server redundancy; use cloud hosting for higher uptime	Technical	20-Nov-24	10-Dec-24	Level 2 – Limited	1-Unlikely	2

2.2. Requirements

2.2.1 Functional Requirements

University Training and Seminar Management System functional requirements this includes users including admin, departmental admin and faculty/staff

Admin *(CITL)*	
Login	Admin must be able to log in to the system.
Manage User Account	Admin must be able to create, edit, and delete user accounts
Manage Events	Admin must be able to view, create, edit, and delete all events created by the admin and departmental
View History (Completed Events)	Admins must be able to view all completed events created by both the admin and departmental admins in the history.
Generate Reports	Admin must be able to generate detailed reports of event attendance, and feedback.
View Activity Logs	Admin must be able to view activity logs, including details such as who logged in, who created an event, who deleted an event, and other relevant actions.
Departmental Admin	
Login	Departmental admin must be able to log in to the system.
Manage Events	Departmental admins must be able to create, edit, and delete their events, meaning they can only view, edit, and delete the events they created.
View History (Completed Events)	Departmental admins must be able to view all completed events they have created.
Generate Reports	Departmental admin must be able to generate detailed reports of event attendance, and feedback.
Users - Faculty/Staff	
Login	User must be able to login to the system
Register for Events	Users must be able to register for available events or seminars and add them to their Google Calendar.

Cancel Registration	Users can cancel their event registration, which will also remove the event from their Google Calendar
View Event Details	Users must be able to view detailed information about events, including date, time, location, and organizer.
Provide Feedback	Users must be able to submit feedback for events or training sessions after their registered event has been completed.
Download Certificates	Users must be able to download certificates for completed seminars or training.
Update Profile	Users must be able to update their profile information, including name, email, college, department and contact details.
Kanban	Users can view their registered upcoming events, ongoing events, and completed events.
Notification	Users can receive notifications in their user interface notification list when they are invited to an event.

2.2.2 Non-Functional Requirements

Performance	The system should handle up to 100 simultaneous admin & users without performance degradation.
Response Time	The system should respond to user requests within 1-5 second time frame, to ensure a smooth user experience.
Usability	*The dashboard should have an intuitive and user-friendly interface to minimize training requirements.*
Accessibility	Make the system accessible to user with disabilities by following accessibility standards
Availability	The portal must be accessible 99.9% of the time, with scheduled downtimes for maintenance occurring during off-peak hours.
Security	Permissions must be role-based, ensuring only authorized admins can perform sensitive actions. Data should be encrypted during transmission and at rest.
Backup and Recovery	In the event of system failure, recovery should be completed within 1 hour.
Compatibility	The system should be accessible from modern web browsers (e.g., Chrome, Edge, Firefox) and mobile devices.

2.2.3 Database Schema

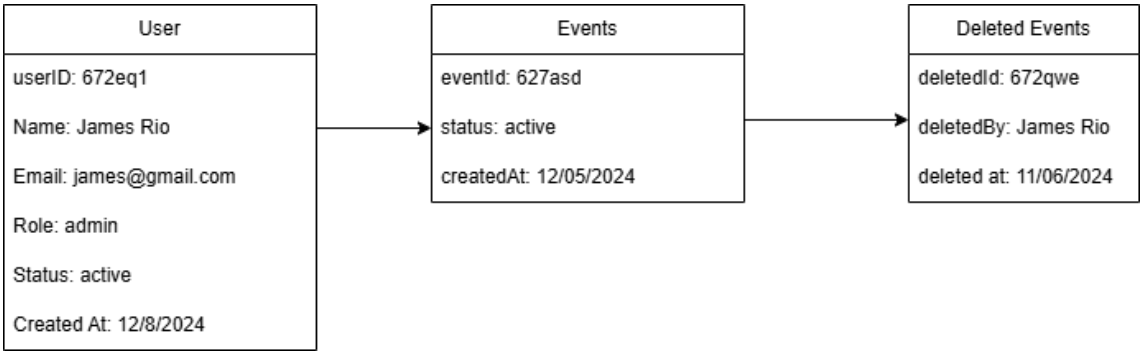


Figure 2.1 *Dashboard Schema*

The **Dashboard Schema** aggregates and retrieves data from three key collections: **Users**, **Events**, and **Deleted Events**. It queries these collections to calculate and return metrics for display on the dashboard, such as:

1. **Total Users**: Retrieved by counting records in the **Users** collection. Active user status is filtered for the *User Table*.
2. **Total Events**: Retrieved from the **Events** collection.
3. **Upcoming Events** and **Successful Events**: Filtered based on event status (e.g., upcoming, successful).
4. **Canceled Events**: Counted from either a canceled status in the **Events** collection or the **Deleted Events** collection.

The data is aggregated, processed, and returned to populate sections such as statistics (tiles), user lists, and event metrics on the dashboard interface.

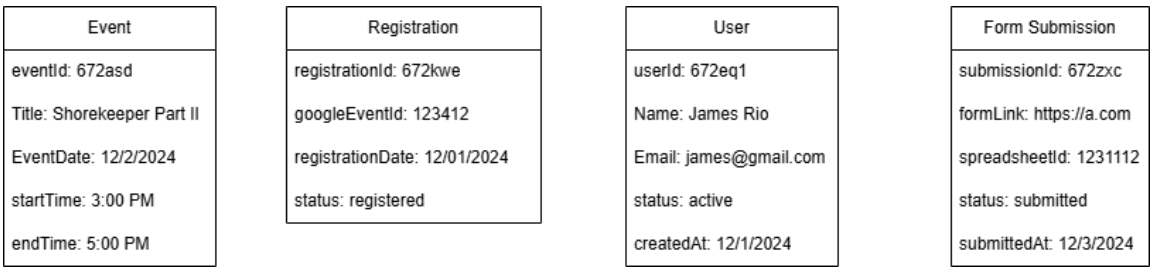


Figure 2.2 *Event Reports History Schema*

The **Event Reports History Schema** is used to generate detailed event reports by querying and aggregating data related to specific events. It retrieves information from the following collections:

1. **Event Details**:
    - Includes the event name, date, time, duration, and location.
    - Displays metadata such as **Event Duration** and **Event Date**.
2. **Participant Data**:
    - Fetches registered participants, attendance status (attended/absent), and role (e.g., faculty/staff).
    - Provides real-time counts for **Attendees Registered**, **Participants Attended**, and **Participants Absent**.
3. **Feedback Collection**:
    - Queries and aggregates participant ratings and satisfaction scores, displayed graphically.

The schema organizes and structures this data to display a clear and comprehensive summary on the history of an event page.

Figure 2.3 *Activity Logs Schema*

The **ActivityLogsSchema** is used to record and display user actions within the system, ensuring transparency, accountability, and ease of monitoring. It logs all significant user activities, such as logins, logouts, event registrations, and other actions.

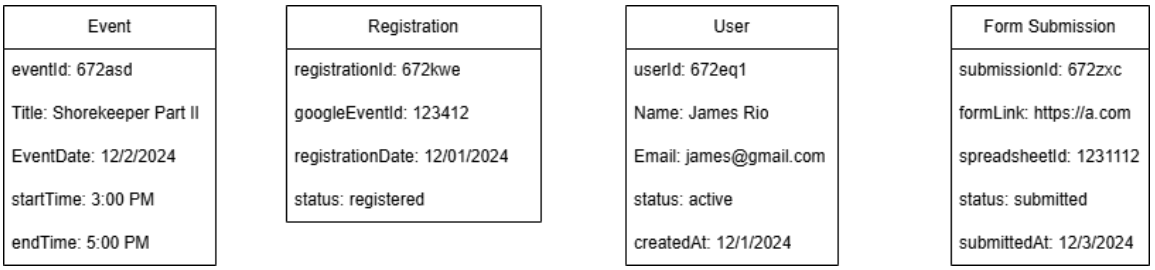


Figure 2.4 *Event Details Schema*

The **EventDetailsSchema** handles the retrieval and display of specific event information when a user accesses an event page. It queries the backend to provide relevant details while dynamically checking the user's registration status and event conditions. Here' s how it works:

1. **Event Details Retrieval**:
    - Queries the database to fetch the event's information.
2. **User Registration Check**:
    - The backend verifies if the user is **registered** for the event.
    - If the user is registered, additional functionalities become available, such as:
        - Displaying their participation status (Registered or Not Registered).
        - Granting access to post-event features.
3. **Post-Event Survey**:
    - Once the event is **over** (determined by comparing the current date/time with the event date), a **survey button** is displayed.

- The button allows registered users to submit feedback or ratings for the event.

4. **Dynamic UI Update**:
    - The schema ensures real-time conditional rendering, adapting the interface based on the user's registration status and event timeline.

2.2.4 System Interface and Execution

The University Training and Seminar Management System provides an intuitive interface for users to interact with the program. The following subsections detail the steps required to initiate and terminate the system, including any prerequisites or parameters for execution.

System Environment Setup

Before running the system, the following environment requirements must be met:

1. **Browser**: A modern web browser (e.g., Google Chrome, Microsoft Edge, Mozilla Firefox) is required for accessing the client interface.
2. **Backend Requirements**:
    - **Node.js**: Version 20.17.0 or later.
    - **MongoDB**: A properly configured Atlas MongoDB database server.
    - Ensure the server environment variables (e.g., API keys, database credentials) are defined in the .env file.

System Execution

Starting the Program

1. **Backend Server**:
    - Open a terminal and navigate to the backend directory:
      - cd server
    - Install dependencies (if not already installed):
      - npm install
    - Start the server using the following command:
      - npm start

   The server will initialize and provide an output indicating it is running at http://localhost:3000

2. **Frontend Client**:

- Open another terminal and navigate to the client directory: bash - cd client
- Install dependencies - npm install
- Start the client application:- npm run dev

The client will initialize and provide an output indicating it is running in http://localhost:5000

Chapter 3
SYSTEM DESIGN & DEVELOPMENT

3.1. Solution Design

The University Training and Seminar Management System provides a centralized platform to manage efficient event processes. Built using React for the frontend and Node.js with Express for the backend, the system ensures proper and efficient management. It features an dynamic interface for online registration, event scheduling to avoid conflicts, digital attendance tracking for accuracy, integrated notifications, and email to keep participants informed. Additionally, the system includes analytics and reporting tools to evaluate event success and improve planning, as well as feedback collection forms for quality assessment. These features collectively address the challenges of manual processes, scattered information, and unreliable communication, ensuring a smooth, scalable, and data-driven event management experience for the university.

3.2. Architectural Views

3.2.1. Logical View (Activity Diagram)

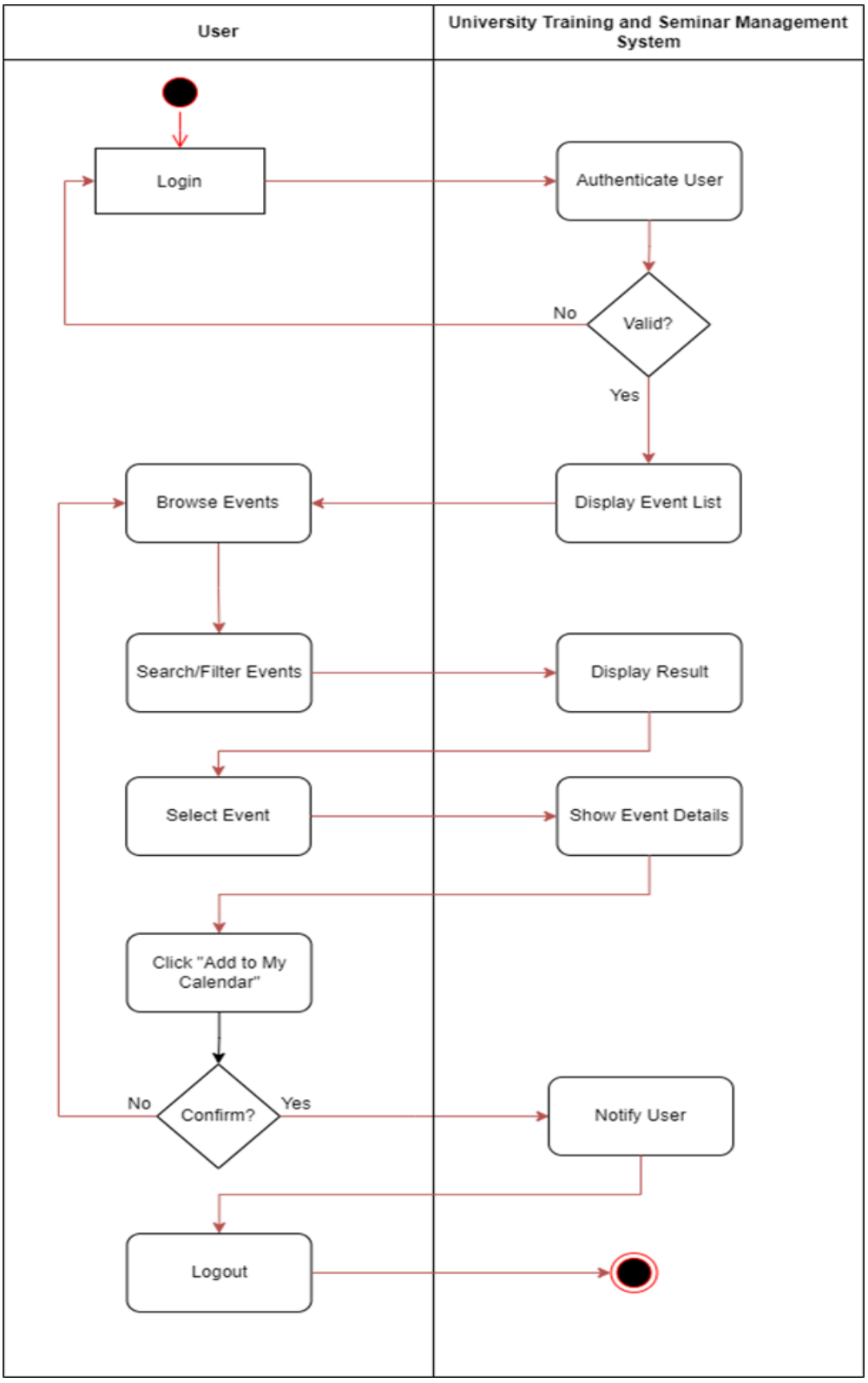


Figure 3.1 *User Event Selection and Calendar Integration Process*

Figure 3.1 illustrates the activity diagram for the user's event selection and calendar integration process. The process begins with the user logging into the system. Upon successful authentication, the user is directed to the homepage, where events provided by the server are displayed. The user can then efficiently browse and search for events, leveraging the system's robust data management capabilities. Selecting an event redirects the user to a detailed event page fetched from the server, which includes comprehensive information about the event. This page also features a registration button that allows users to sign up for the event and optionally add a reminder to their calendar for seamless integration.

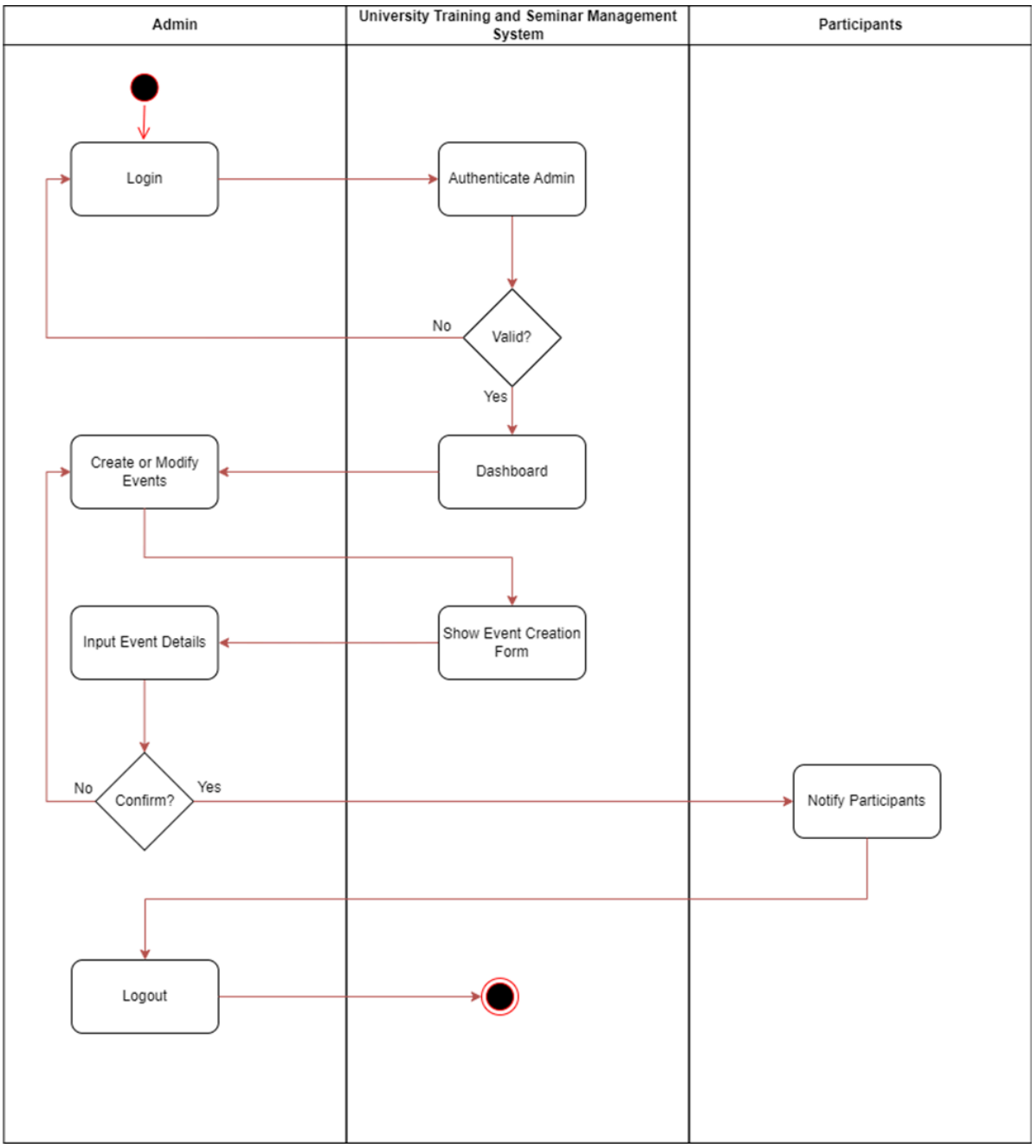


Figure 3.2 *Admins creation to create upcoming events or modify events.*

Figure 3.2 shows the activity diagram for the admin's process of creating or modifying events. The process begins with the admin logging into the system, where the system authenticates the provided credentials. Upon successful authentication, the admin is redirected to the dashboard. From there, the admin navigates to the events page to create or update events. On this page, the admin inputs all the necessary event details. Once the event information is finalized, the admin can publish the event and notify users, ensuring timely and efficient communication.

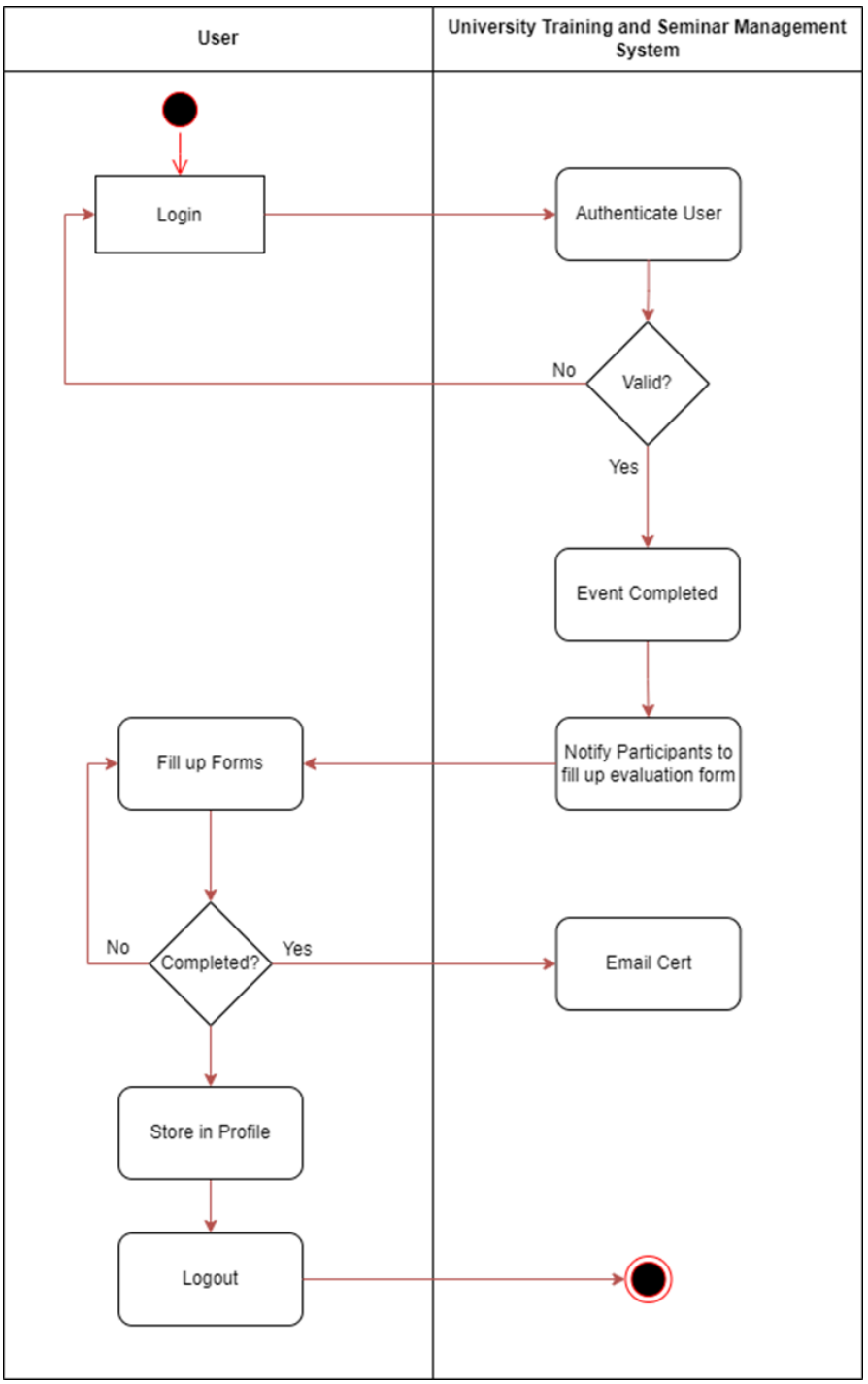


Figure 3.3 *The user navigates to the evaluation form for the completed event.*

Figure 3.3 illustrates the activity diagram detailing how a user accesses the evaluation form following a successfully concluded event. After logging in and completing authentication, the system automatically marks the event as concluded and sends notifications to the participants. The system then prompts the user to complete an evaluation form, allowing them to provide feedback on the event they attended. This ensures a streamlined process for collecting valuable insights from participants.

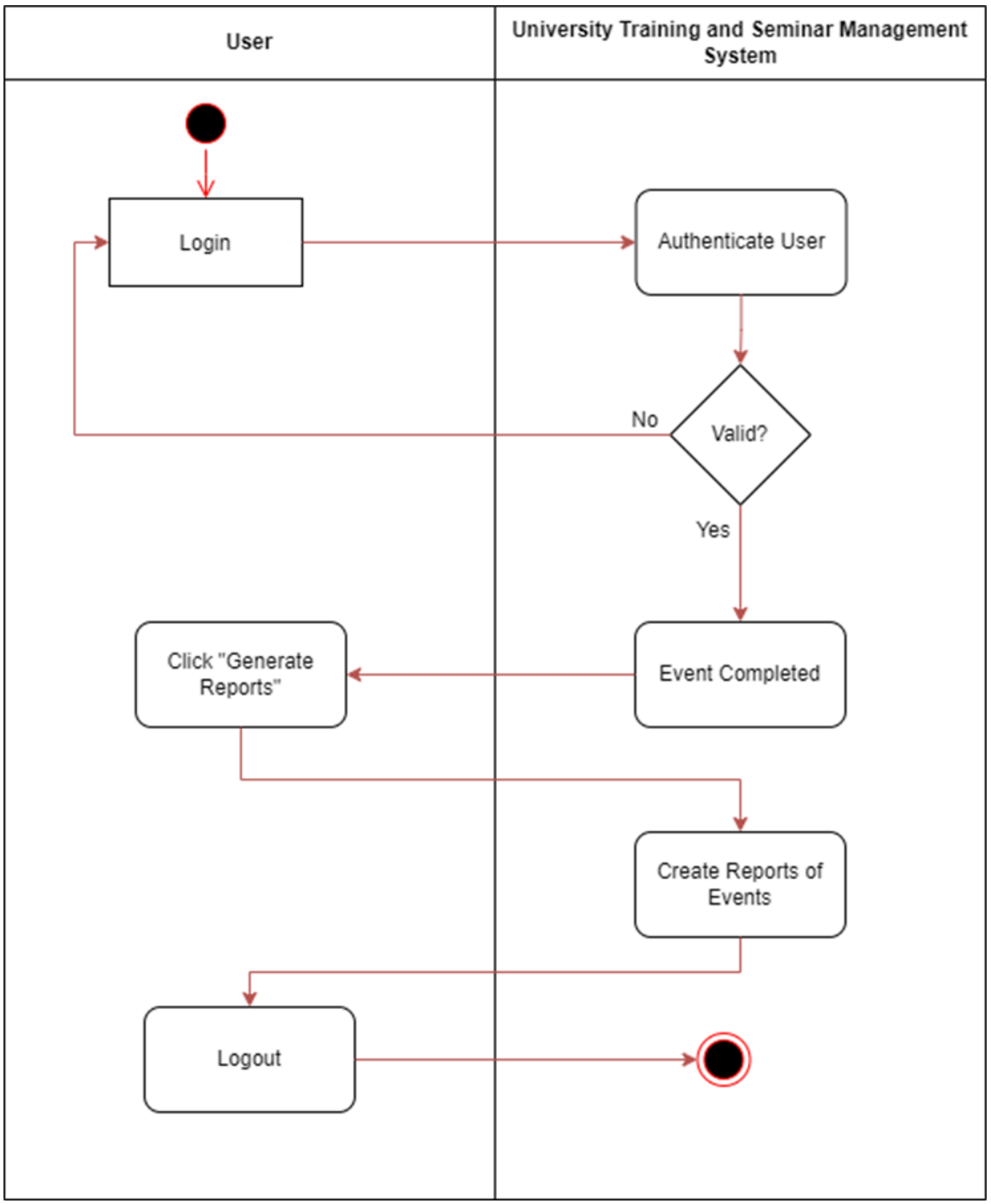


Figure 3.4 *The admin navigates to the reports section for completed events.*

Figure 3.4 shows the activity diagram for the admin's navigation to the report analytics page for completed events. After logging in and successfully authenticating, the admin is directed to the admin page, where they can access the event history. This section displays a list of completed events. Upon selecting a specific event, the system retrieves and presents detailed information about the event from the server. This includes reports and participant evaluations. Additionally, the admin has the option to generate and manage reports for the selected event, enabling comprehensive analysis and documentation.

3.2.2. Process View (User Flow Diagram)

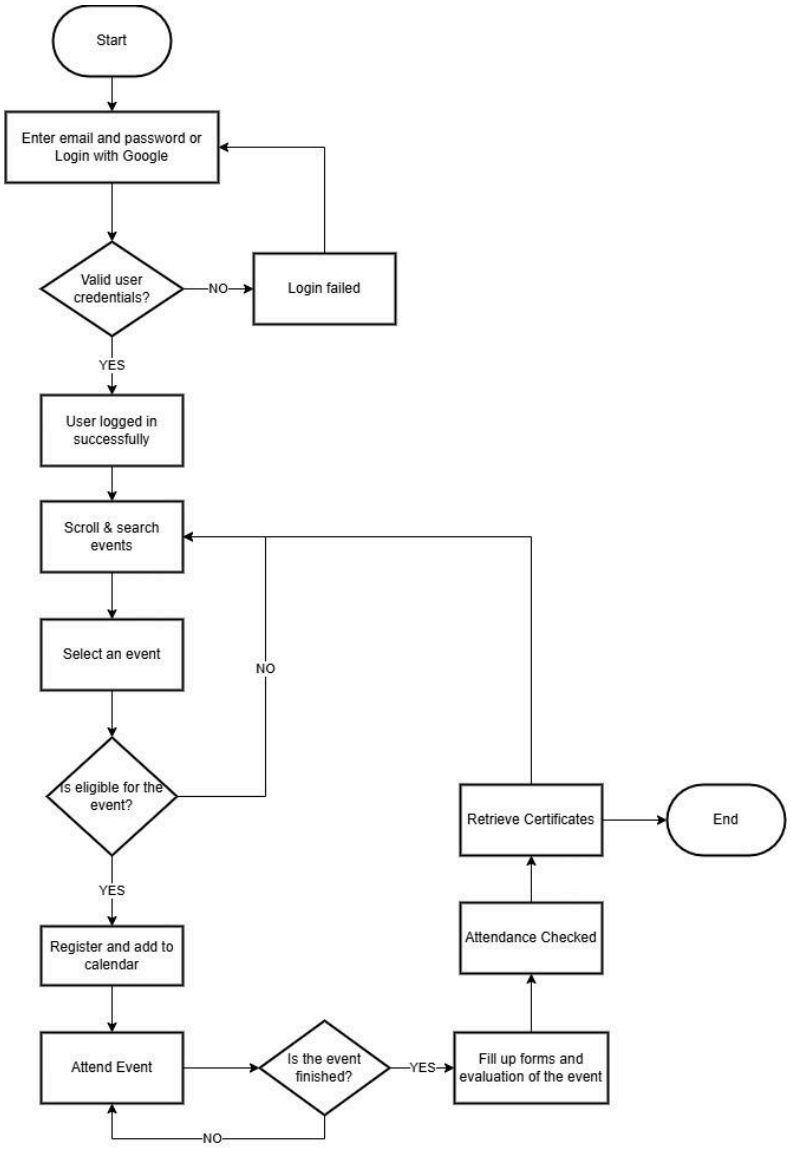


Figure 3.5 *User Flow Diagram*

Figure 3.5 visualizes the user flow through the system. The process begins with the user logging in using credentials provided by the admin. After successful authentication, the user is directed to the homepage displaying a list of events. The user can browse, search, and select an event to view its full details. The system then verifies if the user is eligible based on their invitation or department. If eligible, the user can register, automatically adding reminders to their calendar, including Google Calendar. After attending the event, the user must complete an evaluation form to record their attendance and retrieve any certificates provided.

3.2.3. Scenario View (Use-Case Diagram)

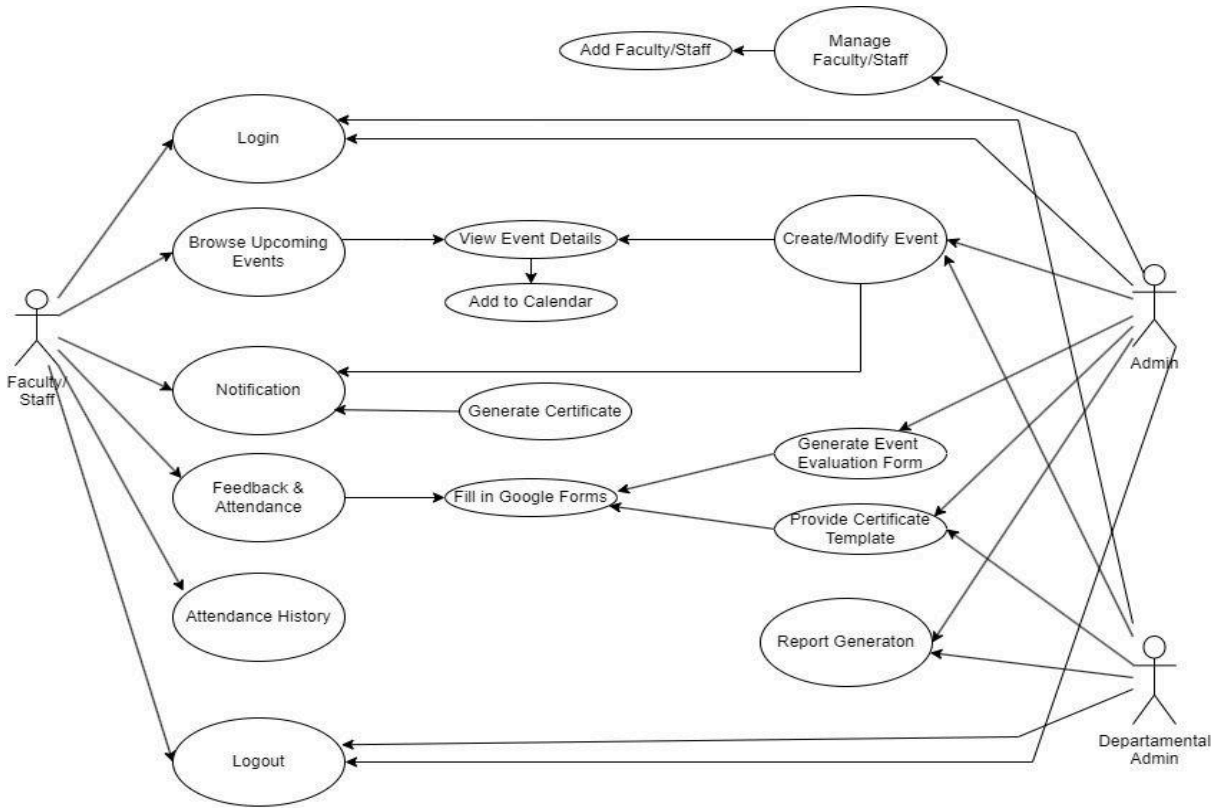


Figure 3.6 *Use-case diagram of the University Training and Seminar Management System.*

The University Training and Seminar Management System is designed to accommodate various user classes, each with distinct needs. Recognizing these user classes is crucial for tailoring the system to effectively manage training and events.

3.4. Project Schedule

	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
Task	**Sept 15 - 21**	**Sept 22 - 28**	**Sept 29 - Oct 5**	**Oct 6 - 12**	**Oct 13 - 19**	**Oct 20 -26**	**Oct 27 - Nov 2**	**Nov 10 - 16**	**Nov 17-23**	**Nov 24-30**	**Dec 1- 7**	**Dec 8 - 14**
Market Research												
Define Specifications												
Overall Archietecture												
Project Planning												
Detail Design												
Software Development												
Test Plan												
Testing and QA												
User Documentation												

Figure 3.7 *Gantt chart*

Figure above represents a 3-month Gantt chart for the development of the *University Training and Seminar Management System*, divided into weekly intervals. The project is structured into five main sections:

1. **Planning** involves market research, defining specifications, overall architecture and project planning in Weeks 1–4.
2. **Design** includes wireframe creation, database schema development, and UI/UX finalization in Detail Design from Weeks 3–4.
3. **Development** focuses on backend and frontend coding, as well as API integration, from Weeks 4–11.
4. **Analysis** covers user requirement analysis, identifying challenges, and preparing use-case diagrams in Weeks 11–12.
5. **Documentation** concludes the project, with user manuals, technical documentation, and final reviews in Weeks 7–12.

3.5. Integration Strategies

3.5.1. API Integrations and Interactions

The University Training and Seminar Management System utilizes six key APIs to enhance functionality, security, and user experience. These integrations streamline processes, improve communication, and ensure efficient data management. The following APIs are integrated into the system:

1. **OAuth**

This system integrates OAuth to enhance user authentication and authorization, providing a secure and seamless login process through Google Sign-In.

A. **Google Sign-In**

Role

Google Sign-In for secure user authentication. Users log in with their Google credentials, and their email is checked against a list of authorized addresses.

Benefits

Controlled Access: Only authorized users are allowed to sign in.

Seamless User Experience: Users can easily log in with Google credentials, eliminating the need for additional usernames or passwords.

Security: OAuth ensures secure authentication, and email verification restricts access to authorized users only.

Conclusion

The integration of Google Sign-In and email verification offers both ease of use and security, ensuring that only authorized users can access the system.

B. **Obtain access token and refresh token**

Role

The system uses two tokens: the access token for authenticating user requests, and the refresh token for obtaining new access tokens when the current one expires. The access token is short-lived, while the refresh token is long-term, ensuring continuous access.

Benefits

Continuous Access: The refresh token eliminates the need for repeated logins by renewing the access token automatically.

Security: OAuth ensures that only tokens (not passwords) are used for authentication, reducing the risk of data compromise.

Improved User Experience: Automatic token renewal provides a smooth, uninterrupted user experience.

Conclusion

The access and refresh tokens work together to ensure secure and continuous access to services, allowing users to stay logged in without frequent authentication.

C. Refresh Access Token using the RefreshToken

Role

When the access token expires, the refresh token is used to request a new one from Google's OAuth server, ensuring continuous access without requiring the user to log in again.

Benefits

Seamless User Experience:Users don't need to log in again after the access token expires.

Security: Refreshing tokens reduces the risk of compromised credentials since the user doesn't need to repeatedly reauthenticate.

Efficiency: The process runs automatically in the background, reducing manual intervention.

Conclusion

Using the refresh token to obtain a new access token provides uninterrupted, secure access and enhances the overall efficiency of the authentication process.

2. **reCAPTCHA**

This system integrates reCAPTCHA v3 to enhance security by detecting and preventing bot activity. It works in the background to analyze user interactions and

assigns a score to assess whether the user is a human or a bot.

A. **Bot Prevention**

Role

reCAPTCHA v3 helps prevent bots from accessing or interacting with the system by assigning a score based on user behavior. A low score indicates suspicious or automated activity, allowing the system to take appropriate action (e.g., blocking or requiring further verification).

Benefits

- No User Disruption: Unlike previous versions, reCAPTCHA v3 does not interrupt the user experience with puzzles or challenges, ensuring seamless interaction.
- Advanced Detection: Uses machine learning to detect complex bot behavior and more accurately distinguish between human and automated users.
- Customizable Security: The system allows the setting of a threshold score for different actions, enabling tailored responses based on the perceived risk level.

Conclusion

By integrating reCAPTCHA v3, the system can efficiently prevent bot interactions while maintaining a smooth and uninterrupted user experience. This not only secures the system but also enhances the user journey by avoiding disruptive security challenges.

3. Gmail API

This system integrated the Google Mail (Gmail) API. It significantly enhances communication with patients and streamline operations. Here's an overview of how Google Mail can play a crucial role in various aspects in the University Training and Seminar Management System;

A. Account Activation

Role: After the admin, who is the CITL, creates a new account for faculty/staff or departmental admins, an email is sent through Gmail to activate their account. This email includes a temporary password that they must use to log into the system and change their password. This process helps ensure that only legitimate users can access the system and protects patient data.

Benefits:

Security:

- Ensures only legitimate users can access the system by requiring email-based account activation and mandatory password updates.
- Protects sensitive data by preventing unauthorized access through temporary credentials.

User Experience:

- Simplifies the onboarding process by guiding users through account activation and password changes.
- Provides clear instructions and a seamless login experience to ensure ease of use.

Conclusion:

The account activation process plays a critical role in maintaining system integrity and security. By requiring email-based activation and a mandatory password change, it ensures only authorized users can access the system, thereby protecting sensitive data. Additionally, the process enhances the user experience by providing a straightforward and intuitive onboarding experience, fostering trust and confidence in the system's usability and security.

B. Emailing Generated Certificates

Role: After users (faculty/staff) complete and verify their Google Forms submissions, the system generates certificates automatically. Using Gmail, these certificates are emailed to the respective users, ensuring a seamless and efficient process for delivering recognition documents. This functionality eliminates manual intervention, saving time and reducing errors while providing users with immediate access to their certificates.

Benefits:

Efficiency

- Automates the certificate distribution process, minimizing manual effort and delays.
- Ensures users receive their certificates promptly after verification.

User Experience

- Provides a hassle-free process where users can directly receive their certificates in their inbox.
- Enhances satisfaction by maintaining a professional and streamlined communication approach.

Conclusion:

The functionality of emailing generated certificates via Gmail demonstrates a commitment to both efficiency and user satisfaction. By automating the certificate distribution process, the system not only ensures prompt delivery but also maintains a professional and user-friendly experience. This feature complements the system's overall goal of enhancing operational effectiveness and user convenience.

4. **Google Sheets API**

This system integrates the Google Sheets API to manage and store survey responses submitted by users. The integration facilitates efficient data collection and organization by automatically saving responses from Google Forms into a Google Sheets document. Here's an overview of how the Google Sheets API plays a crucial role in various aspects of the system:

A. **Survey Data Storage**

Role:

The system uses the Google Sheets API to automatically store survey responses submitted via Google Forms into a Google Sheets document. This ensures a centralized, organized, and accessible repository for all survey data, allowing for easy data retrieval and analysis.

Benefits:

Data Management:

- Stores survey responses in real-time within a Google Sheets document, offering a structured format for easy management and analysis.

- Leverages Google Sheets' built-in functions like sorting, filtering, and formulas to assist in data processing.

Integration and Accessibility:

- Provides real-time synchronization between Google Forms and Google Sheets, ensuring up-to-date data.
- Makes the data easily accessible to authorized personnel for analysis, reporting, and decision-making.

Conclusion

The integration of Google Sheets for survey data storage streamlines the data management process, ensuring that responses are collected, stored, and made accessible in a structured, efficient manner. This promotes effective data handling and simplifies subsequent analysis and reporting.

B. Submission Confirmation

Role

The system uses the Google Sheets API to confirm that a user's form has been successfully submitted. Upon form submission, the status of the submission is updated in the Google Sheet, providing real-time feedback to the user.

Benefits:

Immediate Feedback:

- Updates the Google Sheets document with a submission status field, indicating whether the form has been successfully submitted.
- Provides users with instant confirmation that their survey responses have been recorded.

Transparency and Trust:

- Enhances user confidence by immediately confirming submission, reducing concerns about whether their responses have been received.

Conclusion:

The use of the Google Sheets API for submission confirmation ensures that users receive immediate feedback on their form submission. This not only boosts user

satisfaction by providing transparency but also improves the overall user experience by offering reliable confirmation of data submission.

5. **Google Calendar API**

This system integrates the Google Calendar API to automatically add registered events to the user's calendar. It also provides an option for the admin to configure reminder notifications, alerting users before an event starts. These features work together to ensure users stay informed and on track with their event schedules.

Conclusion

A. Event Registration and Calendar Integration

Role:

The system automatically adds event details to the user's Google Calendar when they register for an event. This includes the event name, date, time, and any relevant information, ensuring that users can easily access their event details from their calendar.

Benefits:

Automation:

- Automatically adds events to the user's calendar without requiring manual input.
- Reduces the risk of errors and ensures that event details are accurately stored.

User Convenience:

- Streamlines the event registration process by eliminating the need for users to manually add events to their calendar.
- Helps users stay organized by providing a clear view of upcoming events directly on their calendar.

Conclusion:

By integrating the Google Calendar API, the system simplifies event registration by automatically adding events to the user's calendar. This enhances user experience and ensures that all event details are readily accessible, helping users stay organized.

B. Event Reminder Notification

Role

The system allows the admin to configure reminder notifications for users before an event starts. These reminders can be set to alert users at customizable intervals, such as 1 hour, 1 day, or 1 week before the event. The reminder notifications are sent via Google Calendar, ensuring users are adequately prepared for the event.

Benefits:

Customizable Reminder Intervals:

- Admins can set reminders at specific times, such as 1 hour, 1 day, or 1 week before the event, based on user preferences or event importance.
- Helps ensure that users are reminded at the right time to adequately prepare for the event.

Timely and Effective Alerts:

- Reminders provide users with ample time to plan ahead, whether it's 1 hour, 1 day, or even 1 week before the event.
- Ensures that users are well-informed and reduces the chance of them missing the event due to forgetfulness or poor time management.

Conclusion:

The customizable event reminder notification feature enhances the user experience by ensuring timely alerts at adjustable intervals. Admins can set reminders that best suit the event and user needs, offering users enough time to prepare and attend events without worry.

6. **Google Forms API**

This system integrates the Google Forms API to collect and manage survey responses from users. When users complete a survey or feedback form, the responses are automatically stored in a Google Sheet, providing an efficient way to analyze and manage data. This

integration streamlines the process of gathering and organizing feedback, making it easier for administrators to monitor user responses.

A. Survey Data Collection

Role:

The system utilizes the Google Forms API to gather survey responses from users. Once a user submits a form, their responses are automatically recorded and stored in a connected Google Sheet. This eliminates the need for manual data entry and ensures that the data is organized and accessible for further analysis.

Benefits:

Efficiency:

- Automatically collects and stores user responses in a Google Sheet, saving time and reducing human error.
- Allows for easy access and management of survey data in a centralized location.

Real-Time Data:

- Provides real-time updates of survey responses, ensuring that administrators have immediate access to the latest data.
- Facilitates quick decision-making based on the responses collected from users.

Ease of Use:

- Users can easily fill out and submit forms, while administrators can effortlessly monitor and analyze the data in an organized format.

Conclusion

Integrating the Google Forms API simplifies the process of collecting and managing survey data, automating data entry and ensuring that responses are easily accessible and ready for analysis. This improves overall efficiency and provides administrators with real-time data for better decision-making.

3.5.2. Integration of Subject Components

The University Training and Seminar Management System operates as a multiple unit through the integration of its various subject components. These subject components include:

- Frontend and Backend Integration - The frontend client, responsible for user interaction, communicates with the backend services via RESTful APIs. This enables seamless data retrieval, updates, and user management processes, ensuring that users can interact with modules like event registration, feedback, and attendance tracking in real-time.
- Database Interaction - The backend services are tightly coupled with the MongoDB database, which acts as the central repository for storing and managing system data such as user profiles, events, feedback, and attendance records. This integration ensures data consistency and availability across all components.
- Authentication and Authorization - The authentication module secures access by validating user credentials through third-party providers like Google OAuth. It ensures only authorized users access specific features, coordinating with the backend to enforce role-based permissions for administrators, employees, and students.
- Notification System - The notification component works with the backend to send reminders and updates to users via email or in-app notifications.
- Analytics and Reporting - The reporting module compiles data from various parts of the system, such as user attendance, event feedback, and registration statistics. It relies on data aggregation and visualization tools in the backend, providing insights to administrators for decision-making.

3.5.3. Data Flow and Communication

The University Training and Seminar Management System's data flow and communication processes are designed to ensure efficient, secure, and reliable information exchange across various system components. These pathways and protocols form the foundation of seamless interaction among the system's modules including:

- Authentication and Authorization - User credentials are securely transmitted to the backend, which verifies them against the database. Successful authentication generates a session token for subsequent requests.

- Event Management - Event details entered by administrators are sent to the backend, processed, and stored in MongoDB. Users and admin retrieve event data through API calls, ensuring they view only relevant and authorized information.
- Feedback Submission - Participants submit feedback forms via the google forms. The backend validates and stores this data in the database, enabling administrators to access evaluation reports.
- Error Handling - In cases of invalid inputs or system errors, the backend generates detailed error messages, which are communicated to the front-end for user notification and debugging purposes.

Chapter 4
SECURITY POLICY

4.1. IS Policy Document

The Information Security (IS) Policy Document is a critical component of our proposal for the University Training and Seminar Management System (UTSMS). It outlines the security measures necessary to protect sensitive information and ensure the integrity of the system as it supports training sessions and events conducted by the Center for Innovative Teaching and Learning (CITL).

4.1.1. Hardware Security Policy

- **Objective**: This policy ensures that all hardware components used in the BukSU Engage system are physically secured and protected from unauthorized access, tampering, or theft. The policy includes:
    - Secure storage of servers, workstations, and network devices.
    - Physical access control with only authorized personnel having access to critical hardware.
    - Regular audits and checks to ensure hardware integrity.
    - Backup systems and disaster recovery measures for hardware failures.

4.1.2. Software Security Policy

- Objective: This policy outlines the principles for ensuring the security of software applications, including the BukSU Engage platform. Key areas include:
    - Secure coding practices to prevent vulnerabilities (e.g., SQL injection, XSS).
    - Regular software updates and patches to address security vulnerabilities.
    - Licensing compliance and software integrity checks.
    - Access control for software configuration and management.
    - Monitoring and logging of software activities for suspicious actions.

4.1.3. Network Communications Policy

- Objective: To protect data transmitted over networks associated with UTSMS.
- Measures:
    - Use of encryption protocols such as SSL/TLS for data transmission.
    - Secure Virtual Private Network (VPN) connections for remote access.
    - Monitoring network traffic for unauthorized access or anomalies.

- Restricting access to sensitive data through network segmentation and firewalls.
- Regular audits to check network security.

4.1.4. Data Policy

- Objective: This policy ensures the proper handling, storage, and disposal of data within the BukSU Engage system, including user data and event-related information. Key areas include:
  - Data encryption at rest and in transit to prevent unauthorized access.
  - Clear guidelines for data retention, backup, and disposal.
  - Data access control based on user roles (faculty, staff, and admin).
  - Protection of personal and sensitive information in accordance with applicable data protection laws (e.g., GDPR, PH Data Privacy Act).
  - Regular data integrity checks and audits.

4.1.5. Process Policy

- Objective: This policy defines the procedures and workflows for maintaining the security and efficiency of processes within the BukSU Engage system. Key areas include:
  - Secure user authentication and authorization for all platform functions.
  - Systematic scheduling, event management, and reporting processes.
  - Incident management processes for addressing security breaches or system failures.
  - Regular training for administrators and staff on security best practices.
  - Process reviews and audits to ensure security compliance.

4.1.6. People Policy

- Objective: This policy defines the procedures and workflows for maintaining the security and efficiency of processes within the BukSU Engage system. Key areas include:
- Measures:
  - Role-based access control for system users (e.g., admins, faculty, staff).

- Ongoing user awareness programs about security risks and best practices.
- Background checks for users with access to critical administrative functions.
- Procedures for reporting suspicious activities or breaches within the system.
- Regular performance reviews to ensure compliance with security policies.

4.2. IT Security Management Framework

The IT Security Management Framework provides a structured approach to managing information security risks associated with UTSMS.

4.2.1. Legal Compliance

- Objective: BukSU Engage complies with various legal standards to protect user data and ensure secure operations. This includes:
  - Data Protection Laws: Adhering to the Data Privacy Act of 2012 to protect personal information.
  - Intellectual Property Laws: Following copyright and licensing regulations for software and content.
  - Regulatory Compliance: Meeting higher education and university policies for academic and administrative systems.
  - Cybersecurity Laws: Ensuring compliance with the Cybercrime Prevention Act of 2012 to address cyber threats.
  - GDPR Compliance: Following the General Data Protection Regulation (if applicable) for protecting data of EU users.

4.2.2. Security Governance Standard

- Objective: Security governance ensures IT security is integrated into operations, aligning with BukSU Engage's goals and security objectives. Key elements include:
  - Security Policies and Procedures: Clear guidelines for data protection, access control, and incident response.
  - Accountability and Responsibility: Defining roles for IT security and ensuring users understand their security obligations.
  - Compliance Auditing: Regular audits to ensure adherence to policies, legal standards, and identify vulnerabilities.
  - Incident Management: Structured processes for handling and learning from security breaches.

    - Security Training and Awareness: Ongoing programs to keep users informed of security risks and best practices.
    - Continuous Improvement: Regular updates to security practices based on feedback and evolving threats.

4.3. Network Security Testing

Network security testing is vital for identifying vulnerabilities within the infrastructure supporting UTSMS.

4.3.1. Network and Server Profiling

- Objective: To regularly assess network configurations and server setups to ensure compliance with security standards.
- Measures:
    - Configuration Assessment: Evaluating network and server setups for vulnerabilities.
    - Traffic Monitoring: Observing network traffic for abnormal patterns or threats.
    - Access Control: Ensuring only authorized users have access to critical network resources.
    - Security Measures: Verifying that firewalls, encryption, and other security tools are in place.

4.3.2. CVSS

- Objective: To evaluate the severity of vulnerabilities affecting systems used by UTSMS.
- Measures:
    - Severity Scoring: Evaluating the severity of vulnerabilities based on their exploitability and potential impact.
    - Prioritization: Helping prioritize remediation efforts based on CVSS scores.
    - Risk Assessment: Providing a standardized way to assess and communicate the risk associated with vulnerabilities.

Table 4.1 *Vulnerability Assessment Table*

Vulnerability	CVSS Score	Severity	What Can Happen	What to Do
Authentication Bypass	9.8	Critical	Remote exploit, no authentication required	Unauthorized access to sensitive data (user schedules, events)
SQL Injection	9.0	Critical	Hackers can mess with the database, steal or delete data (like event details and feedback).	Use safer coding practices like prepared statements to avoid this kind of attack.
Sensitive Data Exposure	8.0	High	Hackers could intercept sensitive data (like user details) if it's sent over an unencrypted connection.	Use HTTPS to encrypt data and prevent hackers from stealing it.
Access Control Issues	5.0	Medium	Regular users could access parts of the system they shouldn't, like event management.	Make sure only authorized users can access certain parts of the system.

4.4. Solution Design

Solution design is a critical phase in the development of the University Training and Seminar Management System (UTSMS). It encompasses the architectural elements necessary for implementing secure systems that meet both functional and security requirements.

4.4.1. Policy Design

- Objective: Develop comprehensive security policies that align with organizational goals and industry best practices.
- Details:

- **Access Control Policy**: To ensure that only authorized users (faculty, staff, and administrators) can access certain features or data, role-based access control (RBAC) will be used. Each user type will have access only to the relevant data and functionalities.
- **Data Protection Policy**: This policy will dictate how personal information, including names, emails, event details, and attendance data, will be collected, stored, and processed. Personal data will be encrypted both at rest and during transmission.
- **Incident Response Policy**: This policy will outline the steps to follow in case of a data breach or other security incidents. It will include reporting mechanisms, damage assessment, and communication procedures for affected users.

4.4.2. Logical Design

- Objective: Define how various components of UTSMS will interact securely while maintaining data integrity and confidentiality.
- Details:
  - **Role-Based Access Control (RBAC)**: Logical design will incorporate RBAC to define and manage user roles. For example, an administrator may have full access to all system features, while a faculty member may only have permission to create and manage events.
  - **Encryption**: Sensitive data, such as user details, event details, activity logs, submissions and certificates, in the BukSU Engage system will be encrypted using industry-standard encryption protocols, such as Mongoose.
  - **User Authentication**: The system will use **JWT (JSON Web Tokens)** for user authentication. This ensures that once users log in, their credentials are securely transmitted and stored, and they can access system functionalities based on their role.

4.4.3. Physical Design

- Objective: Focus on creating a secure physical environment for housing hardware components.
- Details:

- **Server Setup**: Secure servers for hosting **BukSU Engage**, ensuring they are protected by firewalls and intrusion detection systems.
- **Data Storage**: Ensure secure physical storage of databases, with redundant backups for disaster recovery.
- **Network Security**: Implement network segmentation and use secure protocols like HTTPS to prevent unauthorized access over the internet.

4.4.4. IS implementation Design (if applicable)

- Objective: Detail strategies for integrating information security measures into existing systems with minimal disruption.
- Details:
  - **Integration Planning:** Develop a phased approach for integrating security measures into current workflows without affecting ongoing operations.
  - **Testing Environments:** Set up testing environments to evaluate the effectiveness of security measures before full-scale implementation.
  - **User Training:** Provide training sessions for users on new security protocols and tools to facilitate smooth adoption.

4.4.5. Intrusion and Detection Systems

- Objective: Implement systems designed to monitor network traffic continuously for suspicious activities.
- Details:
  - **reCAPTCHA Integration:** To prevent bot-driven attacks and unauthorized automated access, **reCAPTCHA** is integrated into the system. It helps to verify that interactions on the platform are from legitimate users, effectively blocking automated scripts from submitting malicious data or trying to exploit vulnerabilities.
  - **Logging and Monitoring**: Set up logging systems to track user activity and system events for audit and analysis.

4.4.6. Secure Process Model

- Objective: Establish secure processes throughout the event management lifecycle in UTSMS.
- Details:

- **Security Testing**: Regular security testing, such as penetration testing, will be conducted during the development phase and after updates to ensure that new vulnerabilities are not introduced.
- **Ongoing Security Maintenance**: Once the system is deployed, ongoing monitoring will be performed to identify emerging security threats, with regular updates and patches to address vulnerabilities. Security best practices will be followed to ensure continuous protection of the system.

4.5. Business Continuity Plan

The Business Continuity Plan is essential for ensuring that UTSMS can maintain operations in case of disruptions or disasters.

4.5.1 Incident Response Plan

- Objective: Outline procedures for effectively responding to security incidents.
- Details:
  - **Identification:** Develop methods to detect and identify potential security incidents in real-time.
  - **Containment:** Implement strategies to contain the incident to prevent further damage. For example, isolate affected systems or disable user accounts involved in suspicious activity.
  - **Eradication:** Remove the root cause of the incident, such as deleting malicious files, updating software vulnerabilities, or blocking compromised IP addresses.
  - **Recovery:** Ensure that impacted systems are restored to normal functionality with minimal downtime. This may involve restoring from backups, applying patches, or re-enabling affected accounts.
  - **Communication:** Communicate with stakeholders (faculty, staff, and administrators) regarding the incident's impact and actions being taken.

4.5.2. Disaster Recovery Plan

- Objective: Detail recovery steps necessary for restoring critical systems and data following catastrophic events impacting UTSMS operations.
- Details:

- **Risk Assessment:** Identify potential risks that could disrupt operations, including natural disasters (earthquakes, floods), cyberattacks, or hardware failures.
- **Recovery Strategies:** Define recovery strategies for critical components of the **BukSU Engage** system, such as servers, databases, and network infrastructure. This includes having **off-site backups** and cloud-based systems for redundancy.
- **Backup and Redundancy:** Implement regular data backups and ensure redundancy in key system components. This will allow for quick restoration of the system in case of a disaster. Cloud storage solutions can be used for off-site backups.
- **Testing and Drills:** Regularly test recovery processes to ensure that the plan is effective and that team members are prepared for a disaster situation. Simulated disaster recovery drills can help staff become familiar with recovery procedures.
- **Communication:** Ensure that communication channels are in place to notify all stakeholders in case of a disaster. This includes informing the **BukSU Engage** team, faculty, staff, and students about the incident and the recovery status.

Chapter 5

CONCLUSION

5.1. Reflection on Architecture Decisions

The architecture of our system relied heavily on RESTful APIs, with a focus on integrating Google's API suite, including Google Forms, Sheets, Calendar, OAuth, Gmail, and reCAPTCHA. This decision streamlined functionality, making monitoring and usage more intuitive and efficient. Utilizing Google APIs ensured robust data handling and a user-friendly experience, while also enabling smooth integration with various system components.

However, some choices posed challenges. For example, while the integration of Google's core APIs proved beneficial, we initially planned to include the Google Analytics API. During development, we recognized that it offered limited relevance to our project's objectives and decided to remove it. This pivot highlighted the need to evaluate tool utility in real-world applications.

Overall, the architectural approach supported scalability, security, and efficiency, aligning with our project's goals. Building from scratch allowed us greater flexibility, but it also presented challenges in responsiveness and design optimization.

5.2. Difficulties Encountered

The development process was not without its hurdles. Bugs and fixes during API integration required substantial effort. We faced challenges understanding how to utilize the APIs effectively within our system's framework. Additionally, balancing multiple Google API integrations in a way that maintained smooth functionality and user experience demanded iterative testing and adjustments.

Dropping the Google Analytics API mid-development was a difficult decision but necessary to maintain focus on essential functionalities. This experience underscored the importance of aligning project scope with practical needs.

5.3. Improvements and Extensions

One key area for improvement lies in the responsive UI design. As we developed the website from scratch rather than using pre-existing templates, ensuring a fully responsive interface across devices remains a priority. Future efforts should focus on refining the UI/UX for better adaptability and accessibility.

Potential extensions for the system could include incorporating more advanced analytics features, such as integrating custom dashboards for event insights. Additionally, exploring automation tools or adding AI capabilities for event scheduling or feedback analysis could further enhance the system's functionality.

5.4. Conclusion

Completing this project has been both a challenging and rewarding experience. We are proud of the outcome and satisfied with the functionality and performance of the system we developed. Through this project, we gained valuable insights into the MERN stack, including React, Node.js, Express, and MongoDB, and the intricacies of integrating RESTful APIs, particularly Google's robust suite. These technical lessons have enriched our understanding of full-stack development and API-driven systems.

Effective time management was key to our success, allowing us to navigate challenges and deliver the project as planned. For future researchers or developers working on similar projects, we recommend a strong emphasis on planning and aligning tool sets with the project's specific needs. This focus ensures streamlined development and avoids unnecessary complexity.

References

IBM i 7.3. (2024). https://www.ibm.com/docs/en/i/7.3?topic=system-example-disaster-recovery-plan

CVSS v4.0 Examples. (2024). FIRST — Forum of Incident Response and Security Teams. https://www.first.org/cvss/examples

GeeksforGeeks. (2024). *Coding standards and guidelines*. GeeksforGeeks. https://www.geeksforgeeks.org/coding-standards-and-guidelines/

Jona Tarlengco. (2024, November 15). SafetyCulture. https://safetyculture.com/topics/security-risk-assessment/

Pulsifer, E. (2003). What is an API? Use Cases and Benefits. *Kong Inc.* https://konghq.com/blog/learning-center/what-is-api

Appendices

Table 5 . *Summary of Achievements*

Objective	Achievements	Date
Planning	● The interview with the CITL was successfully conducted. ● Analyzed the system requirements based on the interview and the advice provided by the adviser. ● Assigned team members to specific tasks within the system.	Sep 1 to 17, 2024
Designing	● Figma was used as prototype user interface ● Developed use case, component, and activity diagrams for the system. ● Planned the system architecture, database schema, concurrency control, and role-based access control.	Sep 18 to Oct 18 ,2024
Development	● Developed the system frontend using React. ● Developed the system backend using Node.js. ● Utilized bootstrap and css for styling ● Utilized RESTful API for the system. ● Implemented 6 APIs. ● Used MongoDB Atlas as the database. ● Integrated concurrency control for event creation by admins.	October 22 to December 6, 2024
Testing and Debugging	● Integrated testing and debugging processes for the system.	December 6 - 9, 2024
Documentation	● Prepared and finalized all project documentation.	December 9 - 13, 2024

Table 6. *Member's Contributions and Roles*

Team Member	Roles	Contributions
James Rio Abaquita	● Backend developer ● Database Administrator ● Security Specialist ● API Integration Specialist	● SR.1: Arrange Routes (Admin) ● ADREQ.1: Admin Dashboard Page (backend) ● SR.4: Login Form with Google Auth (Backend) ● ADREQ.2: Add Personnel Account Page (Backend) ● SR.2: Creation of GIT Repository (Creation of Branches) ● SR.3: System Connection to the Database ● USREQ.7: User Event Registration (backend) ● SR6: Feedback Collection (backend) ● SR7: Automated Certificate Generation and Delivery ● SR9: Send Email to Users' Temporary Passwords ● USERQ.9 Cancel Event Registration ● ADREQ.13 Activity Logs (backend) ● SR10: Encryption
Vivian Bangcoyo	● UI/UX Designer ● Frontend developer ● Usability Tester	● SR.4: Login Form with Google Auth (front end) ● USREQ.2 - Integration of Users Event Calendar (Front End) ● USREQ.3: User Profile (Front End & Backend) ● USREQ.4: Certificate Gallery (Front End & Backend) ● USREQ. 5: Kanban Events (Front End & Backend) ● USREQ.7: User Event Registration (Front End) ● USERQ. 8: View Events (Front End & Backend) ● SR5: Log Out ● SR6: Feedback Collection (Front End)
Feby Angela H. Felices	● Project Manager ● Backend Developer ● Quality Assurance Tester	● ADREQ.1: Admin Dashboard Page (Front End) ● ADREQ.4: Creation of Events (Front End) ● ADREQ.5: Integration of Admin Event Calendar (Front End)

		● ADREQ.3: Admin Users Table Page (Front End & Backend) ● SR.2: Creation of GIT Repository ● USREQ.3: User Profile (Backend) ● USREQ.6: User Notification System (Backend) ● ADREQ.6: Events Modification (Backend) ● ADREQ.7. Edit Personnels (Frontend & Backend) ● ADREQ.9 Cancel Event (Frontend & Backend) ● ADREQ.10: Archive User (Frontend & Backend) ● ADREQ.12: Admins Concurrency Control (Frontend & Backend)
Keith Einlou J. Pogoy	● Full-Stack Developer ● UI/UX Designer ● Quality assurance testing for both frontend and backend	● SR.1: Arrange Routes (Users) ● ADREQ.1: Admin Dashboard Page (Front End) ● USREQ.1: Home Page (Front End & Backend) ● ADREQ.4: Creation of Events (Front End) ● ADREQ.2: Add Personnel Account Page (Front End) ● ADREQ.5: Integration of Admin Event Calendar (Frontend & Backend) ● USREQ.6: Upcoming Events ● ADREQ.8: Event Report Analysis ● USREQ.6: User Notification System (Front End) ● SR8: Modals/Alerts ● ADREQ.11: Event History ● ADREQ.13 Activity Logs (Front End)

API Documentation

1.1 USER MODULE API

1.1.1 Module Description

This encompasses individuals such as faculty and staff members at Bukidnon State University. It takes to define the functionalities, access points, and operational workflows that allows the users to efficiently manage including browse an event, register an event, modify their user

The API have the following endpoints:

Profile and User Information Endpoints

http://localhost:3000/u/profile
http://localhost:3000/u/certificates
http://localhost:3000/u/certificates/:certificateID
http://localhost:3000/u/history

Event User Endpoints

http://localhost:3000/u/events
http://localhost:3000/u/events/:id
http://localhost:3000/u/calendar
http://localhost:3000/u/events/:id/check-registration
http://localhost:3000/u/events/:id/cancellation

Form Endpoints

http://localhost:3000/u/events/:id/form-submission
http://localhost:3000/u/events/:id/form-status
http://localhost:3000/u/events/:id/form-analytics
http://localhost:3000/u/events/:id/certificate-status

Notifications Endpoints

http://localhost:3000/u/notification/items
http://localhost:3000/u/notification/update/:id
http://localhost:3000/u/notification/remove/:id'

1.1.2 Retrieve User Profile

Version: 1.0

Date: October 30, 2024

Description: This API endpoint will retrieve the current user's profile in session from the database

Endpoint: http://localhost:3000/u/profile

Method: GET

Configurations:

The API request should be validated using a token.

Parameters:

- **authToken –** This is the JWT token retrieved from sessionStorage, used for authorization in API requests, and decoded to obtain the current user data details.

Response codes of this API endpoint:

Code	Message	Description
200	Request Successful (OK)	The request was valid and successfully retrieved data from the database by decoding the user's session token.
401	Access Denied. No Token Provided (Unauthorize)	This is because the token was not provided, meaning it does not have authorization to retrieve the user's profile data.
403	Invalid Token (Invalid Request)	The user token was either invalid or expired
500	Error Retrieving User Details (Internal Server Error)	This is because an error occurred on the server side.

Valid and Invalid Request Responses

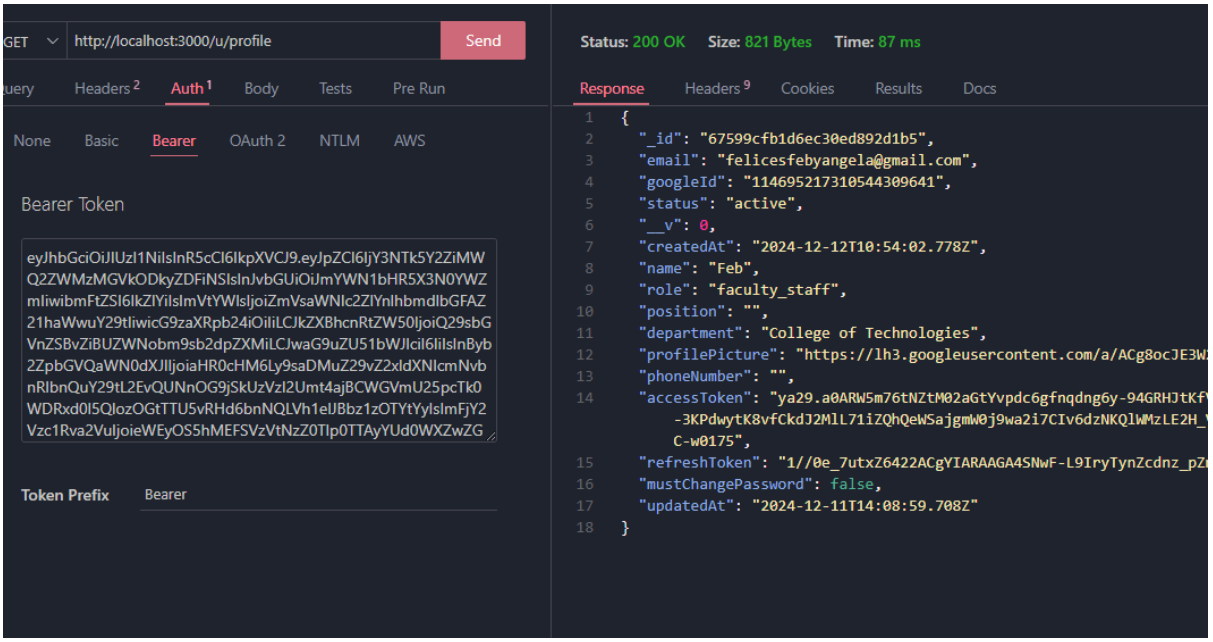


Figure 1. *User Profile GET Valid Request*

This figure shows a valid GET request code 200 for decoding the user session token in order to display the data, such as profile picture, name, email, phone number, department and position

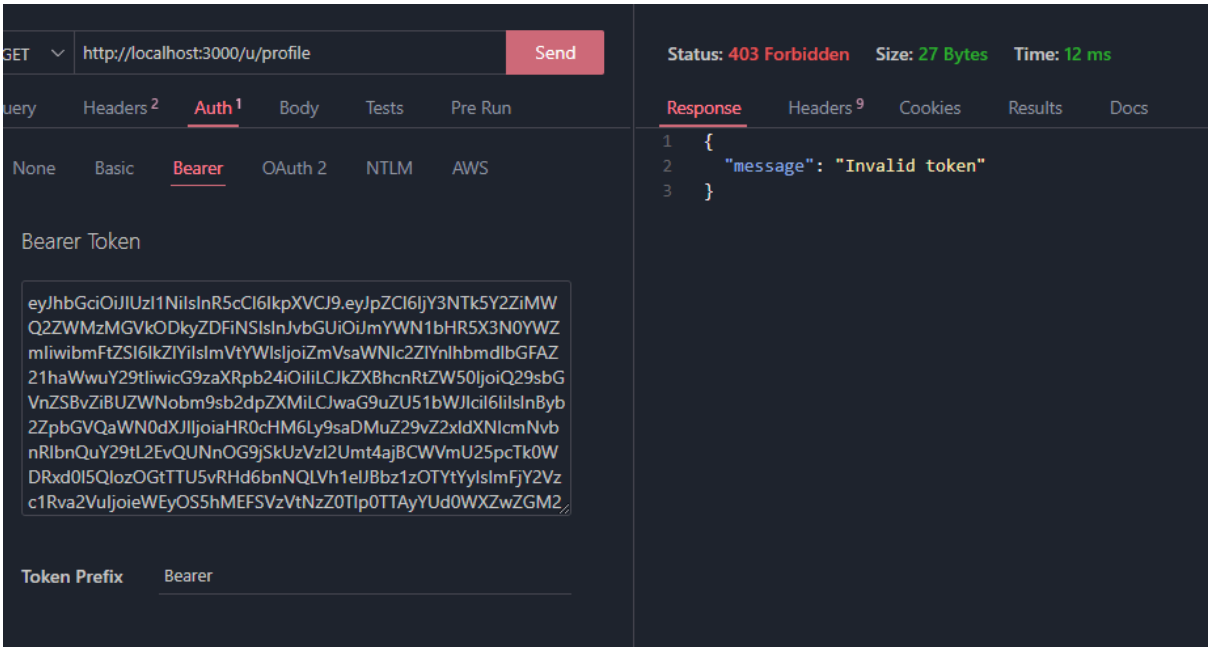


Figure 2. *User Profile GET Invalid Request*

This figure shows the invalid GET request code 403 for the user profile. It could be due to an invalid token, meaning the token is either incorrect or has expired

Response Format: JSON

Response:

This API response is to display the user profile data, including their name, email, department, and position.

Different response Screenshots

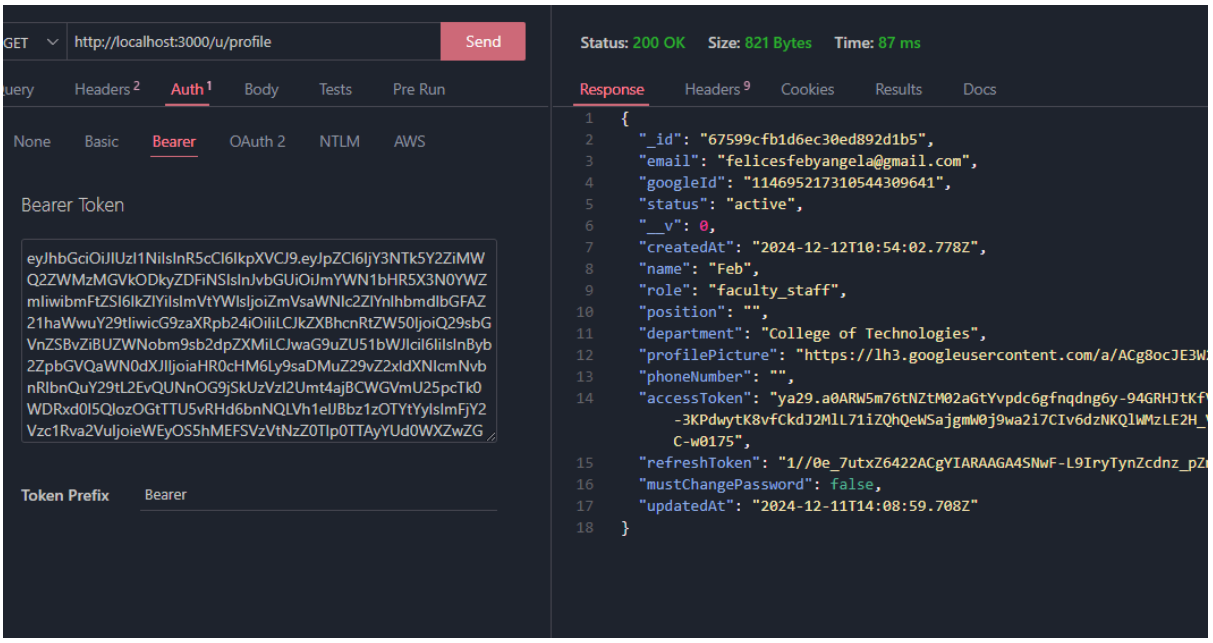


Figure 3. *User Profile Success Response code 200*

This figure shows the success response code 200, which indicates that the token was successfully authorized and the user profile data was fetched from the database.

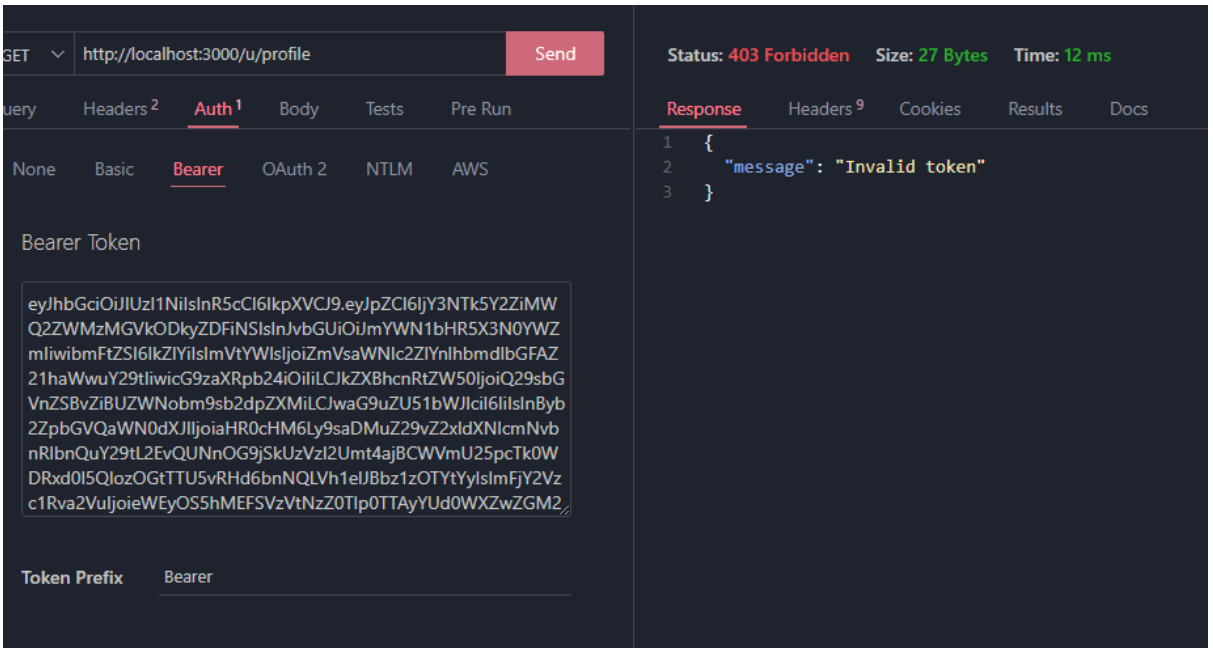


Figure 4. User Profile Error Response code 403

This figure shows the error response code 403, which indicates that the token is invalid or expired, and the user profile data could not be fetched from the database

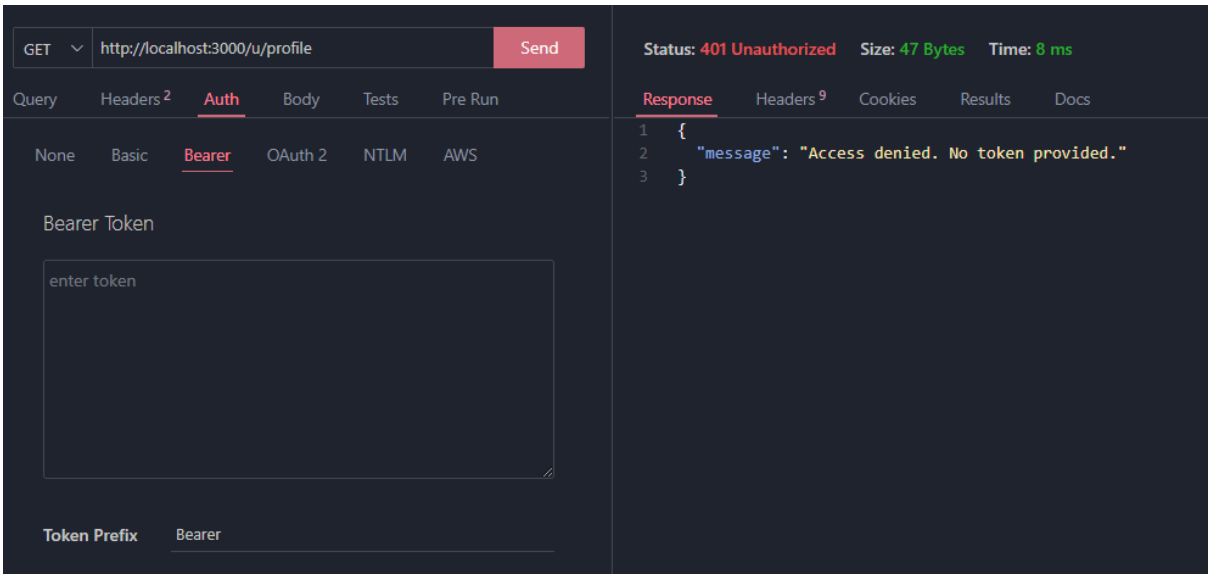


Figure 5. User Profile No Token Provided Response code 401

This figure shows the 'No Token Provided' response code 401, which indicates that the token is unauthorized because it was not provided, and the user profile data could not be fetched from the database.

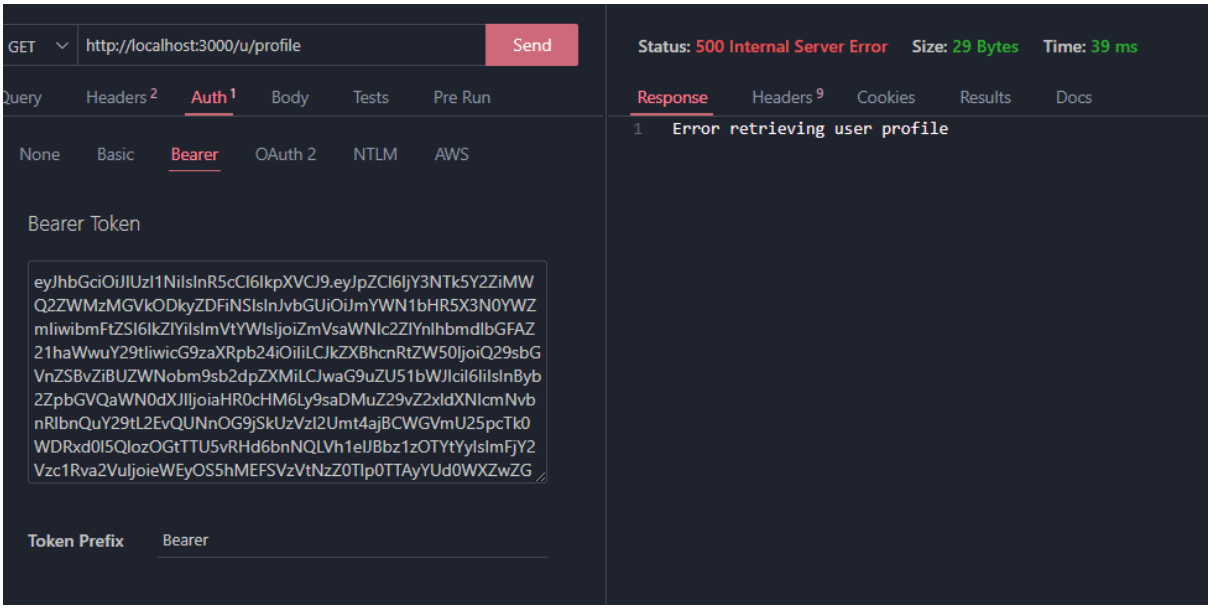


Figure 6. User Profile Server Response Error code 500

This figure shows the server error response code 500, which indicates that the server has encountered a problem of fetching the user details.

1.1.3 Patching User Profile

Version: 1.0

Date: November 3, 2024

Description: This API endpoint will update the user profile data of users in session and then save these updates to the database.

Endpoint: http://localhost:3000/u/profile

Method: PATCH

Configurations: This API requires an active user session and appropriate access privileges to ensure that only authorized updates are made to the user's profile data.

Parameters:

- **authToken:** The session token (JWT) used for user authentication, passed in the Authorization header.
- **department:** (Optional). The department the user belongs to.
- **phoneNumber:** (Optional). The phone number of the user.
- **position** (Optional) The position or role of the user.

Response codes of this API endpoint:

Code	Message	Description
200	Request Successful (OK)	This request is valid and successfully updated the user profile details.
400	No data provided for update (Bad Request)	This request is invalid due to missing data required for the update.
500	Error Updating Profile (Internal Server Error)	The server encountered an error

Valid & Invalid Responses

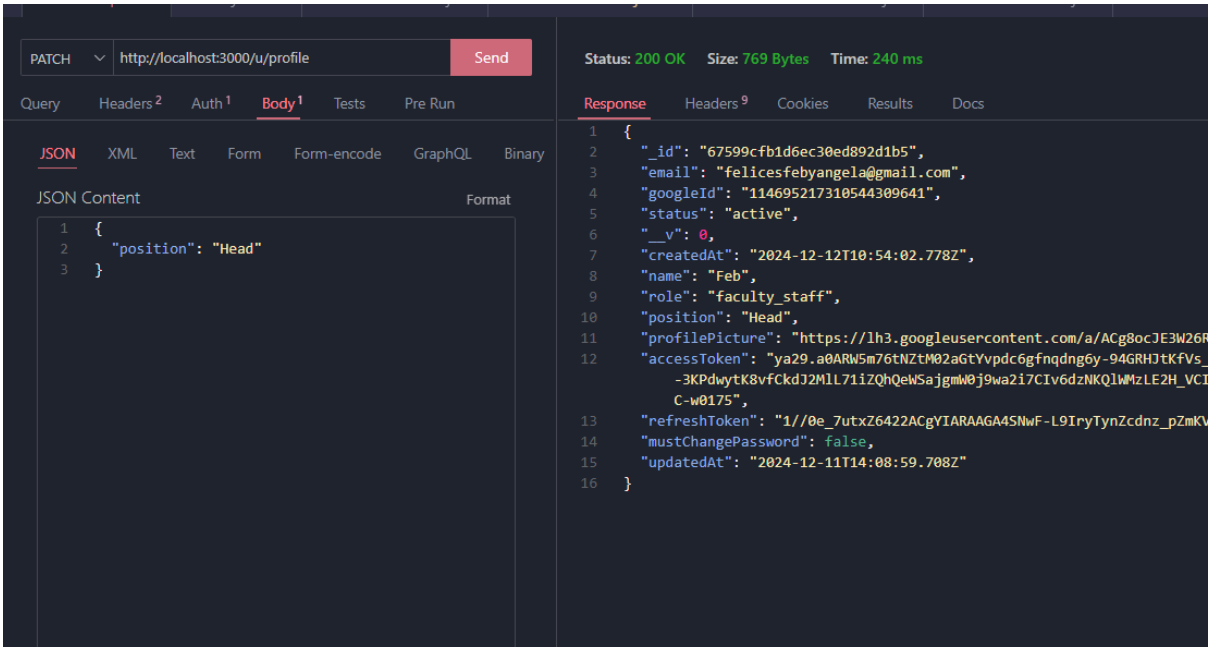


Figure 7. User Profile PATCH Valid Request

This figure shows a valid PATCH request, which successfully updated the desired field. Based on the figure, the position was changed from 'Head of Technologies' to 'Dean.

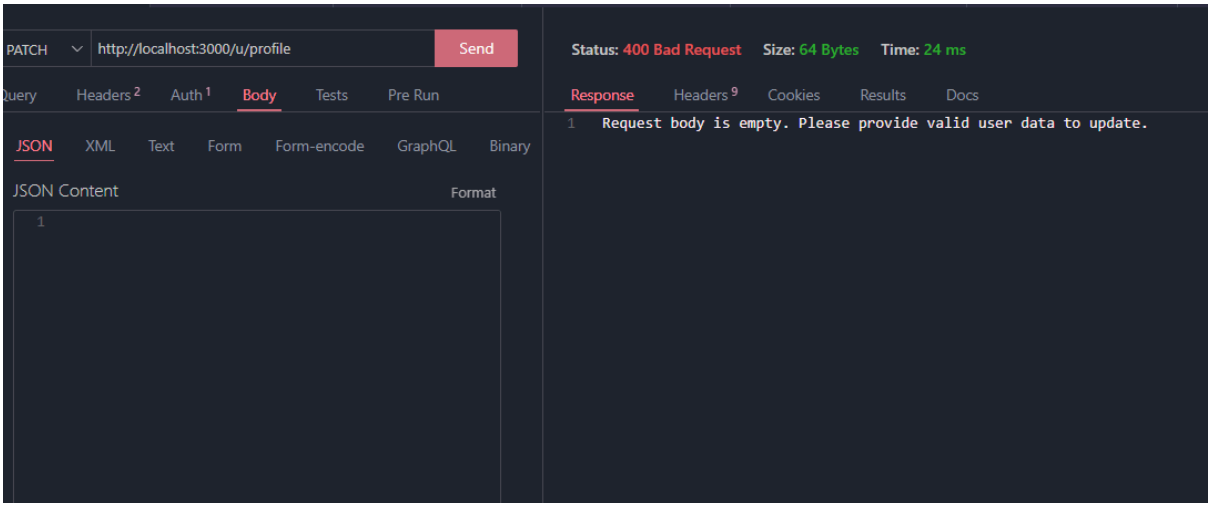


Figure 8. User Profile PATCH Invalid Request

This figure shows an invalid PATCH request. Based on the figure, the body is empty, so the system could not determine which field needed to be updated (e.g., position, department, or phone number). As a result, it returned a Bad Request 400 response.

Response Format: JSON

Response:

The API response is to update the user profile data of those who are in session by only updating the department, phone number, and position.

Different response Screenshots

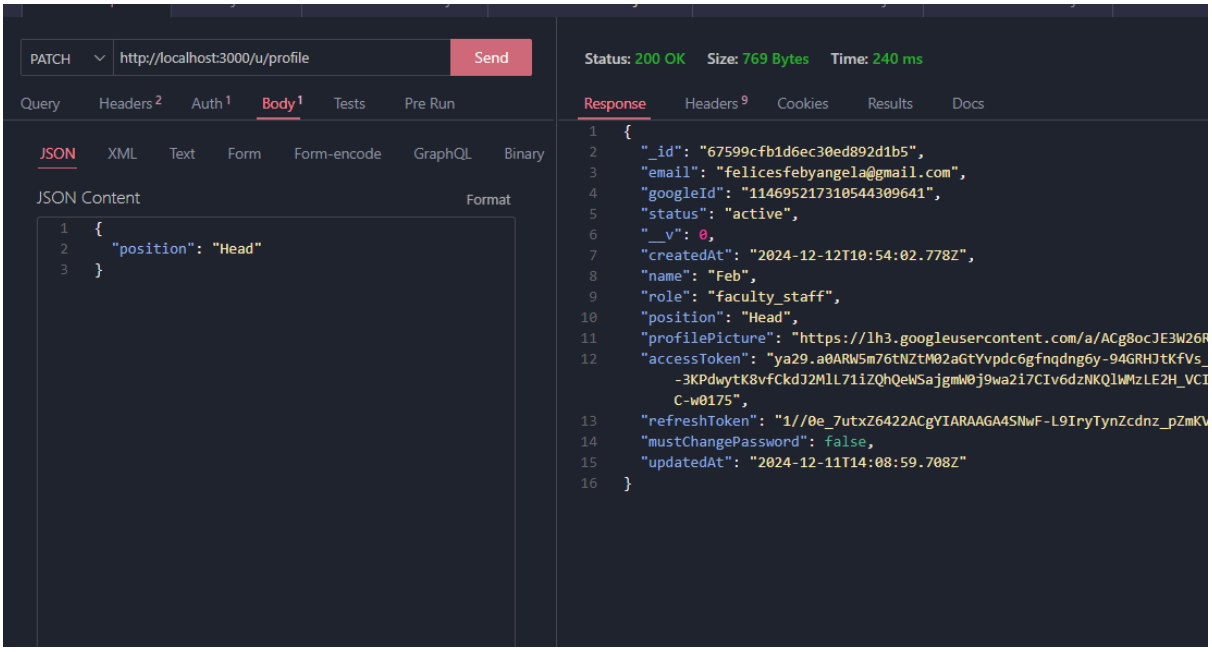


Figure 9. Updating User Profile Success Code 200

This figure shows the success response code 200, which indicates that the token was successfully authorized and the desired field (e.g., position, department, or phone number) was updated successfully.

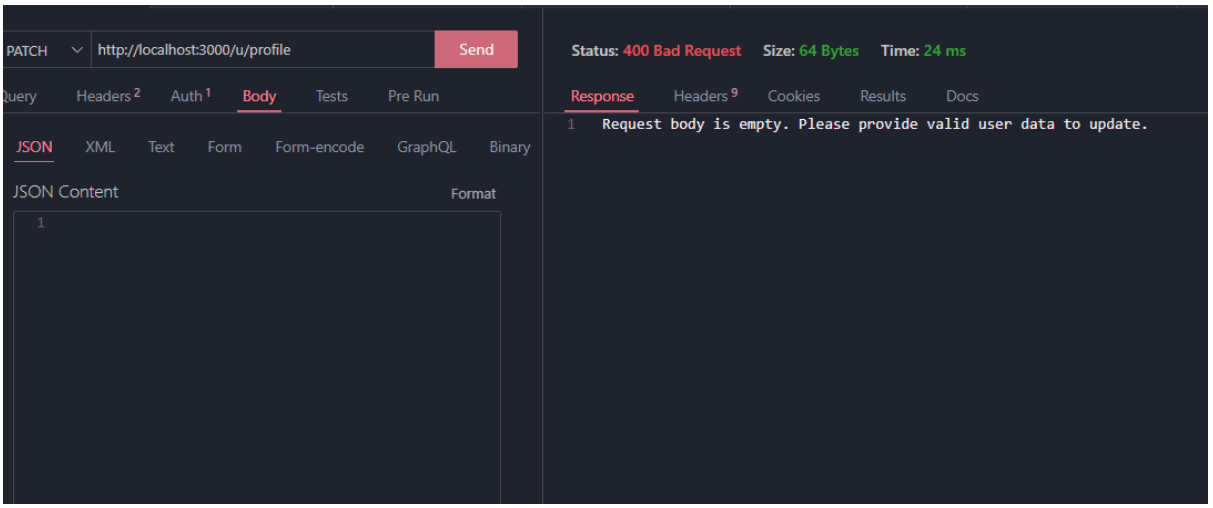


Figure 10. Updating User Profile Error Response Code 400

This figure shows the error response code 400, where the body is empty. The optional fields (e.g., position, department, or phone number) are missing, indicating that no changes were made to the database, which resulted in an error

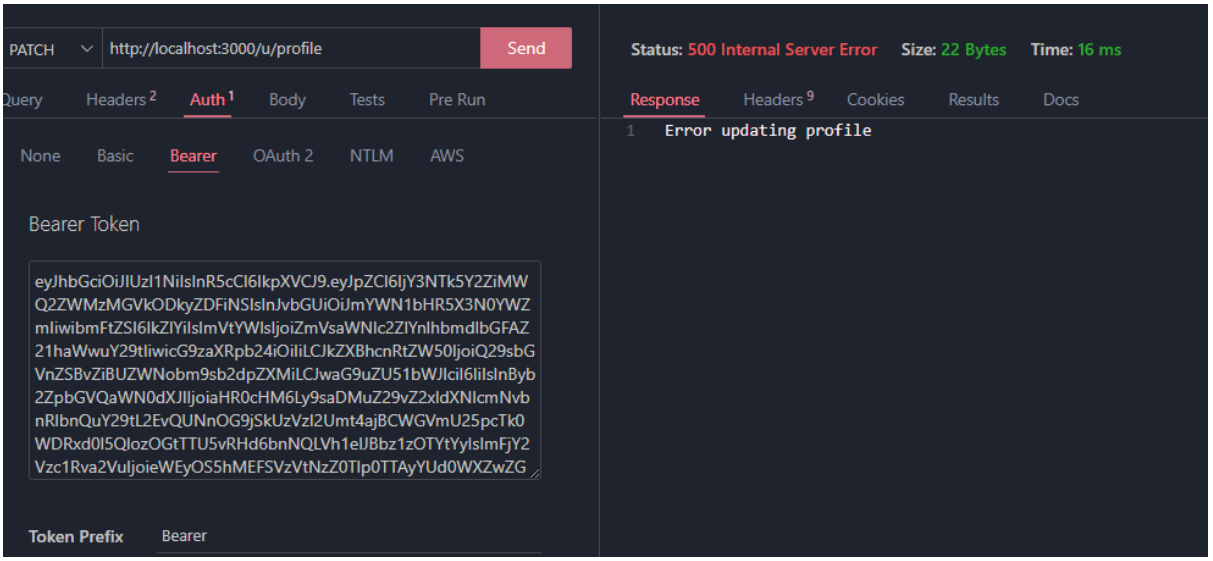


Figure 11. Updating User Profile Server Error Response Code 500

This figure shows the error response code 500, which means an error occurred on the server, resulting in this internal server error.

1.1.4 Retrieving User Certificates

- **Version**: 1.0
- **Date**: November 30, 2024
- **Description**: This API endpoint will fetch certificates achieved by users who are currently in the session.
- **Endpoint**: http://localhost:3000/u/certificates
- **Method**: GET
- **Configurations**: The API retrieves data based on the current session information. It requires a valid session token to identify the active user.
- **Parameters**:
- **authToken –** The session token, It is decoded to obtain the current userId and to list the certificates that this user has.

Response codes of this API:

Code	Message	Description
200	OK	The request was successful, and the list of user certificates was retrieved without any issues.
403	Invalid Token	The request failed due to an invalid token or insufficient permissions to access the resource.
500	Error Retrieving Certificates	The server encountered an unexpected condition that prevented it from fulfilling the request.

Valid and Invalid Request Responses

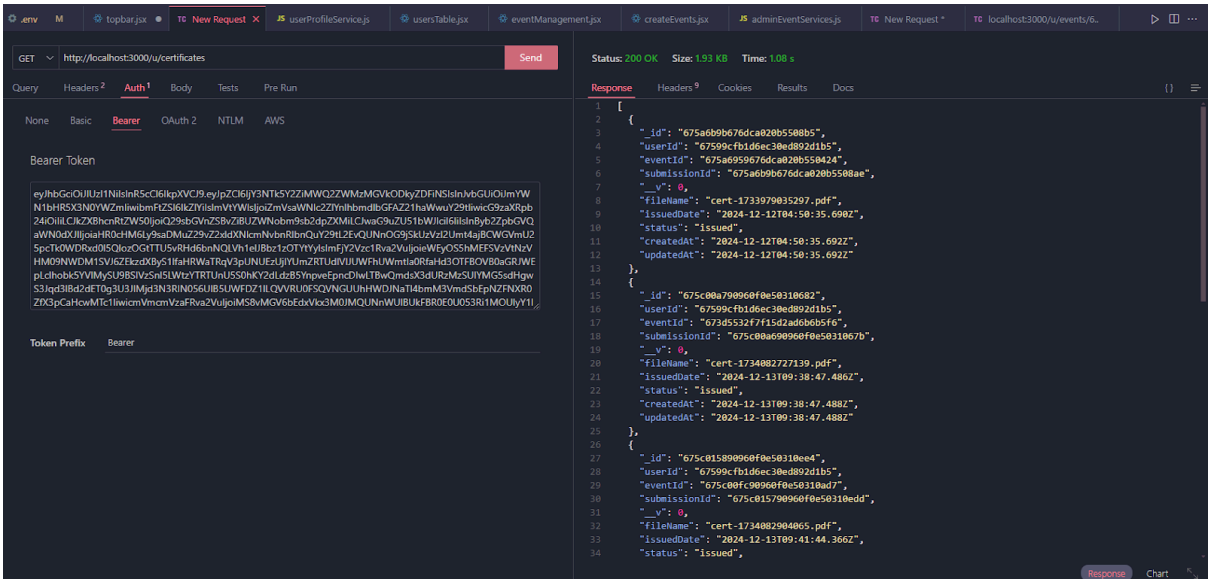


Figure 12. User Certificate List Valid GET Request

This figure shows a successful GET request to retrieve the user certificate list. The request includes a valid token, allowing the server to process it successfully. The server responds with an HTTP 200 Success code, along with the certificate list in the response body.

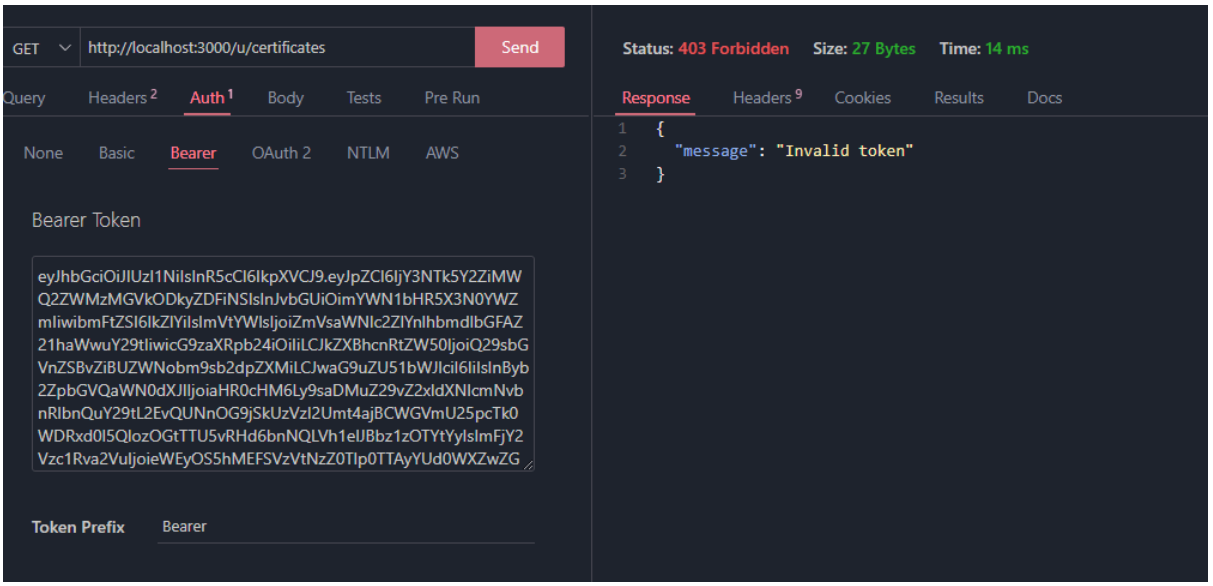


Figure 13. User Certificate List Invalid GET Request

This figure shows a failed GET request to retrieve the user certificate list due to an invalid token. The token may be expired, incorrect, or malformed. As a result, the server responds with an HTTP 403 Forbidden code, denying access to the resource.

Response:

Response Format: JSON

Response: The API response is to list their certificates achieved by users who are currently in the session.

Different response Screenshots

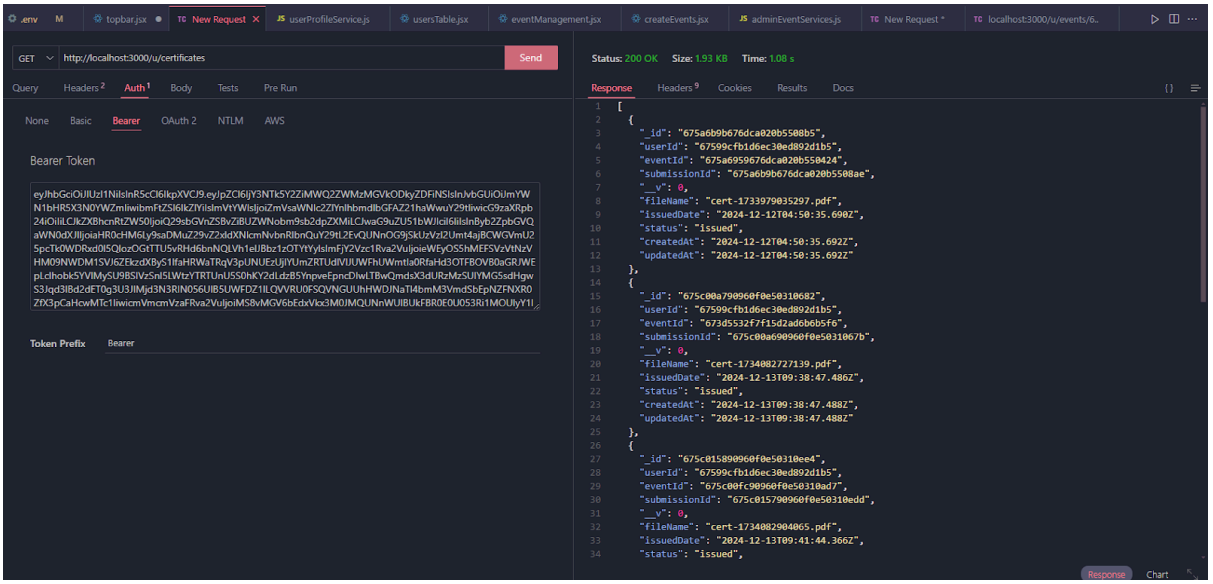


Figure 14. Retrieving Users Certificate List Success Code 200

This figure represents a scenario where the server successfully processes a GET request to retrieve the user's certificate list. The server returns an HTTP 200 Success code, along with the requested certificate list in the response.

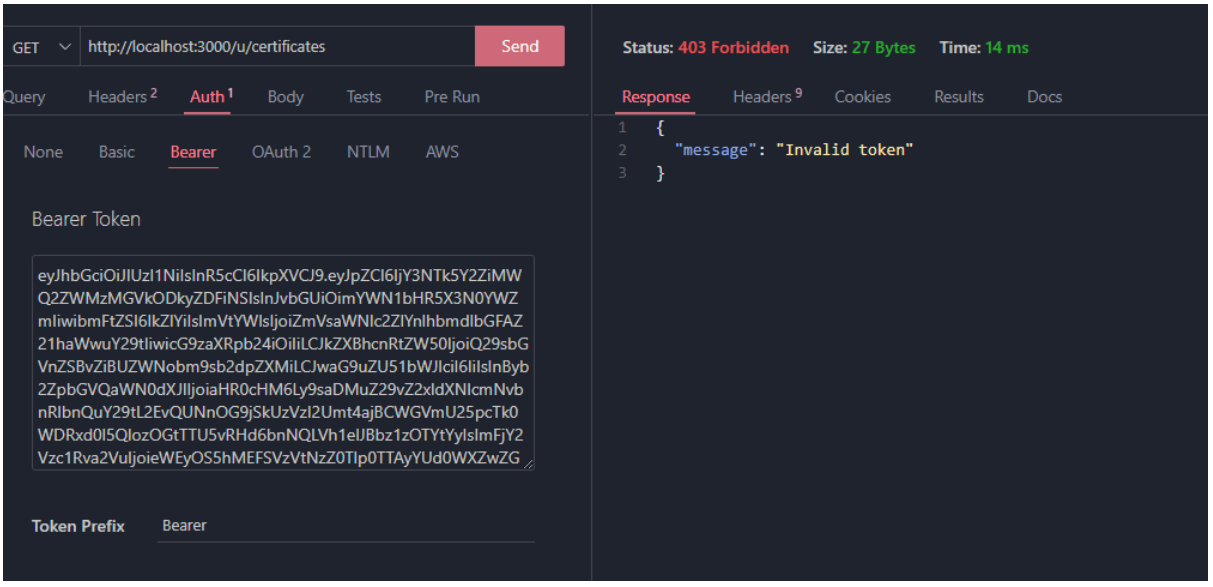


Figure 15. Retrieving Users Certificate List Failure Code 403

This figure illustrates a failed GET request due to insufficient permissions or an invalid token. The server responds with an HTTP 403 Forbidden code, indicating that the client is not authorized to access the requested resource.

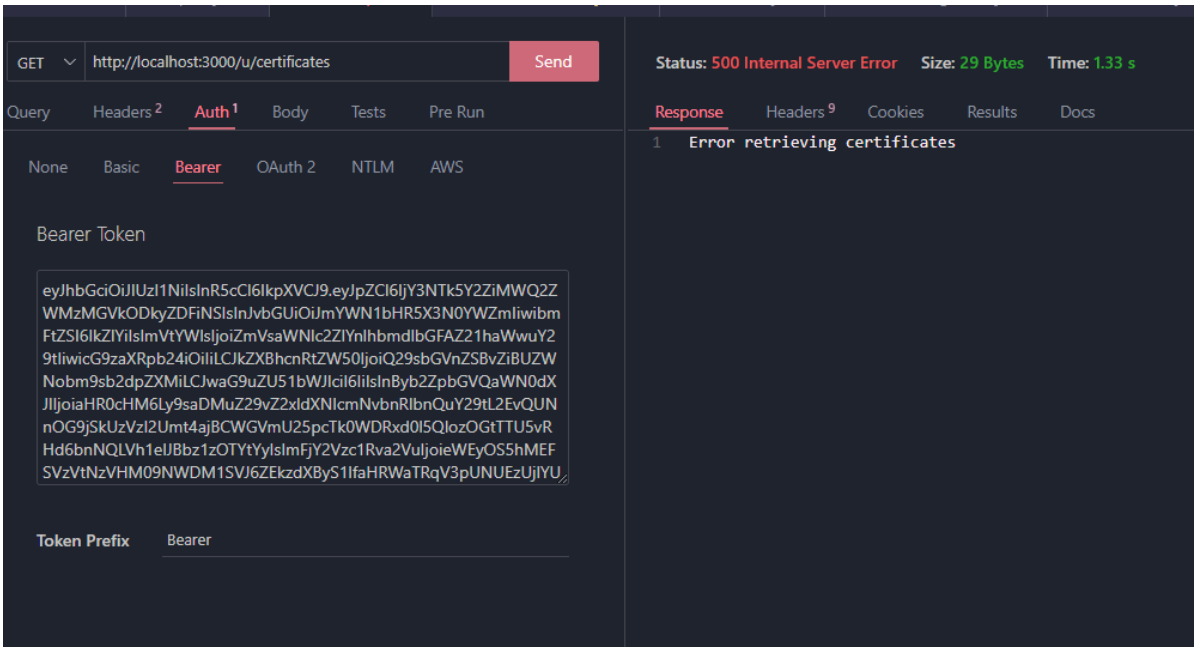


Figure 16. Retrieving Users Certificate List Server Failure Code 500

This figure depicts a scenario where the server encountered an internal error while processing the GET request for the user's certificate list. The server returns an HTTP 500 Internal Server Error code, indicating an unexpected issue that prevents it from fulfilling the request.

1.1.5 Retrieving Specific User Certificates

Version: 1.0

Date: November 30, 2024

Description: This API endpoint will fetch specific certificates achieved by users who are currently in the session.

Endpoint: http://localhost:3000/u/certificates/:certificateID

Method: GET

Configurations: The API requires a valid session token and the certificate ID to fetch detailed information about the requested certificate.

Parameters:

- **certificateID**: The unique ID of the certificate to be retrieved.
- **authToken:** The session token used for user authentication.

Response codes of this API:

Code	Message	Description
200	OK	The request was successful, and the certificate was retrieved without any issues.
403	Invalid Token	The request failed due to an invalid token or insufficient permissions to access the resource.
404	Certificate Not Found	The specified certificate could not be found in the system.
500	Error Retrieving Certificate	The server encountered an unexpected condition that prevented it from fulfilling the request.

Valid and Invalid Request Responses

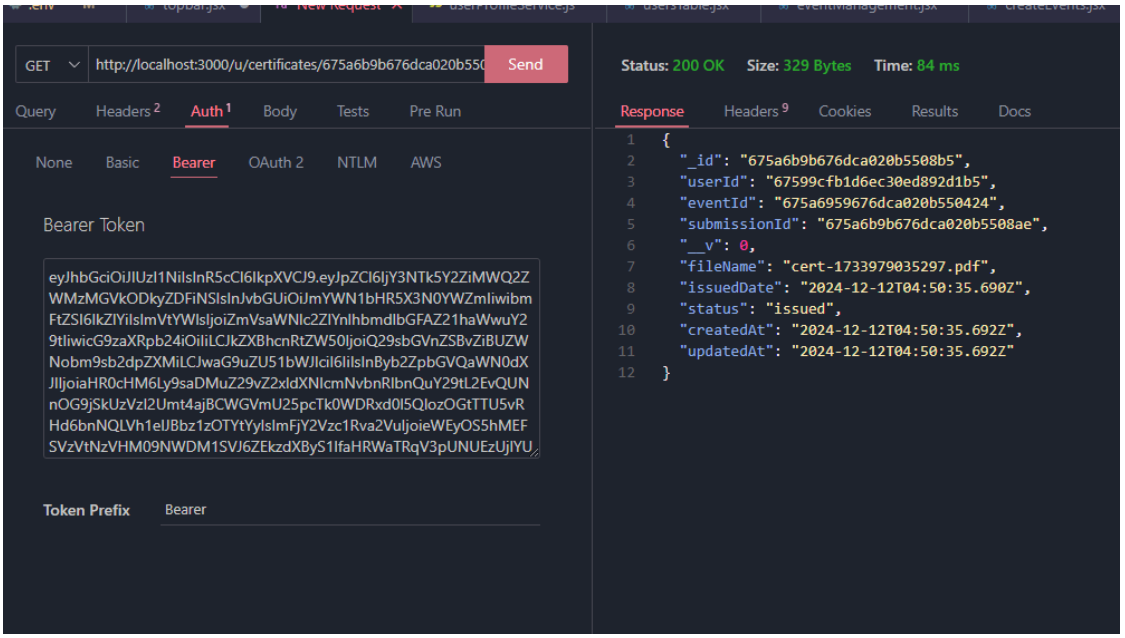


Figure 17. User Specific Certificate Valid GET Request

This figure shows a successful GET request to retrieve a specific user's certificate. The request is processed with valid credentials and a correct certificate ID. The server responds with an HTTP 200 Success code and the certificate details in the response.

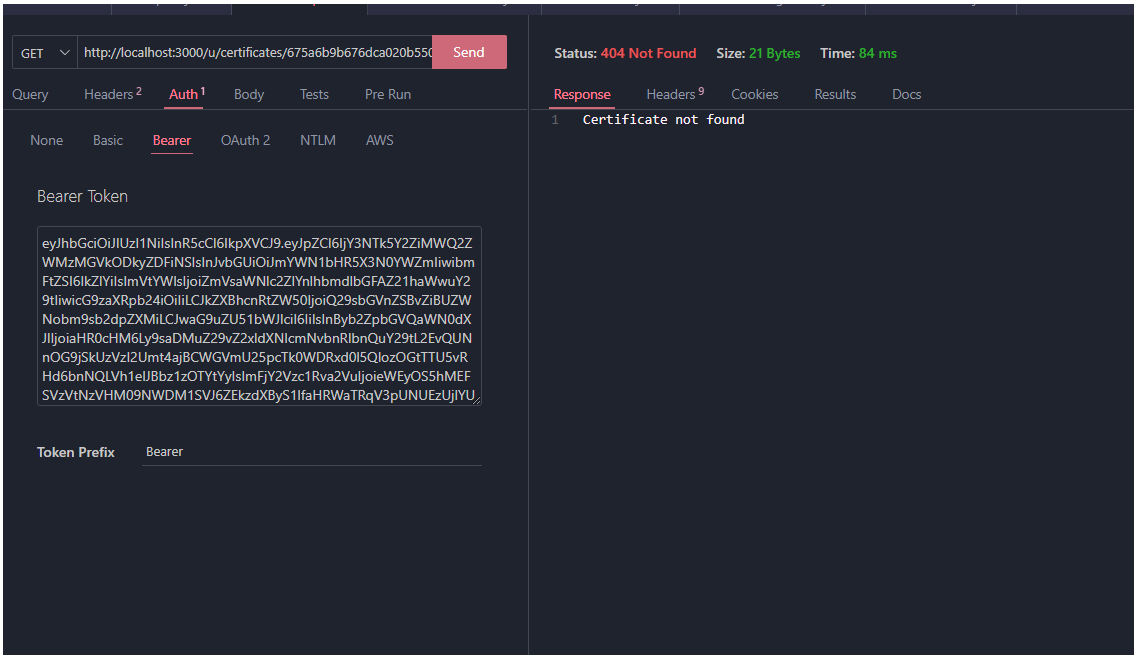


Figure 18. User Specific Certificate Invalid GET Request

This figure depicts a failed GET request to retrieve a specific user's certificate due to the certificate not being found. This could occur if the certificate ID is incorrect or does not exist

in the system. The server responds with an HTTP 404 Not Found code, indicating the resource could not be located.

Response Format: JSON

Response:

The API response shows specific certificates achieved by users who are currently in the session, which means it redirects to the details of that certification.

Different response Screenshots

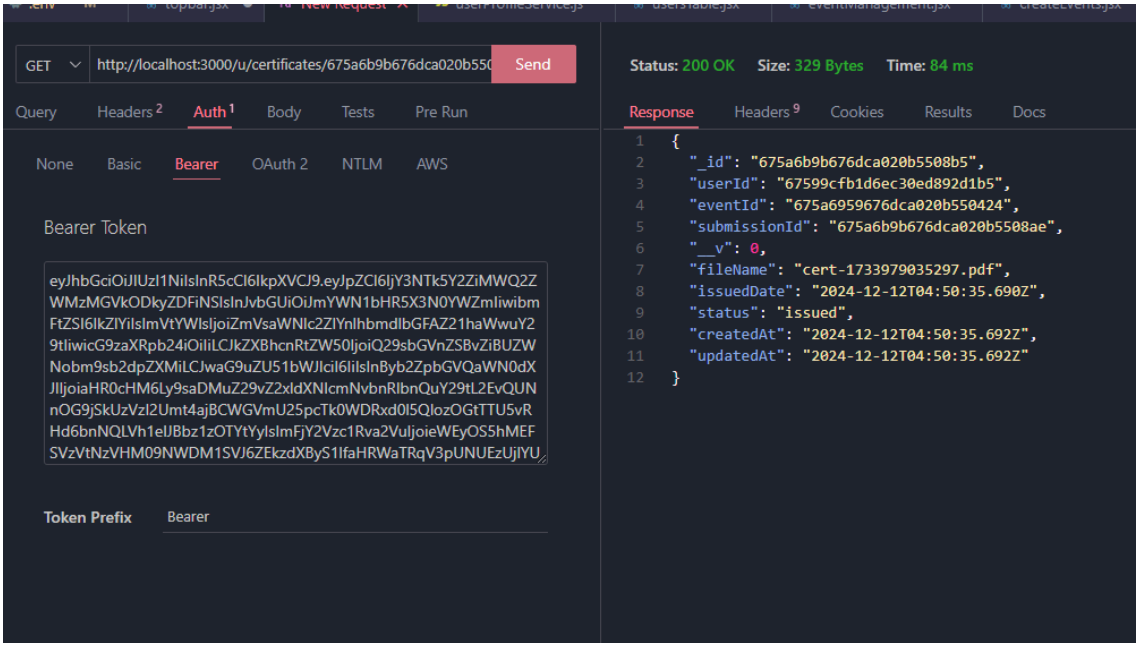


Figure 19. Retrieving Specific Certificate Success Code 200

This figure shows a successful retrieve of a specific certificate. The request is properly authenticated and processed, and the server responds with an HTTP 200 Success code, returning the requested certificate details.

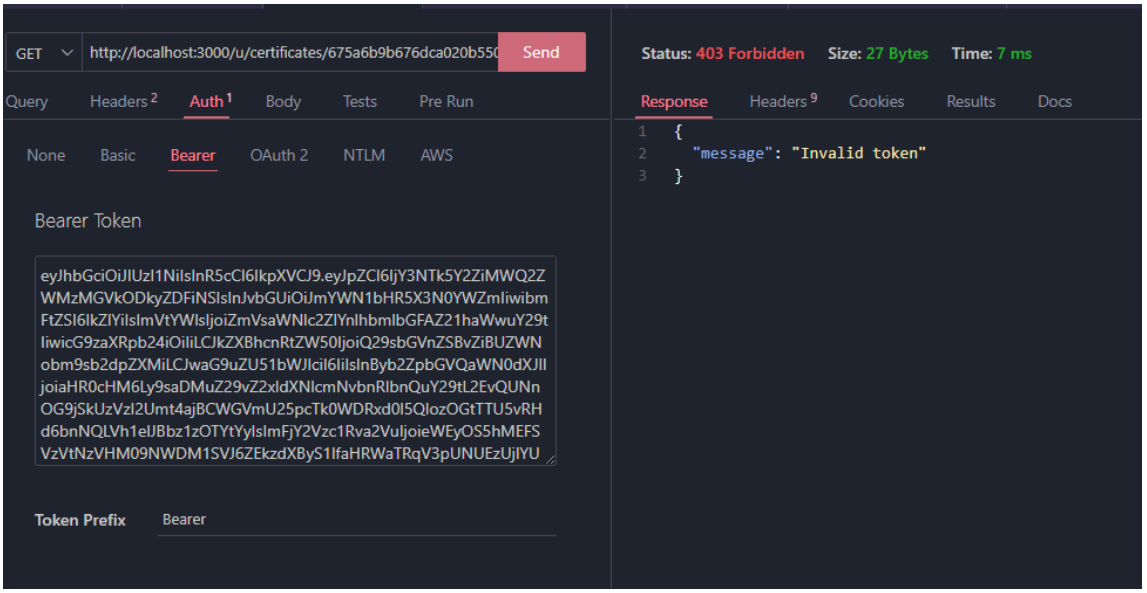


Figure 20. Retrieving Specific Certificate Success Code 403

This figure depicts a GET request to retrieve a specific certificate that fails due to an invalid token or insufficient permissions. The server responds with an HTTP 403 Forbidden code, denying access to the requested certificate.

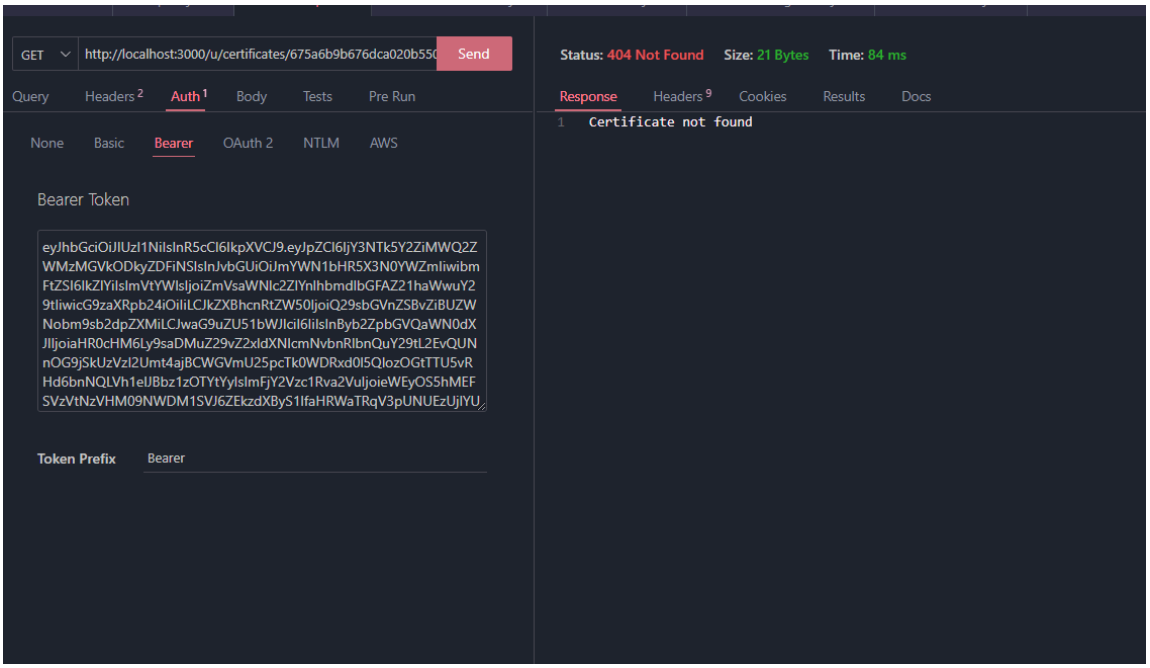


Figure 21. Retrieving Specific Certificate Failure Code 404

This figure illustrates a failed GET request where the specific certificate cannot be found. The server responds with an HTTP 404 Not Found code, indicating that the requested certificate does not exist in the system.

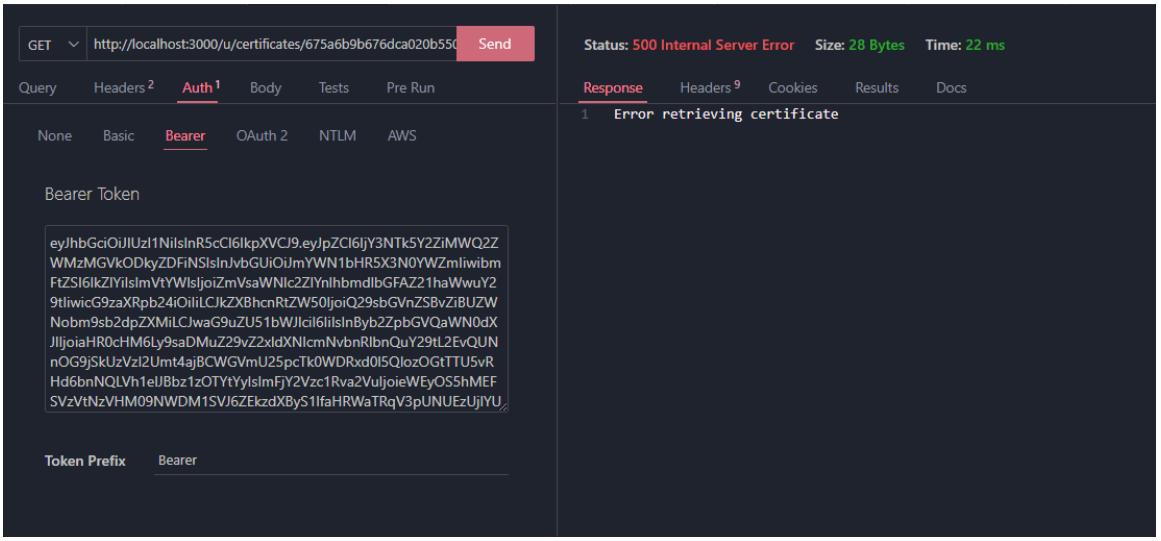


Figure 22. Retrieving Specific Certificate Server Failure Code 500

This figure represents a GET request failure due to an internal server issue. The server responds with an HTTP 500 Internal Server Error code, indicating that something went wrong on the server side, preventing the request from being fulfilled.

1.1.6 Retrieving User Events

Version: 1.0

Date: November 14, 2024

Description: This API endpoint will fetch events from the user's event page.

Endpoint: http://localhost:3000/u/events

Method: GET

Configurations: No authentication or session token is required, and no query parameters are needed to retrieve all events.

Parameters:

- No authentication or session token is required, and no query parameters are needed to retrieve all events.

Response codes of this API endpoint:

Code	Message	Description
200	Request Successful (OK)	This request is valid and successfully retrieve all events from the database
500	Error Fetching Events (Internal Server Error)	The server encountered an error

Valid and Invalid Request Responses

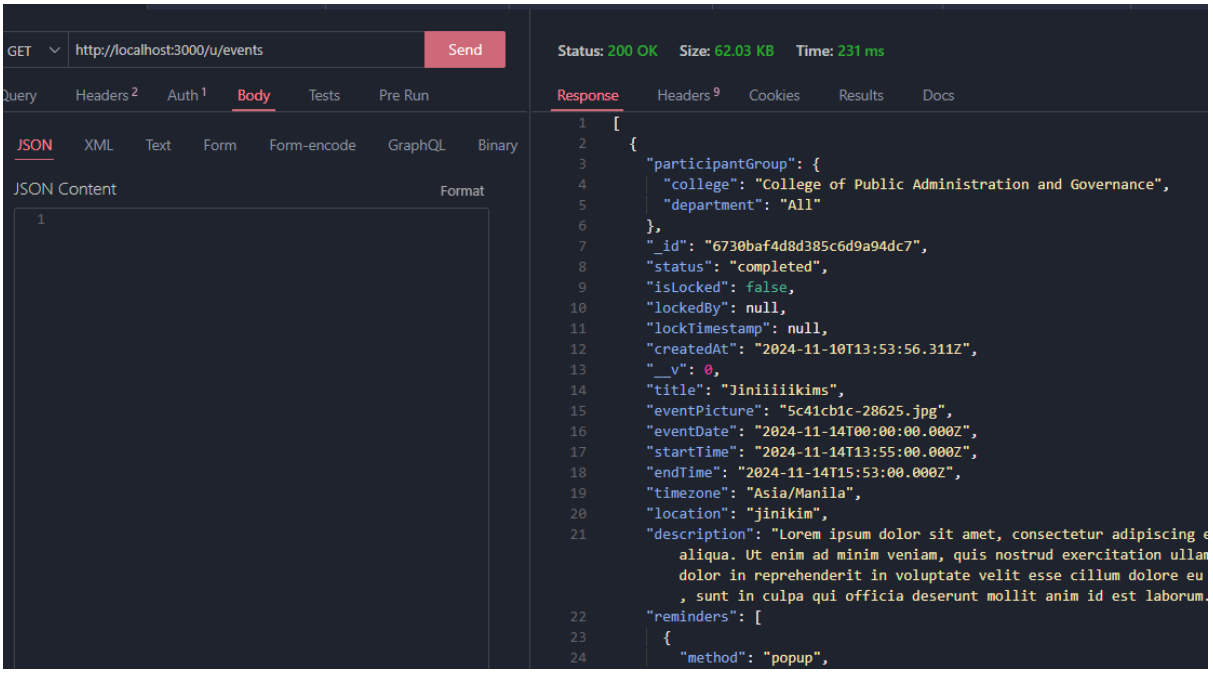


Figure 23. Valid GET request of all events on user interface

This figure shows a valid GET request code 200 for listing all events available in the database to the user interface

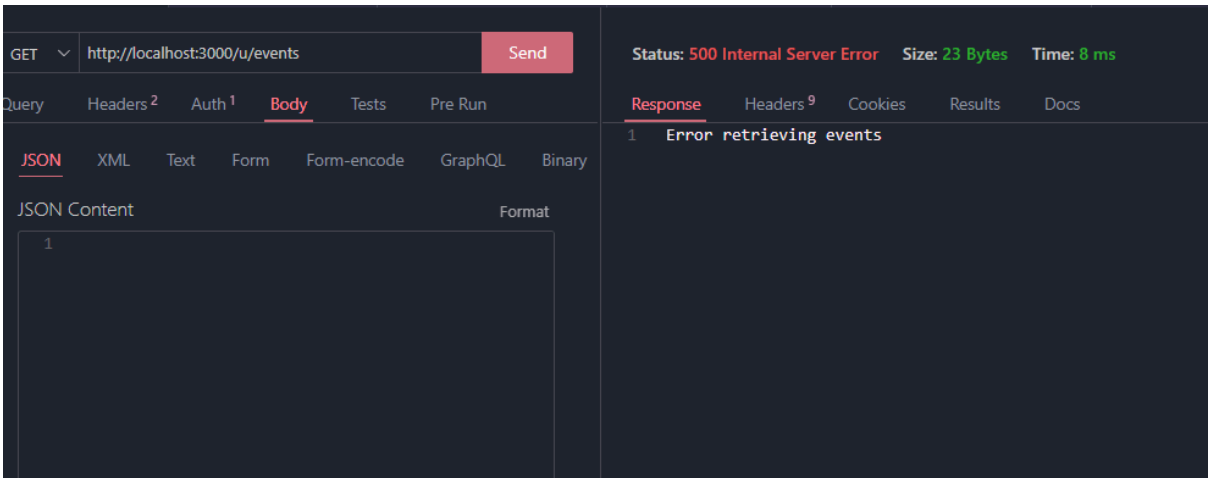


Figure 24. No authentication or session token is required, and no query parameters are needed to retrieve all events.

This figure shows an invalid GET request with a 500 error code for listing all events available in the database to the user interface, which is due to a server problem.

Response Format: JSON

Response: The API response is to list upcoming events to the user's event page

Different response Screenshots

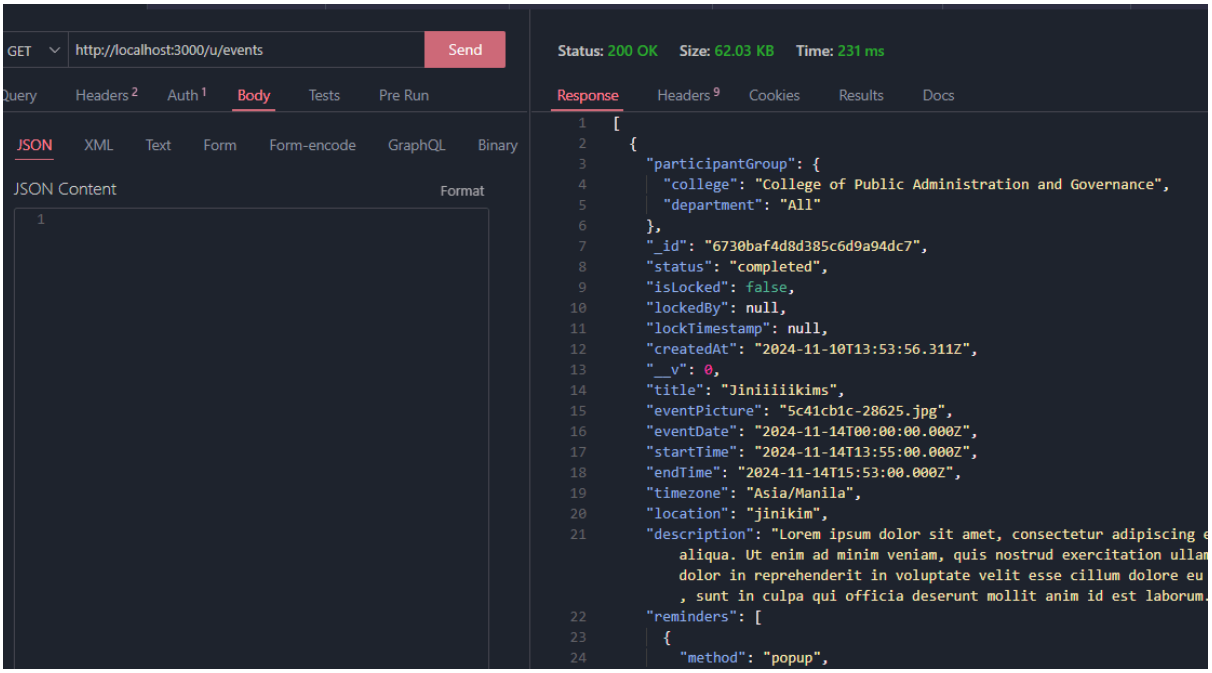


Figure 25. Retrieving Events List Success Code 200

This figure illustrates the successful retrieval with code 200 of the event list created by both the admin and departmental admin.

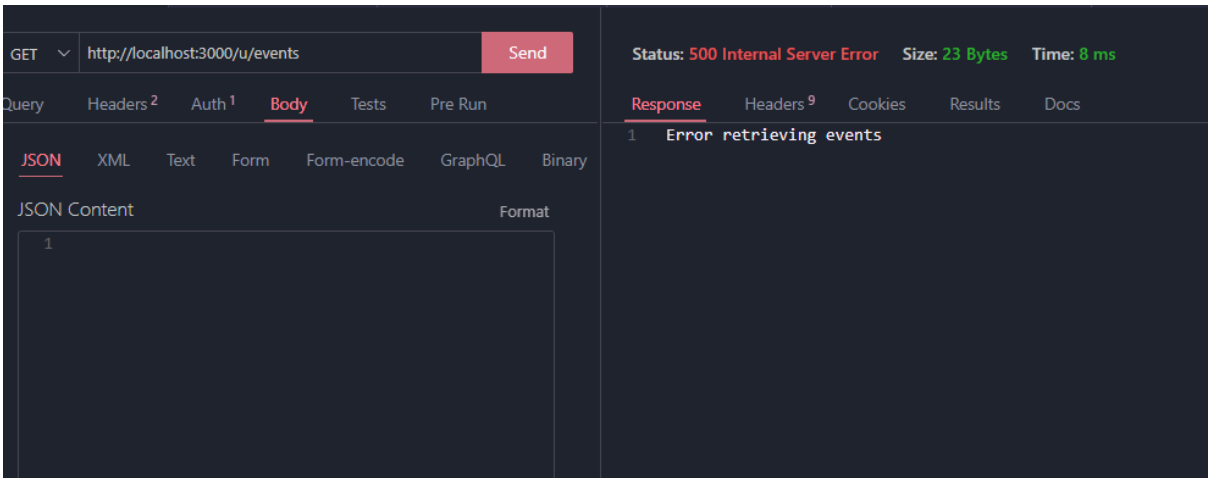


Figure 26. Server Error Events List Retrieval Code 500

This figure shows that the failure to retrieve the list of events, code 500, typically indicates that something went wrong on the server while processing the request. In the specific case of retrieving the event list, the error may occur due to several reasons.

1.1.7 Retrieving Specific Event

Version: 1.0

Date: November 13, 2024

Description: This API endpoint will fetch specific events from the user's event page to view details such as the event picture, title, date, time, location, and registration options.

Endpoint: http://localhost:3000/u/events/:id

Method: GET

Configurations: The API requires a valid session token for authentication to retrieve the event details associated with the user.

Parameters:

- **eventId:** The unique identifier of the event being retrieved.

Response codes of this API endpoint:

Code	Message	Description
200	OK	The request was successful, and the event data was returned.
404	Event not found	The event with the specified ID was not found in the database.
500	Error retrieving event	An internal server error occurred while attempting to retrieve the event.

Valid & Invalid Responses

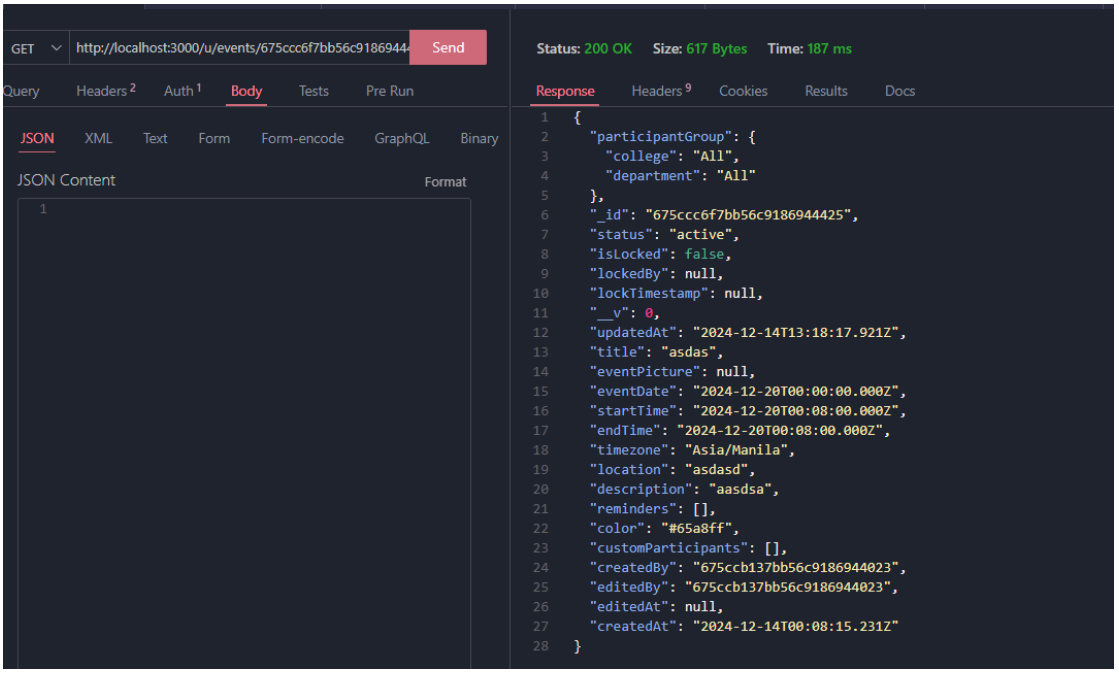


Figure 27. Valid GET request of specific events on user interface

This figure shows a valid GET request with a 200 status code for viewing specific event data based on the eventID.

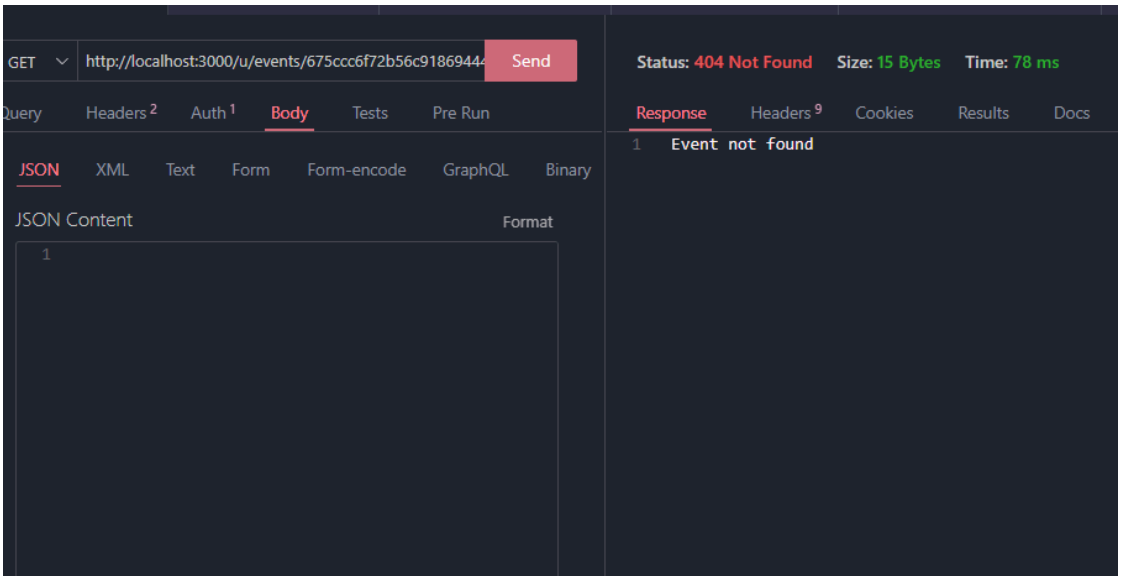


Figure 28. Invalid GET request of specific events on user interface

This figure shows an invalid GET request with a 404 status code for viewing specific event data based on the eventID. This is because the eventID is invalid, meaning it is not found in the database.

Response Format: JSON

Response:

This API endpoint displays user-specific events from the user's event page, allowing them to view the details of each event.

Different response Screenshots

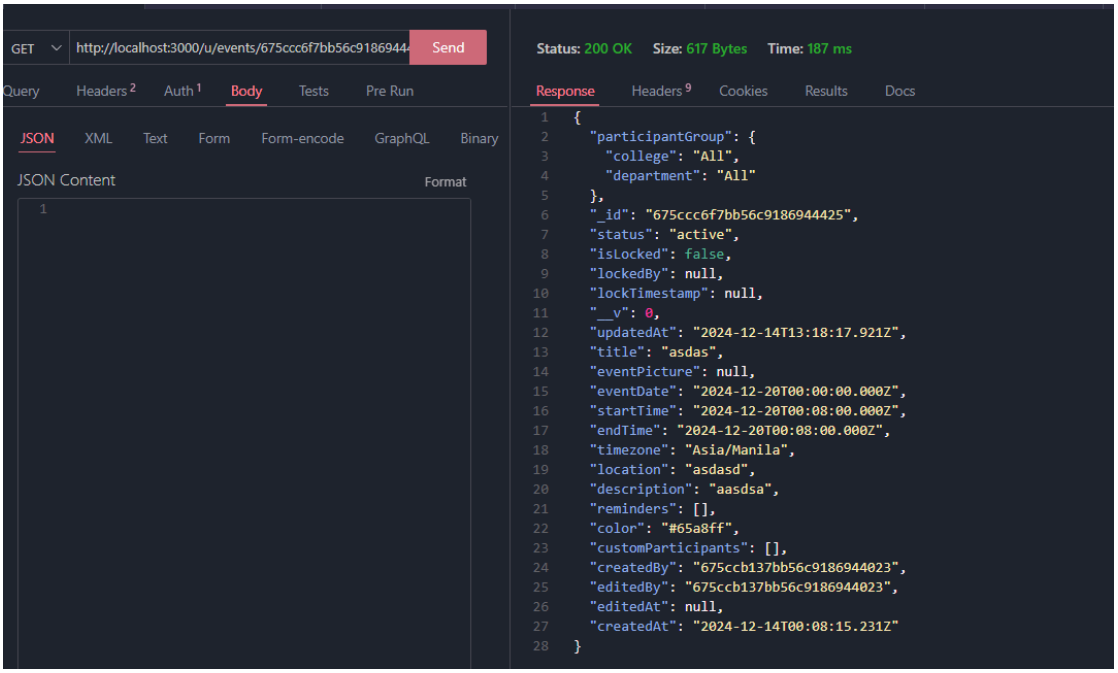


Figure 29. Retrieving Specific Event Success Code 200

This figure shows the successful retrieval with code 200 of the specific event created by both the admin and departmental admin. It displays the details of that specific event.

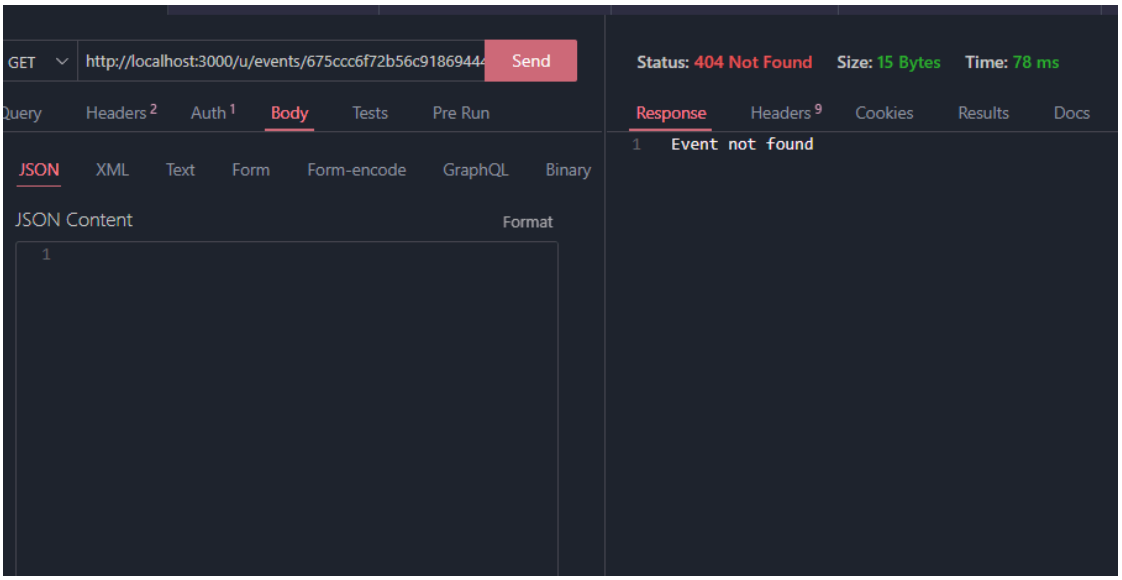


Figure 30. Retrieving Specific Event Failure Code 404

This figure shows the failure to retrieve the specific event due to a "404 Not Found" error. This error occurs when the requested event cannot be found in the system, possibly due to an incorrect event ID or the event being deleted.

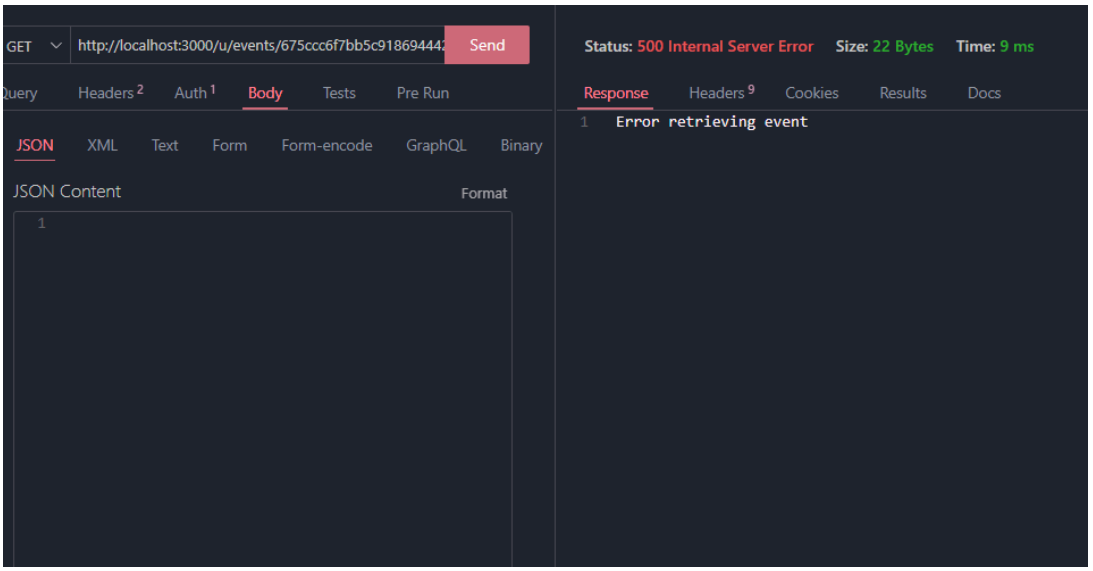


Figure 31. Retrieving Specific Event Server Failure Code 500

This figure shows the failure to retrieve the specific event due to a "500 Internal Server Error." This error occurs when there is an issue on the server while processing the request, preventing the event from being retrieved.

1.1.8 Register User for Specific Event

Version: 1.0

Date: November 13, 2024

Description: This API endpoint will enable users to register for specific events they want to join or participate in.

Endpoint: http://localhost:3000/u/events

Method: POST

Configurations: The API requires a valid session token to authenticate the user and allow them to register for an event.

Parameters:

- **authToken:** The session token (JWT) used for user authentication and passed in the Authorization header.
- **eventId:** The unique identifier of the event the user wants to register for.

Response codes of this API endpoint:

Code	Message	Description
201	User ${userId} registered for event ${eventId}	The user has successfully registered for the event.
403	Invalid Token	The authentication token provided is invalid or expired.
404	Event not found	The specified event ID was not found in the database.
500	Google Calendar Error: Invalid Credentials	An error occurred with Google Calendar due to invalid credentials.

Valid and Invalid Request Responses

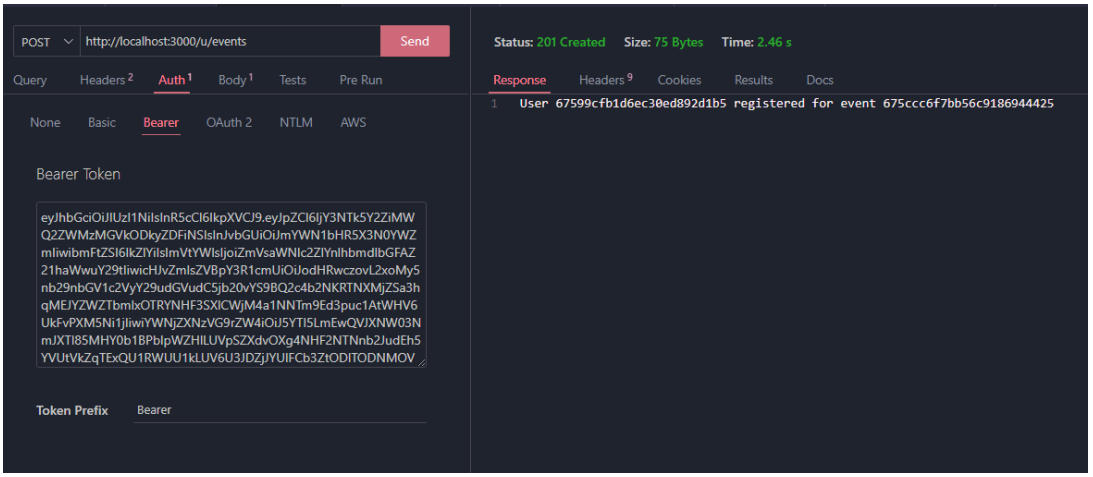


Figure 32. A valid POST request for a specific event on the user interface

This figure shows a valid POST request with a 201 status code for a user to register an event based on the eventID. This will successfully create a new resource and save it to the database.

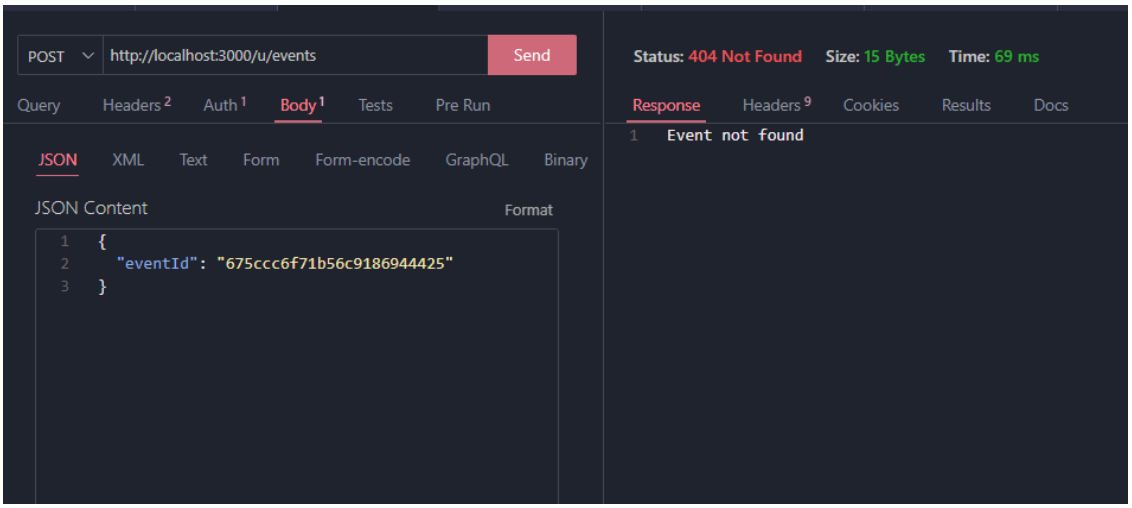


Figure 33. A Invalid POST request for a specific event on the user interface

This figure shows an invalid POST request with a 404 status code, as the eventID is not found in the database, resulting in an invalid response.

Response Format: JSON

Response:

The API response enables users to register for specific events they want to join or participate in and saves this information in the database.

Different response Screenshots

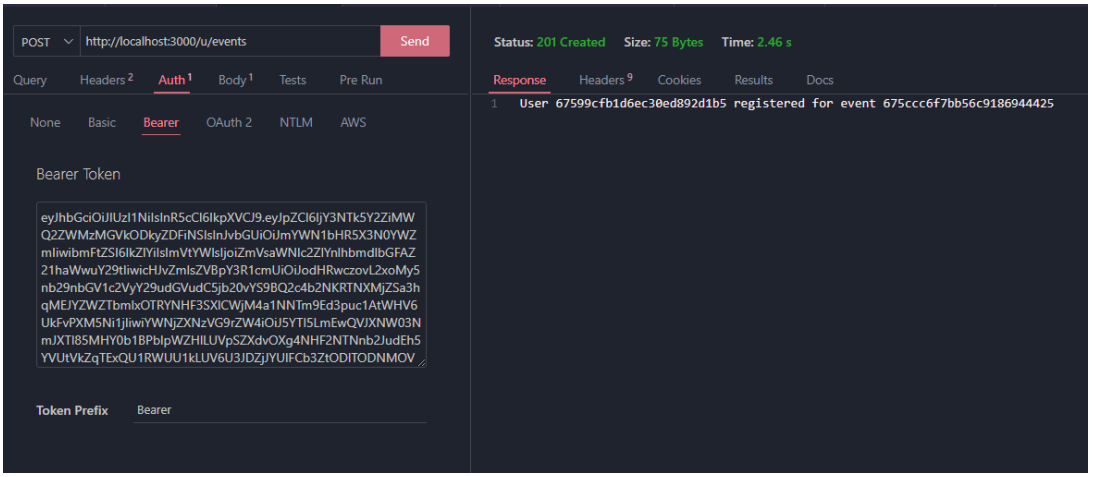


Figure 34. User Successfully Registers an Event with Status Code 201

This figure shows that the response code 201 indicates the registration was successfully created and saved in the database, confirming that the user is now registered for a specific event.

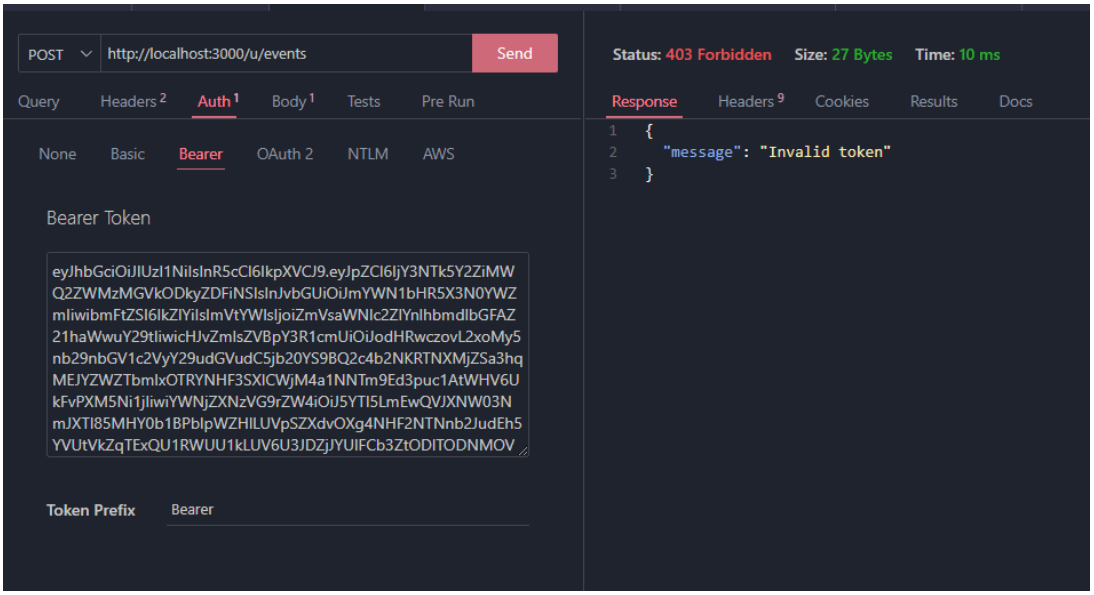


Figure 35. User with Invalid Token Fails to Register an Event with Status Code 403

This figure shows that the response code 403 indicates the registration failed to create because the user token is invalid, preventing the system from determining which user wants to register for this event

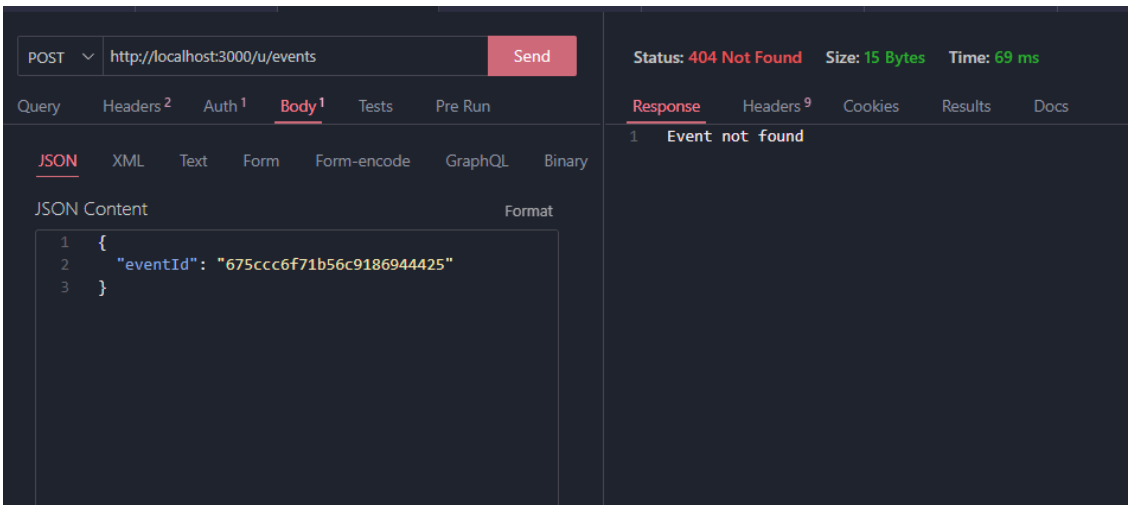


Figure 36. User Failed to Registers an Event with Status Code 404

This figure shows that the response code 404 indicates the registration failed to create because the eventID was not found in the database, resulting in a 'not found' event.

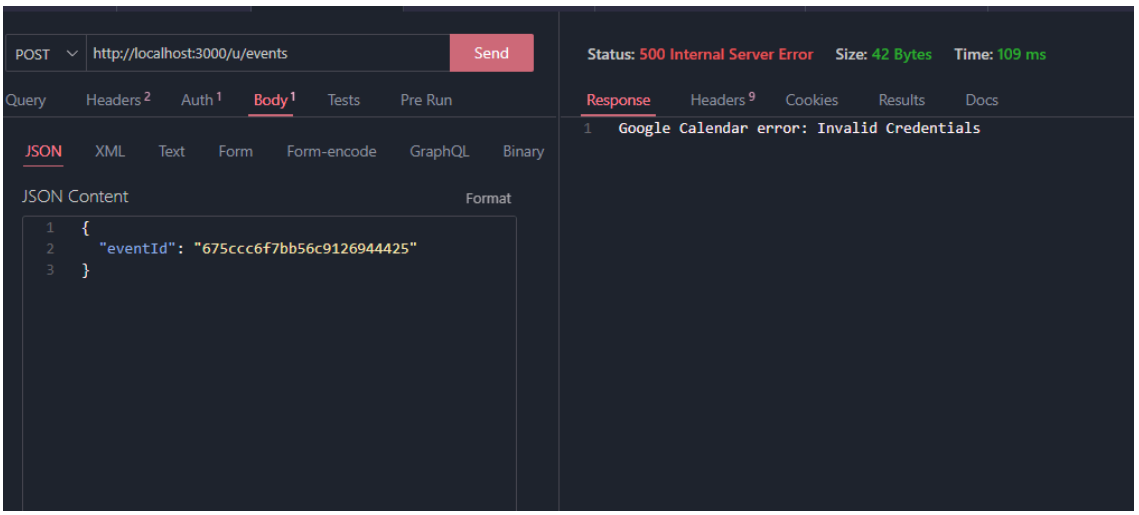


Figure 37. User Fails to Register an Event Due to a Server Error with Status Code 500

This figure shows that the response code 500 indicates the registration failed to create due to a server error caused by invalid Google Calendar credentials.

1.1.9 Checking User Registered Events

Version: 1.0

Date: November 10, 2024

Description: This API endpoint will check whether a user is registered for this event.

Endpoint: http://localhost:3000/u/events/:id/check-registration

Method: GET

Configurations: This API requires a valid session token to authenticate the user and determine their registration status for the event.

Parameters:

- **authToken**: The session token (JWT) used for user authentication, passed in the Authorization header.
- **eventId**: The unique identifier of the event being checked for user registration.

Response codes of this API endpoint:

Code	Message	Description
200	● The user is already registered for this event ● User is not registered for this event	● The registration status has been successfully retrieved, and the user **is registered** for the event. ● The registration status has been successfully retrieved, and the user **is not registered** for the event.
403	Invalid Token	The request was denied due to an invalid or expired token, indicating unauthorized access to check the registration status.
500	Error checking registration status	An error occurred while checking the registration status, typically due to a database issue.

Valid and Invalid Request Responses

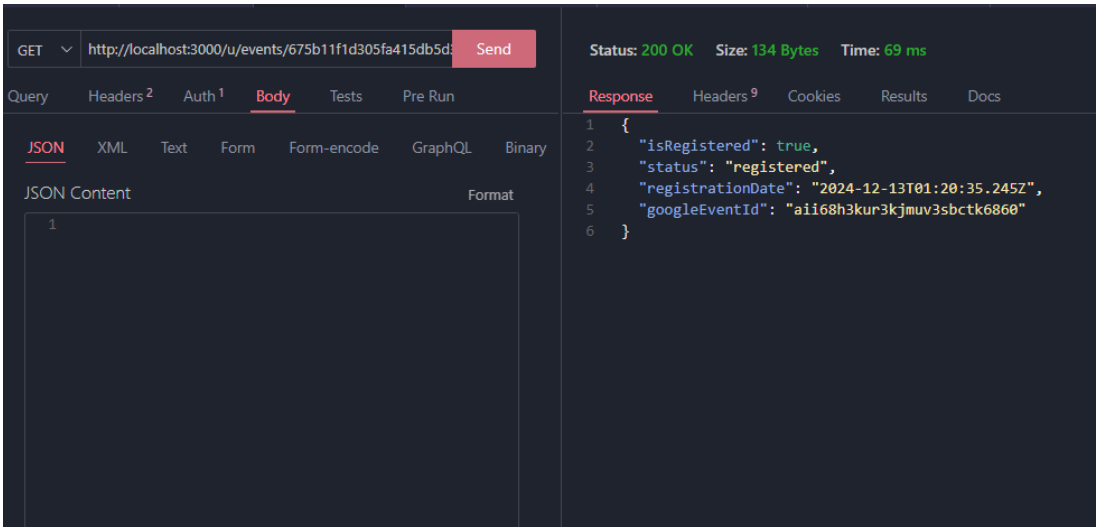


Figure 38: A Valid GET Request for Checking User Event Registration

This figure illustrates a valid GET request, which returns a 200 status code indicating successful processing. The request checks the user's registration status based on the provided eventID and retrieves the associated user registration details.

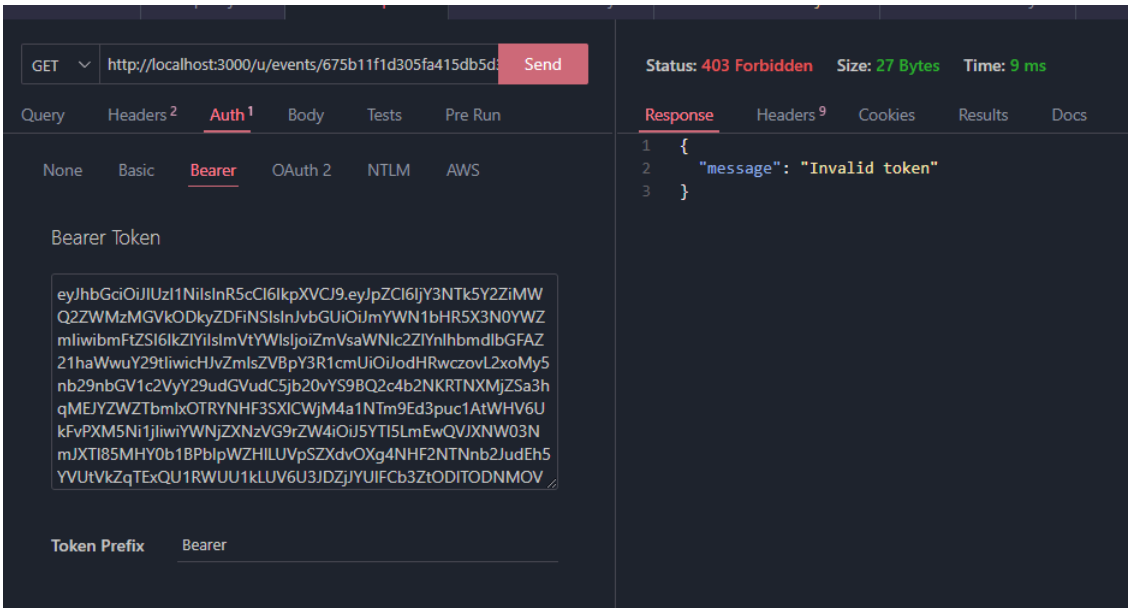


Figure 39: Invalid GET Request for Checking User Event Registration

This figure illustrates an invalid GET request when checking the user's event registration status. The failure is due to an invalid token, which prevents access to the system.

Response Format: JSON

Response:

This API response is to check whether a user is registered for this event.

Different response Screenshots

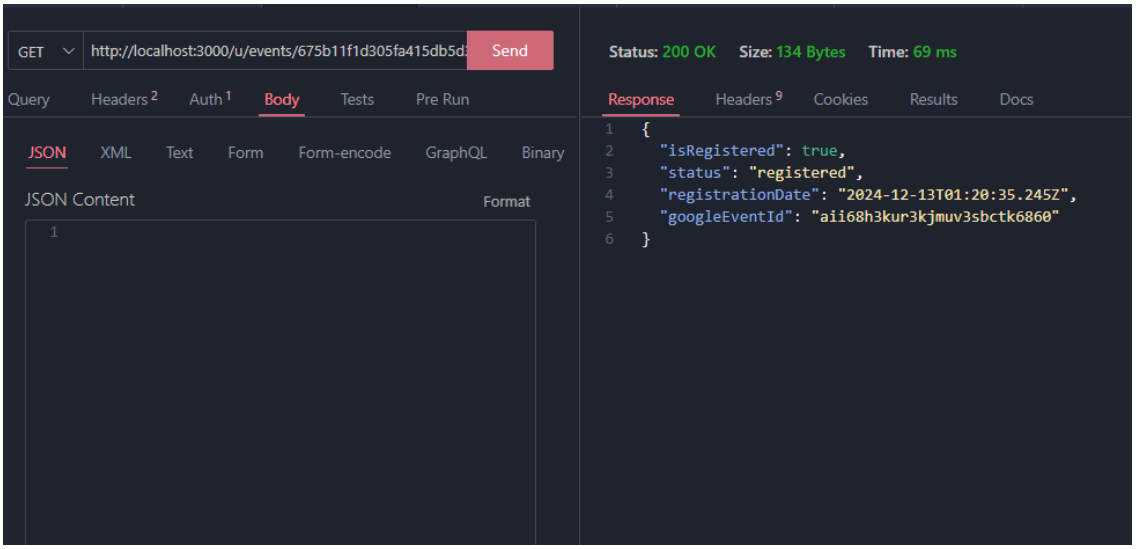


Figure 40: Successful Retrieval for Checking User Event Registration with Status Code 200

This figure demonstrates the successful retrieval of user event registration details, as indicated by a status code of 200, confirming that the request was successfully processed and the data was returned without errors.

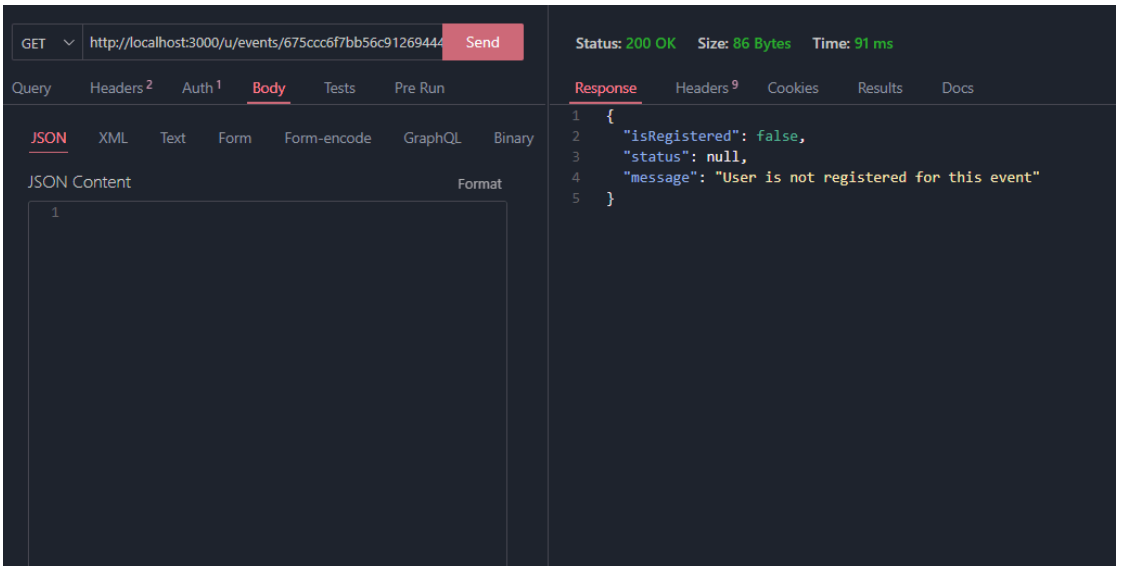


Figure 41: Successful Retrieval for Checking User Event Registration with Status Code 200

This figure shows a successful retrieval process for checking the user's event registration status, with a status code of 200. It indicates that the system successfully retrieved the user's status, confirming that they are not registered for the event.

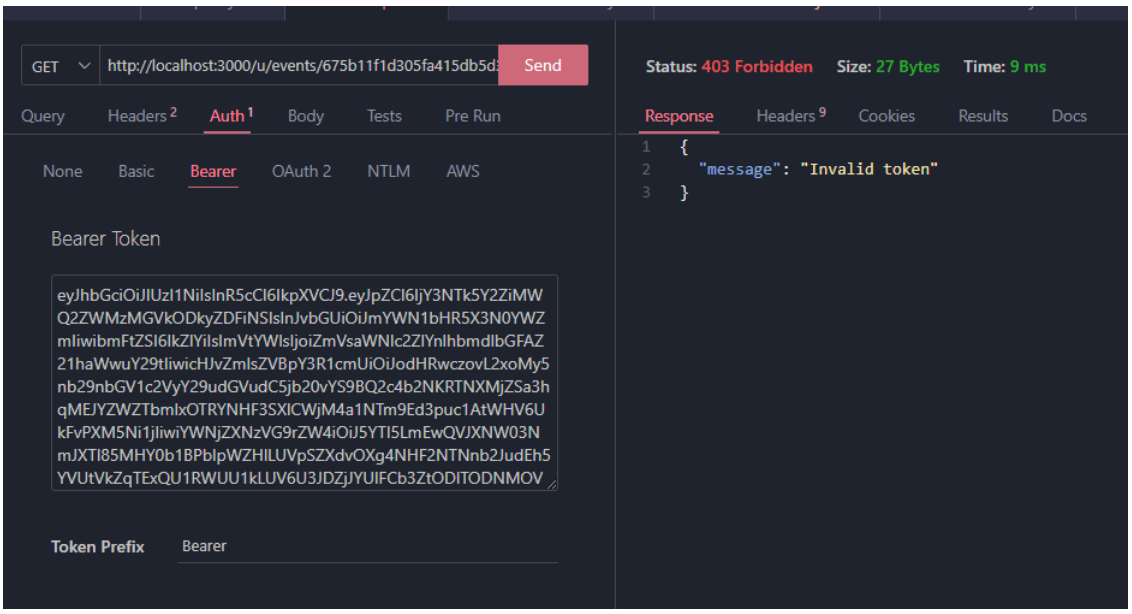


Figure 42. Failed to Retrieve User Registration Status Code 403

This figure shows a failed attempt to retrieve the user's event registration status, resulting in a status code of 403. This error occurs due to an invalid or expired token, indicating unauthorized access.

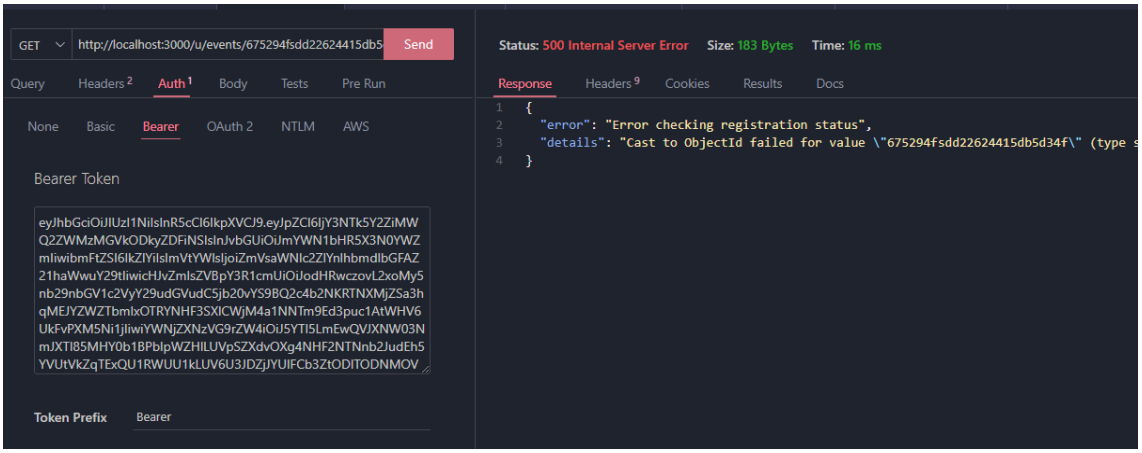


Figure 43. Failed to Retrieve User Registration Status Due to Server Error 500

This figure shows the response returned when an error occurs while attempting to retrieve the user event registration details from the server. The server responds with a 500 Internal Server Error, indicating an unexpected error such as invalid eventID that prevented the retrieval of the data.

1.1.10 User Cancellation of an Event

Version: 1.0

Date: November 11, 2024

Description: This API endpoint will enable users to cancel their registration for that specific event.

Endpoint: http://localhost:3000/u/events/:id/cancellation

Method: DELETE

Configurations: This API requires a valid session token for user authentication to authorize the cancellation request.

Parameters:

- **authToken**: The session token (JWT) used for user authentication, passed in the Authorization header.
- **eventId:** The unique identifier of the event the user wishes to cancel their registration for.

Response codes of this API endpoint:

Code	Message	Description
200	Registration cancelled successfully	The user's registration has been successfully cancelled, and any related event data in Google Calendar has been removed.
404	Registration not found	The specified user is not registered for the event, so cancellation cannot be processed.
500	Error cancelling registration	A general error occurred while attempting to cancel the registration, usually due to a database or server issue.

Valid and Invalid Request Responses

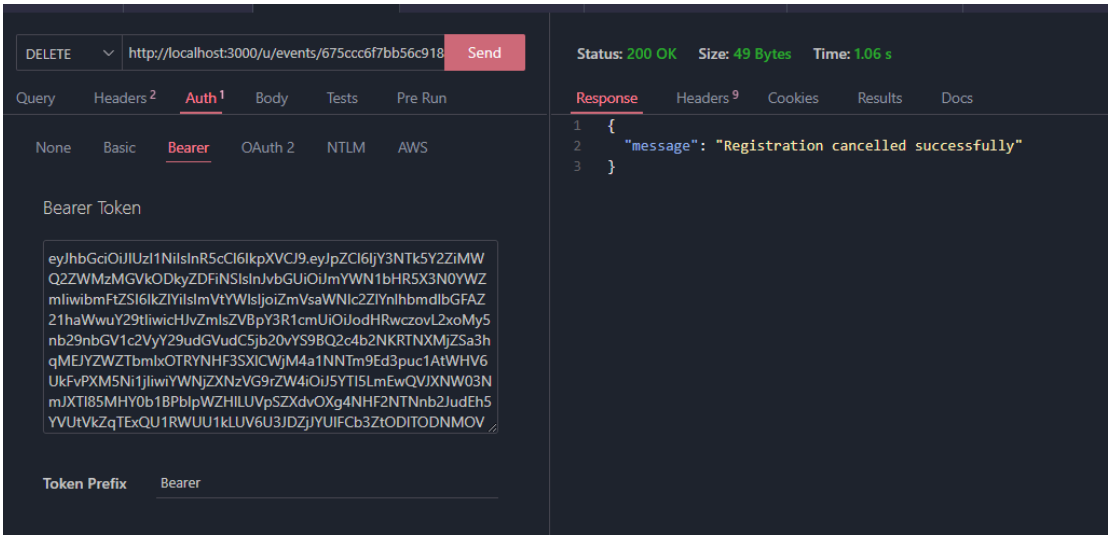


Figure 44. A Valid DELETE Request for Cancelling User Event Registration

This figure shows a valid DELETE request used to cancel a user's event registration. This request removes the user's registration from the event, effectively canceling their participation.

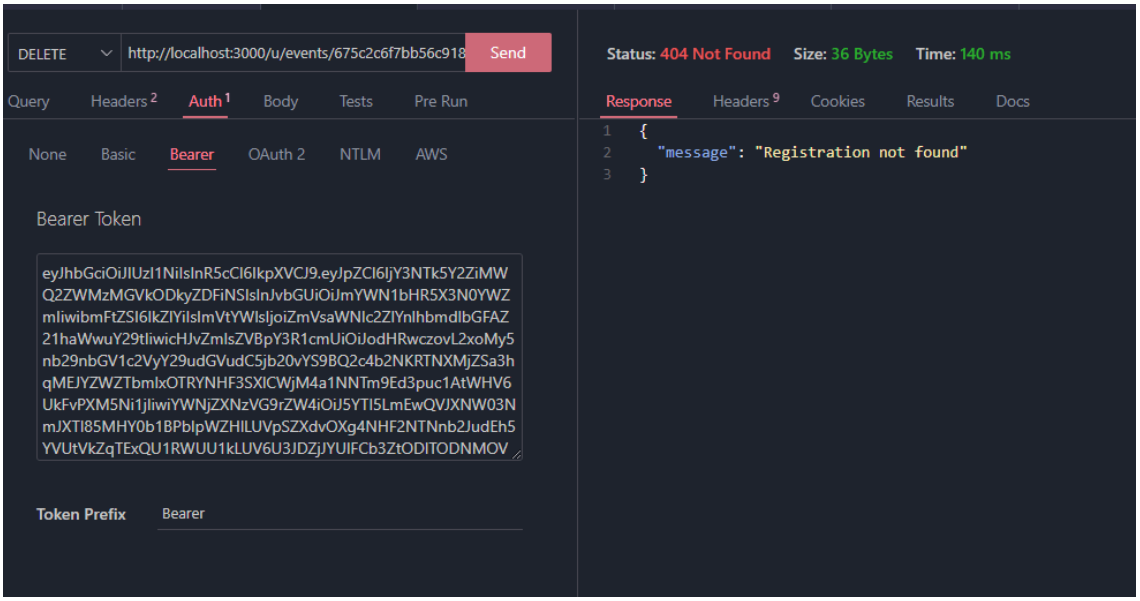


Figure 45. A Invalid DELETE Request for Cancelling User Event Registration

This figure shows an invalid DELETE request to cancel a user's event registration. This may occur if the user is not registered for the event and attempts to cancel it.

Response Format: JSON

Response:

The API response is for users to successfully cancel their registration for that specific event.

Different response Screenshots

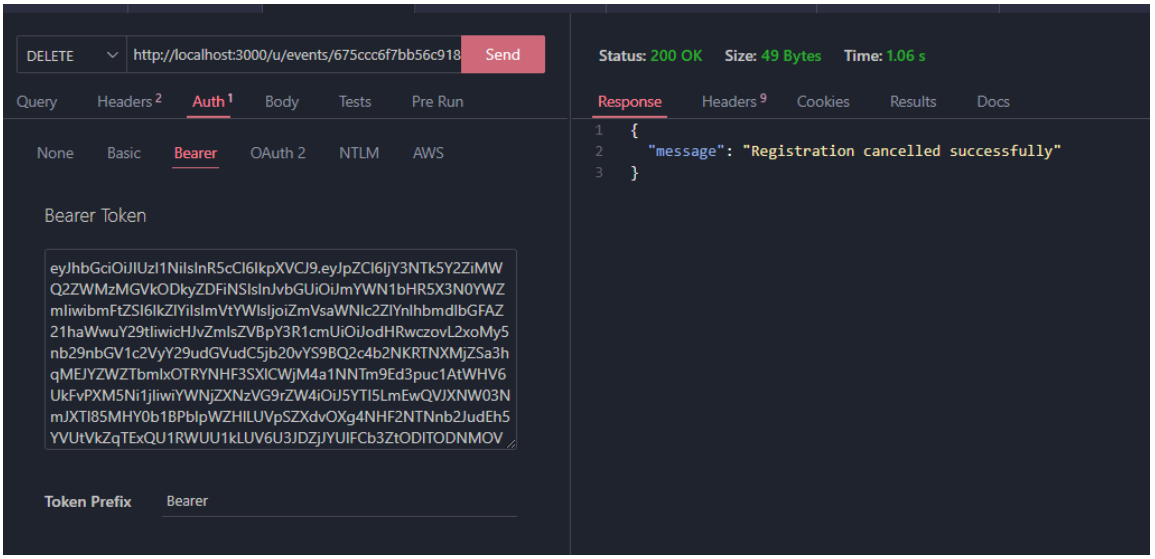


Figure 46. Successful cancellation of an event registration with a status code of 200.

This figure shows the successful cancellation of an event registration. The operation is confirmed with a status code of 200, which indicates that the request was successfully processed by the server.

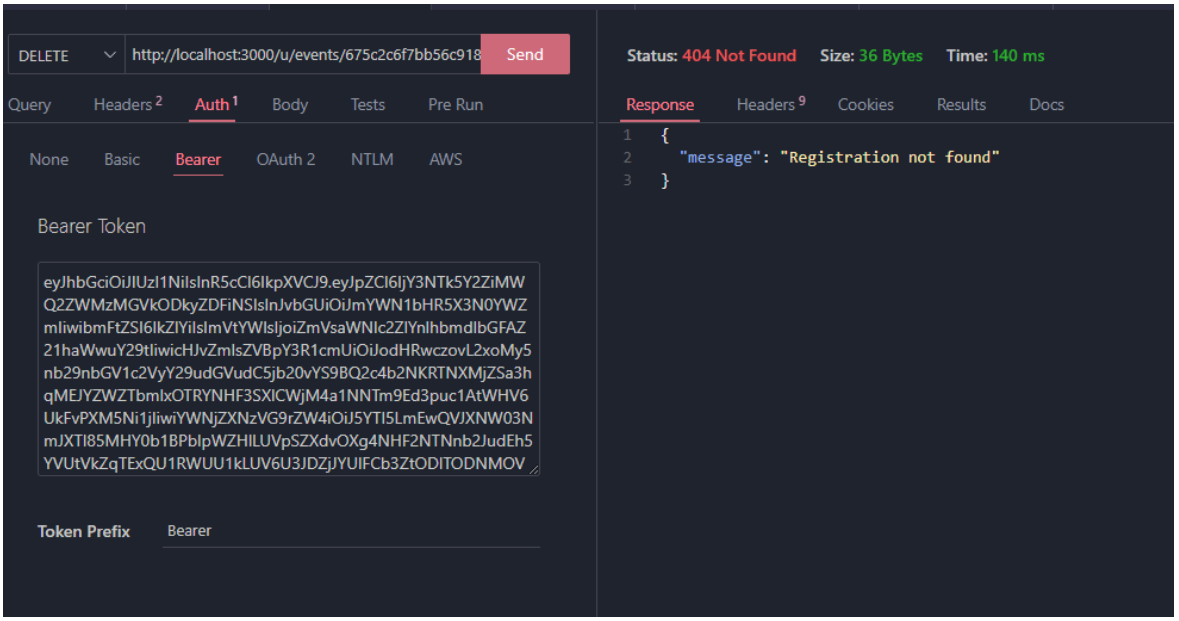


Figure 47. Failed cancellation of an event registration with a status code of 404.

This figure shows a failure to cancel a registered event since the user is not registered for that event, meaning there's nothing to cancel, causing a '404 Not Found' error.

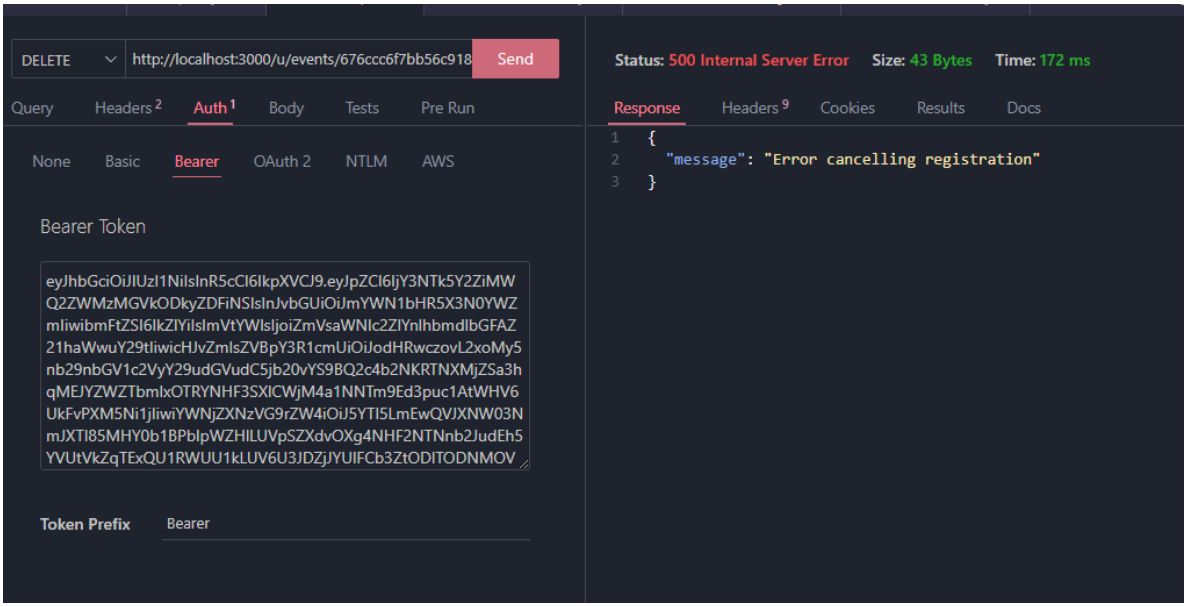


Figure 48. Server error when cancelling an event registration with a status code of 500

This figure shows a server error when attempting to cancel a registered event, usually due to a database or server issue.

1.1.11 Retrieving Users Event History

Version: 1.0

Date: November 13, 2024

Description: This API endpoint will fetch users past registered events, means an event they registered but already ended

Endpoint: http://localhost:3000/u/history

Method: GET

Configurations: The API requires a valid session token to authenticate the request and fetch the event history associated with the user.

Parameters:

- **authToken:** The session token used for user authentication

Response codes of this API endpoint:

Code	Message	Description
200	OK	The request was successful, and an array of registered events for the user was returned.
403	Invalid Token	The token provided in the request is invalid or expired, preventing authorization for the request.
500	Error fetching registered events	An unexpected error occurred while attempting to fetch the user's registered events from the database.

Valid and Invalid Request Responses

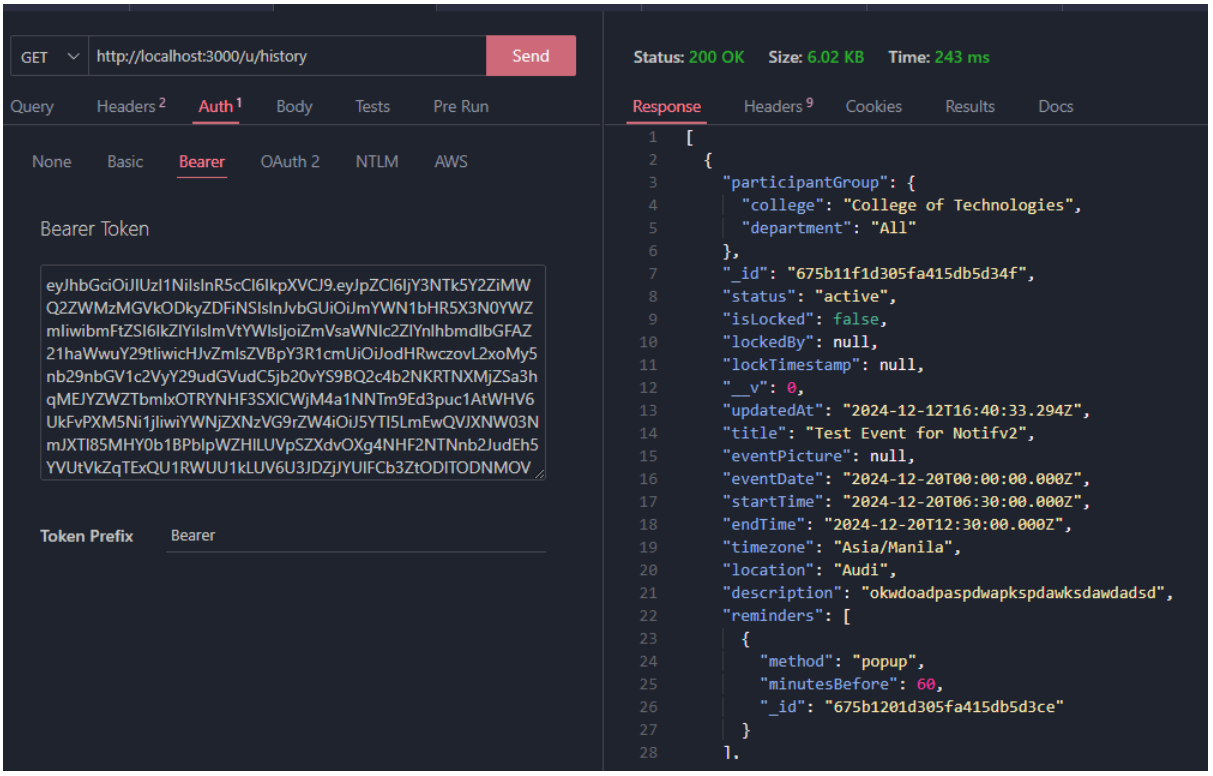


Figure 49. Valid GET Request for Registered Events That Have Already Ended

This figure shows a valid GET request with code 200 for decoding the user session token to display data, such as events that the user had registered for and have already ended.

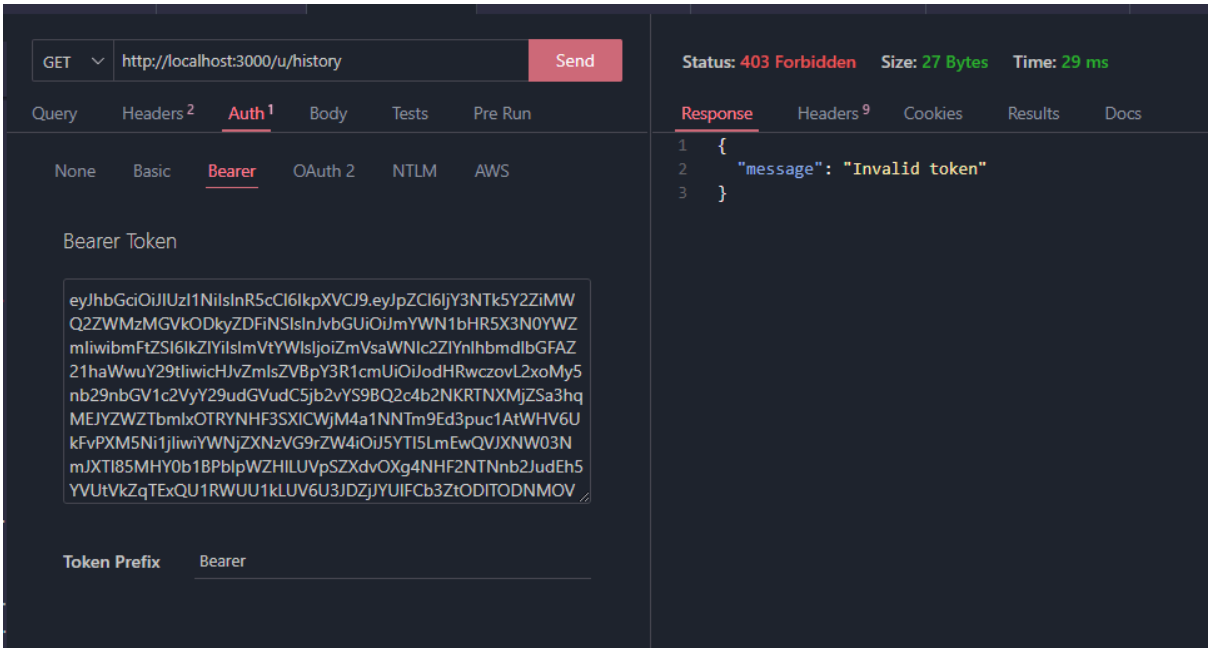


Figure 50. Invalid GET Request for Registered Events That Have Already Ended

This figure shows an invalid GET request with a 403 code, as the token is not valid, causing the request to fail.

Response Format: JSON

Response:

The API response is to list users past registered events, means an event they registered but already ended

Different response Screenshots

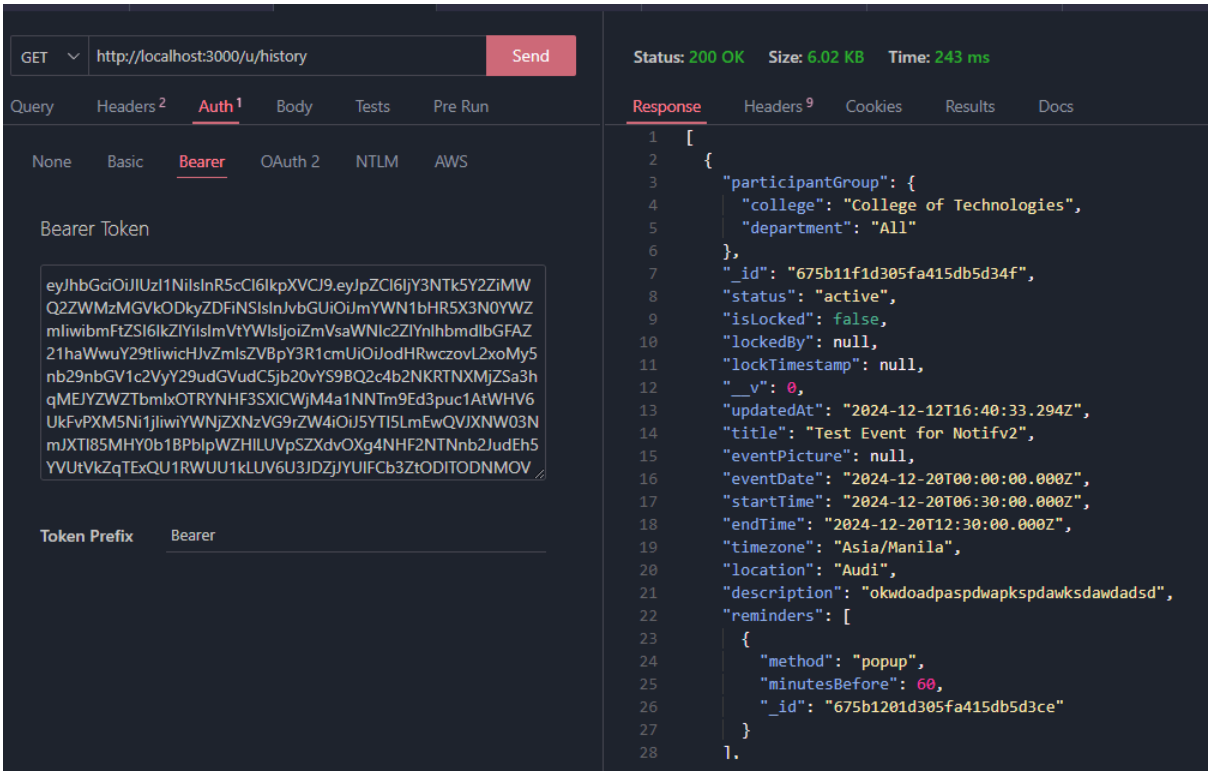


Figure 51. Successful Retrieval of Ended Registered Events with Code 200

This figure shows a 200 status code response, indicating the successful retrieval of events that the user had registered for and that have already ended.

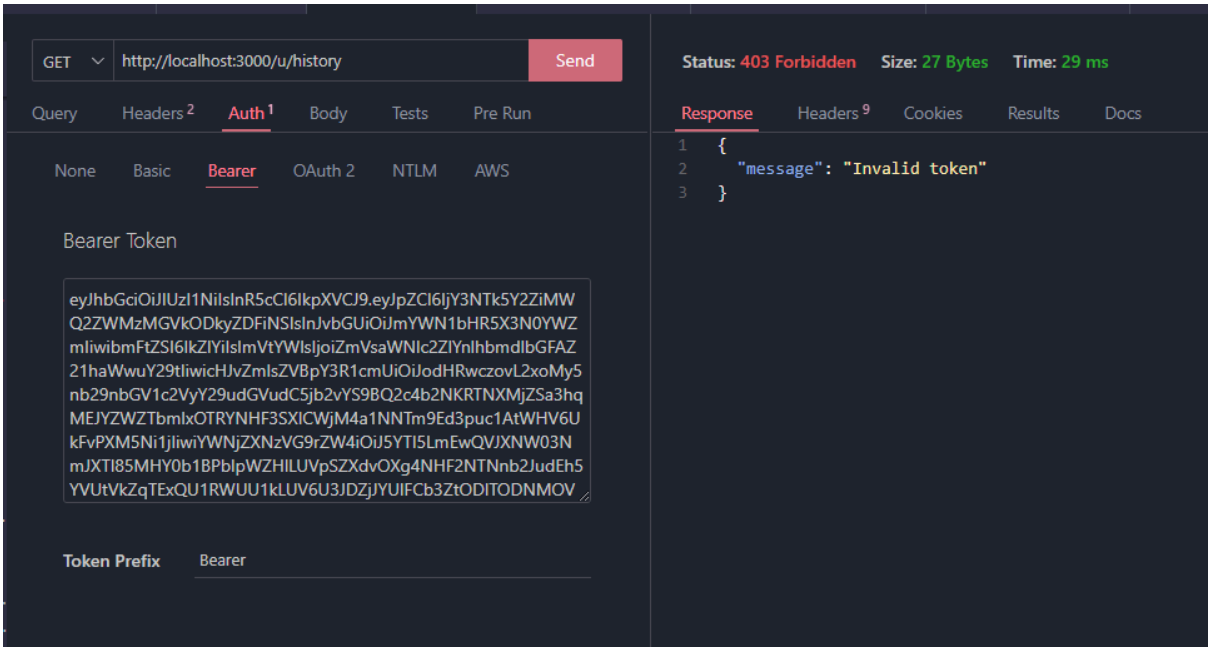


Figure 52. Error Retrieval of Ended Registered Events with Code 403

This figure shows a 403 status code response, indicating that the retrieval of events the user had registered for and that have already ended failed due to an invalid token. The system was unable to determine which user's completed registered events to retrieve.

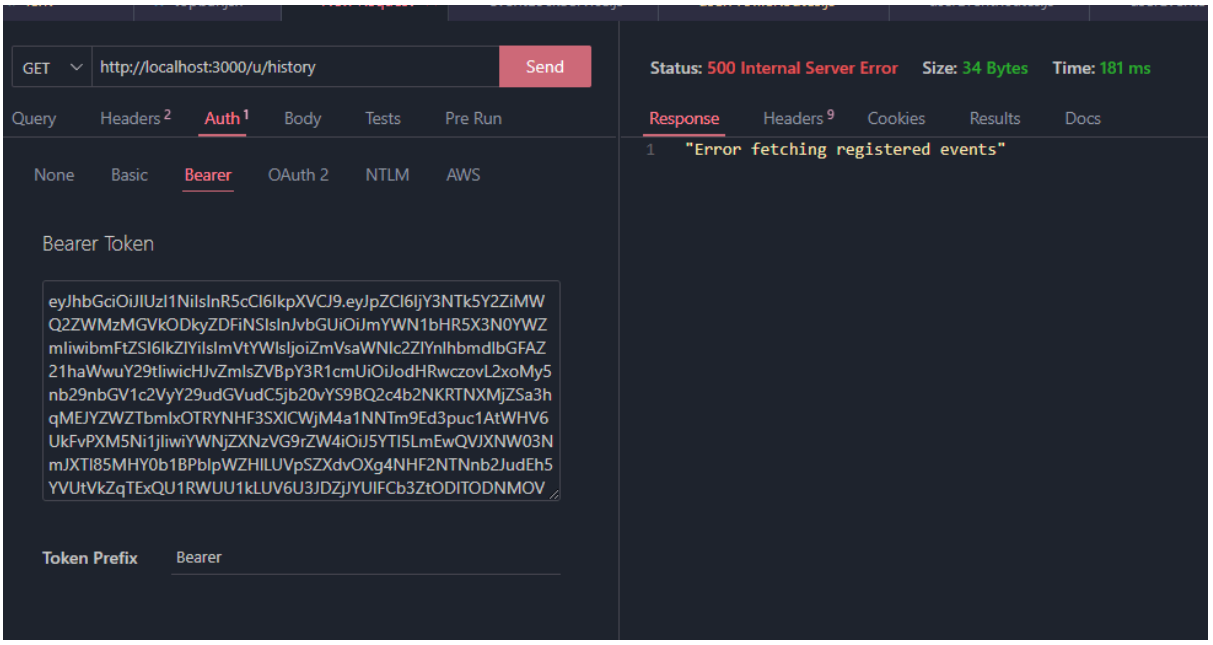


Figure 53. Server Error Retrieval of Ended Registered Events with Code 500

This figure shows a 500 status code response, indicating that the retrieval of the user's registered completed events failed due to a server error.

1.1.12 User Specific Event Form Submission

Version: 1.0

Date: December 2, 2024

Description: This API endpoint will enable users to submit the event form after the event has ended

Endpoint: http://localhost:3000/u/events/:id/form-submission

Method: POST

Configurations: This API requires an active user session for authentication. The user must submit the form only after the event has concluded.

Parameters:

- **authToken:** The session token (JWT) used for user authentication, passed in the Authorization header**.**
- **eventId:** The unique identifier of the event for which the user is submitting the form.

Response codes of this API endpoint:

Code	Message	Description
200	Successful Submission of a Form	This indicates a successful submission where the form is verified, the user's response is found, and the data is recorded. The submission is processed and the certificate generation is initiated.
400	Failed Submission of a Form	This indicates a failed submission, typically due to errors like missing registration, invalid form data, or the user not having submitted the form with the correct email.
500	Server Error for Verifying the Submitted Form	This indicates a server-side error while trying to verify the form submission. Possible causes could include issues with database access, Google Sheets API, or other server-related problems.

Valid and Invalid Request Responses

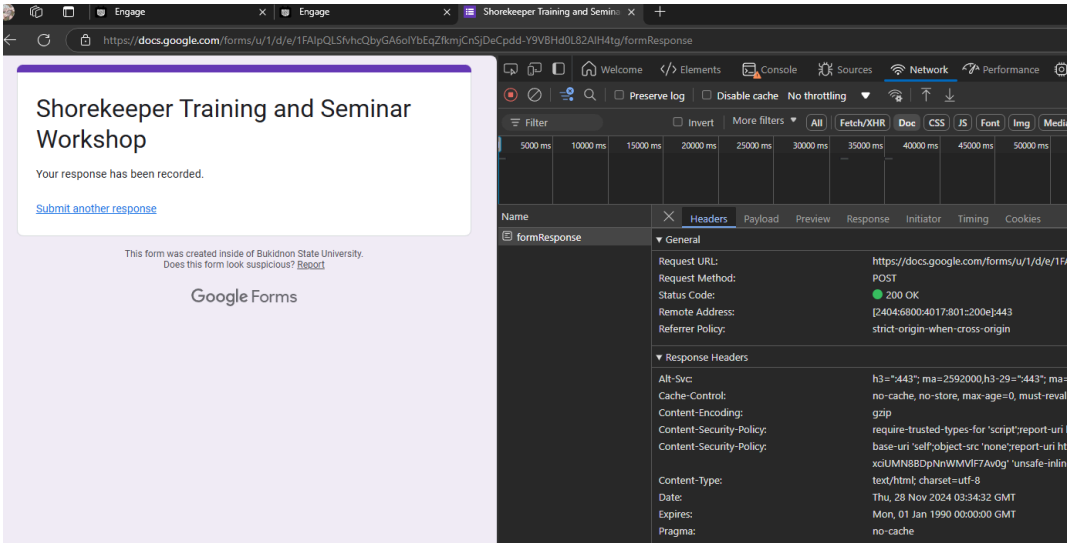


Figure 54. Valid POST Request for Submitting a Form

This figure shows a valid request for submitting a form for an event that the user participated in and has already ended. This enables the system to open a form for the user to evaluate the post-event. The request requires the email of the user who attended, submission of the form, and verification of the form after submission

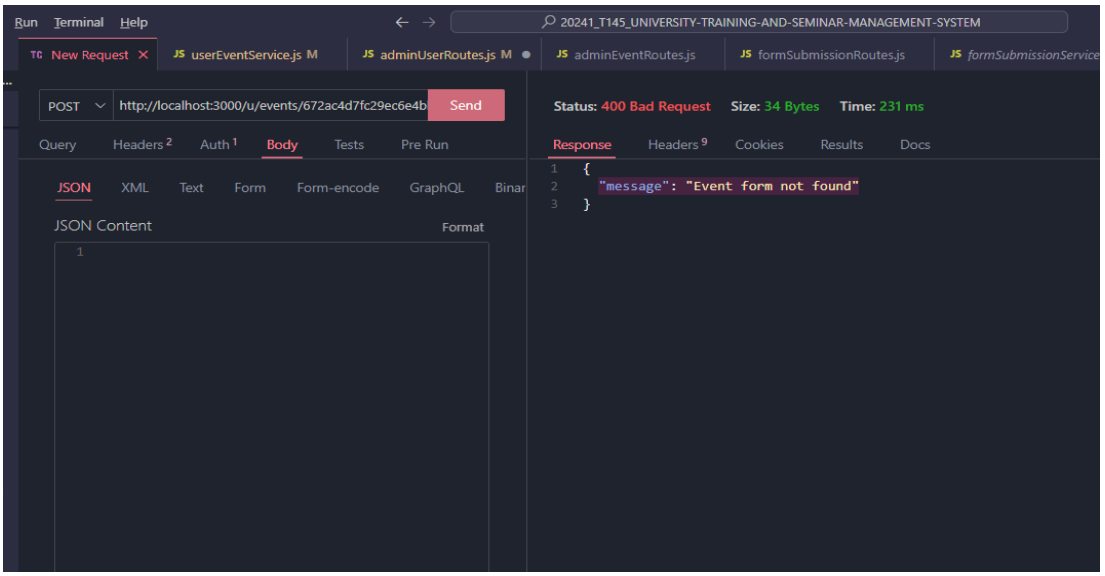


Figure 55. Invalid POST Request for Submitting a Form

This figure shows an invalid request for submitting a post-event form, due to the event form not being found.

Response Format: JSON

Response:

The API response is for users who successfully submit the event form after the event has ended.

Different response Screenshots

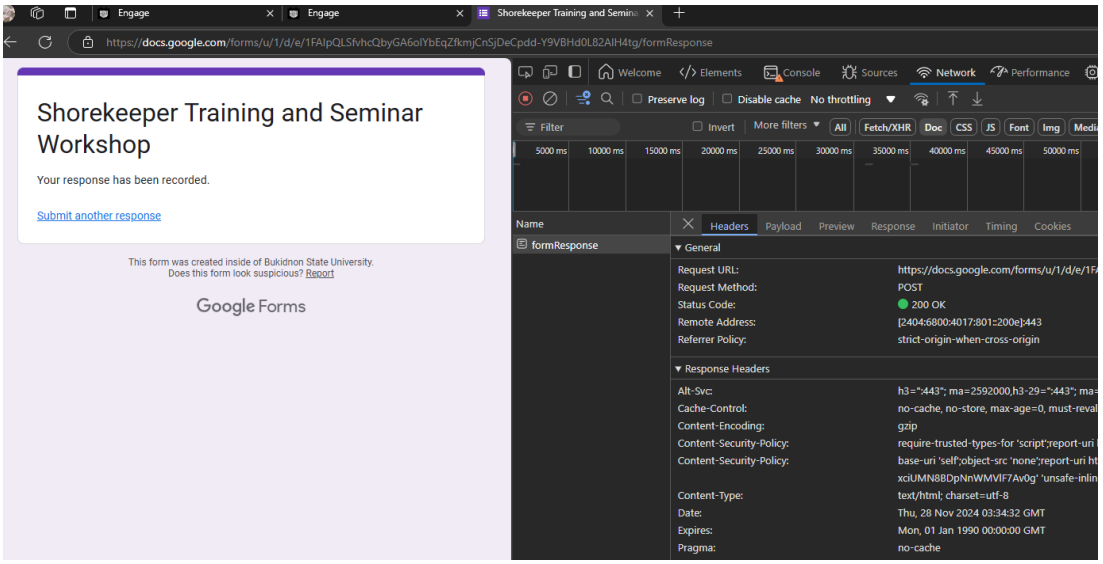


Figure 56. Successful Submission of a Form with Status Code 200

This figure shows the successful submission of a form with a status code of 200, which was then stored in the database.

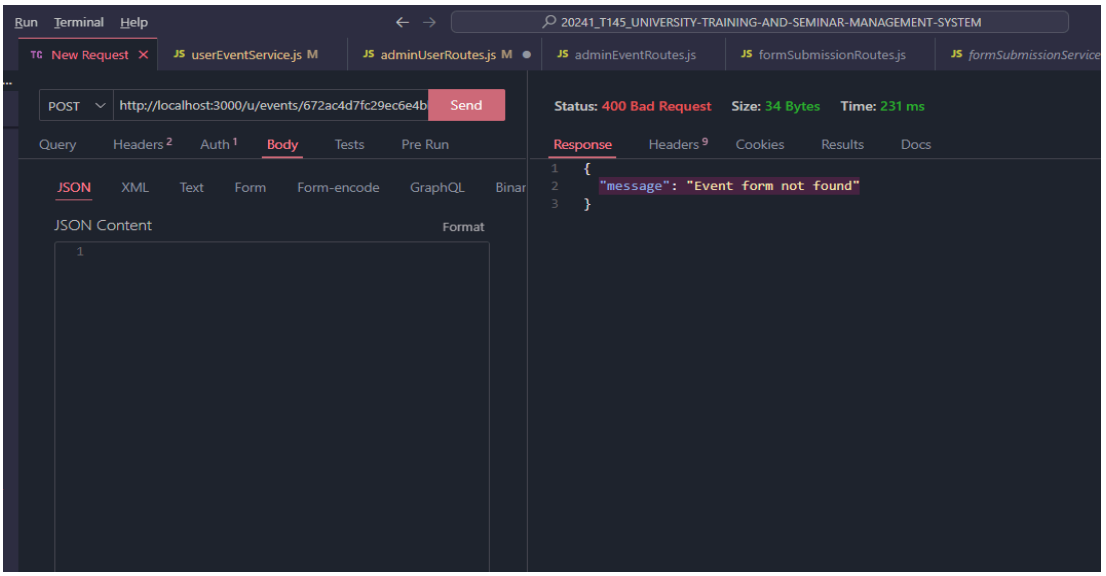


Figure 57. Failed Submission of a Form with Status Code 400

This figure shows the failure of form submission with a status code of 400. This is due to an error, such as the event form not being successfully retrieved or the event form being invalid.

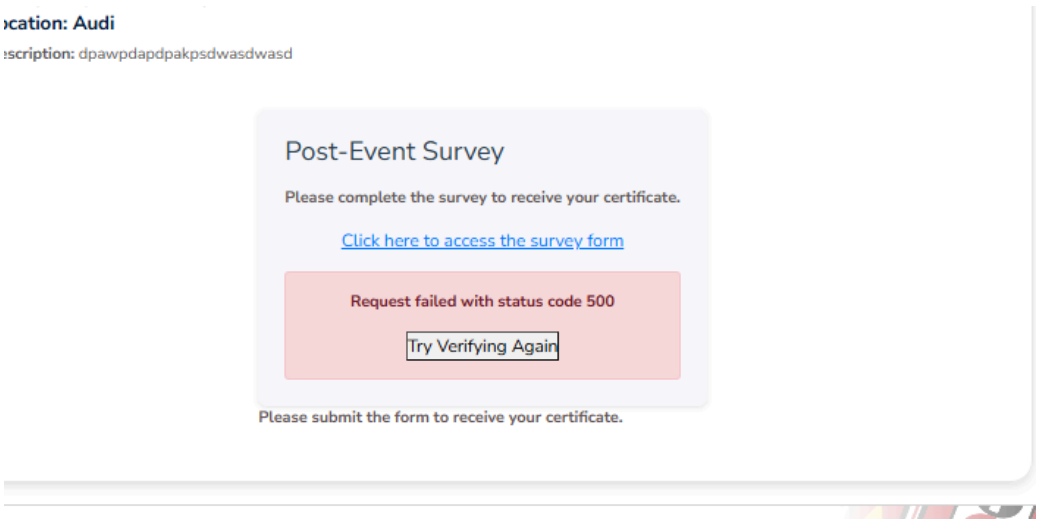


Figure 58. Server Error with Status Code 500 for Verifying the Submitted Form

This figure shows a server error (status code 500) while verifying the submitted form. This is due to a problem on the server, which can occur if Google credentials are exposed, leading to Google disabling them

1.1.13 User Registered Form Status

Version: 1.0

Date: December 2, 2024

Description: This API endpoint will fetch the user's form status to determine whether they were able to submit the status or not.

Endpoint: http://localhost:3000/u/events/:id/form-status

Method: GET

Configurations: This API requires an active user session to ensure that the certificate data is retrieved for the authenticated user only.

Parameters:

- **authToken:** The session token (JWT) used for user authentication, passed in the Authorization header.
- **eventId:** The unique identifier of the event for which the user is submitting the form.

Response codes of this API endpoint:

Code	Message	Description
200	OK	The request was successfully processed, and the form submission status was returned.
403	Invalid Token	The client does not have access rights to the content. This code was not explicitly used in the function.
404	Event Form Not Found	The specified form submission for the given eventId and userId could not be found..
500	Internal Server Error	An error occurred on the server while processing the request, such as during database access.

Valid and Invalid Request Responses

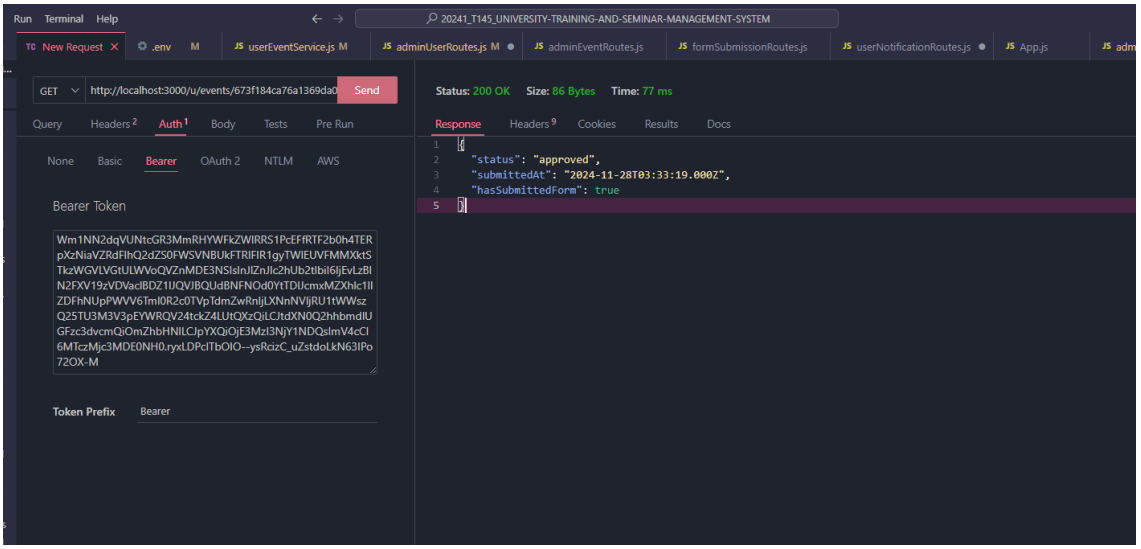


Figure 59. Valid GET Request for Retrieving the Status of a Form

This figure shows a valid request for retrieving the status of the form, indicating whether the user is approved or not. This allows the system to determine the user's approval status.

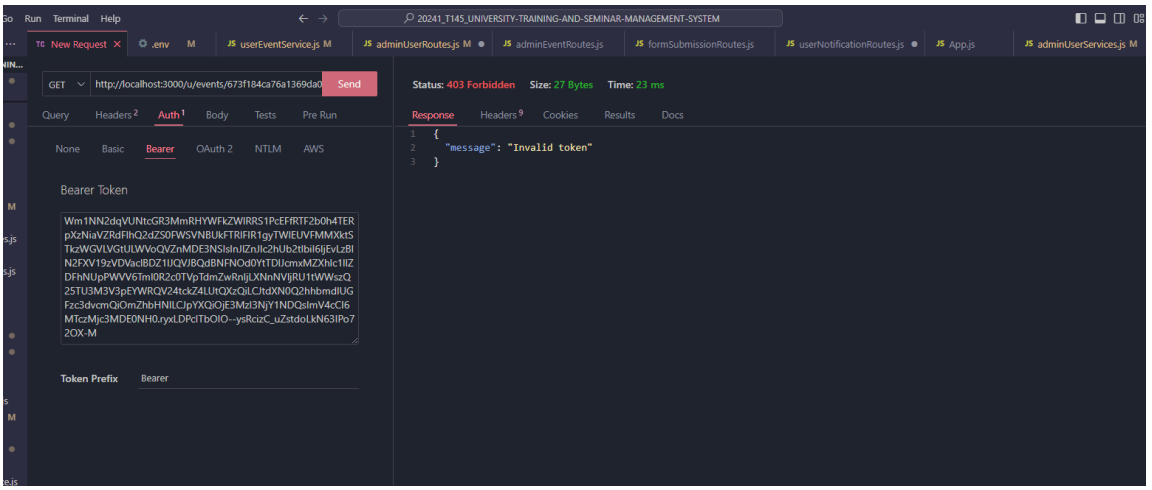


Figure 60. Invalid GET Request for Retrieving the Status of a Form

This figure shows an invalid request for retrieving the status of the form, due to the user token being invalid or expired.

Response Format: JSON

Response:

The API response successfully indicates the user's form status, allowing it to determine whether they were able to submit it or not

Different response Screenshots

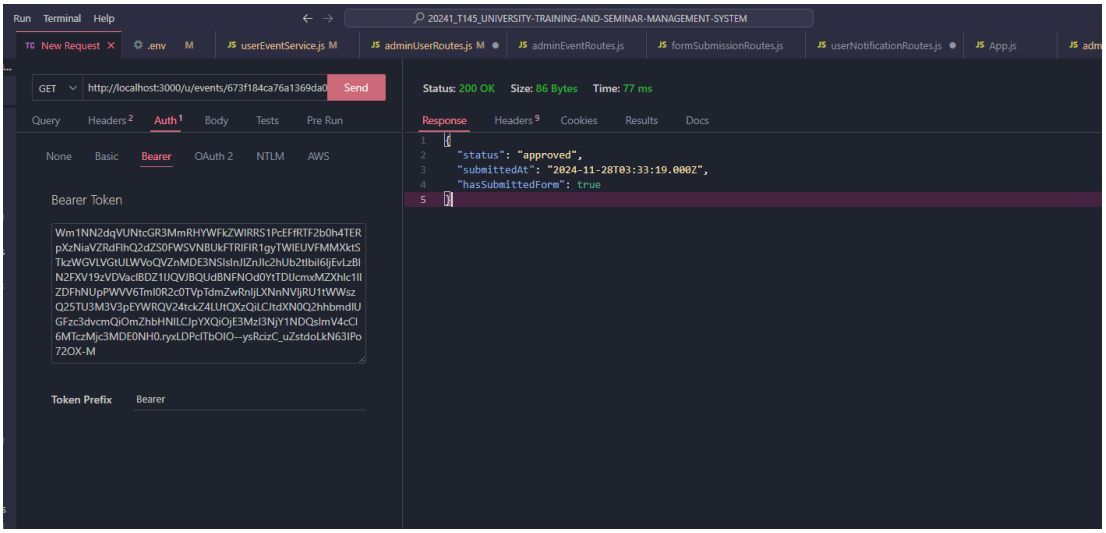


Figure 61. Successful Retrieval of the User Form with Status Code 200

This figure shows the successful retrieval of a user form with a status code of 200. It confirms whether the user's submitted and verified form has been approved

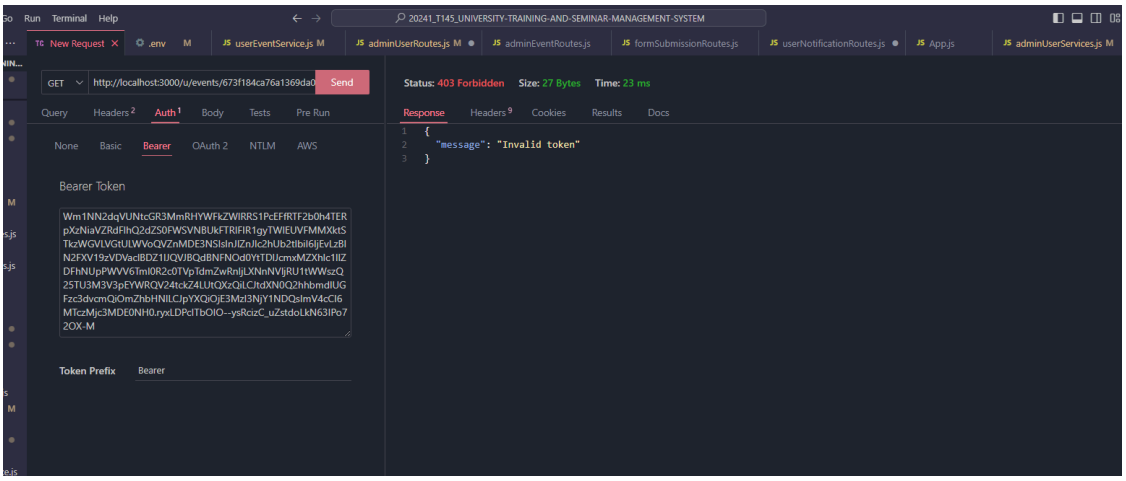


Figure 62. Failure Retrieval of the User Form with Status Code 403

This figure shows the failed retrieval of a user form with a status code of 403. This is due to the user token being invalid, meaning it is either incorrect or expired, which causes the rejection of the form status retrieval.

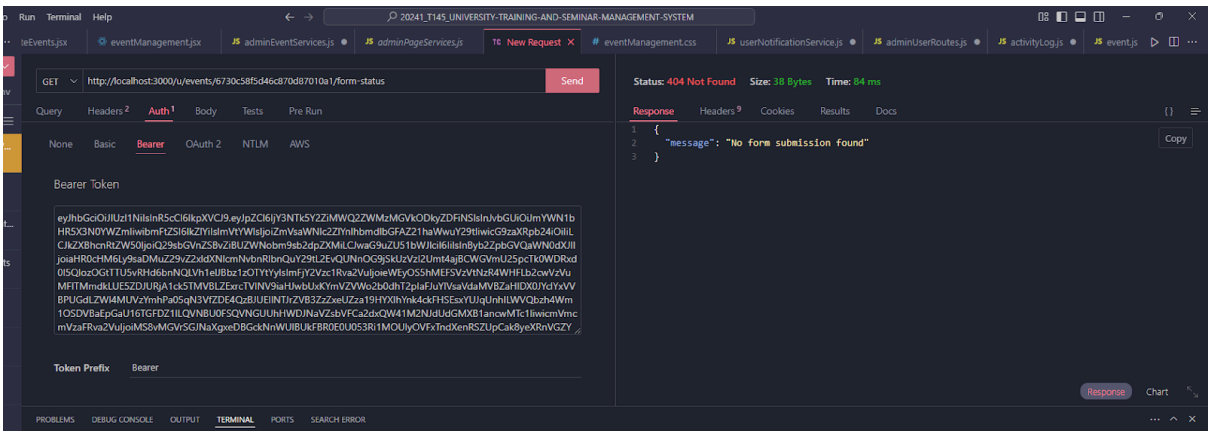


Figure 63. Failure Retrieval of the User Form with Status Code 404

This figure illustrates the failure to retrieve the user form, resulting in a 404 status code. This indicates that the system could not locate the form submitted by the user for this event.

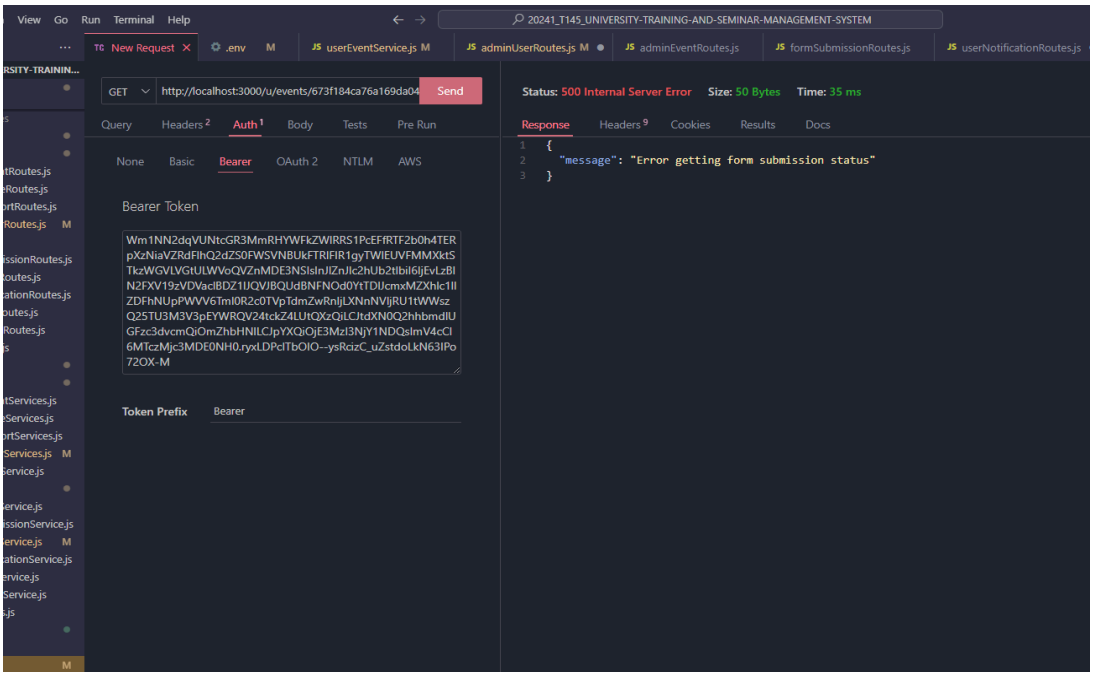


Figure 64. Server Error Response with the Status code 500

This figure shows the server error with a status code of 500. This is due to an issue that has occurred on the server side.

1.1.14 Retrieving Users Form Analytics

Version: 1.0

Date: December 5, 2024

Description: This API endpoint will retrieve analytics for a specific event's form. It fetches data from a linked Google Form, processes responses from a spreadsheet, and provides insights such as total responses, response rate, and question summaries. Authentication is required to access this endpoint.

Endpoint: http://localhost:3000/u/events/:id/form-analytics

Method: GET

Configurations:

The API authenticates requests, retrieves event details and form responses via the Google Sheets API, and processes the data to calculate analytics like total responses and question summaries.

Parameters:

- **authToken:** The session token (JWT) used for user authentication, passed in the Authorization header.
- **eventId:** The unique identifier of the event whose form analytics are being retrieved.

Response codes of this API endpoint:

Code	Message	Description
200	OK	The request was successful, and the form analytics were retrieved and returned.
404	Event Form Not Found	The specified event does not have an associated form or the event itself could not be located.
500	Error Fetching Analytics	An internal server error occurred while processing the request or fetching analytics.

Valid and Invalid Request Responses

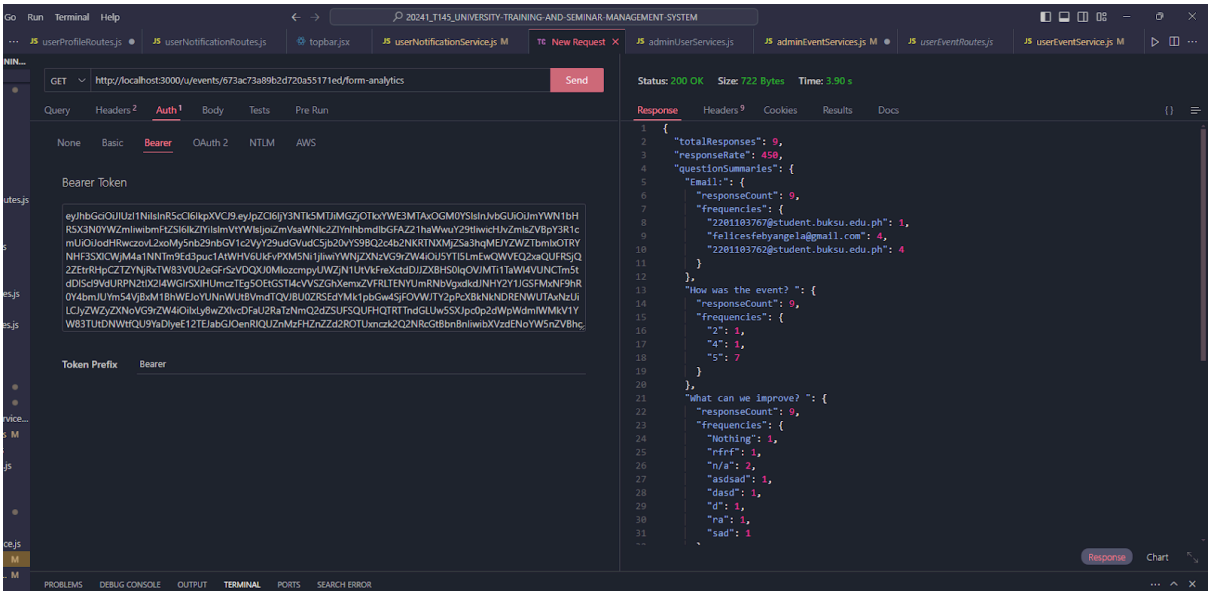


Figure 65. Valid GET Request for Retrieving the User Form Analytics

This figure shows a valid GET request being made to retrieve user form analytics. The request is processed successfully and returns the relevant analytics data for the form responses.

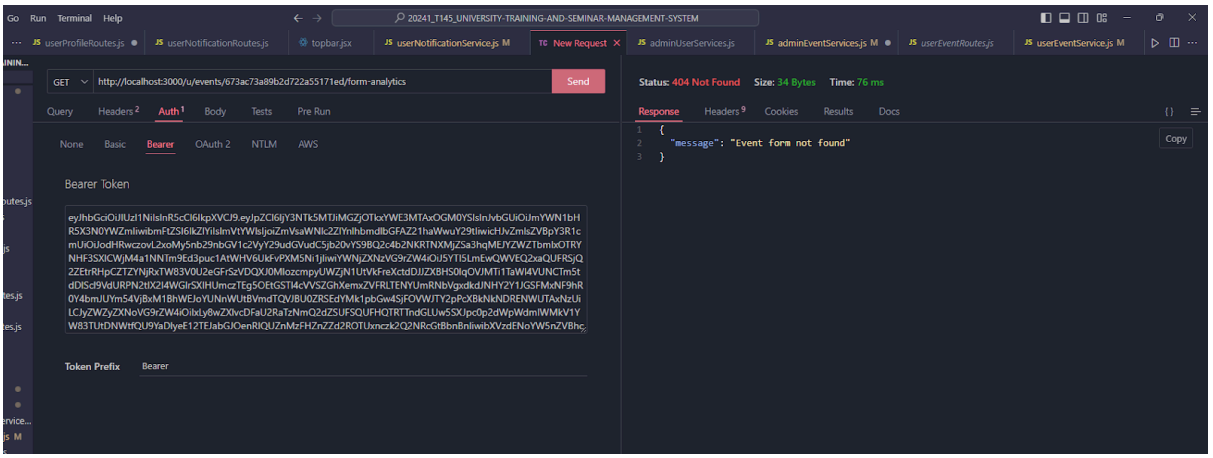


Figure 66. Invalid GET Request for Retrieving the User Form Analytics

This figure illustrates an invalid GET request for retrieving user form analytics. The request fails due to issues such as incorrect parameters or unauthorized access.

Response Format: JSON

Response:

The API response provides analytics including total responses, response rate, and question summaries based on the retrieved form data.

Different response Screenshots

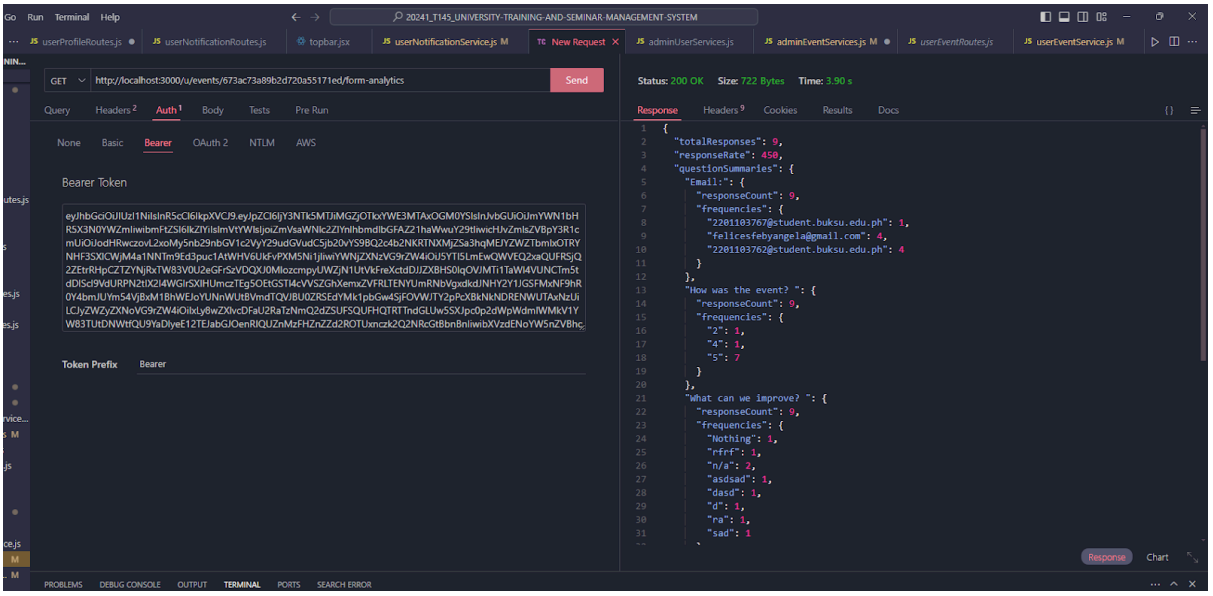


Figure 67. Successful Retrieval of the User Form Analytics with Status Code 200

This figure demonstrates the successful retrieval of the user form analytics, returning a status code of 200, along with the analytics data, including total responses, response rate, and question summaries.

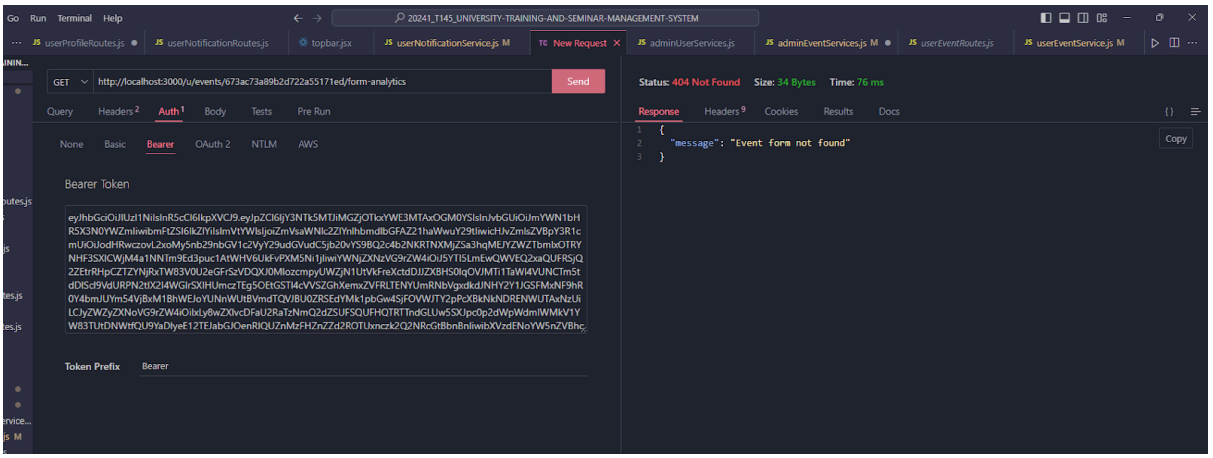


Figure 68. Failed Retrieval of User Form Analytics (Status Code 404)

This figure shows a failed attempt to retrieve user form analytics, with a status code of 404. The failure occurs when the event form is not found or does not exist in the database.

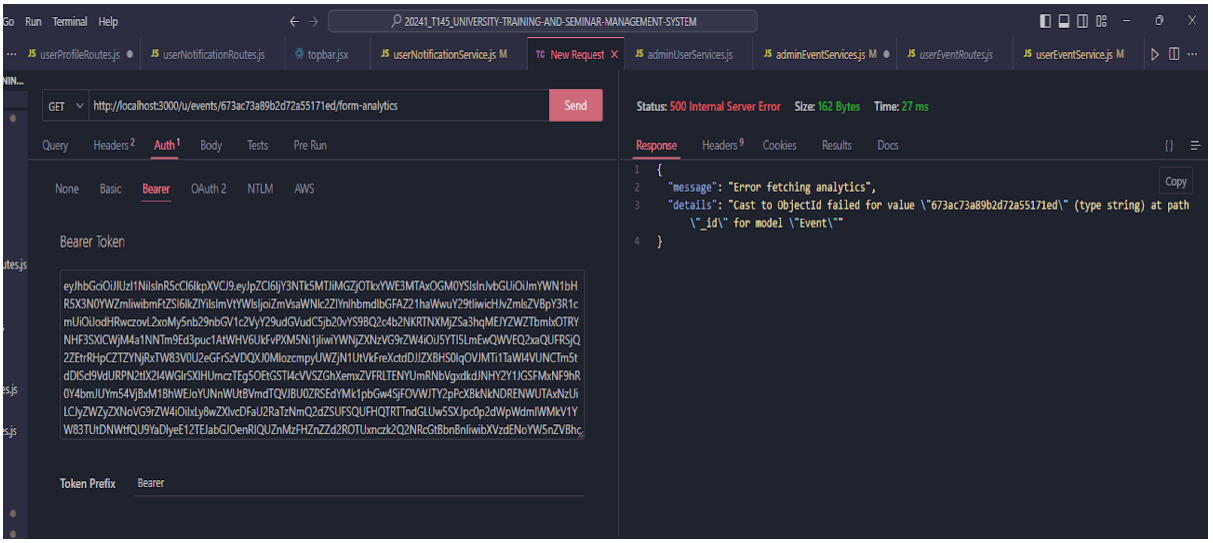


Figure 69. Server Error During Retrieval of the User Form Analytics (Status Code 500)

This figure illustrates a server error when retrieving the user form analytics, returning a status code of 500. The error typically occurs due to a server issue, such as a database problem or unexpected server failure.

1.1.15 Retrieving Users Certificate Status

Version: 1.0

Date: November 30, 2024

Description: The API will retrieve the status of a user's certificate for a specific event. It checks if the certificate has been issued for the user based on the event ID and user ID, and returns a status indicating whether the certificate is "generated" or "pending".

Endpoint: http://localhost:3000/u/events/:id/certificate-status

Method: GET

Configurations:

The API should authenticate the user to ensure secure access, typically using a session token or JWT.

Parameters:

- **authToken:** The session token (JWT) used for user authentication, passed in the Authorization header.
- **eventId:** The unique identifier of the event for which the certificate status is being checked. This is passed as part of the URL.

Response codes of this API endpoint:

Code	Message	Description
200	Certificate Status Retrieved Successfully	The request was successful, and the certificate status has been retrieved, indicating whether it is "generated" or "pending."
403	Invalid Token	The provided token is invalid or expired, preventing access to the certificate status.
500	Error Checking Certificate Status	A server error occurred while attempting to retrieve the certificate status, typically due to a database or internal issue.

Valid and Invalid Request Responses

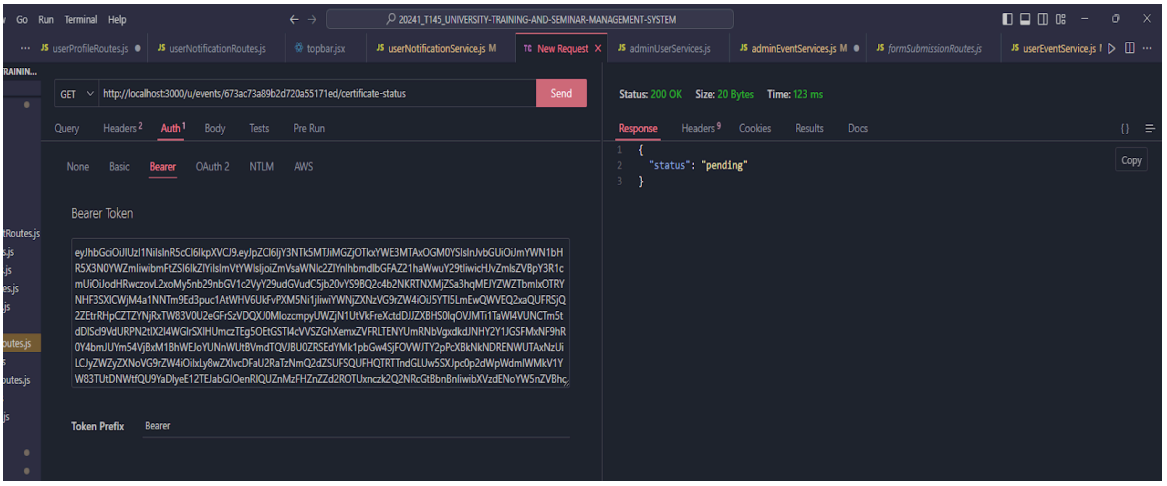


Figure 70. Valid GET Request for Retrieving Users Certificate Status

This figure shows a successful GET request to retrieve the user's certificate status. The status code returned is 200, indicating the certificate status is correctly fetched.

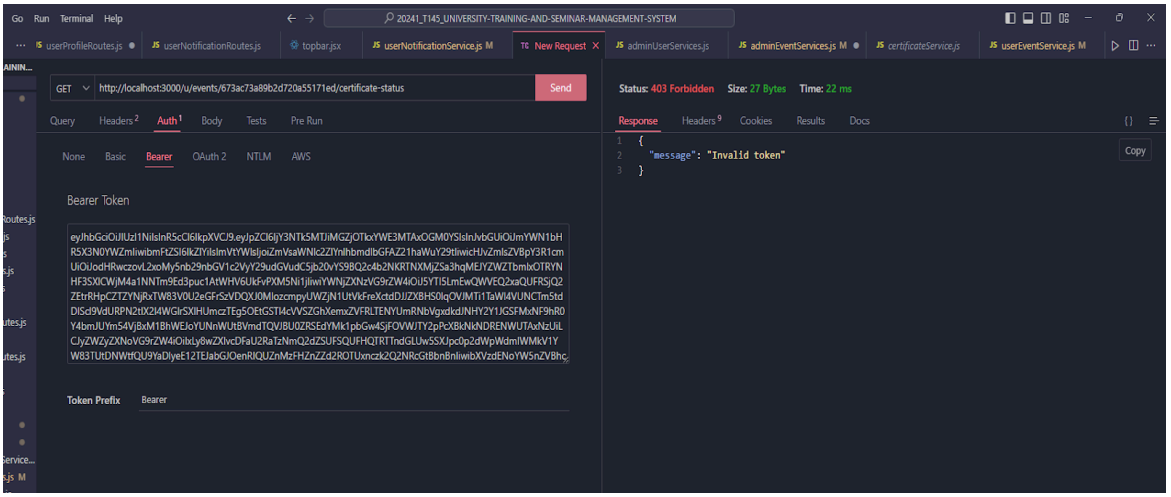


Figure 71. Invalid GET Request for Retrieving Users Certificate Status

This figure demonstrates a failed GET request due to missing or incorrect parameters. The status code returned is 400, indicating a bad request.

Response Format: JSON

Response:

The API will respond with the certificate status indicating whether the certificate has been generated or is pending.

Different response Screenshots

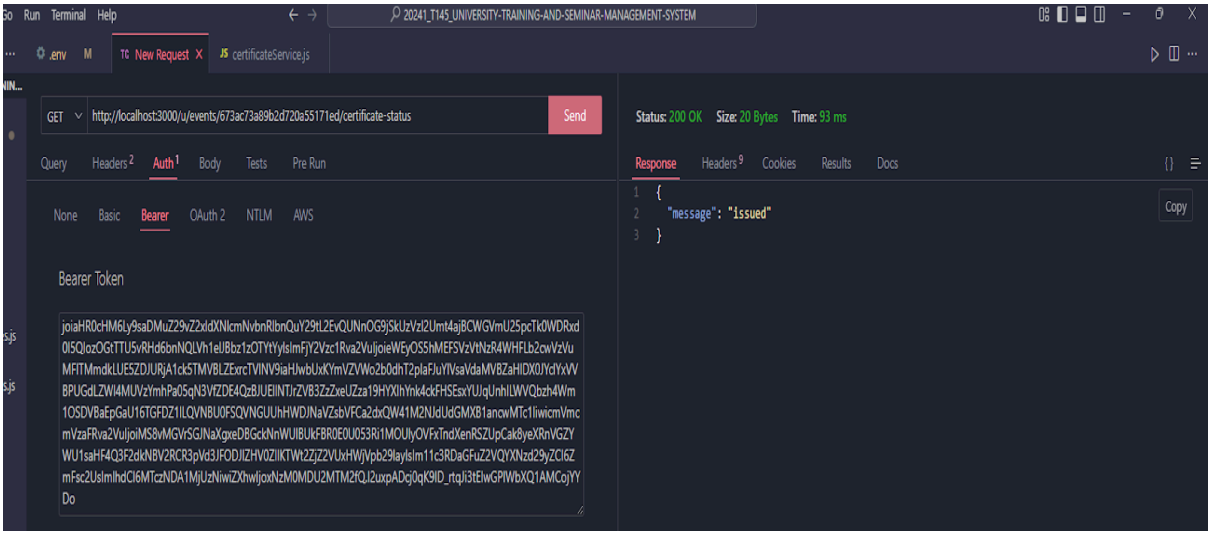


Figure 72. Successful Retrieval of the User's Certificate Status with Status Code 200

This figure illustrates the successful retrieval of the user's certificate status with a status code 200, indicating that the certificate has been successfully generated and issued.

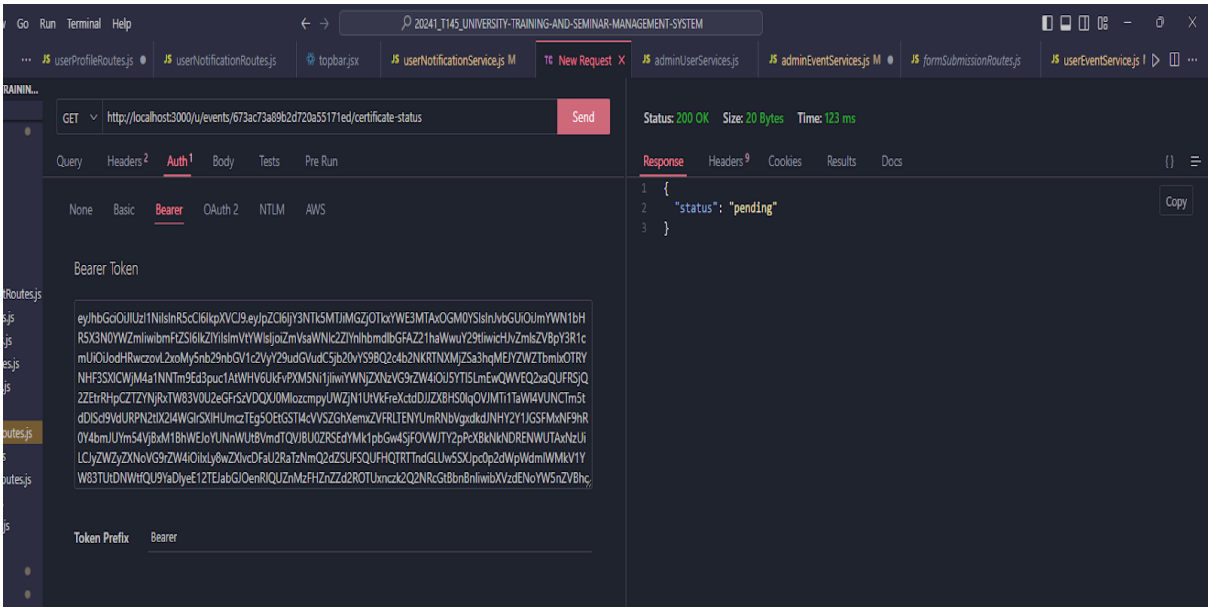


Figure 73. Retrieval of the User's Certificate Status (Status Code 200)

This figure shows the successful retrieval of the user's certificate status, with the status code 200. The certificate status is "pending," indicating that the certificate has not yet been generated.

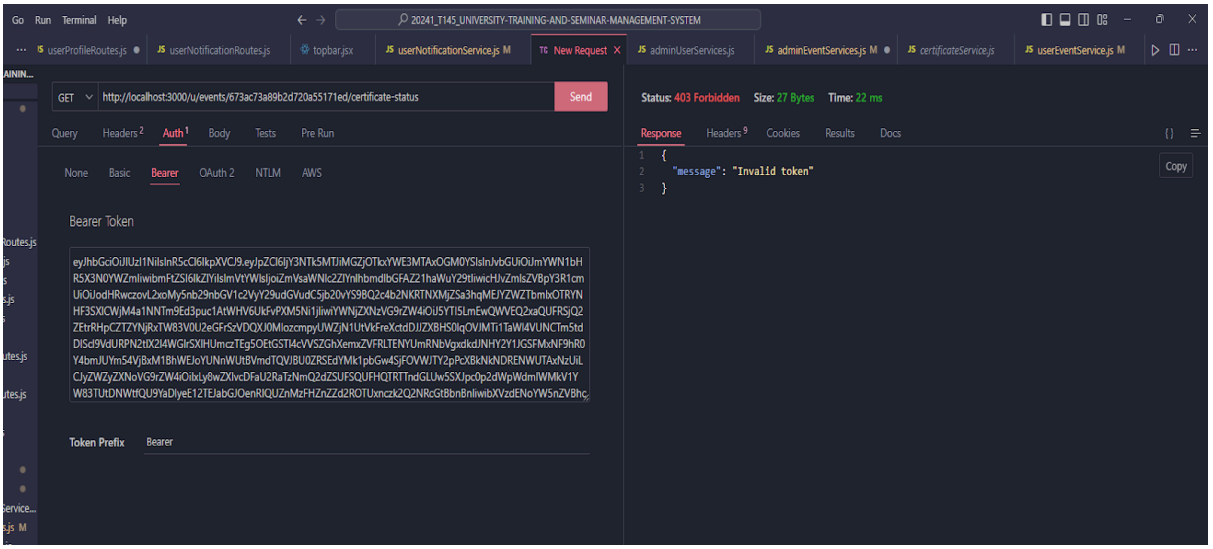


Figure 74. Invalid Token (Status Code 403)

This figure illustrates a failed request due to an invalid or expired authentication token. The server returns a 403 status code, denying access to the certificate status

.

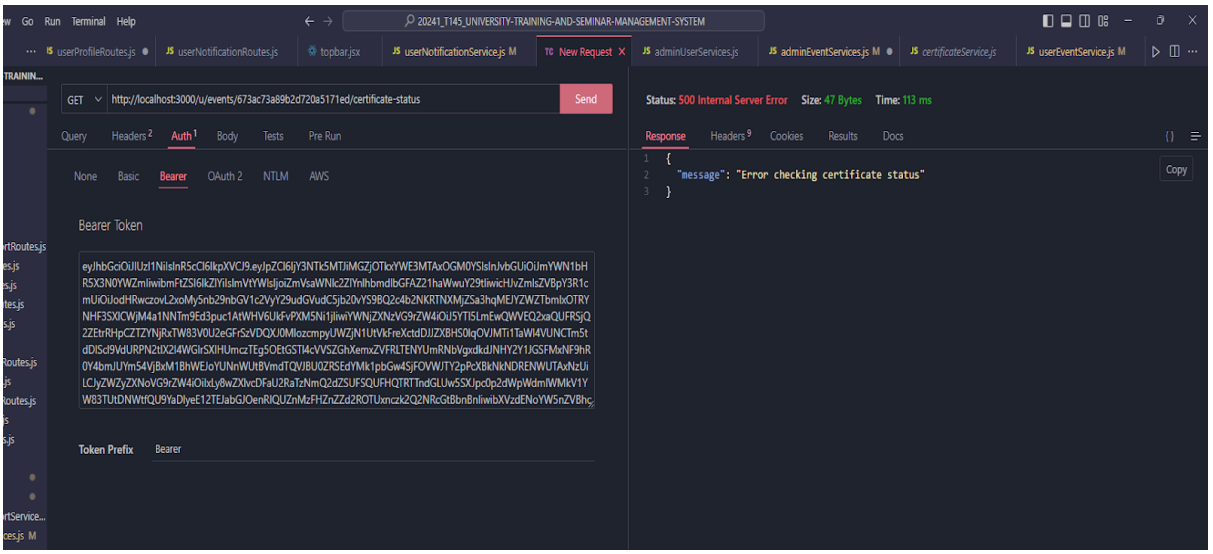


Figure 75. Server Error While Retrieving the User's Certificate Status (Status Code 500)

This figure represents a server error during the retrieval of the user's certificate status. The status code 500 indicates an internal server issue, preventing the request from being processed successfully.

1.1.16 Retrieving User Notification List

Version: 1.0

Date: December 5,2024

Description: This API endpoint will fetch users' notifications, such as notifications from the system informing them that they have been invited to

Endpoint: http://localhost:3000/u/notification/items

Method: GET

Configurations: This endpoint might require user authentication. Ensure that users are authenticated before accessing the notification list.

Parameters:

- **authorizationToken:** The session token (JWT) used for user authentication, passed in the Authorization header.

Response codes of this API endpoint:

Code	Message	Description
200	OK	The request was successful, and the list of items (notifications) was retrieved and returned in the response body.
401	Access denied. No token provided	The request is missing the authentication token, preventing access to the requested resource.
403	Invalid Token	The request lacks valid authentication credentials (e.g., the token is invalid or expired).
500	Error Retrieving Items	An error occurred on the server side while retrieving the items, preventing the request from being fulfilled.

Valid and Invalid Request Responses

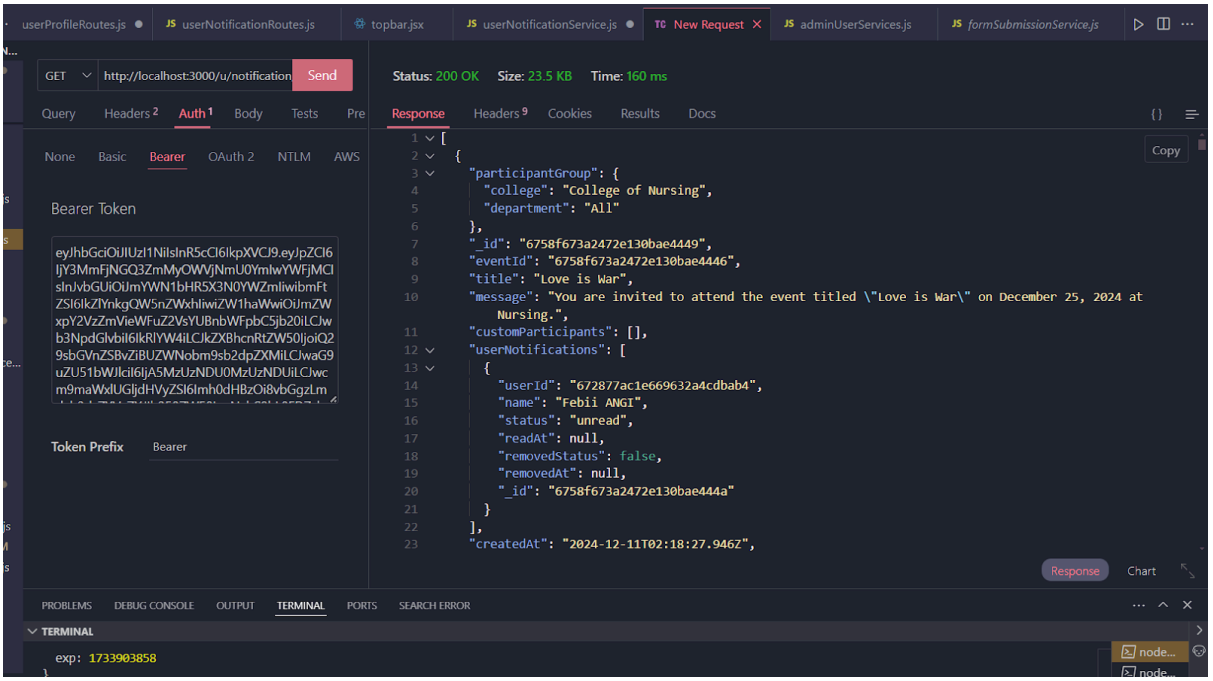


Figure 76. Valid GET Request for Listing All User Notifications

This figure represents a valid request for retrieving all event invitations from the admin. When an event is created and users are invited, they will be notified.

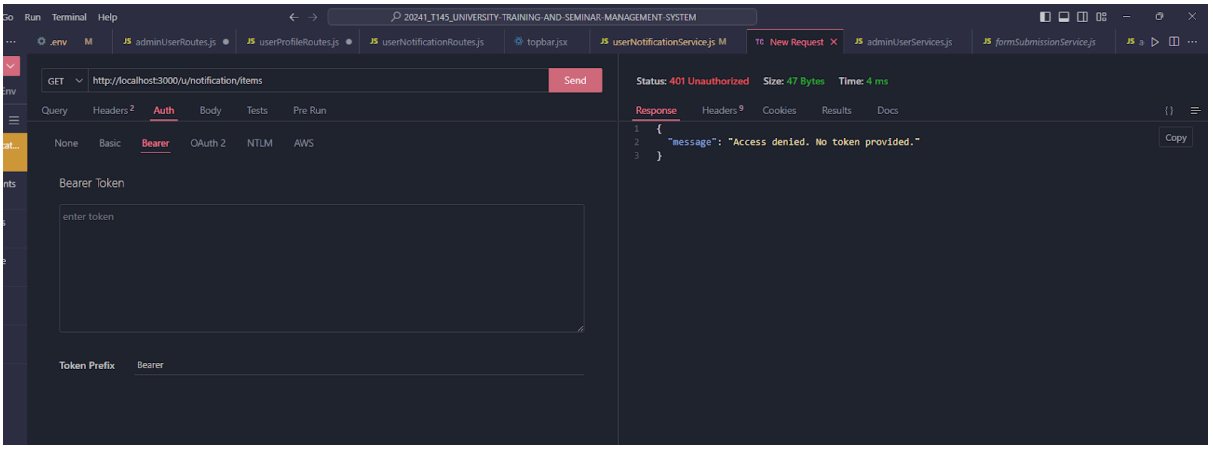


Figure 77. Invalid GET Request for Listing All User Notifications

This figure illustrates an invalid request for retrieving all user notifications from the admin, caused by denied access due to the lack of an authentication token.

Response Format: JSON

Response:

The API response to list notifications, such as notifications from the system informing them that they have been invited to an event.

Different response Screenshots

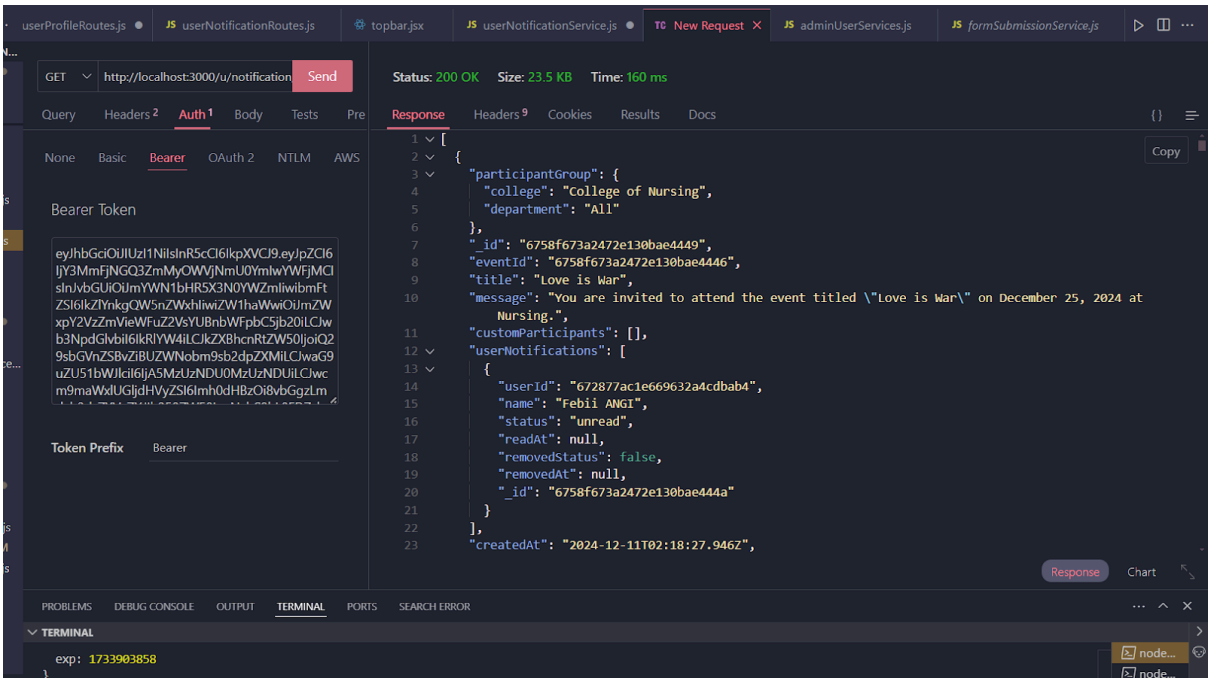


Figure 78. Successful Retrieval of User Notifications List with Status Code 200

This figure indicates that the request was processed successfully, and the server returned the list of user notifications in the response body.

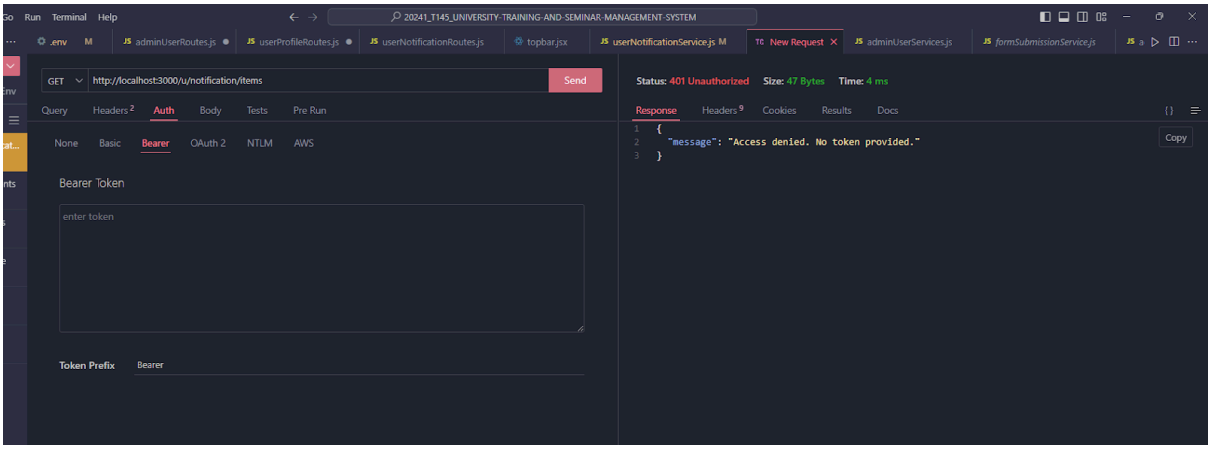


Figure 79. Failure in Retrieving User Notifications with Status Code 401

This figure depicts the failure of a request to retrieve user notifications, resulting in a 401 Unauthorized status code. The failure occurs when the authentication token is either missing, invalid, or cannot be verified, preventing access to the requested resource.

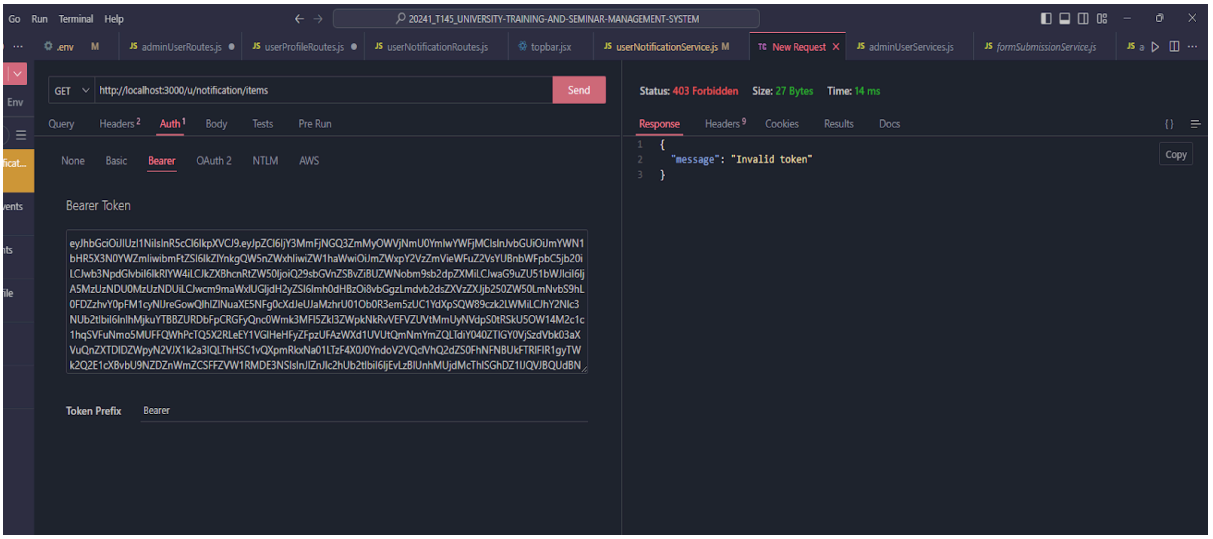


Figure 80. Failure in Retrieving User Notifications with Status Code 403

This figure illustrates the failure of a request to retrieve user notifications, resulting in a 403 Forbidden status code. The failure occurs when the authentication token provided is invalid, meaning the user is either unauthorized or the token cannot be verified, blocking access to the requested resource.

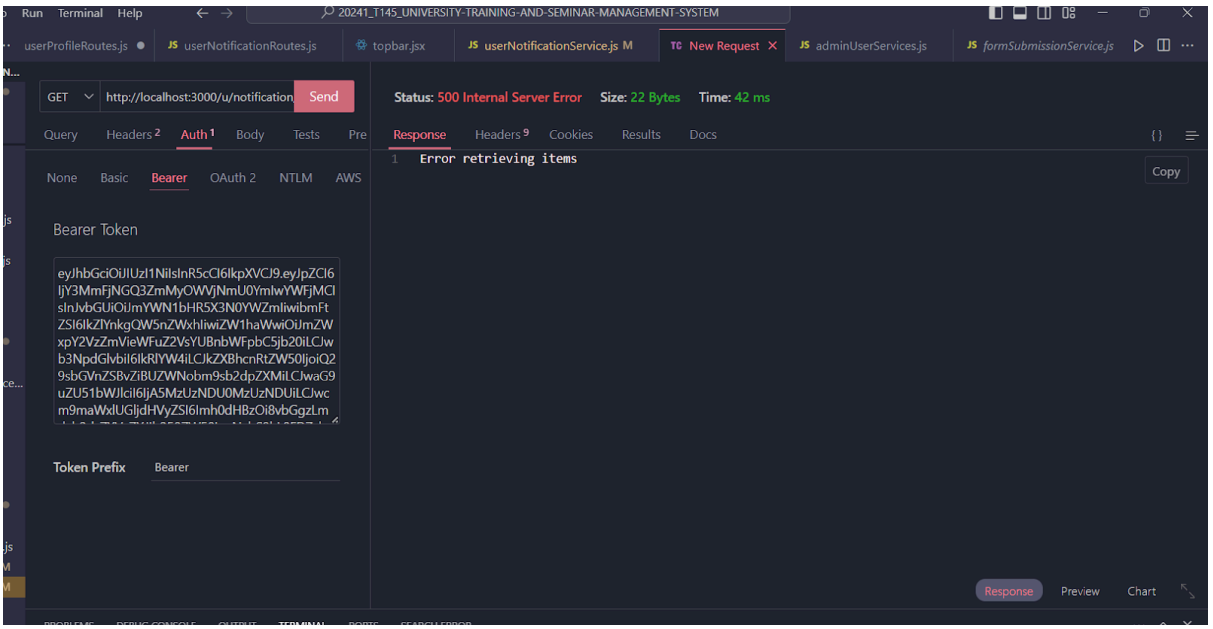


Figure 81. Server Error in Retrieving User Notifications with Status Code 500

This figure depicts a server error that occurred while attempting to retrieve user notifications, resulting in a 500 Internal Server Error status code. The error indicates a problem on the server side, preventing the successful processing of the request.

1.1.17 Updating Notification Read Status

Version: 1.0

Date: December 3,2024

Description: This API endpoint will update the read status of a notification item. If the user reads the notification, its status will be updated.

Endpoint: http://localhost:3000/u/notification/update/:id

Method: PATCH

Configurations: This endpoint might require user authentication. Ensure that users are authenticated before accessing the updates

Parameters:

- **authorizationToken:** The session token (JWT) used for user authentication, passed in the Authorization header.
- **notificationiD:** The ID of the notification that is to be updated, passed as a parameter in the URL.

Response codes of this API endpoint:

Code	Message	Description
200	Notification marked as read	The request was successful, and the notification's status was updated to "read".
401	Access Denied	The request was denied due to missing or invalid authentication credentials (e.g., no token or invalid token).
404	Notification not found	The notification with the specified ID and userId was not found in the system.
500	Error updating notification status	An internal server error occurred while attempting to update the notification status.

Valid and Invalid Request Responses

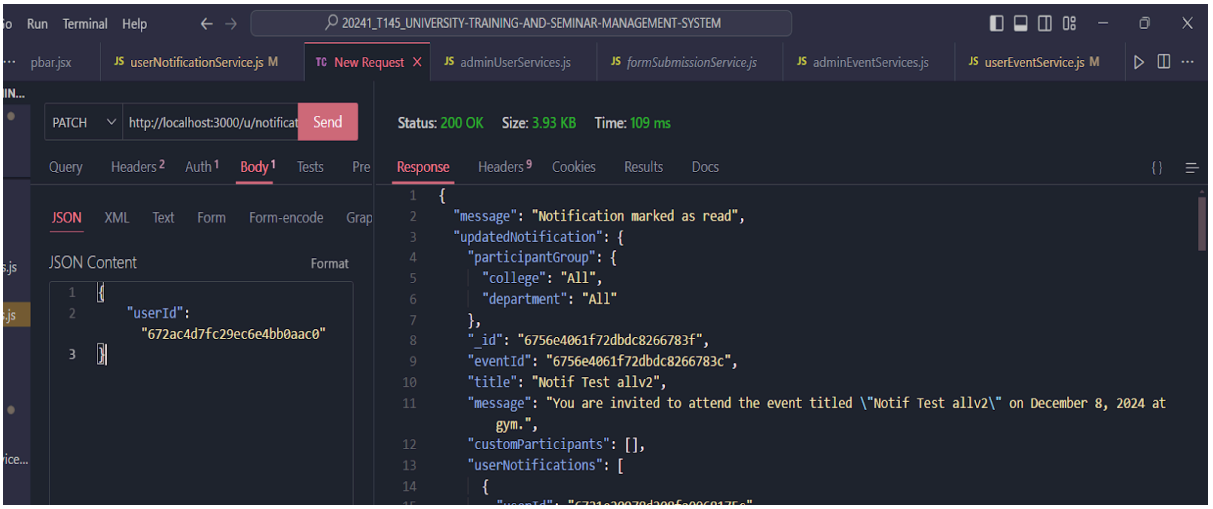


Figure 82. Valid PATCH Request for Updating the Notification Read Status

This figure illustrates a valid PATCH request triggered when a user clicks on a notification. The request updates the notification's status from "unread" to "read," reflecting the user's interaction with the notification.

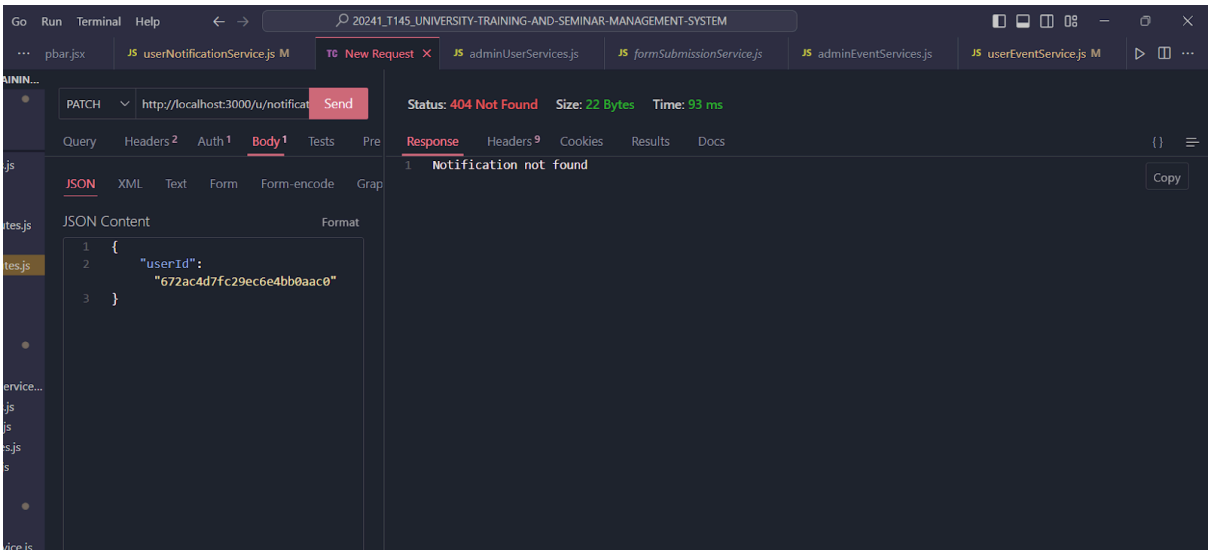


Figure 83. Invalid PATCH Request for Updating Notifications Read Status

This figure illustrates an invalid PATCH request when the system cannot find the specified notification. The 404 Not Found status code is returned, indicating that the requested notification does not exist or is unavailable for updating.

Response Format: JSON

Response:

The API response to update the read status when the user reads the notification item that was sent to them

Different response Screenshots

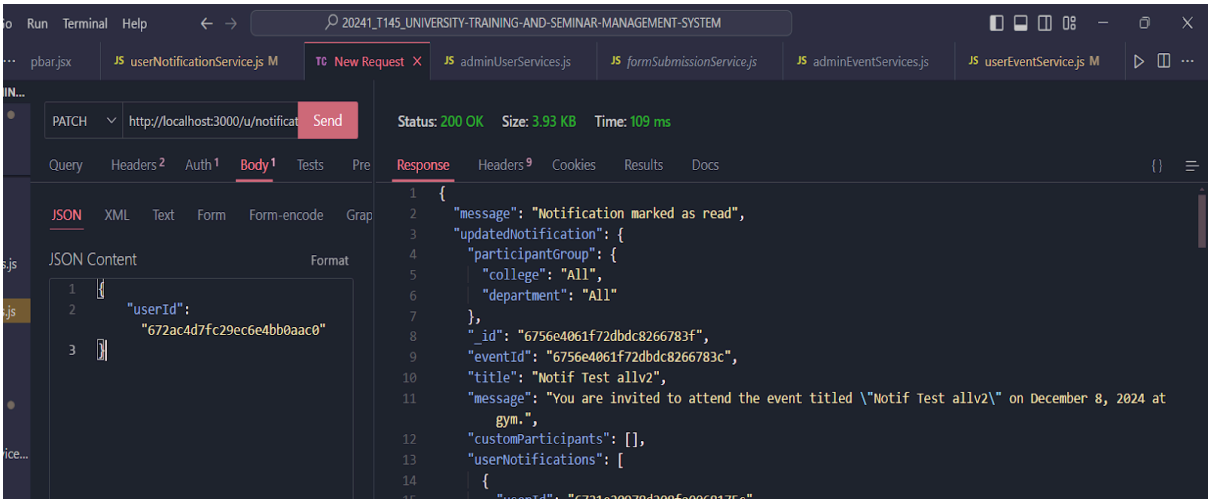


Figure 84. Successful Update of User Notification Read Status, Code 200

This figure illustrates the successful update of a user notification's read status, triggered by a PATCH request. The notification's status is updated from "unread" to "read," and the request is completed with a 200 OK status code, indicating that the operation was successful.

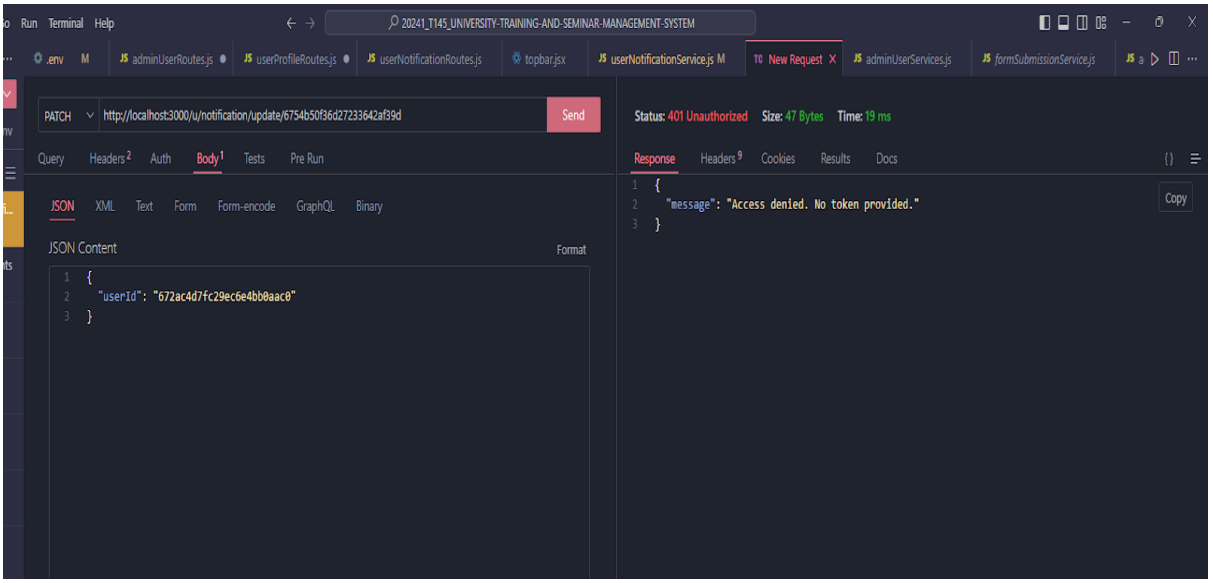


Figure 85. Failure to Update User Notification Read Status, Code 401

This figure illustrates the failure to update a user notification's read status due to an invalid or missing authentication token. A 401 Forbidden status code is returned, indicating that the user is authenticated but does not have permission to perform the update operation, often because the token is invalid or expired.

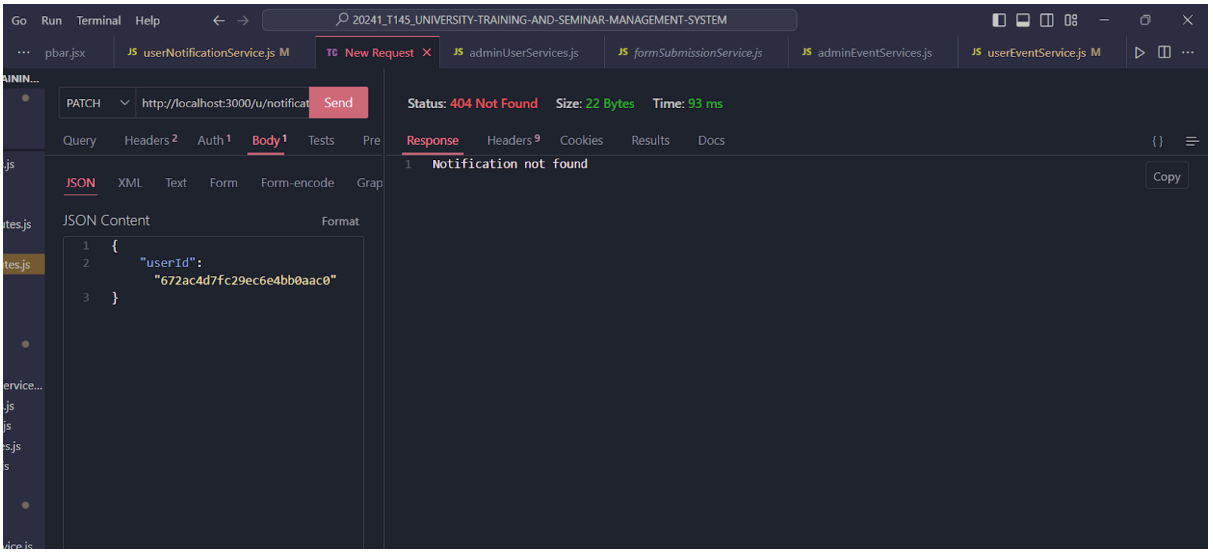


Figure 86. Failure to Update User Notification Read Status, Code 404

This figure illustrates the failure to update a user notification's read status due to the notification not being found. A 404 Not Found status code is returned, indicating that the specified notification could not be located for the update.

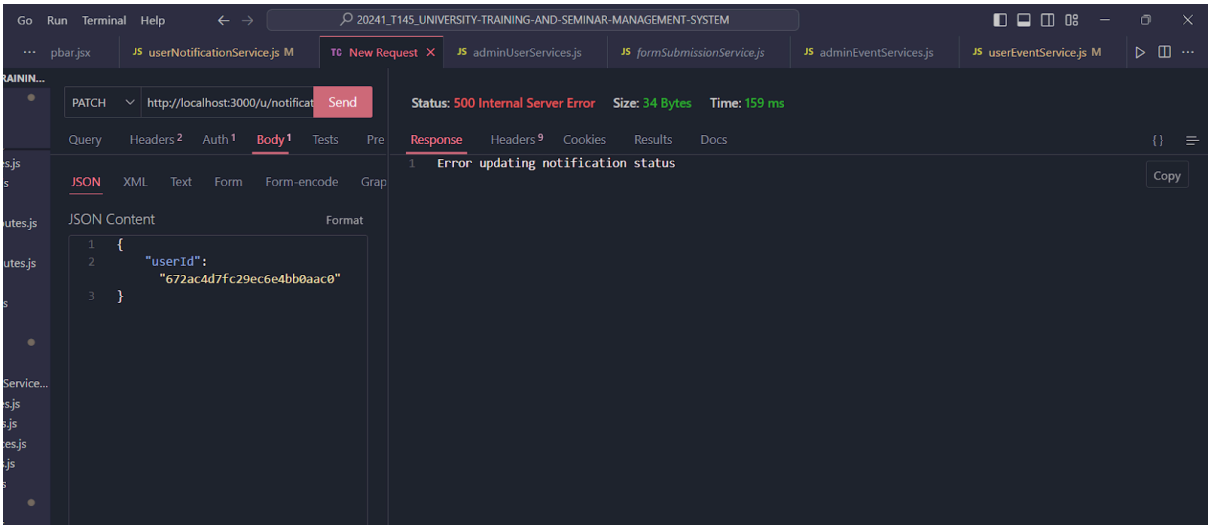


Figure 87. Server Failure to Update User Notification Read Status, Code 500

This figure illustrates a server-side failure when attempting to update a user notification's read status. A 500 Internal Server Error status code is returned, indicating that an issue occurred on the server, preventing the update from being processed successfully.

1.1.18 Updating Notification Status on Removal

Version: 1.0

Date: December 5, 2024

Description: This API endpoint will update the status of a notification to indicate its removal when the user chooses to remove a received notification.

Endpoint: http://localhost:3000/u/notification/remove/:id

Method: PATCH

Configurations: This endpoint might require user authentication. Ensure that users are authenticated before accessing the removal

Parameters:

- **authorizationToken:** The session token (JWT) used for user authentication, passed in the Authorization header.
- **notificationiD:** The ID of the notification that is to be updated, passed as a parameter in the URL.

Response codes of this API endpoint:

Code	Message	Description
200	Notification removed	The request was successful, and the notification's removedStatus was updated to true, marking it as removed.
404	Notification or user not found	The notification or the user with the specified ID could not be found in the system, preventing the update.
500	Error removing notification	An internal server error occurred while attempting to update the notification's removedStatus, preventing the request from being processed.

Valid and Invalid Request Responses

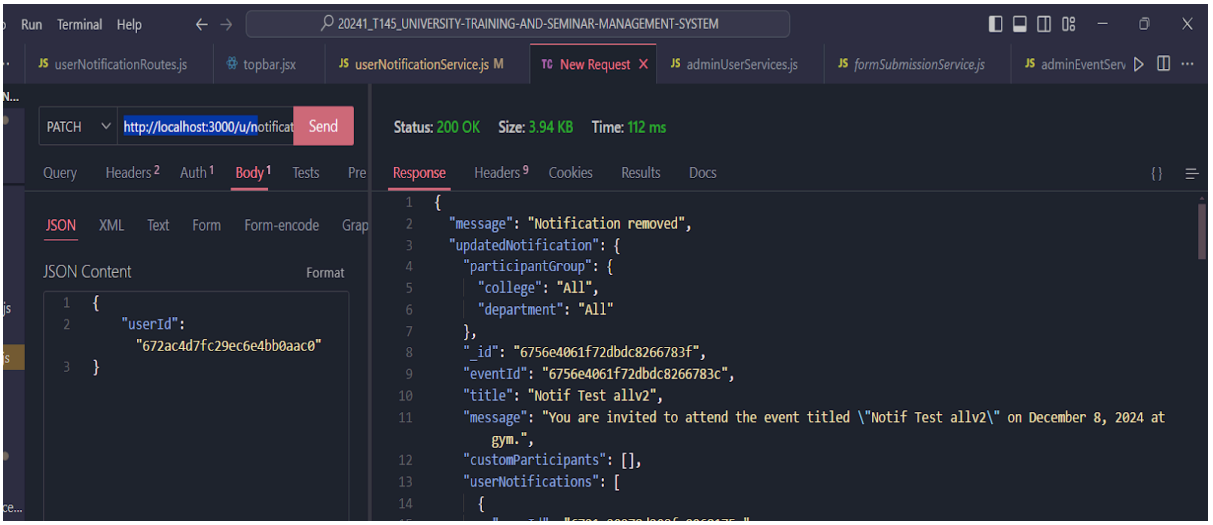


Figure 88. Valid PATCH Request for Updating the Notification Removed Status

This figure illustrates a valid PATCH request for updating the notification's removedStatus to true. The request is triggered when the user removes a specific notification, marking it as removed in the system. The operation is successfully processed, indicating the notification has been properly updated.

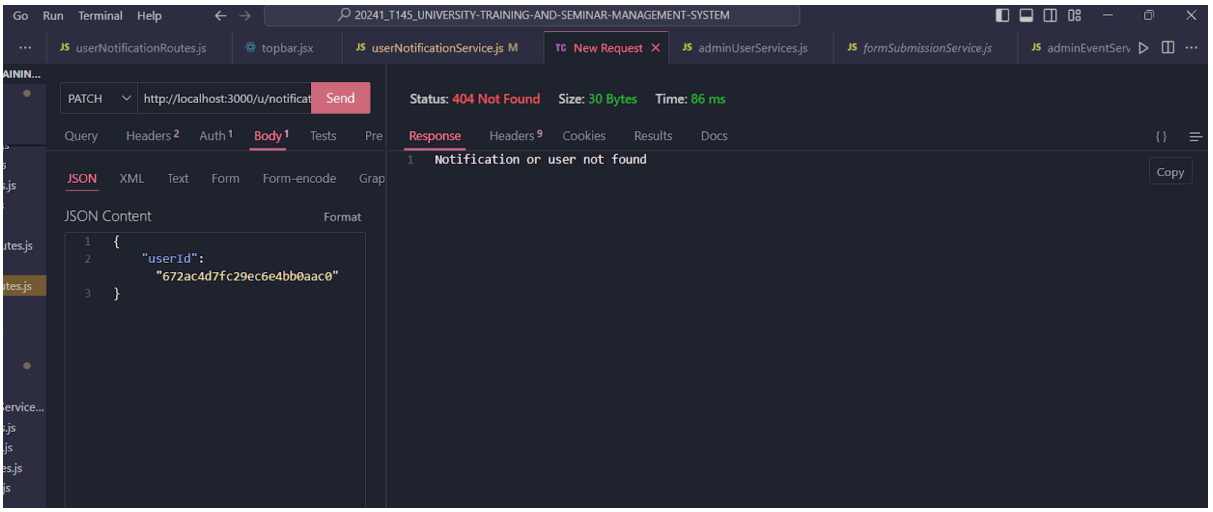


Figure 89. Invalid PATCH Request for Updating Notifications Removed Status

This figure illustrates an invalid PATCH request for updating the notification's removedStatus. The request fails due to an issue, such as the notification not being found, invalid parameters, or lack of proper authorization, preventing the update from being processed.

Response Format: JSON

Response:

The API response to update the removal status parameters in the user notification array depends on the authenticated user's ID.

Different response Screenshots

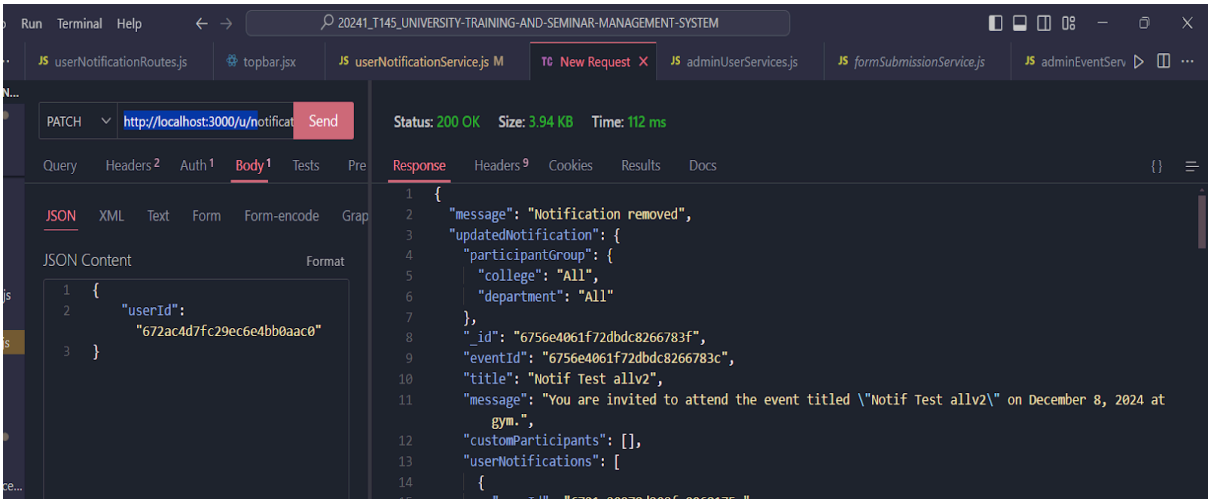


Figure 90. Successful Update of User Notification Removed Status, Code 200

This figure illustrates a successful update of a user notification's removedStatus to true, indicating that the notification has been removed. The 200 OK status code confirms that the update operation was completed successfully.

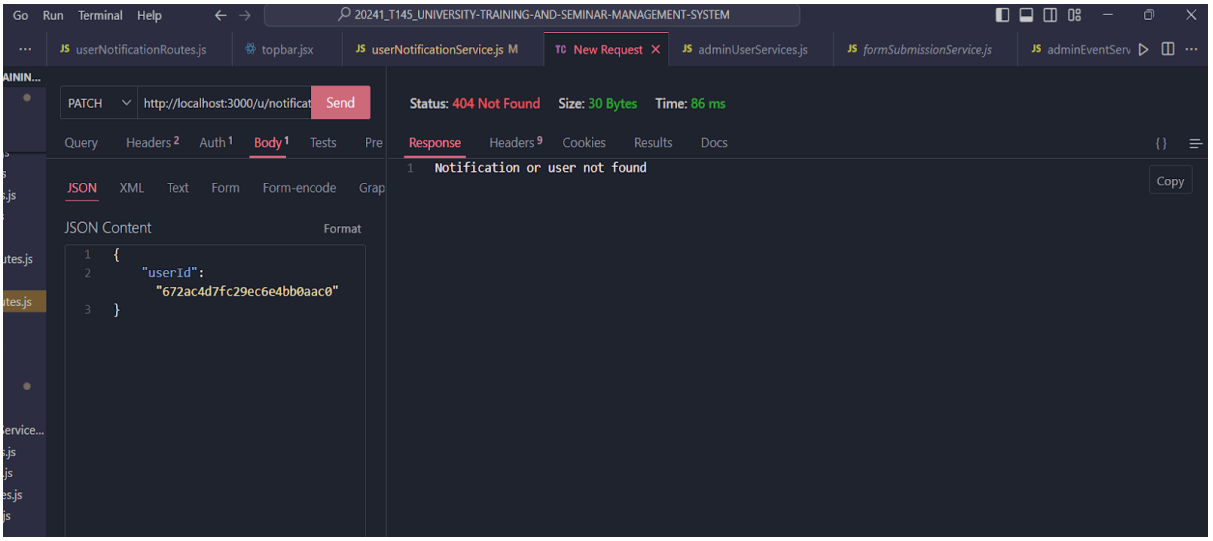


Figure 91. Failure to Update Notification Removed Status, Code 404

This figure illustrates a failure to update the notification's removedStatus due to the notification not being found in the system. A 404 Not Found status code is returned, indicating that the specified notification could not be located for the update.

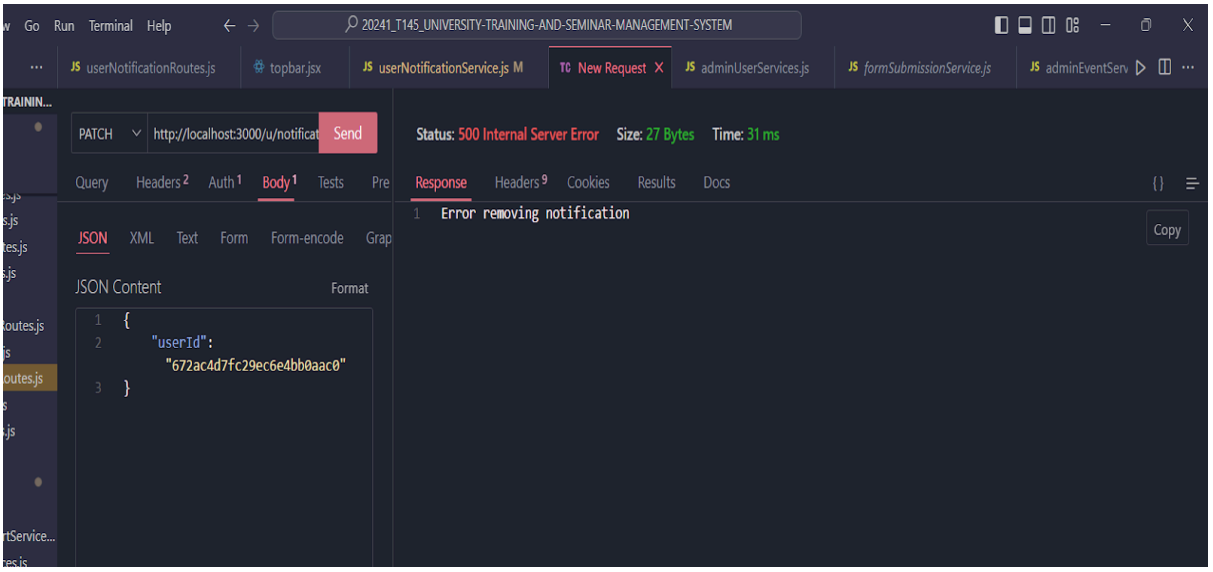


Figure 92. Server Failure to Update Notification Removed Status, Code 500

This figure illustrates a server-side failure when attempting to update the notification's removedStatus. A 500 Internal Server Error status code is returned, indicating that an issue occurred on the server, preventing the update from being processed successfully

1.2 ADMIN MODULE API

1.2.1 Module Description

This section provides a comprehensive exploration of the routes and APIs associated with the administrator role within the University Training and Seminar Management System. It takes to define the functionalities, access points, and operational workflows that allows the administrators to efficiently manage training sessions and seminars.

The API have the following endpoints:

User Management Endpoints

http://localhost:3000/a/users
http://localhost:3000/a/personnel
http://localhost:3000/a/users/:id

Event Administration Endpoints

http://localhost:3000/a/events
http://localhost:3000/a/events/:id
http://localhost:3000/a/events/:id/check-lock
http://localhost:3000/a/events/:id/lock
http://localhost:3000/a/events/:id/unlock

Event Report Endpoints

http://localhost:3000/a/event-history
http://localhost:3000/a/event/registrations
http://localhost:3000/a/event/registrations/:id
http://localhost:3000/a/event/form-submissions
http://localhost:3000/a/event/form-submissions/:id

Admin Page Routes Endpoints

http://localhost:3000/a/dashboard
http://localhost:3000/a/activity-logs

1.2.2 Retrieve All Users

Version: 1.0

Date: November 25, 2024

Description: This api endpoint will retrieve all users from the database.

Endpoint: http://localhost:3000/a/users

Method: GET

Configurations: This endpoint requires admin authentication to ensure only authorized administrators can access the list of users**.**

Parameters:

- **authToken:** The session token (JWT) used for admin authentication, passed in the Authorization header. in order for the list of users to show

Response codes of this API endpoint:

Code	Message	Description
200	OK	The request was successful, and the list of users (excluding passwords) was retrieved and returned in the response.
403	Invalid Token	The request lacks valid authentication credentials (e.g., the token is invalid or expired).
500	Error retrieving users	An internal server error occurred while attempting to fetch the user data, preventing the request from being processed.

Valid and Invalid Request Responses

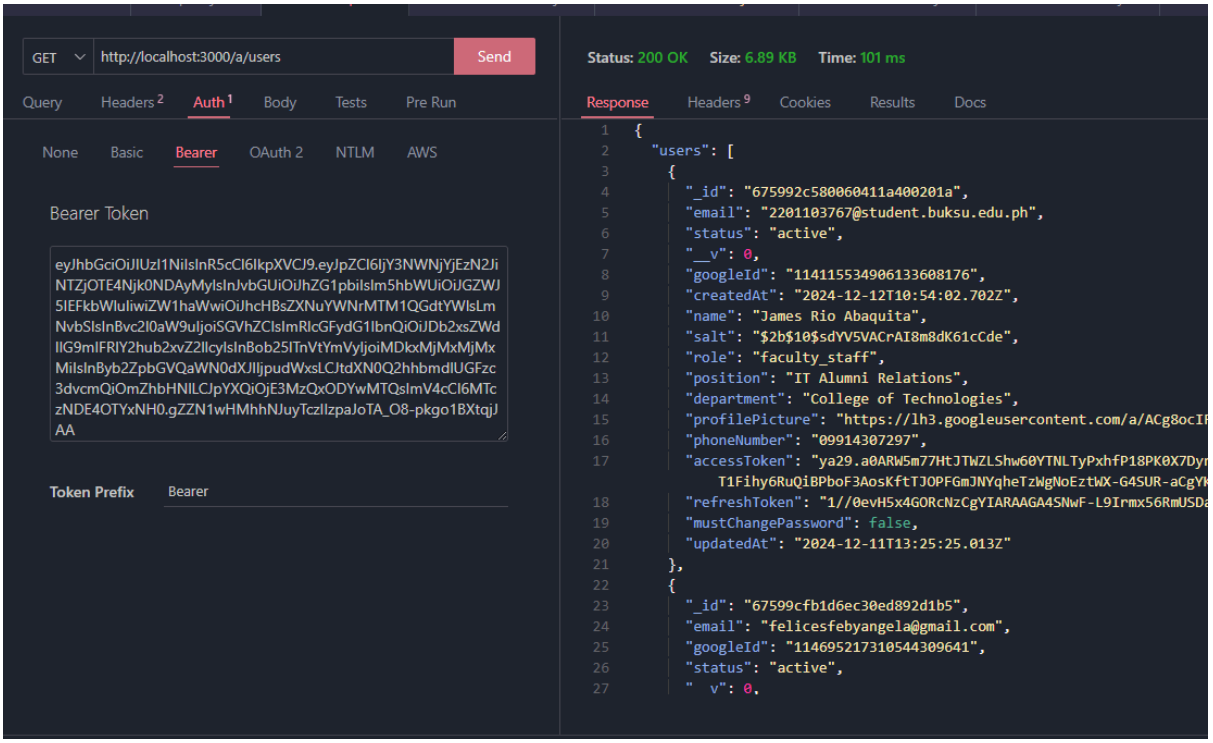


Figure 93. A Valid GET Request for Retrieving all users from the database

This figure shows a valid GET request for retrieving all users stored in the database. It will fetch all data except for the password, as it is private.

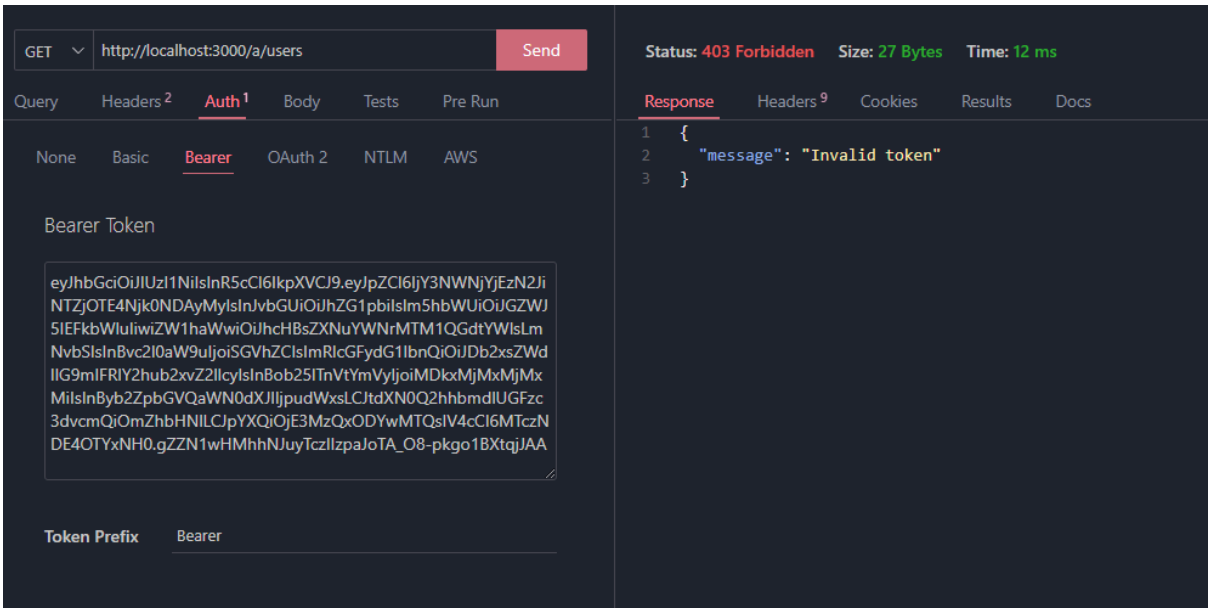


Figure 94. An Invalid GET Request for Retrieving all users from the database

This figure illustrates an invalid GET request for retrieving all users from the database. The request required admin authorization, but the admin token provided was either invalid or expired.

Response Format: JSON

Response:

The API response is the list of all users from the database.

Different response Screenshots

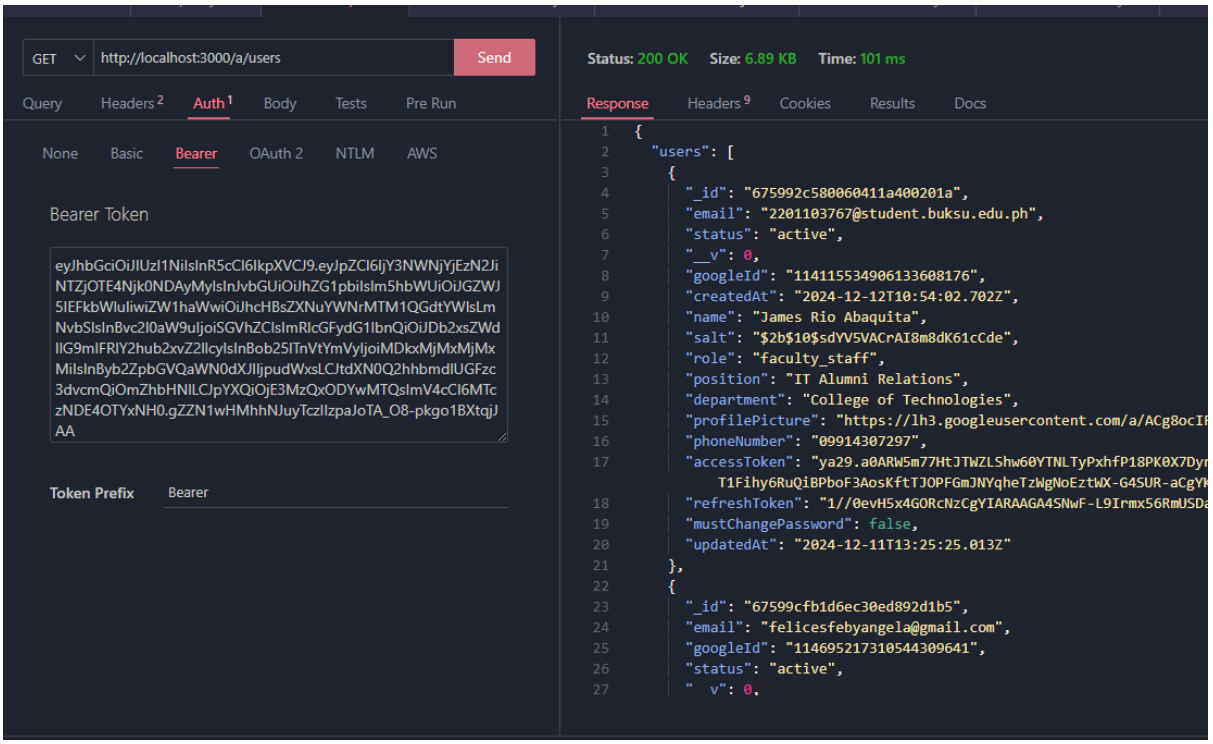


Figure 95. Successful retrieval of all users from the database with a status code 200

This figure shows the successful retrieval of all users from the database, accompanied by a status code 200 response

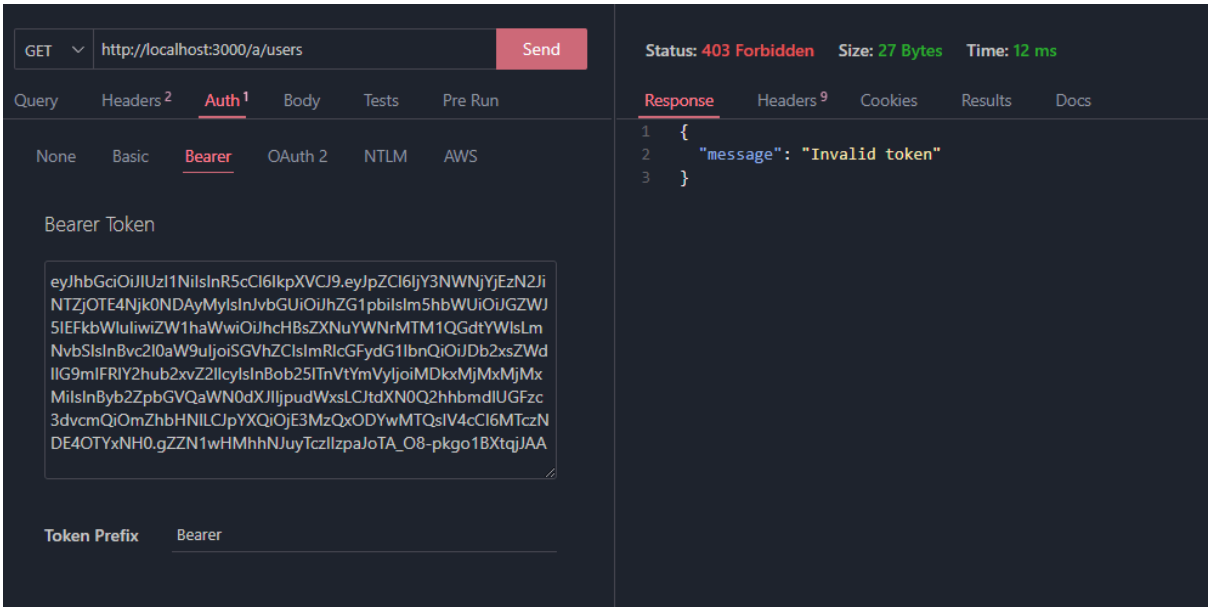


Figure 96. Failure retrieval of all users from the database with a status code 403

This figure shows the failed retrieval of all users from the database, accompanied by a status code 403 response, due to an invalid or expired admin token.

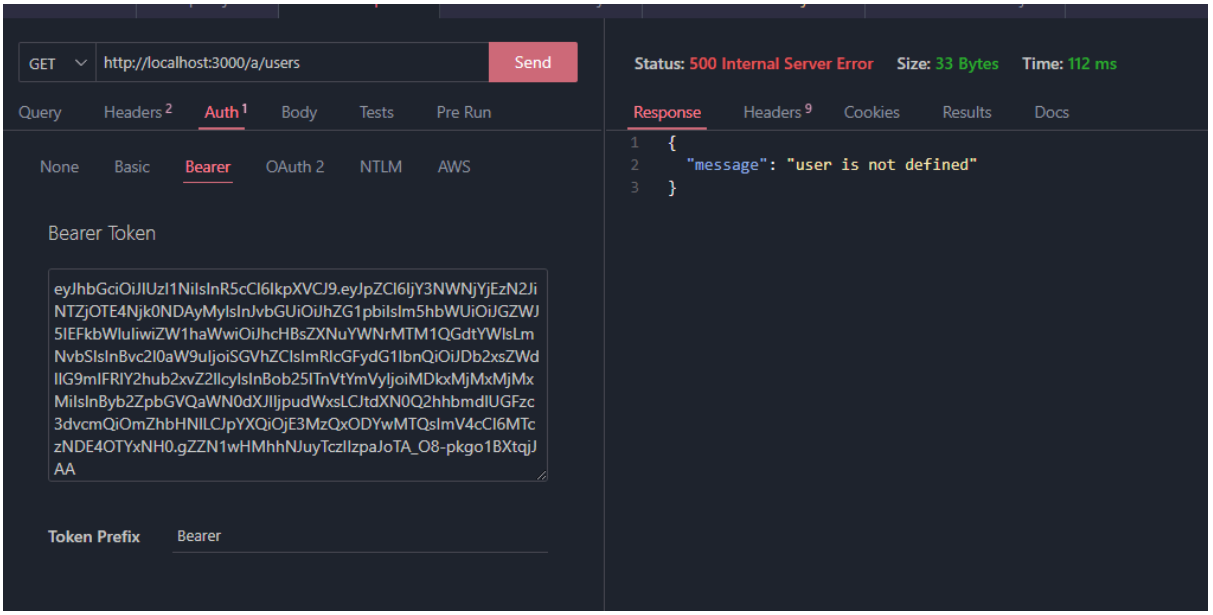


Figure 97. Server Error retrieval of all users from the database with a status code 500

This figure shows the server error when retrieving all users from the database, accompanied by a status code 500 response, due to an issue occurring on the server, causing an internal server error

1.2.3 Add User Account

Version: 1.0

Date: November 25, 2024

Description: This api endpoint will add a user and information into the database.

Endpoint: http://localhost:3000/a/personnel

Method: POST

Configurations: The API request should be validated using a token.

Parameters:

- **authToken:** (Header) A valid session token (JWT) used to authenticate and authorize the request.
- **Body:** (JSON Object) Contains the following fields:
  - **name: (Required, String)** The desired username for the new user.
  - **email: (Required, String)** The email address of the new user.
  - **role: (Required, String)** The role or permission level for the user (e.g., "admin", "user").
  - **department: (Optional, String)** The role or permission level for the user (e.g., "admin", "user").
  - **position: (Optional, String):** Additional information related to the user, such as position
  - **phone number (Optional, String):** Additional information related to the user, such as phone no.

Response codes of this API endpoint:

Code	Message	Description
201	Personnel account created	The request was successful, and the personnel account was created successfully. The response includes the user's details.
400	Failed to Validate	The request failed validation due to an error with the provided data, such as missing or invalid fields.
400	User validation failed: name: Path `name` is required., email: Path `email` is required., role: Path `role` is required."	This indicates missing required fields such as name, email, or role when submitting the request.
400	Missing Token	The request failed because the required authentication token was missing.

400	The User Already Exists	The request failed because a user with the specified email already exists in the system.
403	Invalid Token	The request was denied due to an invalid or expired authentication token.

Valid and Invalid Request Responses

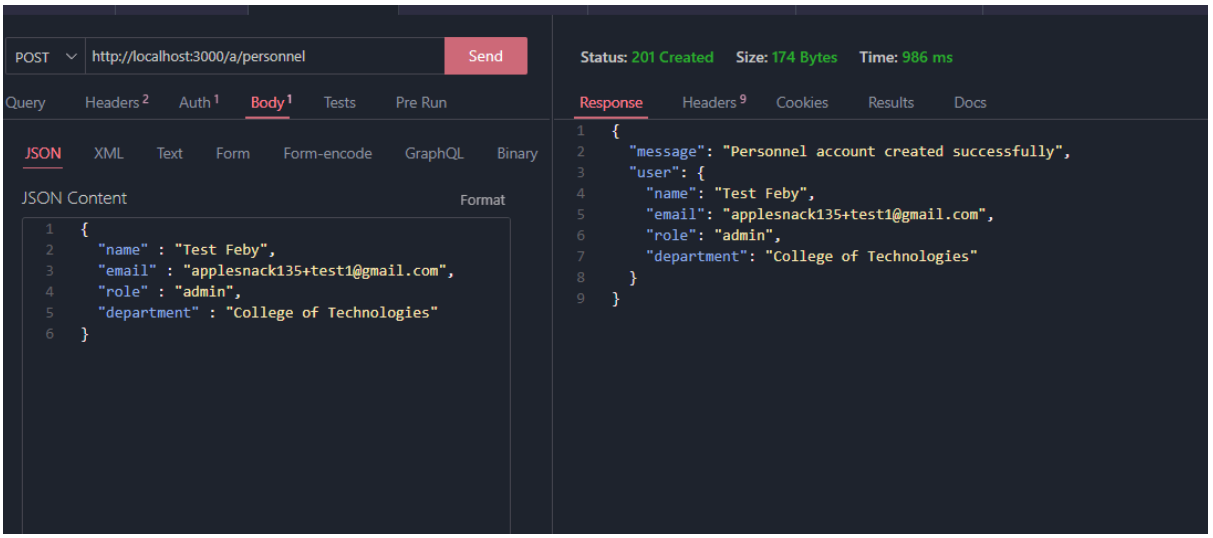


Figure 98. Valid POST Request for Adding a New User Account

This figure shows a valid request for creating a new account with admin privileges, meaning only the 'admin' role can create a user. The request body must contain the name, email, and role, while the department, position and phone number are optional. The password is not included, as the system will automatically generate a temporary hashed password. When users log in with the temporary password, they will be prompted to change it.

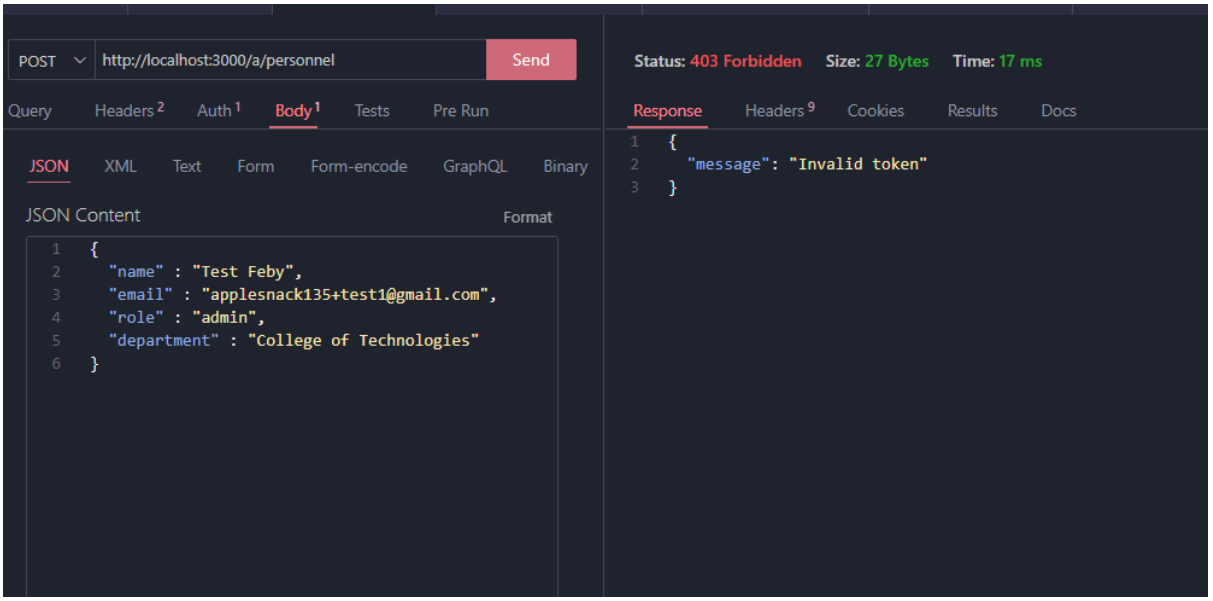


Figure 99. Invalid POST Request for Adding a New User Account

This figure shows an invalid request for creating a new account, as the admin token is either invalid or expired

Response Format: JSON

Response:

The API response will add the details of the currently added user to the database

Different response Screenshots

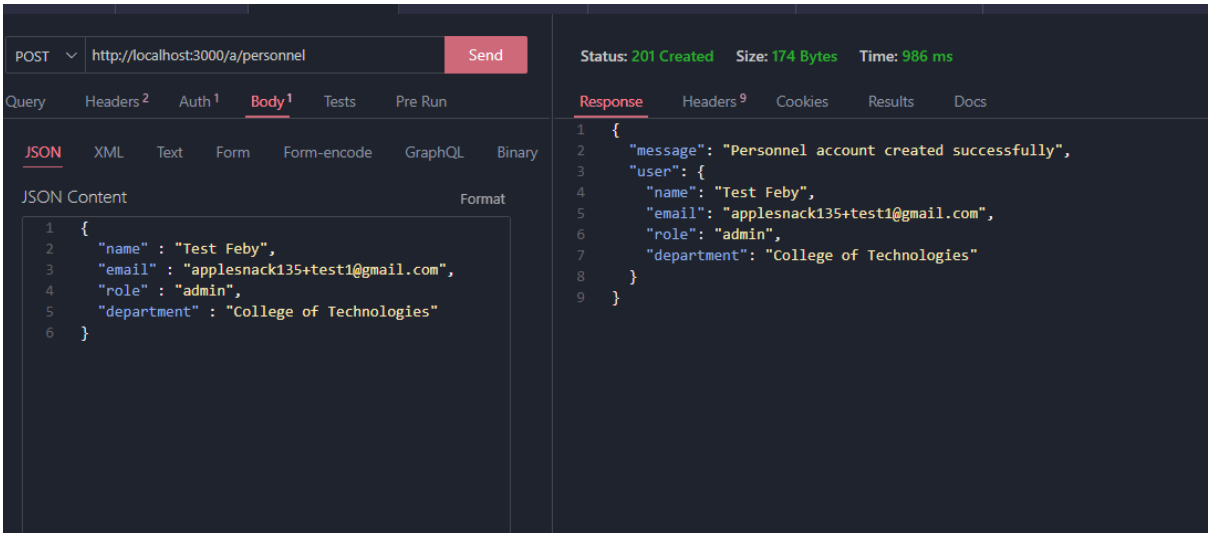


Figure 100. Successfully Created a New User Account with Status Code 201

This figure shows the successful creation of a new user account with a response code 201.

The account has been saved in the database and is awaiting the user's login to change the temporary password that was generated

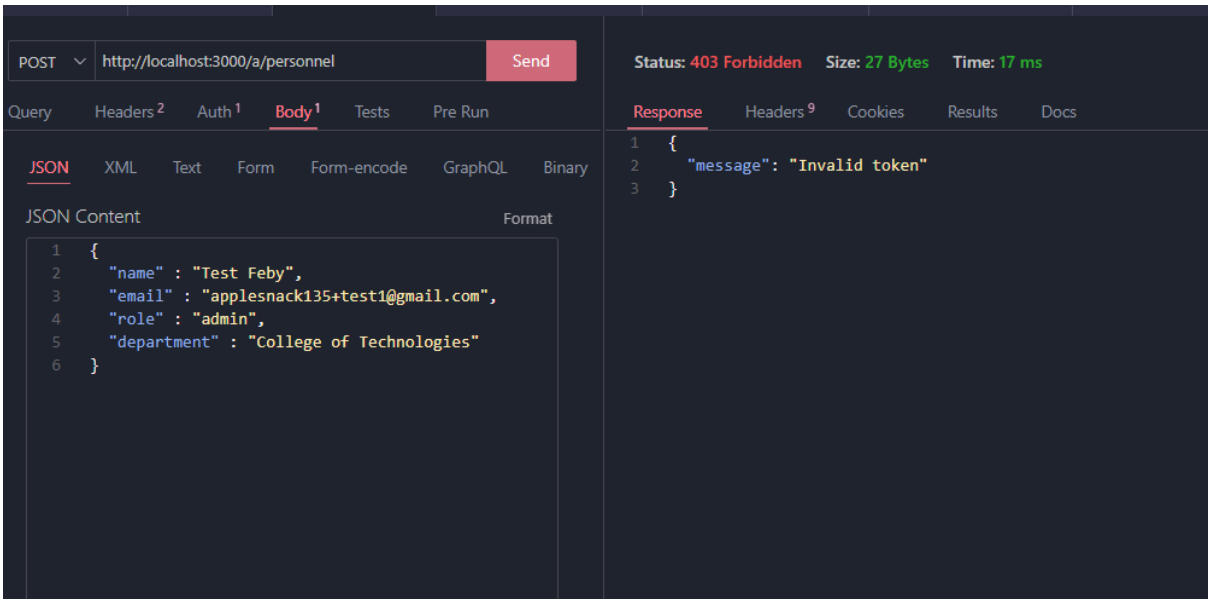


Figure 101. Failed to Create a New User Account with Status Code 403

This figure shows the failure to create a new user account with a response code 403. This is due to the admin token being either invalid or expired

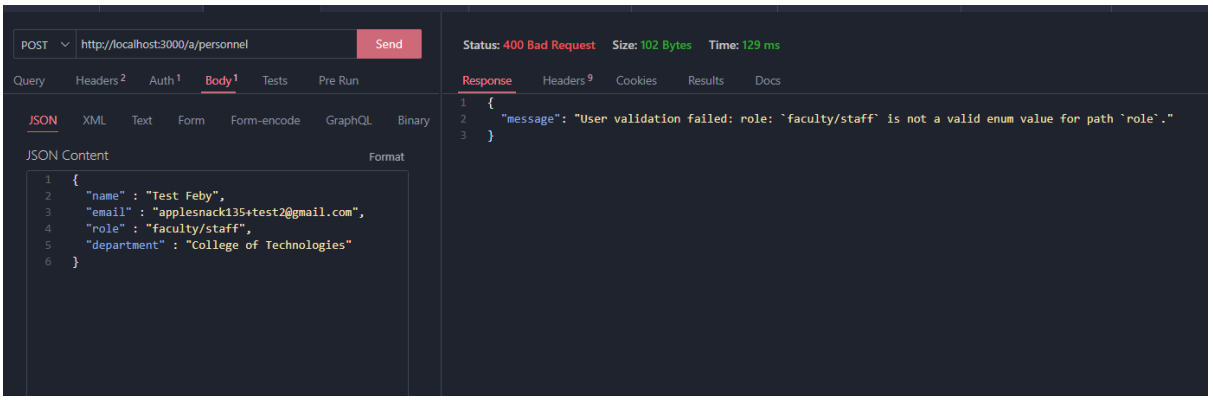


Figure 102. Failed to Validate with Status Code 400

This figure shows a validation failure with a response code 400. This is due to incorrect field data that is not included in the schema(eg. in role: it must be either 'faculty_staff' or 'departmental_admin)

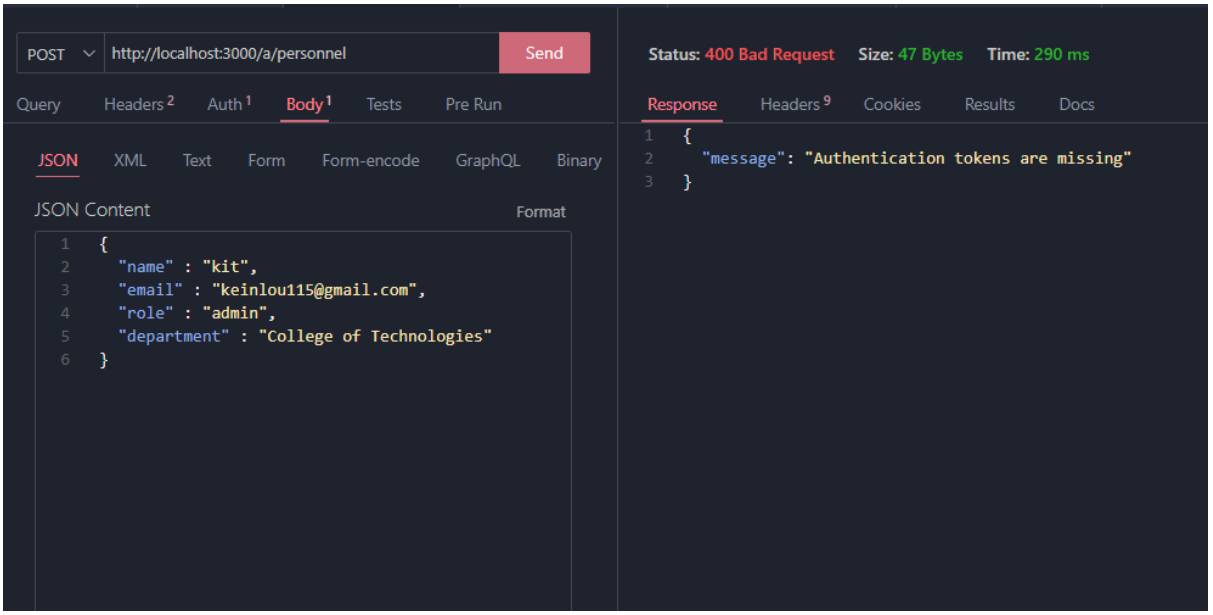


Figure 103. Missing Token with Status Code 400

This figure shows a scenario where the admin token is missing from the request, which results in a failure to authenticate the user. As a consequence, the system returns a response code 400, indicating that the request is malformed or lacks the necessary authorization for processing

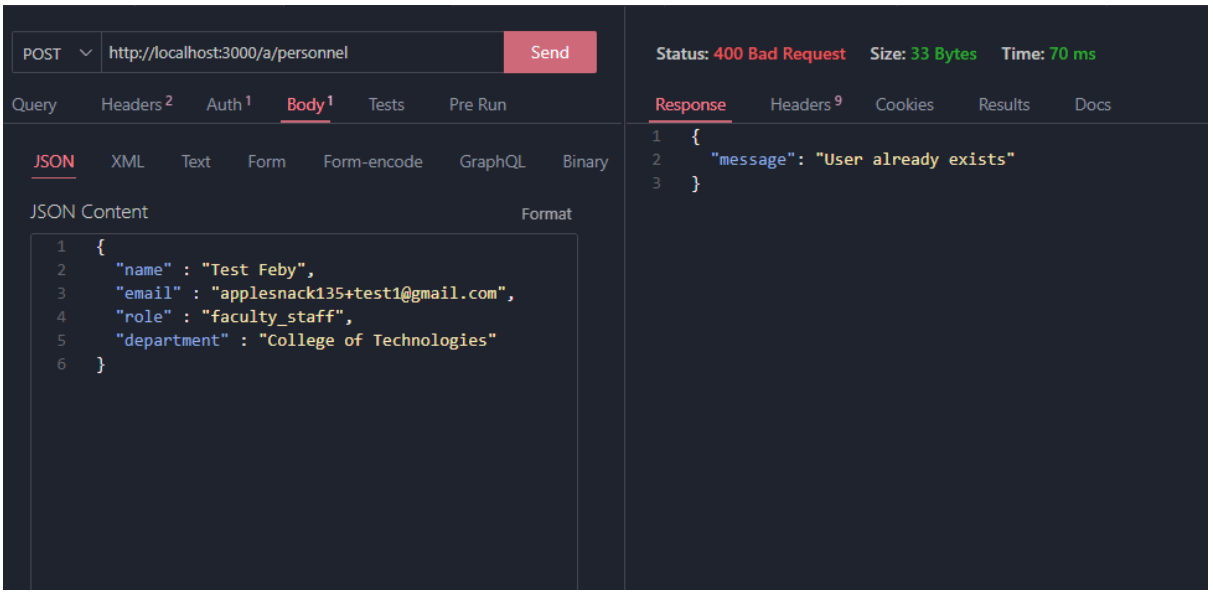


Figure 104. The User Already Exists with Status Code 400

This figure shows a scenario where the user email already exists in the database, resulting in a response code of 400. As a result, the system rejects the POST request, preventing the creation of a duplicate user account.

1.2.4 Modify User Account

Version: 1.0

Date: November 25, 2024

Description: This API endpoint will update the user's data in the database, including their role, position, and department. Only an admin will be able to make these changes, which means that departmental admins do not have the authorization to edit this information directly.

Endpoint: http://localhost:3000/a/users/:id

Method: PUT

Configurations: The API request must include a valid authentication token to verify admin-level access.

Parameters:

- **userId**: The unique identifier of the user whose data is being modified.
- **authToken:** A valid session token (JWT) required for authentication and authorization.
- **Request Body (optional):** Contains the specific fields to update (e.g., {"email", “email", “role”, “department” and “position”}).

Response codes of this API endpoint:

Code	Message	Description
200	User details updated successfully	The request was successful, and the user details were updated in the database. The updated user is returned in the response.
404	User not found	The user with the specified ID was not found in the database, resulting in a failed update attempt.
500	Error updating user details	An internal server error occurred while attempting to update the user's details.

Valid and Invalid Request Responses

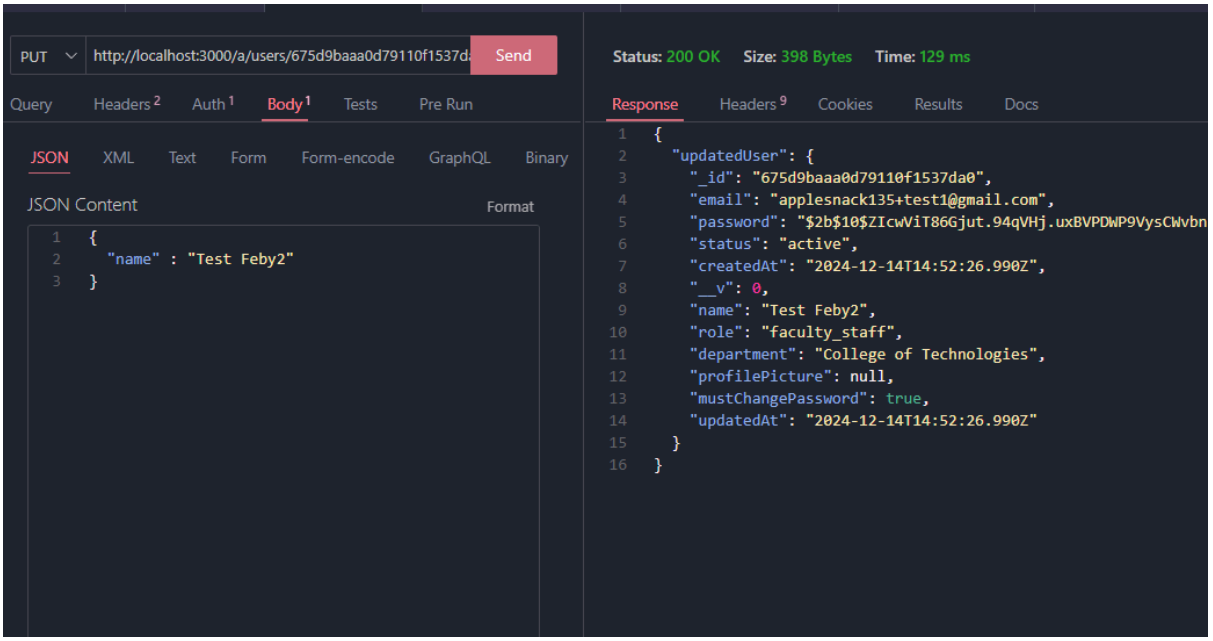


Figure 105. Valid PATCH Request to Update the User

This figure shows a valid request for updating user details with admin privileges, meaning only the 'admin' role can edit a user. The request body includes only the following options: name, email, role, department, and position.

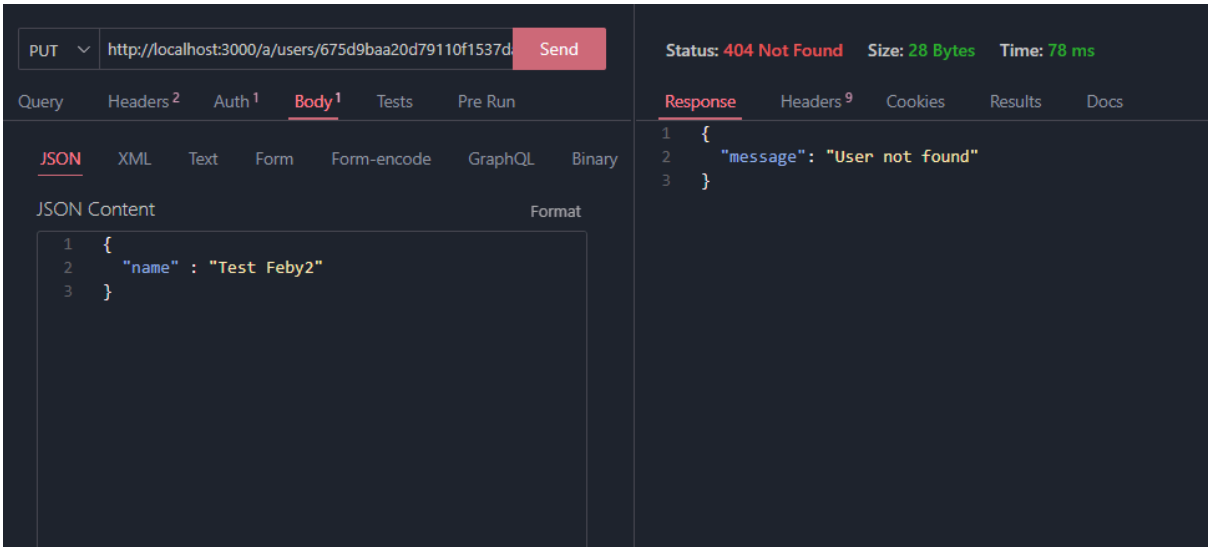


Figure 106. Invalid PATCH Request to Update the User

This figure shows an invalid PATCH request for updating user account details, as this scenario occurs when attempting to update a user that does not exist in the database.

Response Format: JSON

Response:

The API response updated the currently edited user and saved it in the database.

Different response Screenshots

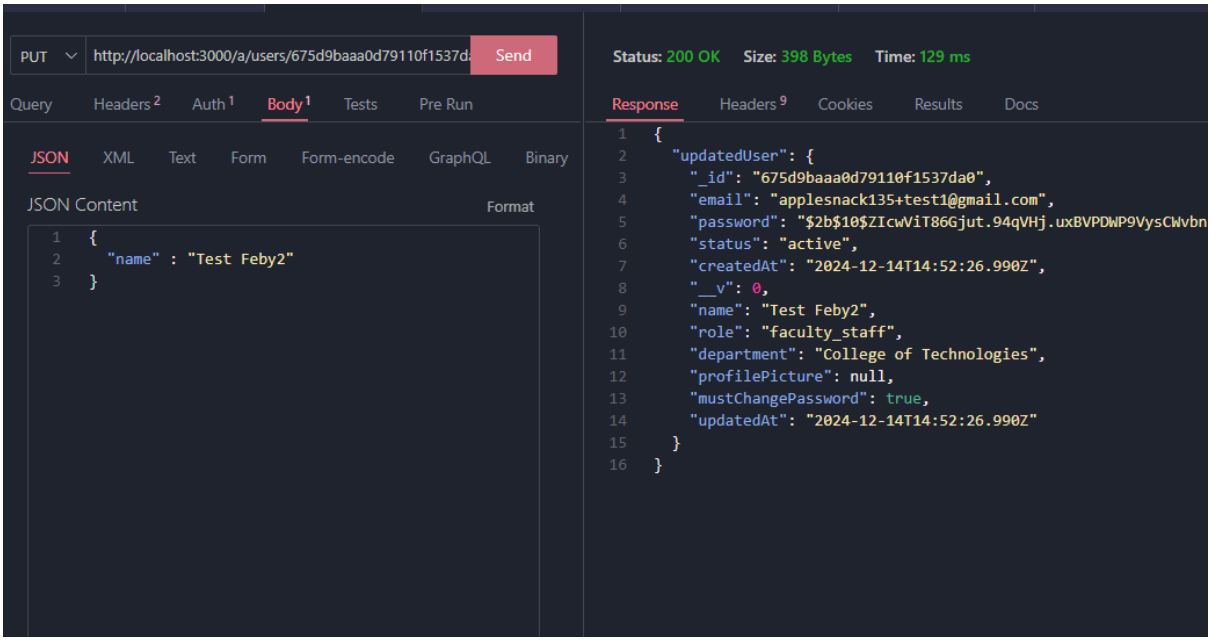


Figure 107. Successfully updating a user account with status code 200

This figure illustrates the successful update of a user account, accompanied by a response code of 200. The updated user details have been successfully stored in the database.

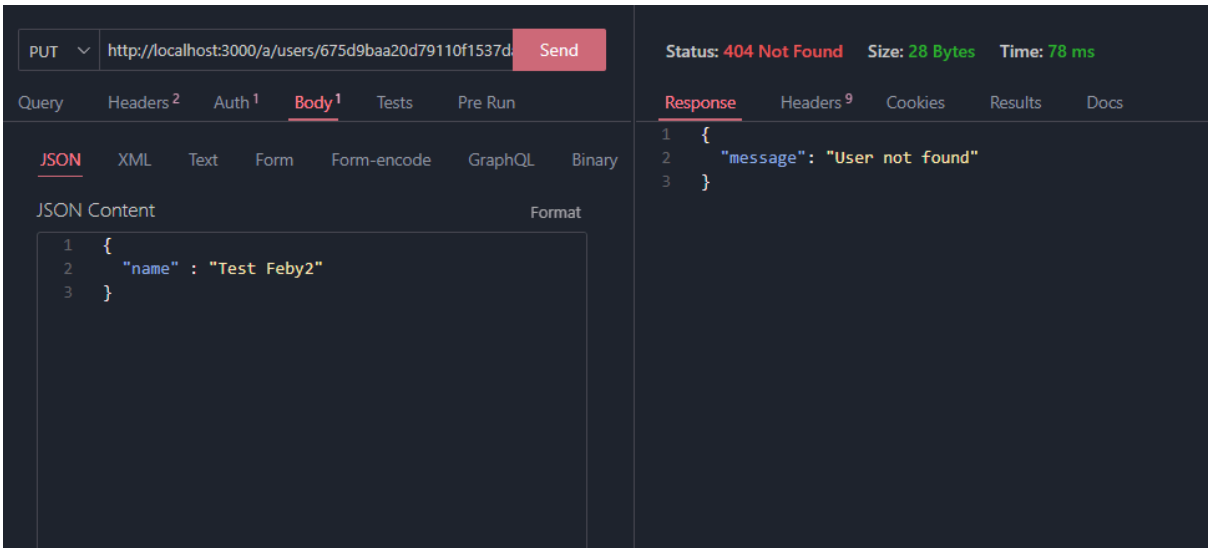


Figure 108. Failed to Update a User Account with Status Code 404

This figure illustrates the failure to update a user account, accompanied by a response code of 404. This occurs when the user is not found in the database based on the user ID, resulting in an error.

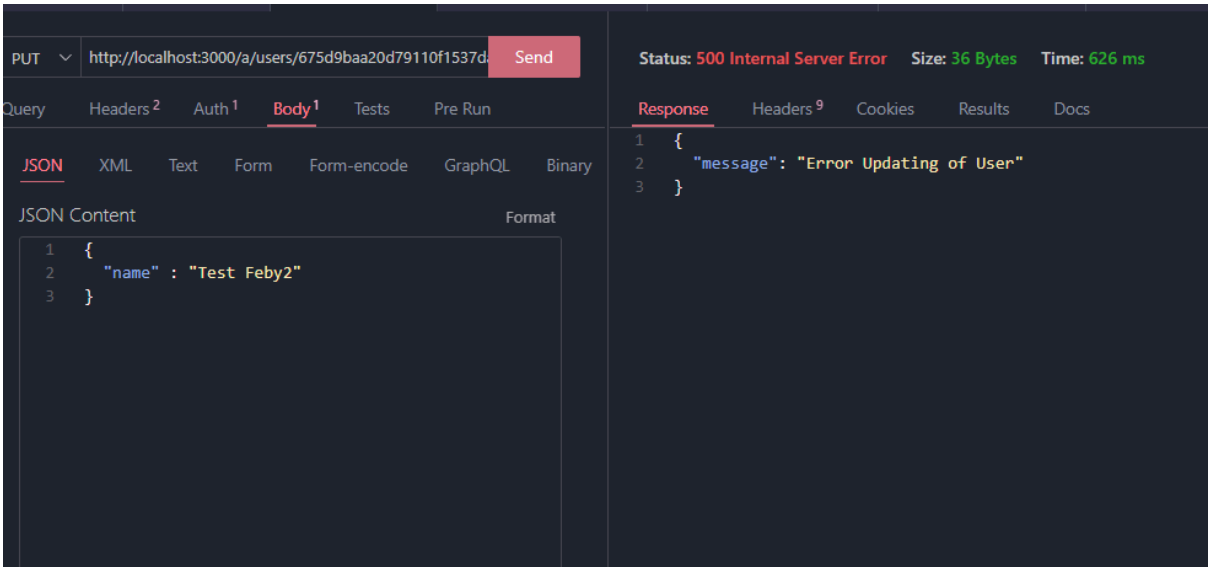


Figure 109. Server Error to Update the User with status code 500

This figure shows the server error that occurs when updating user details in the database, accompanied by a status code 500 response. This error is caused by an issue on the server, such as incorrect field data, resulting in an internal server error.

1.2.5 Add Event

Version: 1.0

Date: November 25, 2024

Description: This API endpoint will add a newly created event by an admin or departmental admin, and it will be saved in the database

Endpoint: http://localhost:3000/a/events

Method: POST

Configurations: The API request should be validated using a token.

Parameters:

- **authToken:** A valid session token (JWT) required for authentication and authorization. The token is used to identify the user and to set the createdBy, editedBy, and lockedBy fields.
- **title: (Required, String):** The title of the event. This field is required.
- **eventDate: (Required, Date)** The date of the event. This field is required.
- **startTime: (Required, Date)** The start time of the event. This field is required.
- **endTime: (Required, Date)** The end time of the event. This field is required.
- **location: (Required, String)** The location of the event. This field is required
- .**eventPicture: (String) (Optional)** A URL or file path to the event's image or poster.
- **description: (String) (Optional)** A description of the event.
- **reminders: (String) (Optional)** Details of any reminders to be sent before the event.
- **formLink: (String) (Optional)** A link to the form for event registration or other participant-related forms.
- **formId: (String) (Optional)** The ID of the associated form for the event.
- **certificateTemplate: (String) (Optional)** A reference to the certificate template used for issuing certificates.
- **participantGroup: (String) (Optional)** Defines the group of participants (e.g., college of technology, college of business and ect.).
- **customParticipants: (Array) (Optional)** A custom list of participants if the group is not predefined.

Response codes of this API endpoint:

Code	Message	Description
200	Successful Creation of an Event	This indicates that the event was successfully created, including the processing of participants and notifications.
400	Failure of Creation of an Event Validation failed due to an invalid data field in the event	This indicates that the event creation failed due to missing required fields, invalid data, or other client-side errors.
400	Validation failed due to an invalid data field in the event	This indicates that the event creation failed due to invalid data in the event fields, such as missing or incorrect values.
500	Server Error in Adding an Event	This indicates a server-side error occurred while trying to create the event, possibly due to issues with database access or external services.

Valid and Invalid Request Responses

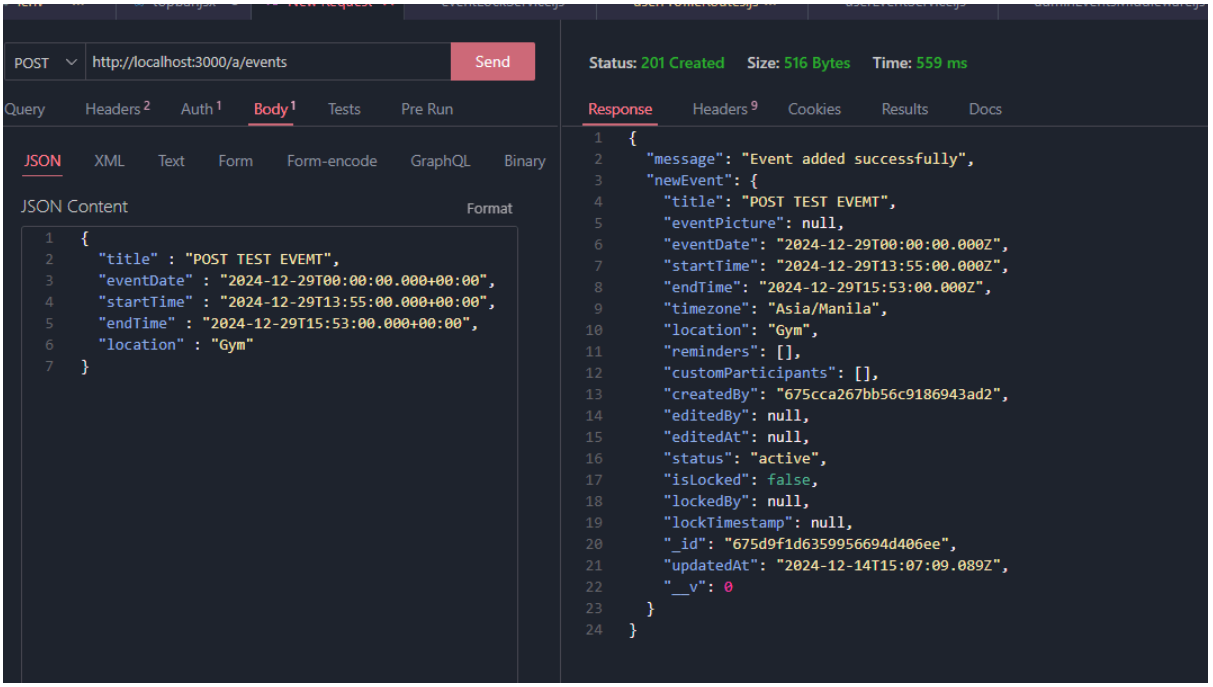


Figure 110. Valid POST Request for Adding a New Event

This figure shows a valid request for creating a new event. The request body must contain the title, event date, start time, end time, and location, while the other fields are optional. It will also include an additional field, 'created by,' which indicates the admin or departmental admin who created the event.

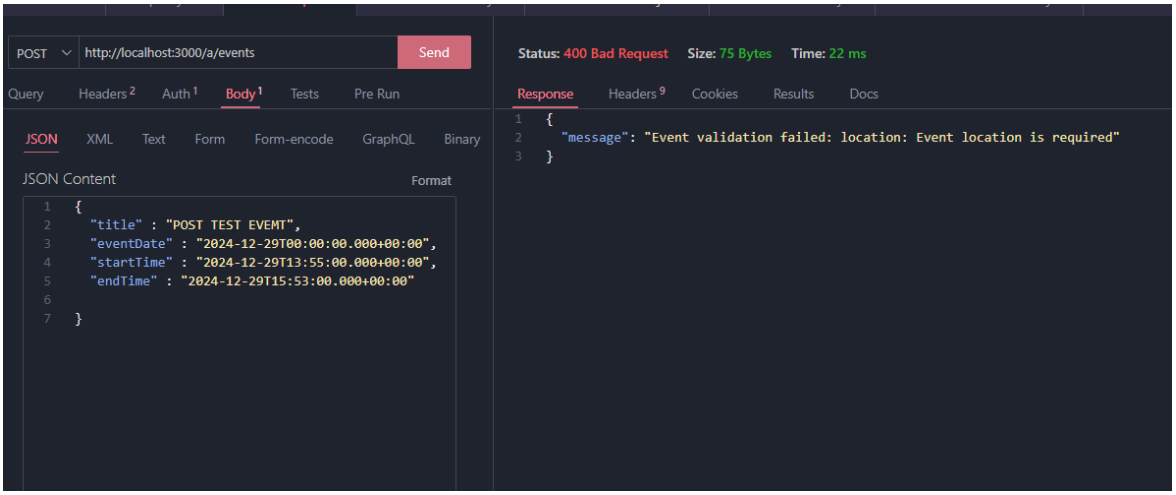


Figure 111. Invalid POST Request for Adding a New Event

This figure shows an invalid request for creating a new event due to incomplete required data fields. The request body must contain the title, event date, start time, end time, and location to create an event, while other fields are optional.

Response Format: JSON

Response:
The API response will add a newly created event to the database.

Different response Screenshots

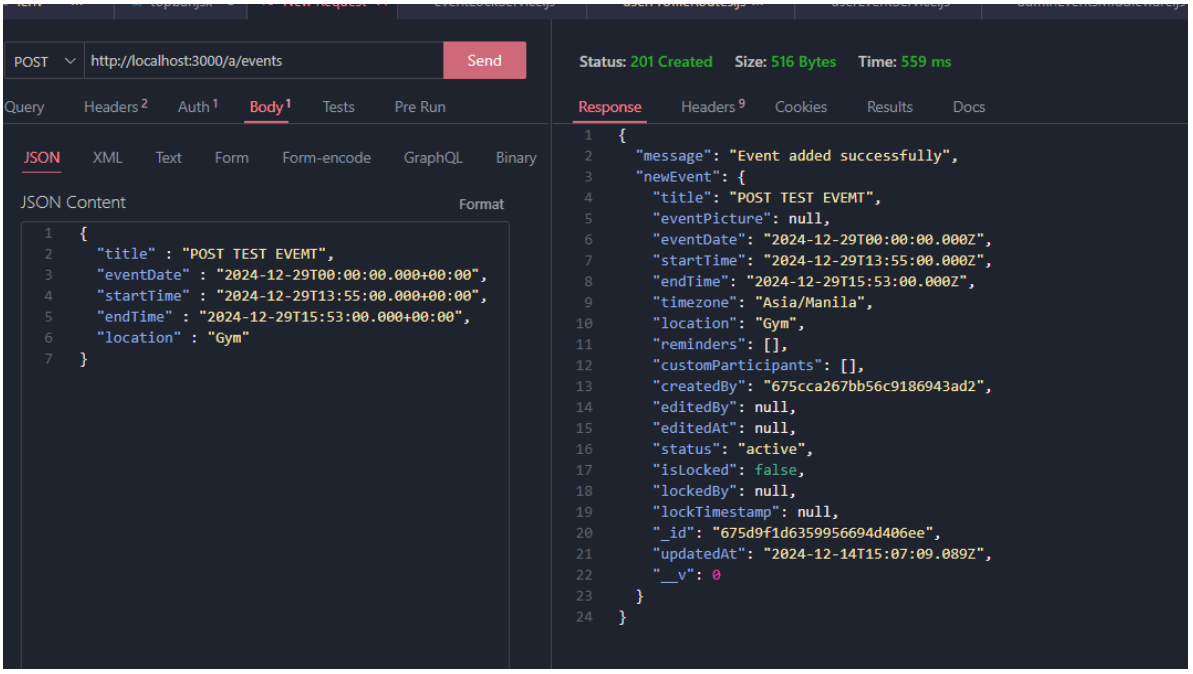


Figure 112. Successful Creation of an Event with status code 201

This figure illustrates the successful creation of an event, accompanied by a response code of 201. The required request body was provided, and as a result, the event has been successfully stored in the database

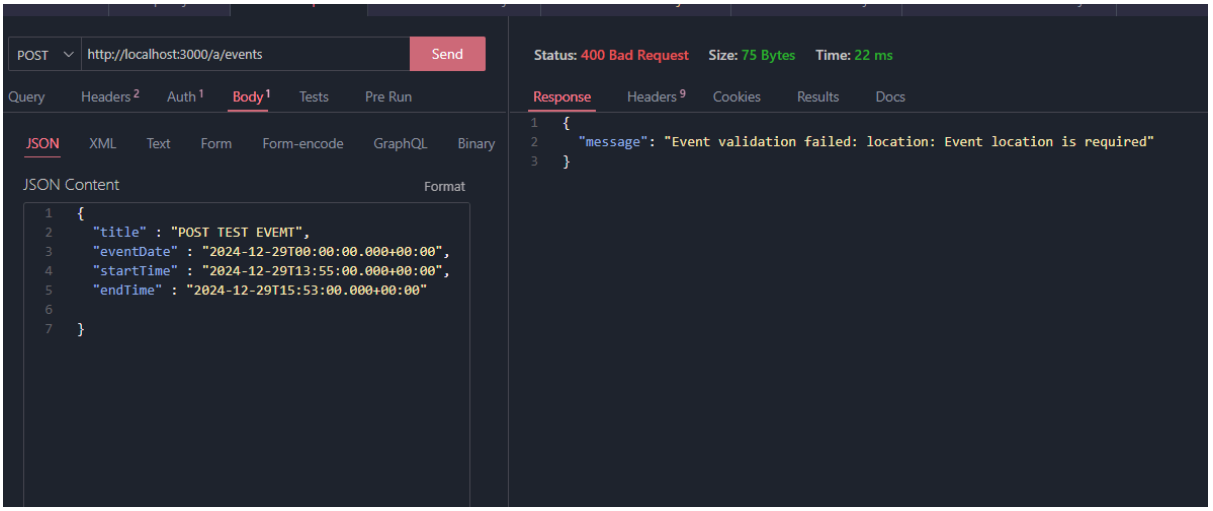


Figure 113. Failure of Creation of an Event with status code 400

This figure illustrates the failed creation of an event, accompanied by a response code of 400. Since the request body is missing a required field, it must contain all the necessary fields as specified in the schema, including the title, event date, start time, end time, and location.

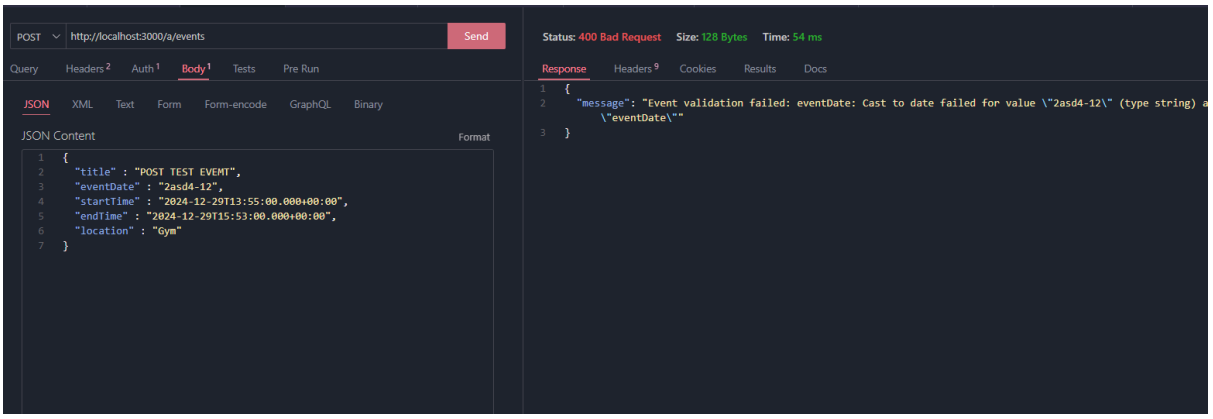


Figure 114. Validation failed due to an invalid data field in the event, resulting in a status code of 400.

This figure illustrates a validation error with a status code of 400. This can occur when an expected data field is incorrect.

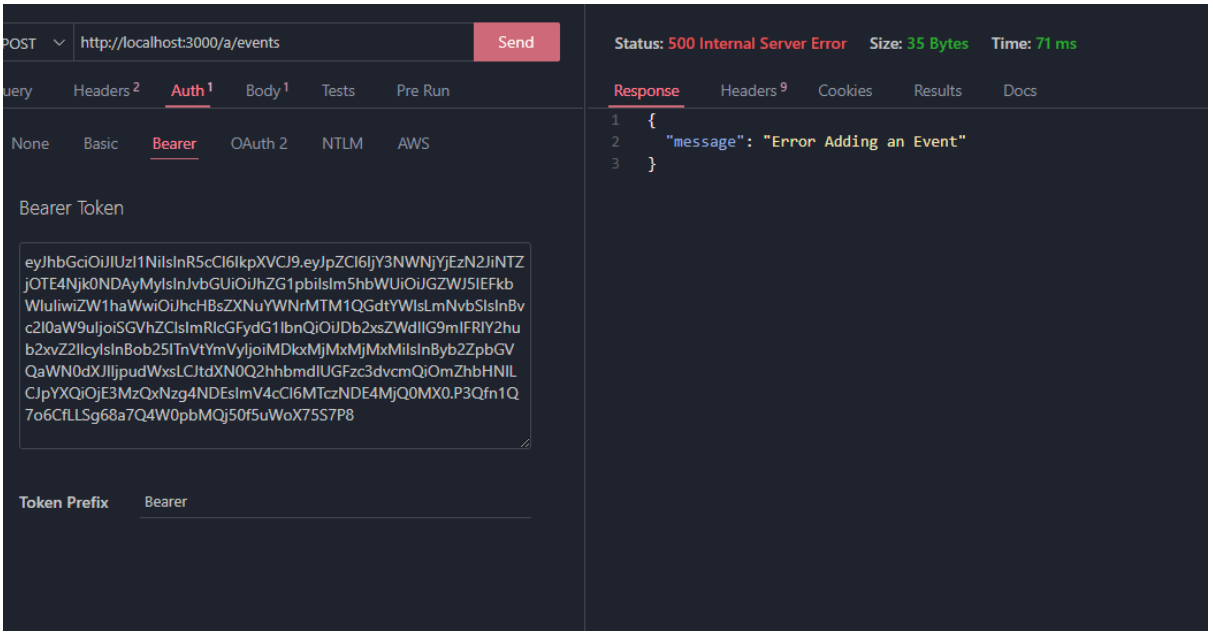


Figure 115. Server Error in Adding an Event with Status Code 500

This figure represents a scenario where a server error occurs during the process of adding an event. The status code 500 indicates that an internal server error was encountered, preventing the event from being successfully created. This could be caused by issues like database connection failures, server misconfigurations, or unexpected runtime errors.

1.2.6 Retrieve All Events

Version: 1.0

Date: November 25, 2024

Description: This api endpoint will fetch and retrieve all events listed in the database.

Endpoint: http://localhost:3000/a/events

Method: GET

Configurations: The API request should be validated using a token.

Parameters:

- **authtoken: (String)** A valid session token (JWT) that will be used to authenticate and authorize the admin making the request.

Response codes of this API endpoint:

Code	Message	Description
200	Events retrieved successfully	The request was successful, and the list of events was retrieved from the database and returned.
404	No events found	The request was successful, but no events were found in the database.
500	Internal Server Error	An internal server error occurred while trying to retrieve the events, indicating a potential server issue.

Valid and Invalid Request Responses

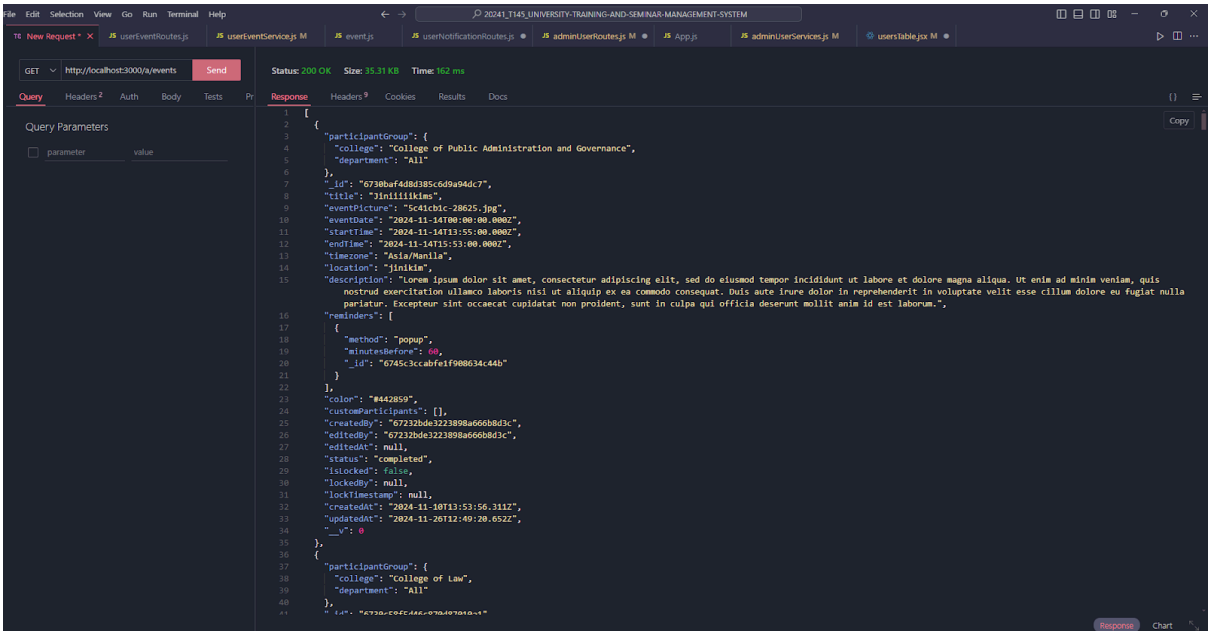


Figure 116. Valid GET Retrieval of List of Events from the Database

This figure shows a valid GET request code 200 for listing all events available in the database to the admin interface.

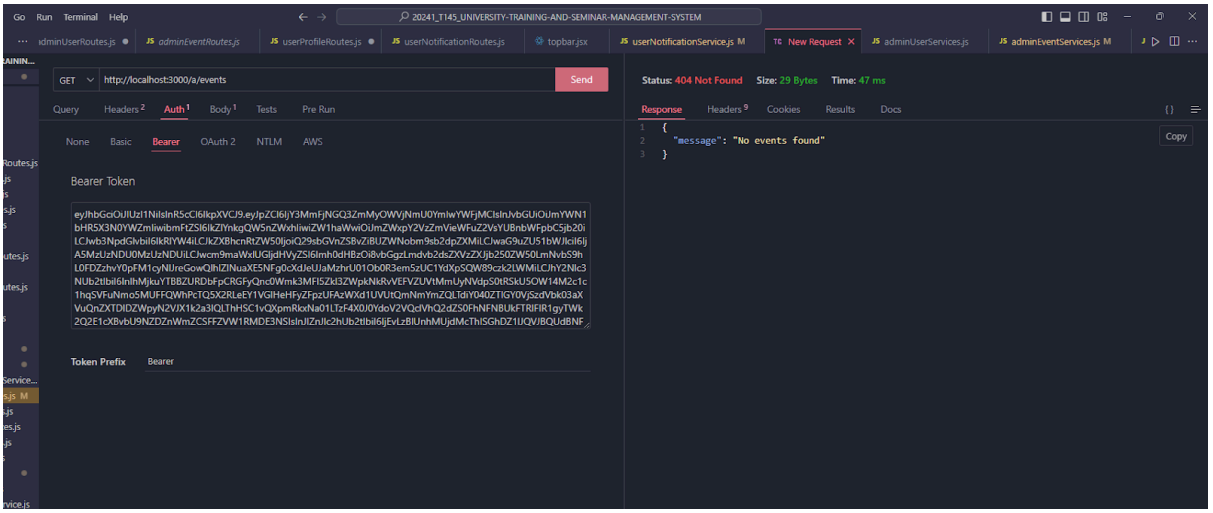


Figure 117. Invalid GET Retrieval of the List of Events from the Database

This figure illustrates an invalid GET request for retrieving the list of events, resulting in a 404 error. This occurs because no events were found in the database. The system checks for the existence of events, and when none are found, it returns an error message indicating that no events are available.

Response Format: JSON

Response:

The API response fetches all events from the database.

Different response Screenshots

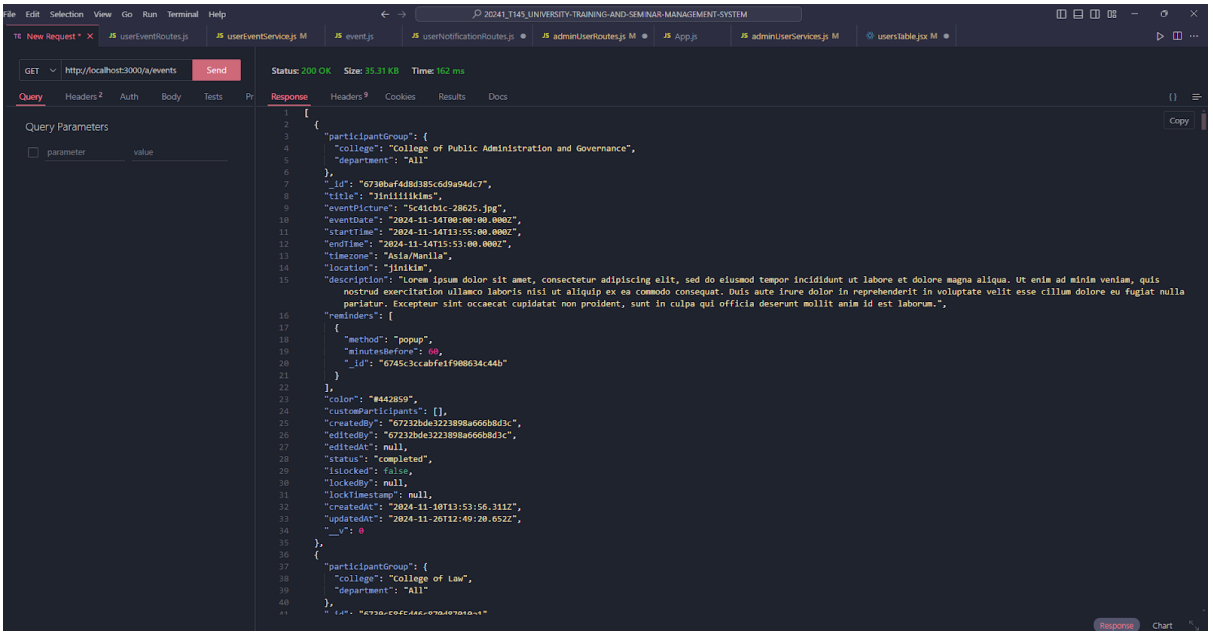


Figure 118. Successful Retrieval of Event List with Status Code 200

This figure shows a successful response (200) when retrieving the event list from the database. It confirms that the request was processed correctly and the event data was returned without errors.

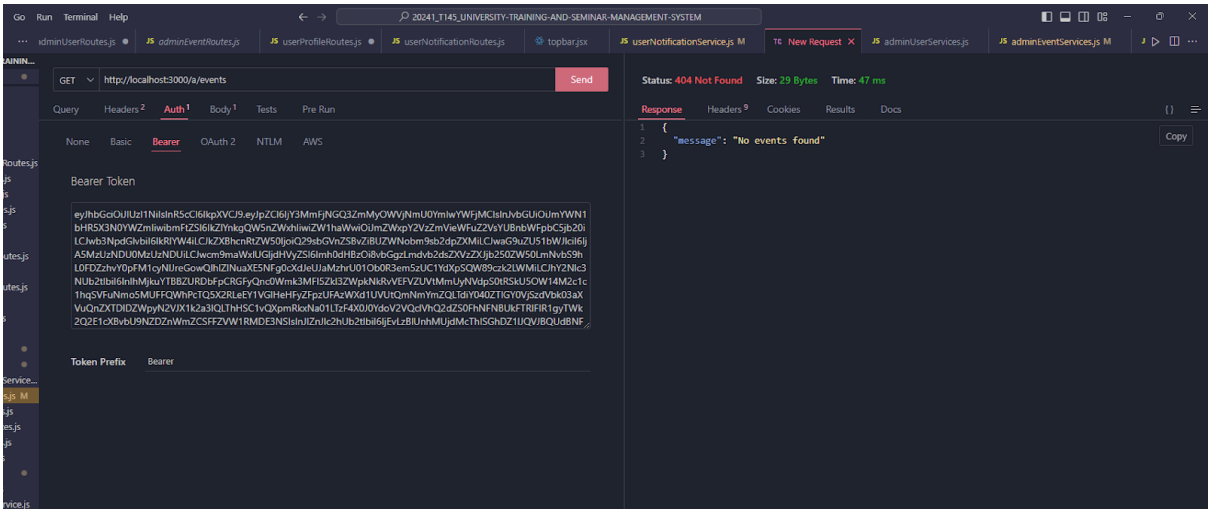


Figure 119. Failure Retrieval of Event List with Status Code 404

This figure demonstrates the failure to retrieve the event list due to the absence of events in the database, resulting in a 404 error. The system attempted to fetch the events but returned an error message indicating that no events were found.

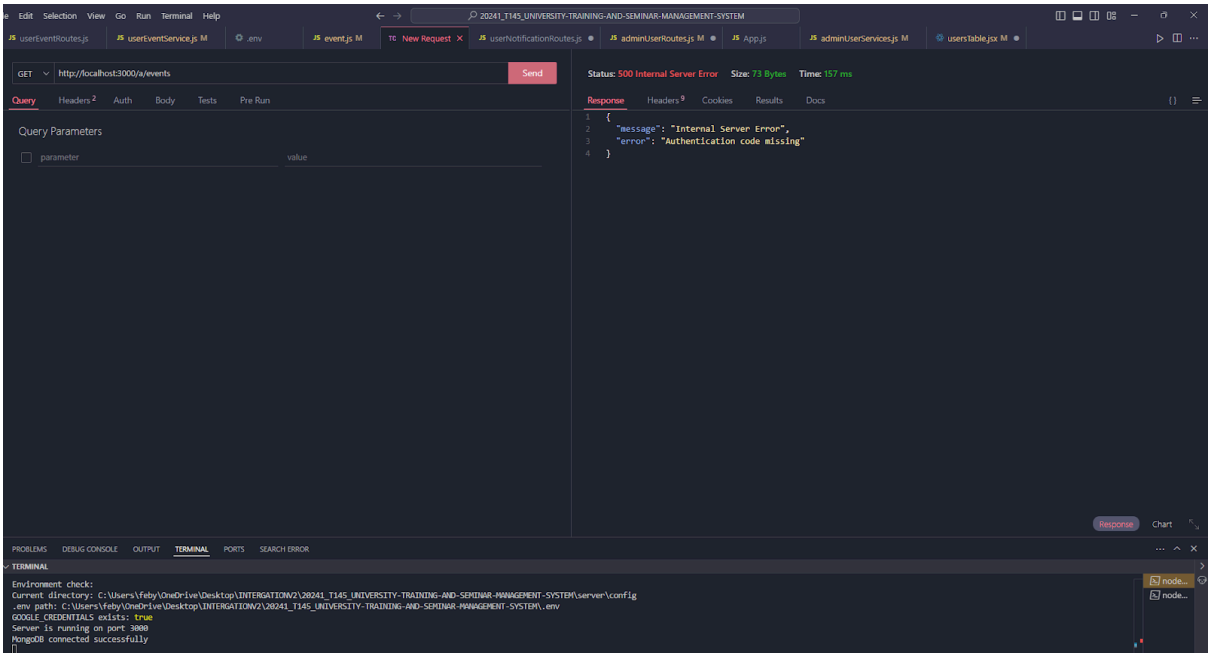


Figure 120. Server Error in Retrieving an Event List with Status Code 500

This figure shows a server error (500) when retrieving the event list from the database. It indicates an internal server issue during the data retrieval process.

1.2.7 Edit or Update an Event

Version: 1.0

Date: November 25, 2024

Description: This api endpoint will edit data of a specific event and will modify the data in the database.

Endpoint: http://localhost:3000/a/event/:id

Method: PUT

Configurations: The API request should be validated using a token.

Parameters:

- **eventId: (String)** The unique identifier of the event to be updated.
- **authToken: (String**) A valid session token (JWT) required for authentication. It should be included in the Authorization header to authorize the request.
- **title: (string, optional):** The new title for the event.
- **eventDate: (date, optional):** The new event date.
- **startTime: (date, optional):** The new start time of the event.
- **endTime: (date, optional):** The new end time of the event.
- **location: (string, optional):** The updated location of the event.
- **eventPicture: (string, optional):** The new picture associated with the event.
- **description: (string, optional):** The updated description of the event.
- **reminders: (array, optional):** Any changes to reminder settings for the event.
- **formLink: (string, optional):** A new or updated form link.
- **formId: (string, optional**): A new or updated form ID.
- **certificateTemplate:** (string, optional): A new or updated certificate template.
- **participantGroup: (string, optional):** Updated participant group.
- **customParticipants: (array, optional):** Any changes to the custom participants

Response codes of this API endpoint:

Code	Message	Description
200	Event updated successfully	Indicates that the event was successfully updated in the database and the Google Calendar event was also updated.
400	Bad request	Returned when there is an issue with the request, such as missing parameters or invalid data.
403	Invalid token	Indicates that the provided token is invalid, preventing the admin from accessing or modifying the event.
500	Internal Server Error	Returned when an unexpected error occurs on the server side, such as database or API issues.

Valid and Invalid Request Responses

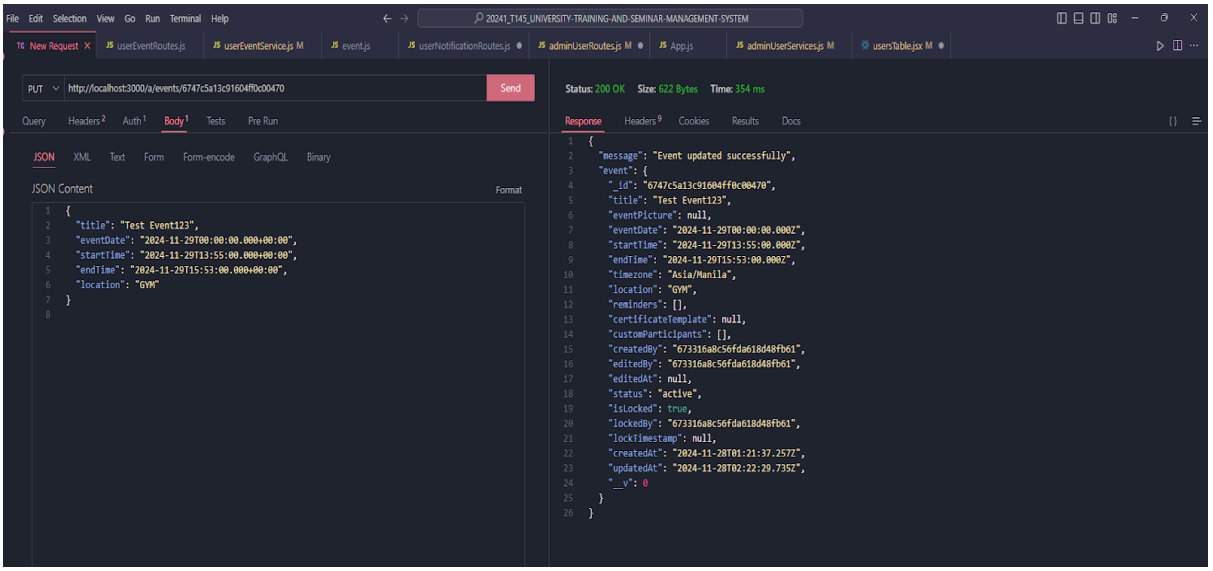


Figure 121. Valid PUT Request of Updating an Event

This figure shows a valid request for updating an event. The request body contains optional data fields, such as the title, event date, start time, end time, location, and others. It also includes new data inputs, 'editedBy' and 'editedAt,' which indicate the admin or departmental admin who edited the event.

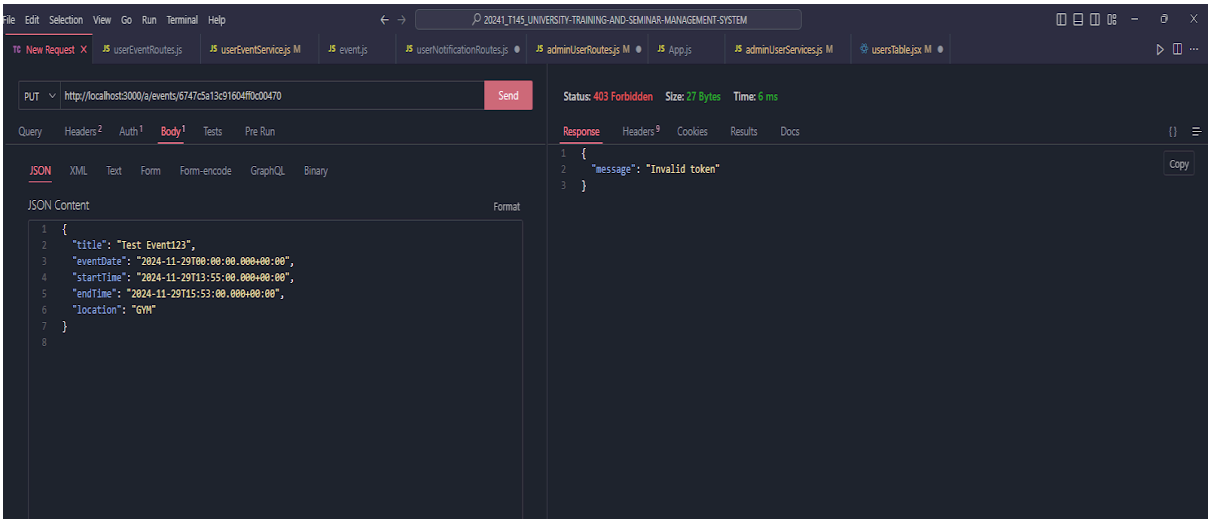


Figure 122. Invalid PUT Request of Updating an Event

This figure shows an invalid request for updating an event due to an invalid admin token.

Response Format: JSON

Response:

The API response will update the newly edited event and save it to the database.

Different response Screenshots

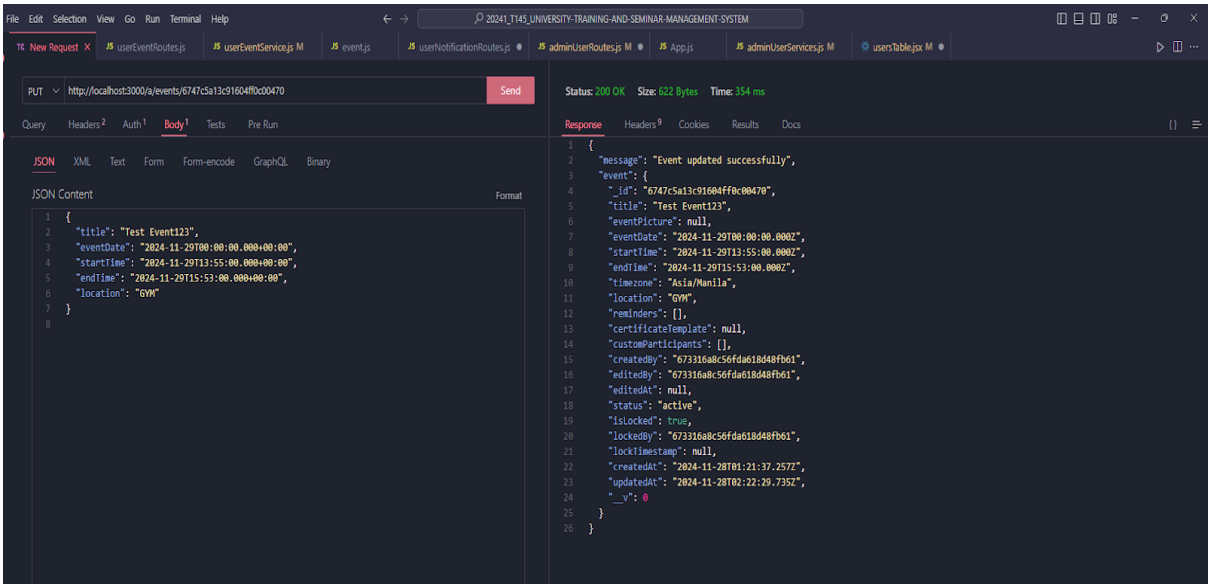


Figure 123. Successful Update of an Event with Status Code 200

This figure illustrates the successful update of an event, accompanied by a response code of 200. The desired request body was provided, and as a result, the event has been successfully updated in the database.

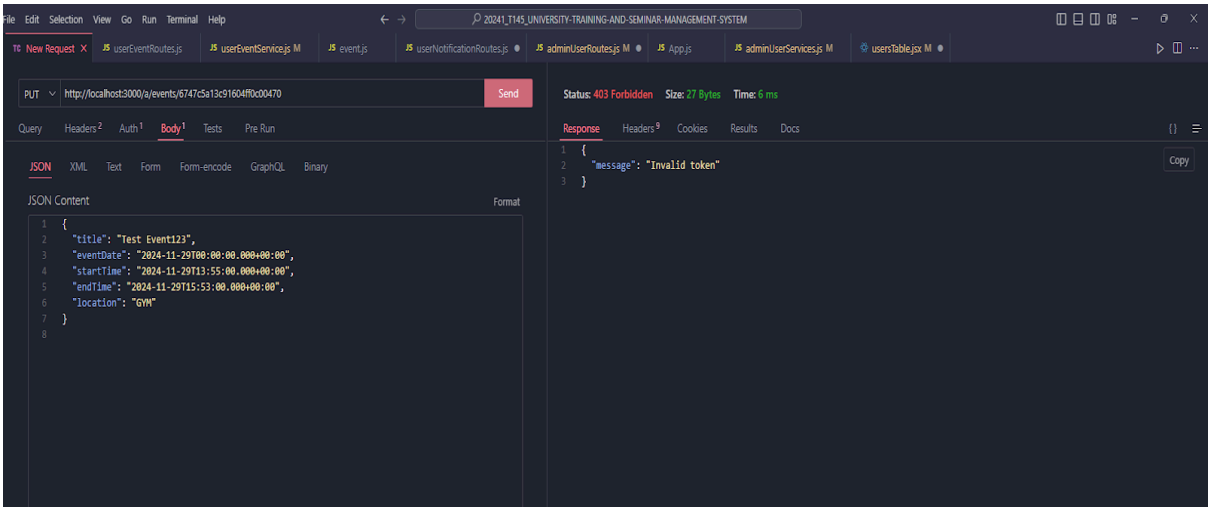


Figure 124. Failed to Update of an Event with Status Code 403

This figure illustrates the failure to update an event, accompanied by a response code of 403. This is because the admin or departmental admin token is either invalid or expired.

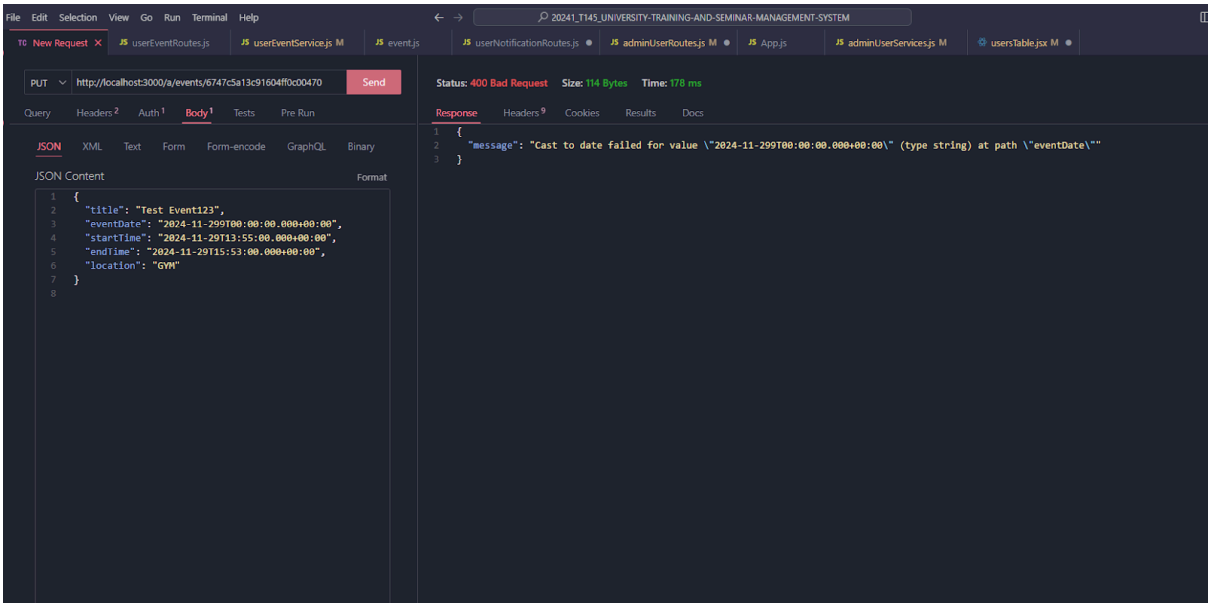


Figure 125. Invalid Format of an Event with Status Code 400

This figure illustrates the failure to update an event, accompanied by a response code of 400. This is because the data fields are not correctly formatted.

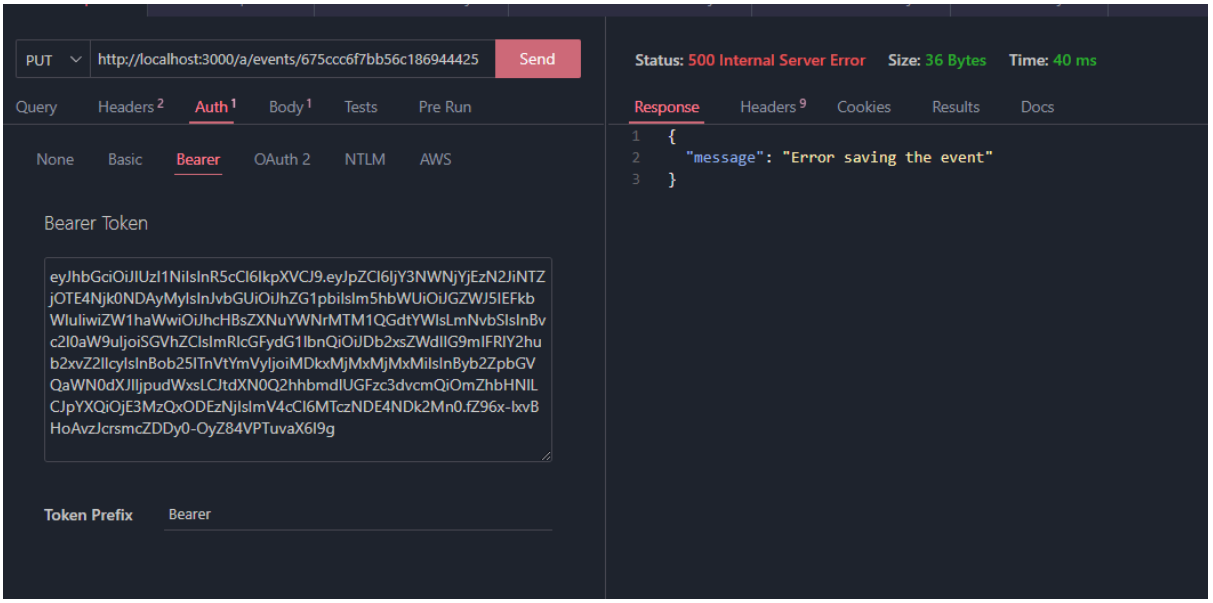


Figure 126. Server Error of an Event with Status Code 500

This figure illustrates a 500 Internal Server Error, indicating a server-side issue that prevents fulfilling a request during an event. It signifies unexpected conditions like unhandled exceptions, misconfigurations, or dependency failures on the server.

1.2.8 Cancel an Event

Version: 1.0

Date: November 25, 2024

Description: This api endpoint will delete an event from the event collection of the database, however it will send a duplicate copy of the deleted event to another collection.

Endpoint: http://localhost:3000/a/events/:id

Method: DELETE

Configurations: The API request should be validated using a token.

Parameters:

- **eventId:** The unique identifier of the event to be deleted.
- **authToken: (String)** A valid session token (JWT) required for authentication. It should be included in the Authorization header to authorize the request.

Response codes of this API endpoint:

Code	Message	Description
200	Event and associated picture deleted successfully	Returned when the event and associated picture are successfully deleted.
404	Event not found	Returned when the event with the specified ID does not exist in the database.
500	Internal Server Error	Returned when an unexpected server-side error occurs, such as database or file system issues.

Valid and Invalid Request Responses

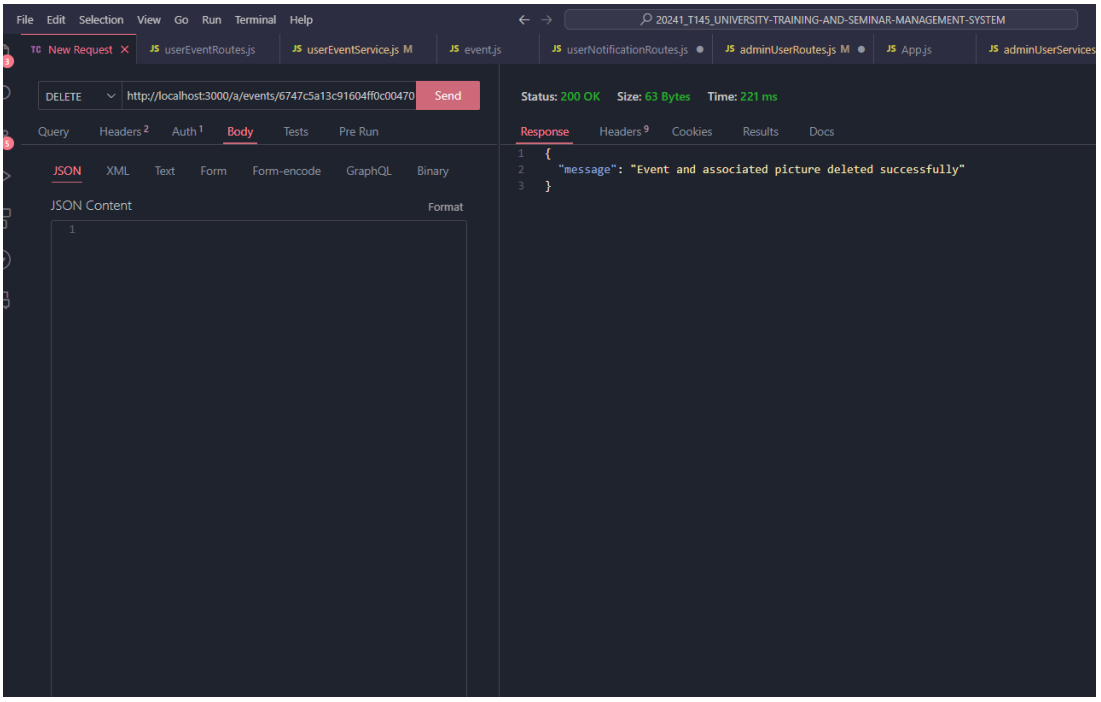


Figure 127. Valid DELETE Request for Cancelling an Event

This figure shows a valid request for cancelling an event. After the event has been successfully deleted, the event details will be removed from the event collection and moved into the deleted event collection, with an added field for 'deletedBy' and 'deletedAt'.

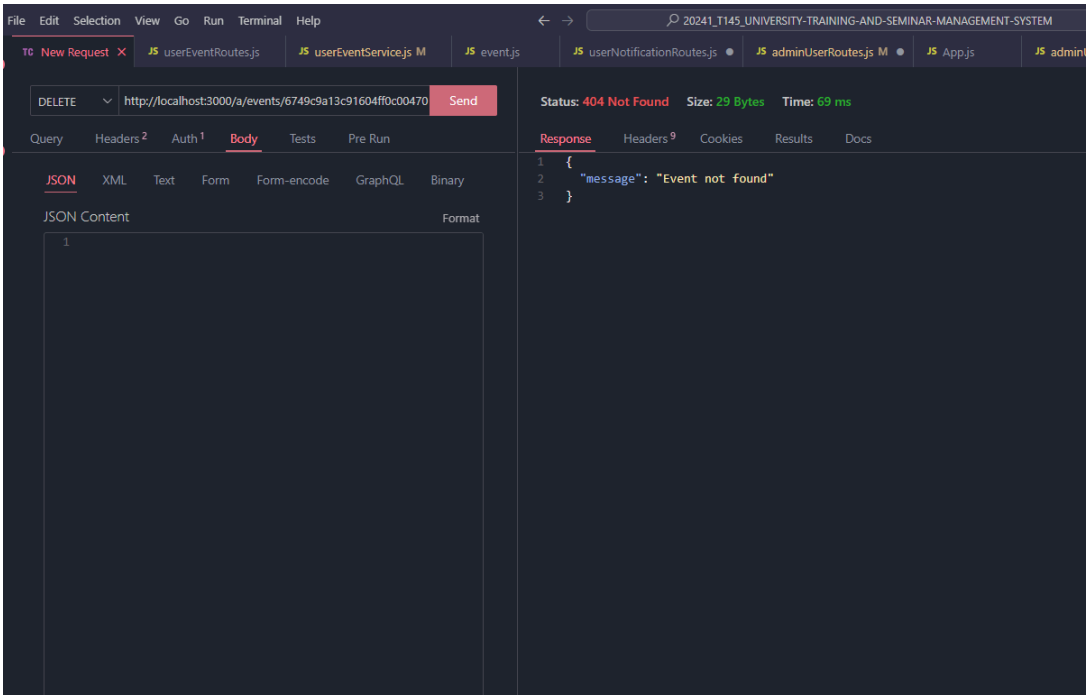


Figure 128. Invalid DELETE Request for Cancelling an Event

This figure illustrates an invalid request for cancelling an event. The error occurred because the event intended for deletion does not exist in the database.

Response Format: JSON

Response:

The API response will delete the selected event from the event collection and save it to the deleted event collection.

Different response Screenshots

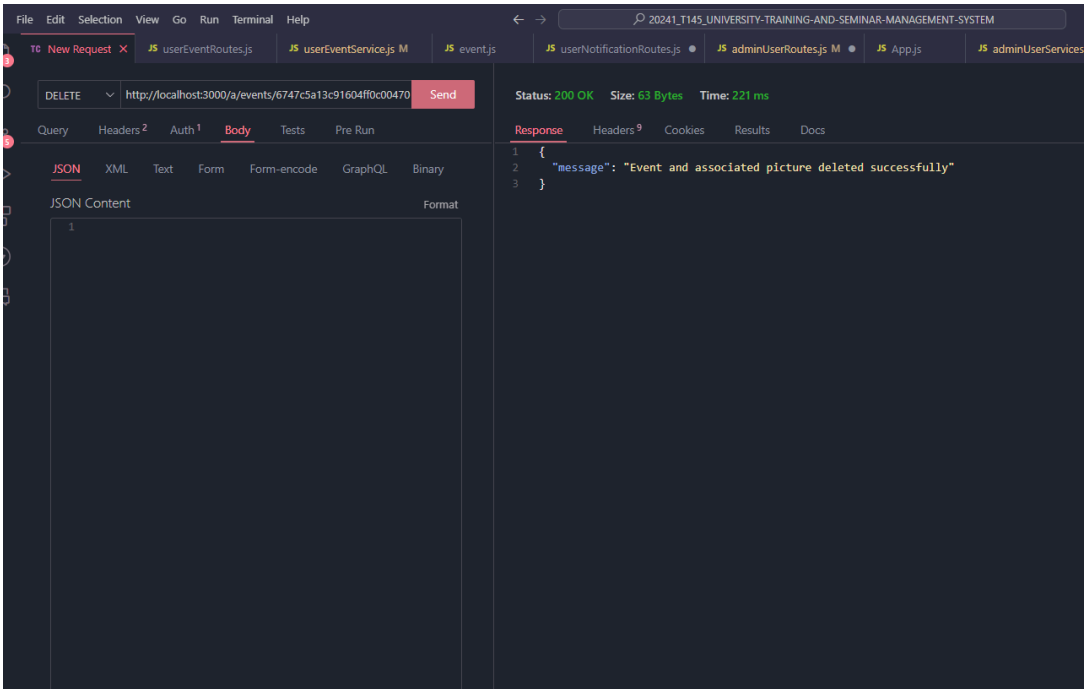


Figure 129. Successfully Cancelling a Specific Event with Status Code 200

This figure illustrates the successful cancellation of a specific event, where the server responds with a status code of 200. It indicates that the event has been deleted successfully, including its associated data (such as notifications and images). The response also includes a confirmation message to inform the user that the cancellation process was completed without issues.

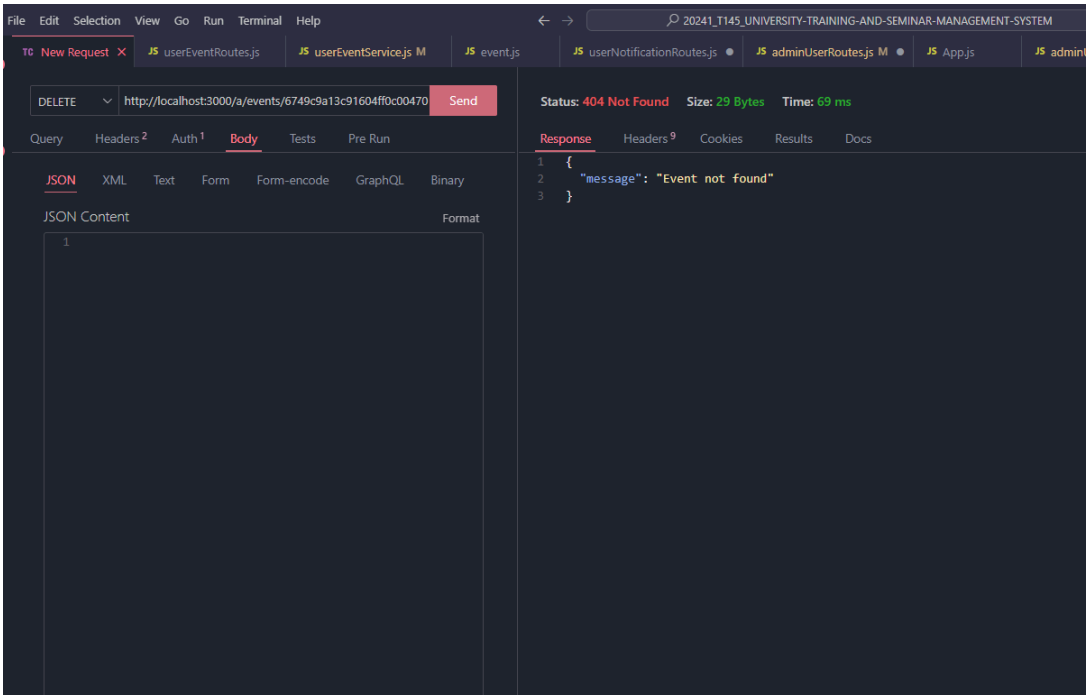


Figure 130. Failure of Cancelling a Specific Event with Status Code 404

This figure shows a failure to cancel an event, with a 404 status indicating the event was not found in the database, and an error message informing the user.

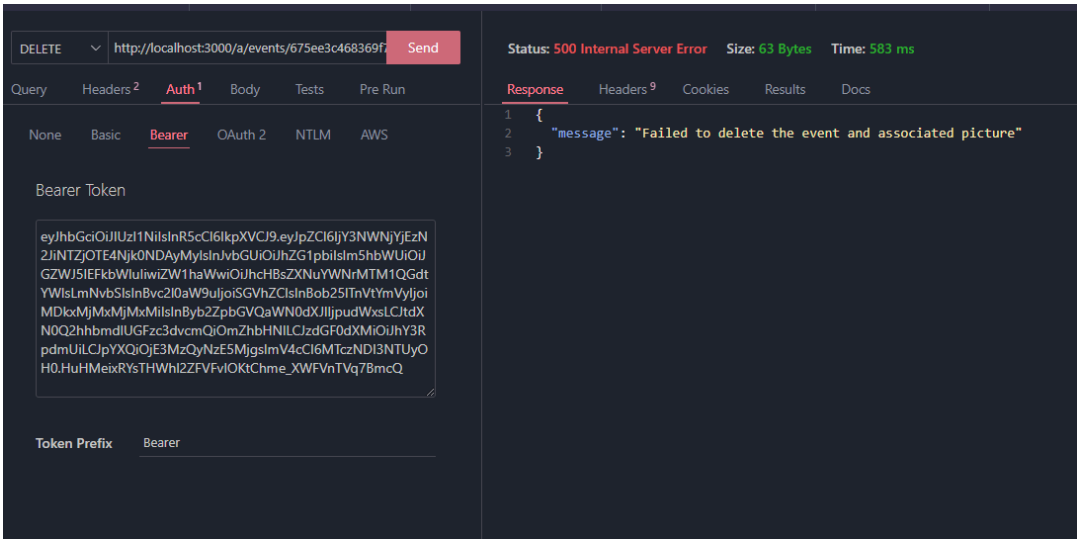


Figure 131. Server Error of Cancelling a Specific Event with Status Code 500

This figure shows a server error (500) during event cancellation due to an internal issue, with an error message provided for further investigation.

1.2.9 Event Concurrency Lock

Version: 1.0

Date: November 25, 2024

Description: This API endpoint will change the status of isLock to true in the database when the admin is editing an event

Endpoint: http://localhost:3000/a/events/:id/lock

Method: PUT

Configurations: The API request must include a valid session token (JWT) for authentication.Only authorized users (e.g., admin or departmental admin) can perform this operation.

Parameters:

- **eventId:** The unique identifier of the event to be locked.
- **authToken:** A valid session token (JWT), included in the Authorization header, is required to authorize the request.

Response codes of this API endpoint:

Code	Message	Description
200	Event locked for editing	Indicates that the event has been successfully locked for editing. The event is now in an exclusive state for modifications by the current user.
403	Invalid Token Event is being edited by another admin	This occurs when the admin provides an invalid token, meaning the token is either expired, malformed, or incorrect.
403	Event is being edited by another admin	This occurs when the event is currently being edited by another admin, and the current admin is prevented from making modifications.
404	Event not found	This response indicates that no event was found for the given eventId, meaning the specified event doesn't exist in the database.
500	Internal Server Error	An unexpected server error occurred during the process of locking the event. This could be due to issues with the server, database, or related services.

Valid and Invalid Request Responses

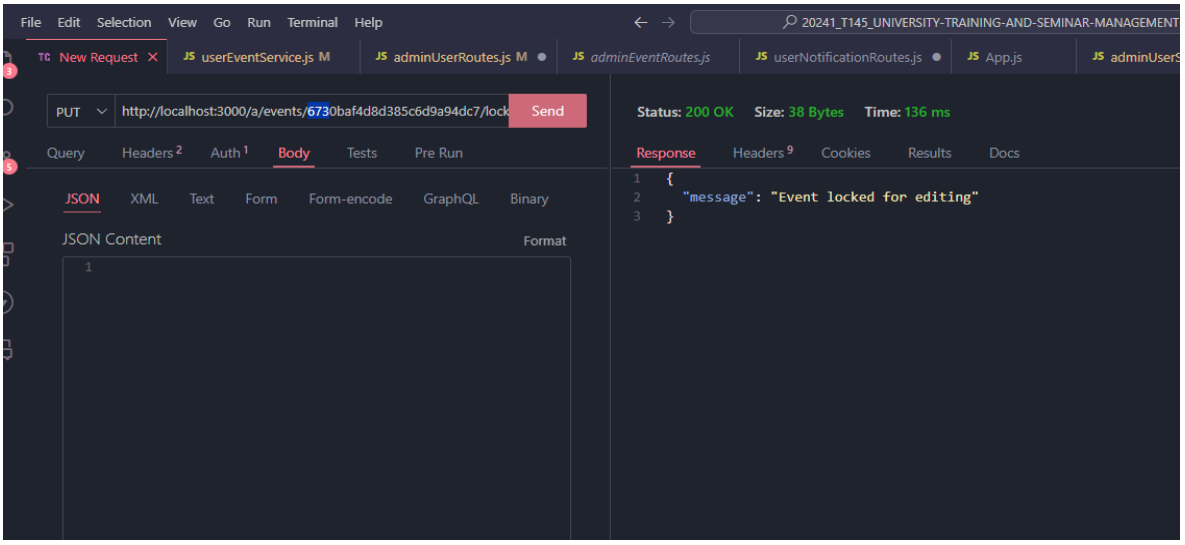


Figure 132. Valid PUT Request for Locking an Event in Edit Mode

This figure illustrates a valid request to lock an event, which updates the 'isLocked' field in the database to true. This action activates a locking mechanism that prevents multiple admins from editing the event simultaneously.

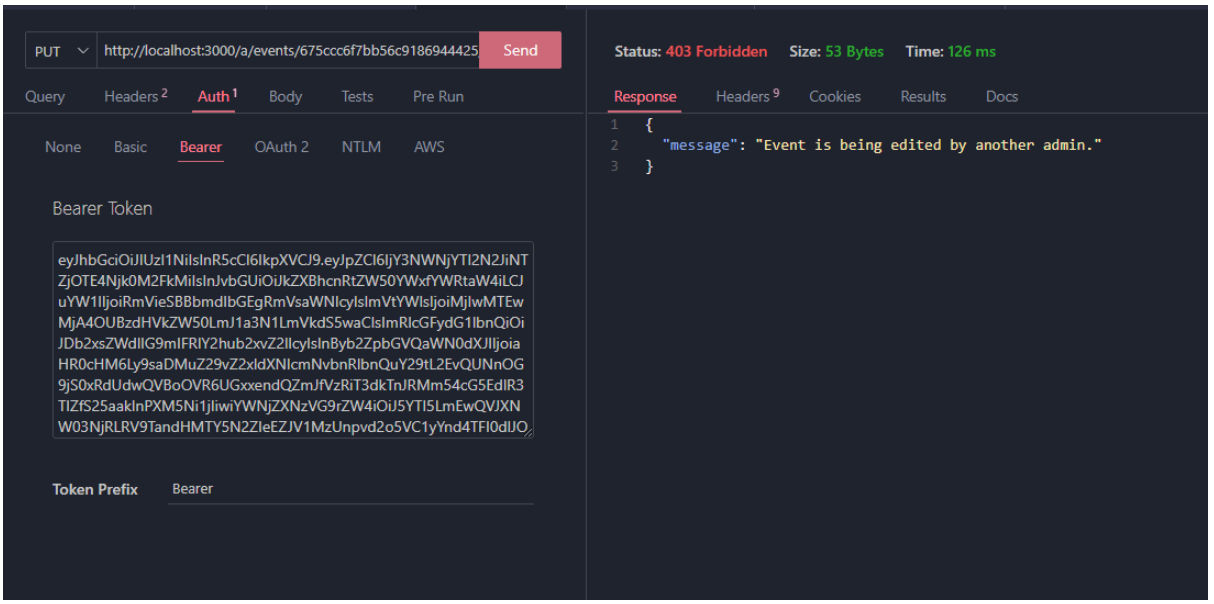


Figure 135. Invalid PUT Request for Locking an Event in Edit Mode

This figure illustrates an invalid request to lock an event, which occurs due to that specific event being edited by another admin.

Response:

The API response updated the status of isLock from false to true in the database

Different response Screenshots

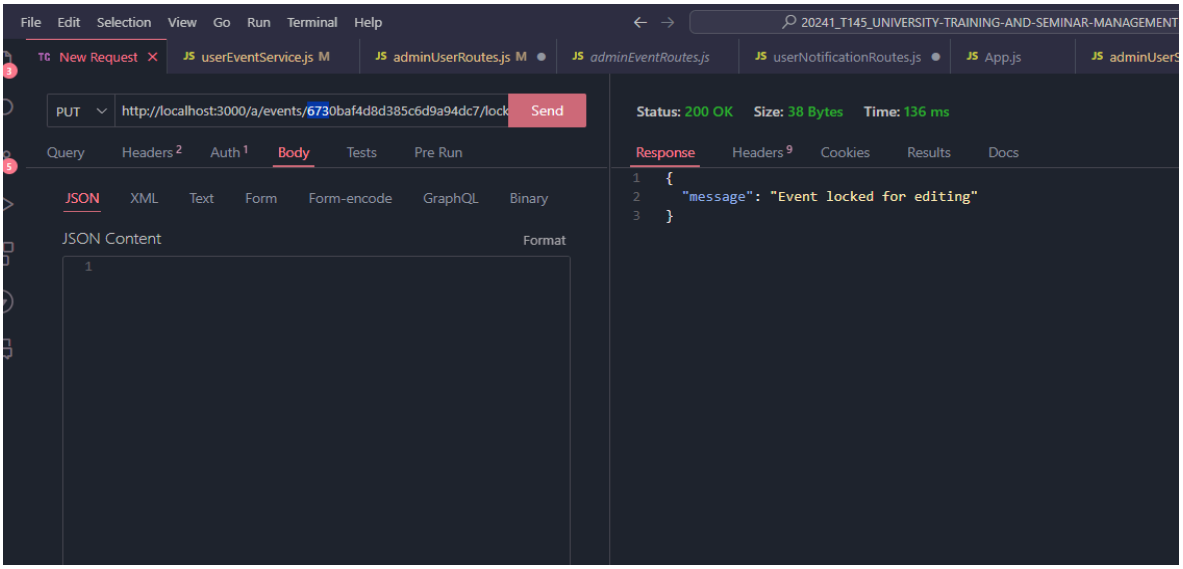


Figure 136. Successful Locking an Event in Edit Mode with Status Code 200

This figure shows a successful event lock for editing, indicating the event is now exclusive to the current user. Status code 200 confirms the request was processed successfully.

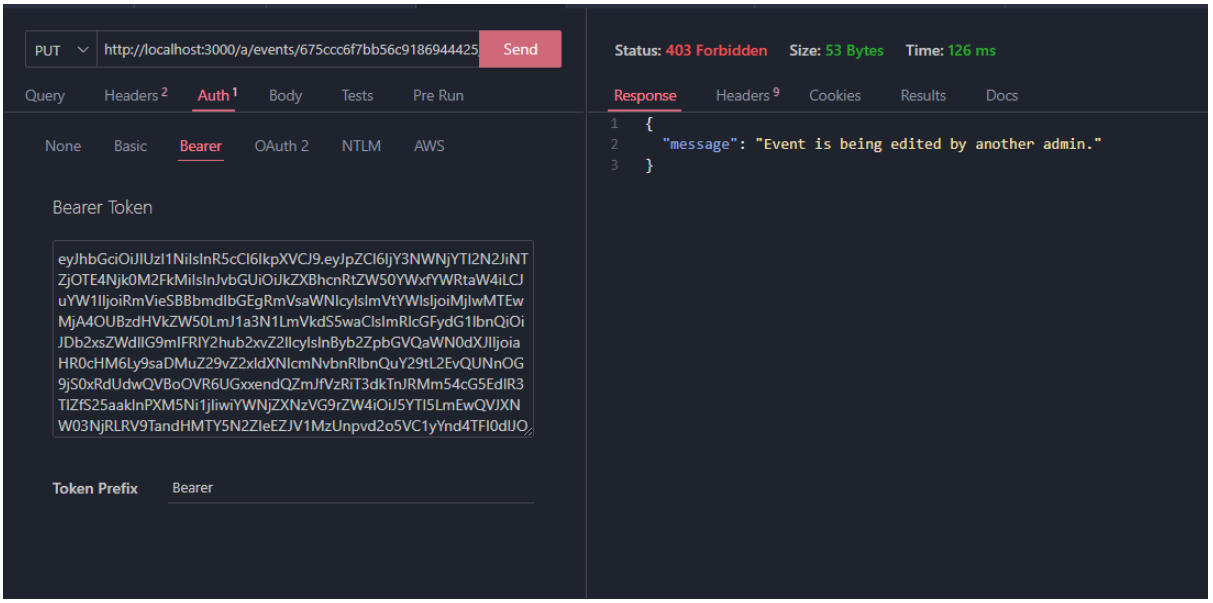


Figure 137. Failure Locking an Event in Edit Mode with Status code 403

This figure demonstrates a failure where the user receives a 403 Forbidden response when attempting to lock an event, typically because that specific event is currently being edited by another admin means other admins can't lock that event yet.

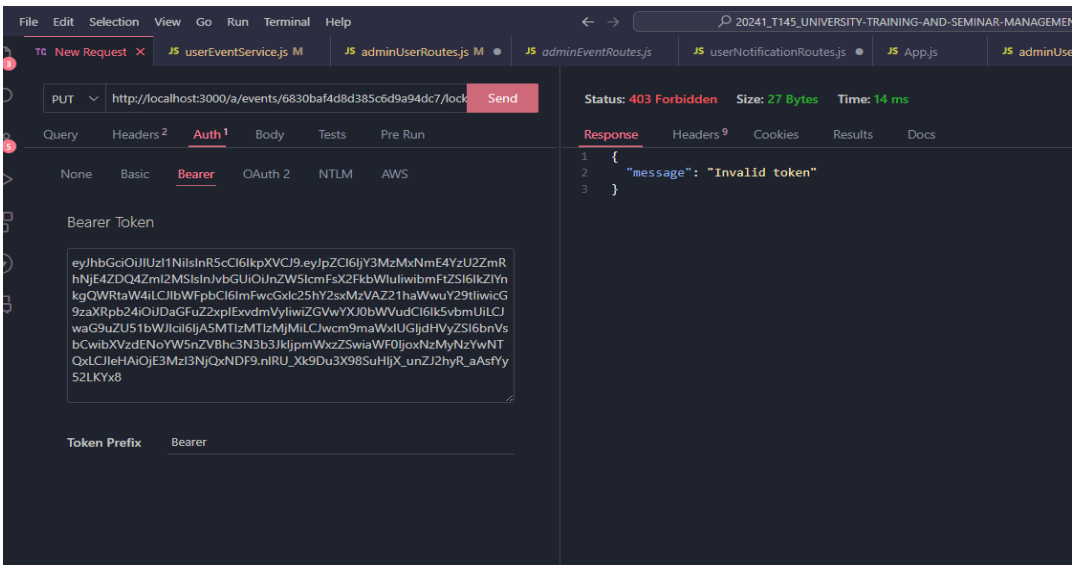


Figure 138. Failure Locking an Event in Edit Mode with Status code 403

This figure demonstrates a failure where the user receives a 403 Forbidden response when attempting to lock an event, typically due to an invalid or expired token. The event locking operation is denied.

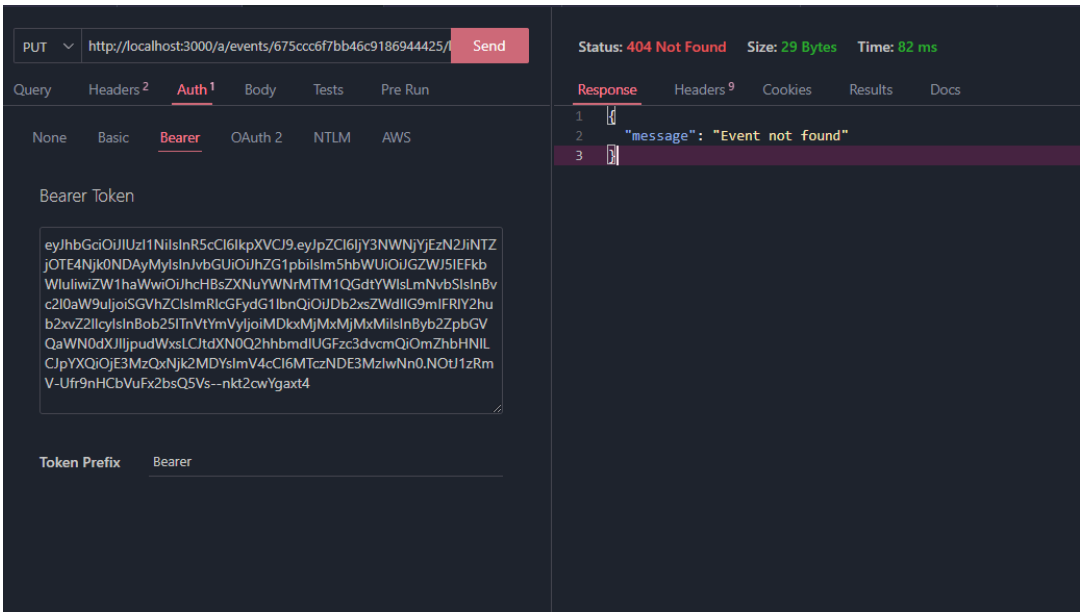


Figure 139. Failure Locking an Event in Edit Mode with Status code 404

This figure shows a failure where the event is not found in the database. The 404 status code indicates that the specified event ID does not exist, preventing the operation from being completed.

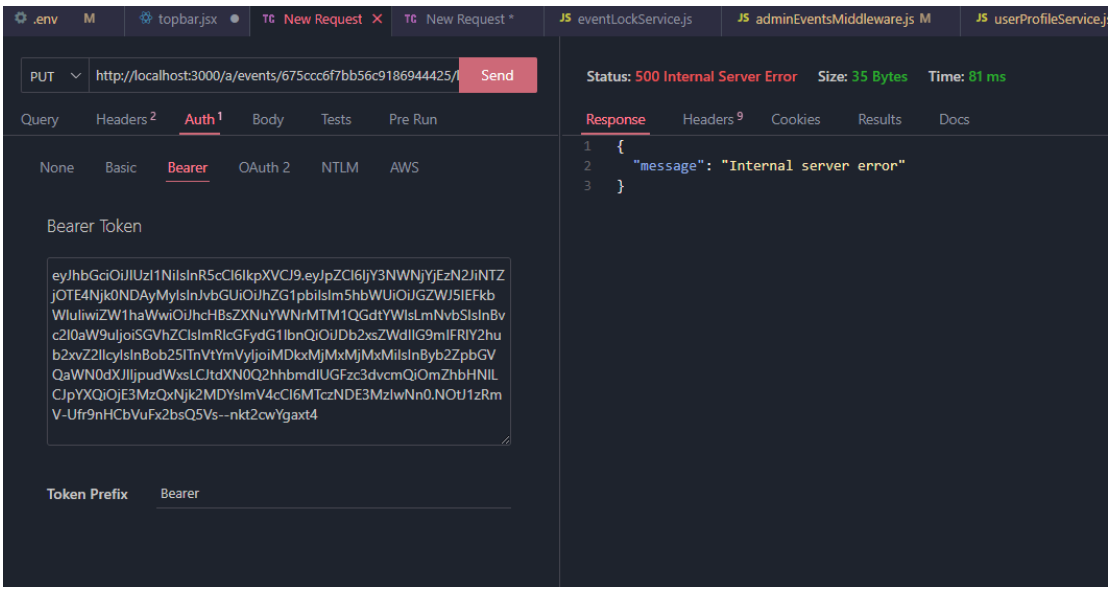


Figure 140. Server Error of Locking an Event in Edit Mode

This figure shows an error where the event lock fails due to a server issue. The 500 status code indicates a general server error, suggesting problems with the database, server, or unexpected failures during the process.

1.2.10 Event Clear Concurrency Lock

Version: 1.0

Date: November 25, 2024

Description: This API endpoint will change the status of isLock to false in the database if either the admin saves the edited details or exits edit mode.

Endpoint: http://localhost:3000/a/events/:id/unlock

Method: PUT

Configurations: The API request should be validated using a token.

Parameters:

- **eventId:** The unique identifier of the event to unlock.
- **authToken**: A valid session token (JWT), included in the Authorization header, is required to authorize the request.

Response codes of this API endpoint:

Code	Message	Description
200	Event unlocked	Indicates that the event has been successfully unlocked and is no longer in an exclusive editing state. The event is now available for edits by other users.
403	Invalid Token	This response signifies that the request failed due to an invalid or expired token, which means the user is not authorized to unlock the event.
404	Event not found	This status code indicates that no event with the specified ID was found, meaning the event does not exist in the database. Therefore, unlocking cannot proceed.
500	Internal Server Error	This response suggests that an unexpected server-side error occurred while trying to unlock the event. The issue may be related to the server or database.

Valid & Invalid Responses

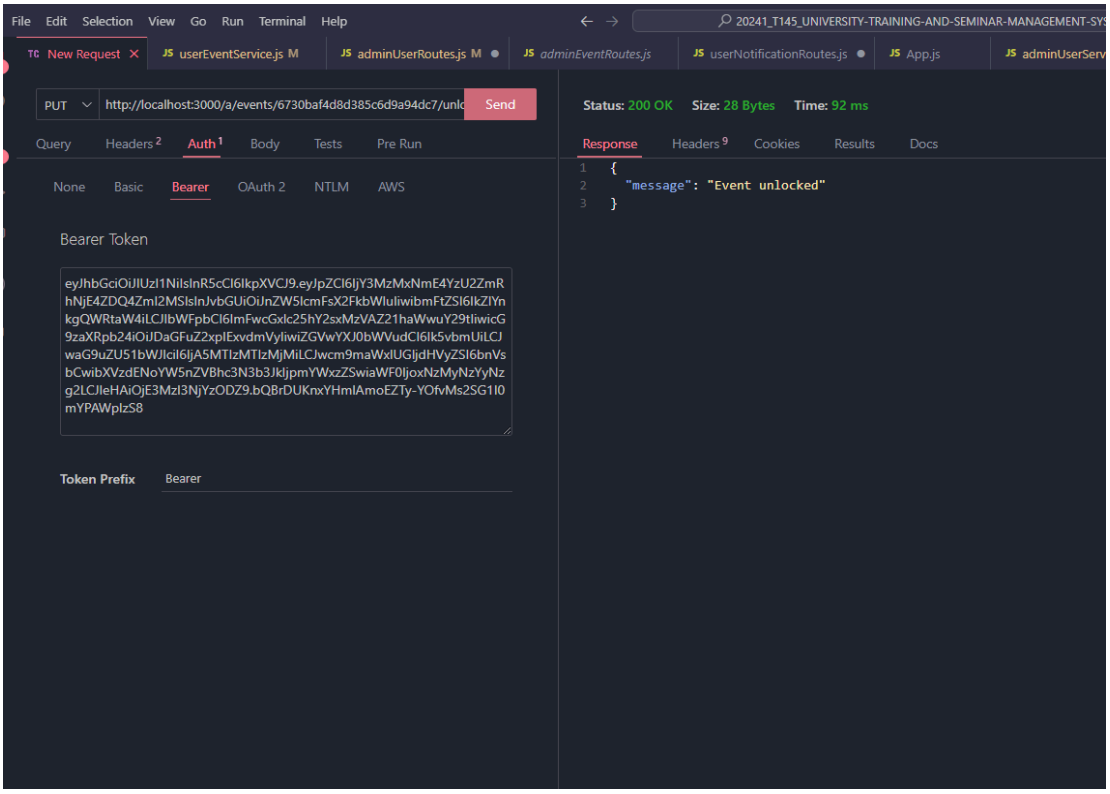


Figure 141. Valid PUT Request for Unlocking an Event in Edit Mode

This figure illustrates a valid request to unlock an event, updating the 'isLocked' field in the database to false. This action disables the locking mechanism, allowing admins to edit the event again.

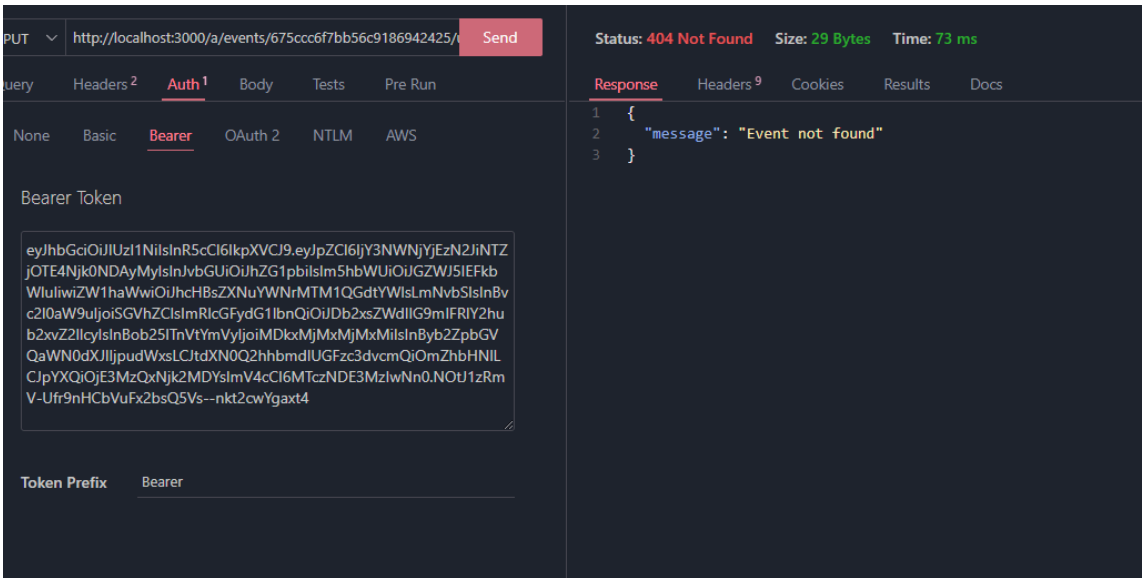


Figure 142. Invalid PUT Request for Unlocking an Event in Edit Mode

This figure illustrates an invalid request to unlock an event, which occurs because the event that was supposed to be unlocked is not found

Response Format: JSON

Response:

The API response updated the status of isLock from true to false in the database

Different response Screenshots

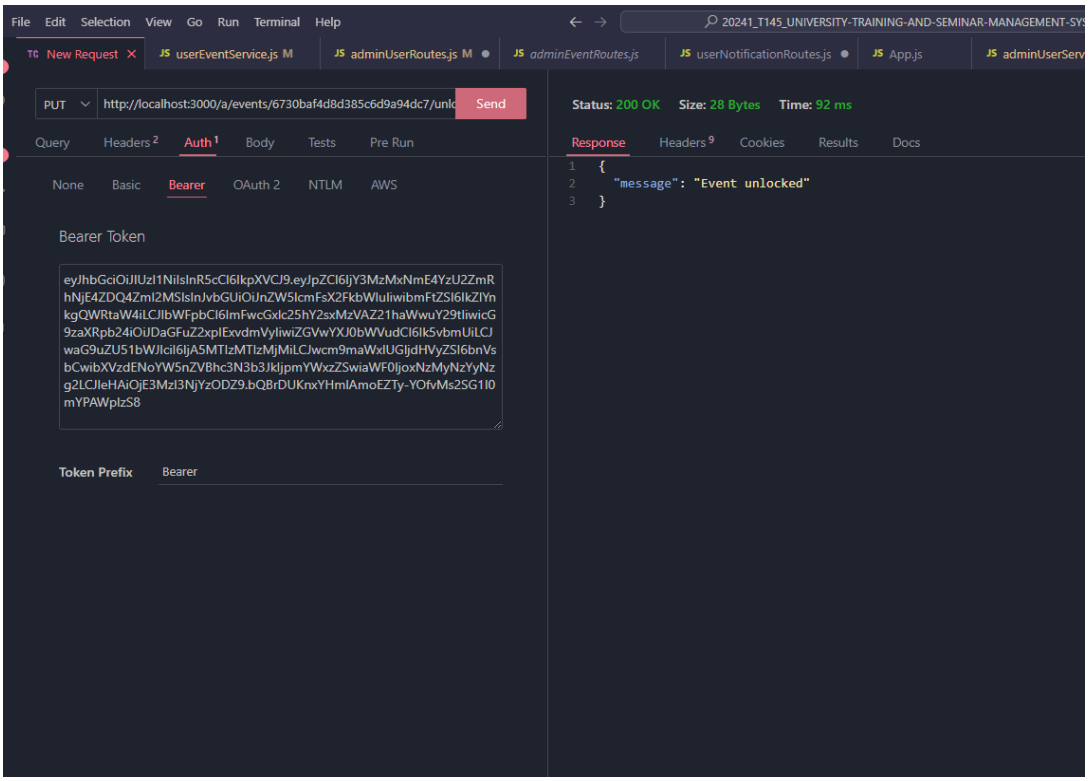


Figure 143. Successful Unlocking an Event in Edit Mode with Status Code 200

This figure indicates that the event has been successfully unlocked for editing, allowing other admins to make changes.

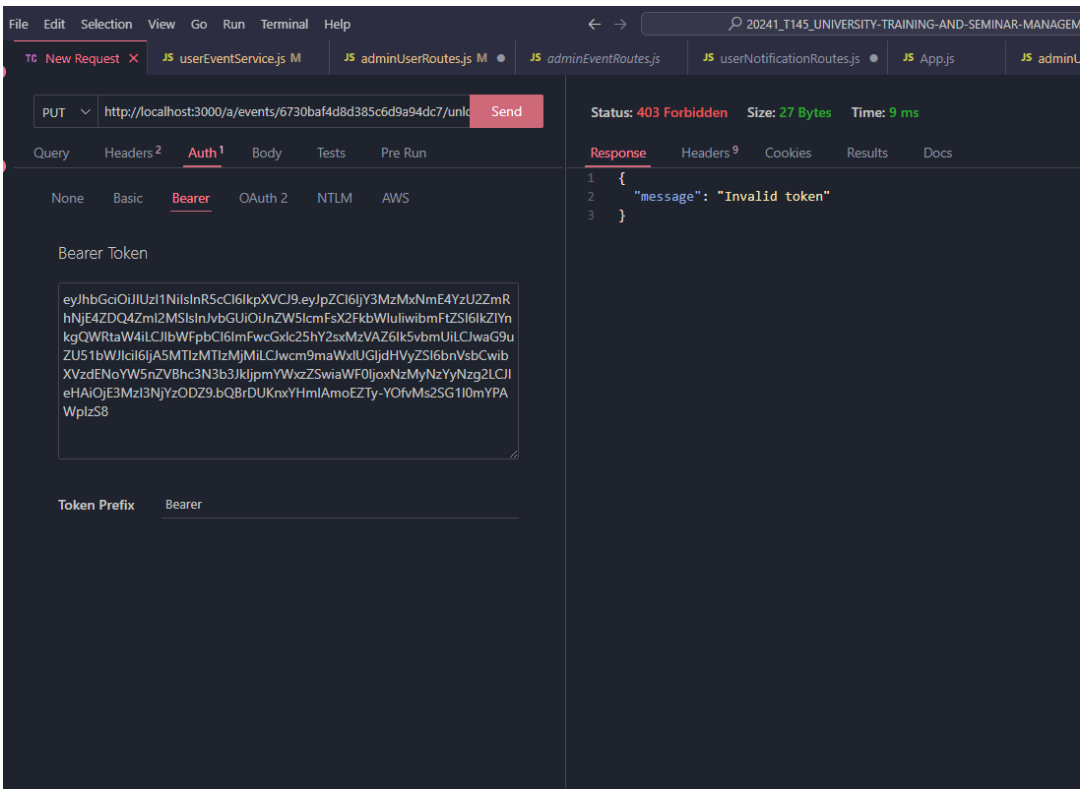


Figure 144. Failure Unlocking an Event with Status Code 403

This figure indicates that the user does not have permission to unlock the event, likely due to an invalid token or lack of authorization.

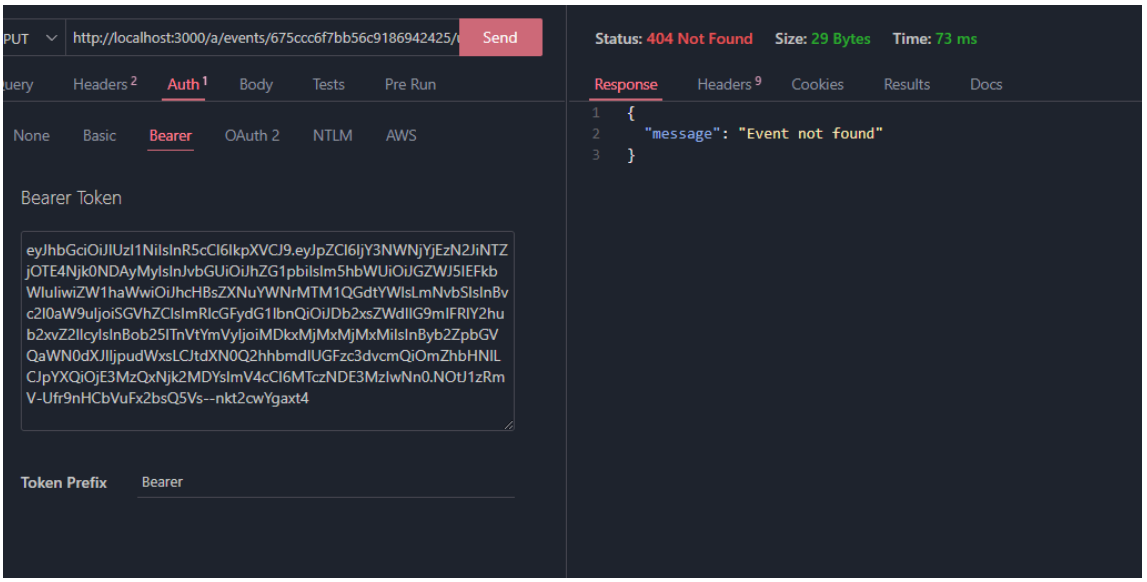


Figure 145. Failure Unlocking an Event with Status code 404

This figure indicates that the event with the specified ID could not be found in the database, meaning the event does not exist.

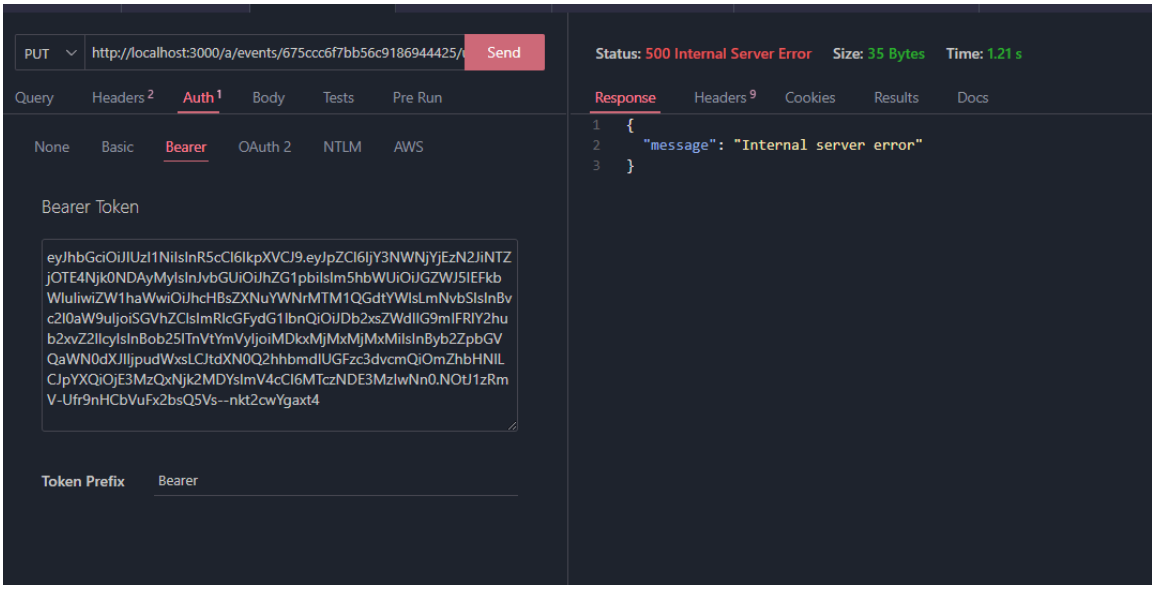


Figure 146. Server Error of Unlocking an Event with Status code 500

This figure represents an unexpected server error when trying to unlock the event, which could be due to server, database, or related service issues.

1.2.11 Retrieve Admin Event History List

Version: 1.0

Date: November 25, 2024

Description: This API endpoint will allow the admin to retrieve the complete list of event history without the need for authentication or session tokens. It provides access to all events recorded in the system.

Endpoint: http://localhost:3000/a/event-history
Method: GET

Configurations: No authentication or session token is required, and no query parameters are needed to retrieve all events.

Parameters:

- None

Response codes of this API endpoint:

Code	Message	Description
200	OK	The request was successful and the event history is returned.
400	Invalid or missing 'status' query parameter	The 'status' query parameter was either invalid or missing in the request.
500	message: error.message	An unexpected error occurred while processing the request.

Valid & Invalid Responses

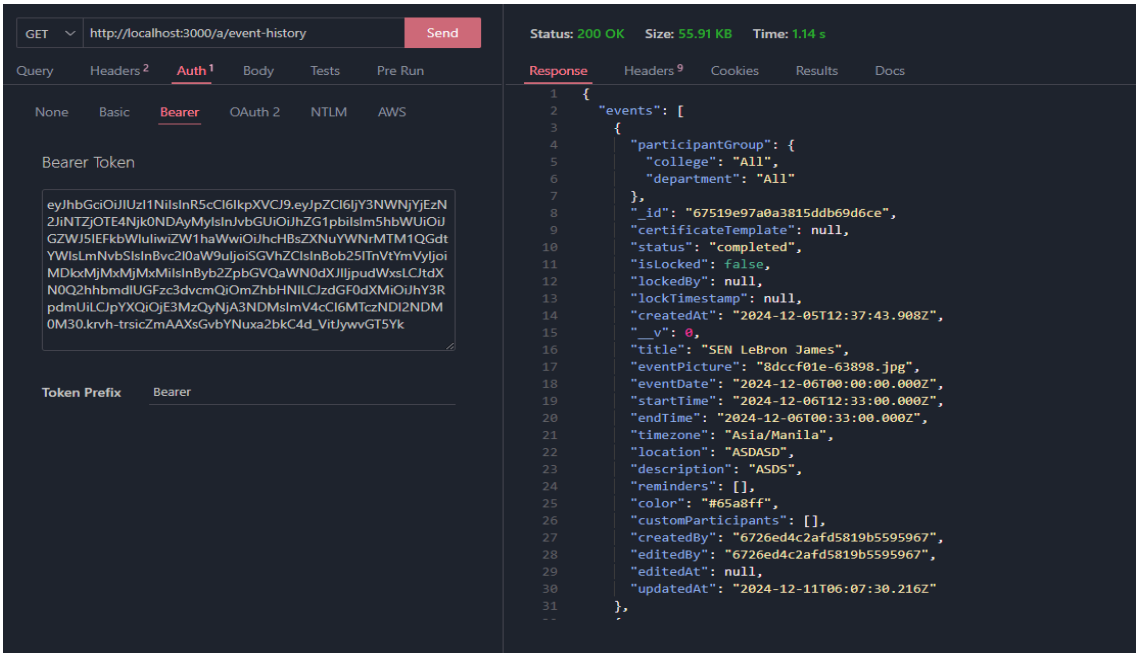


Figure 147. Valid GET Request for List of Event History

This figure shows a valid GET request made to retrieve the list of event history. The request is successfully processed, and the system responds with the appropriate event data.

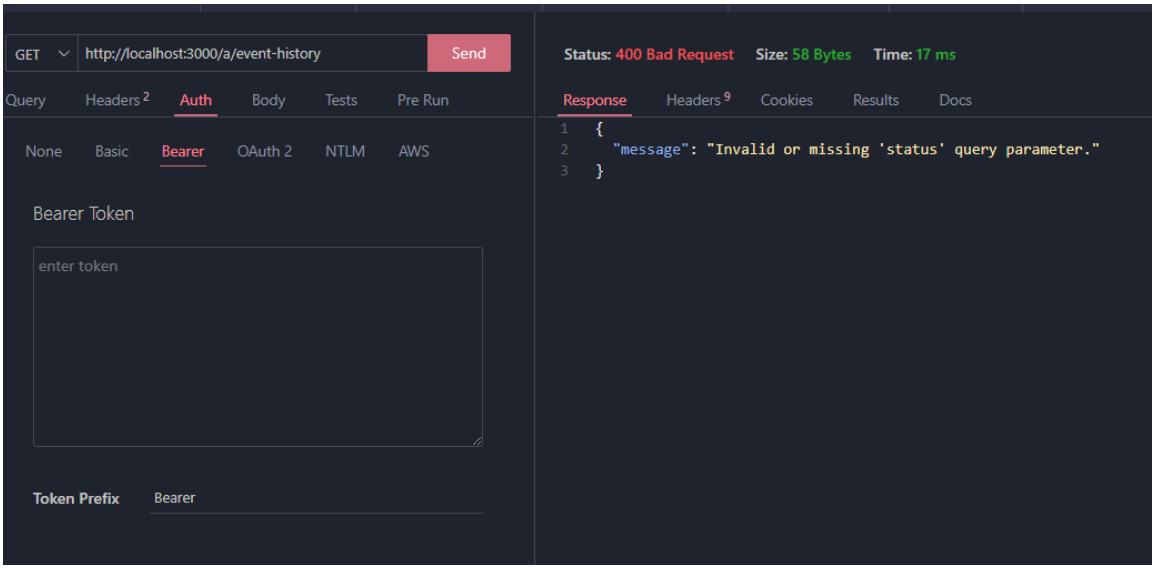


Figure 148. Invalid GET Request for List of Event History

This figure shows an invalid GET request for the list of event history. The response code is 400, indicating that the system cannot detect or identify the requested event status in the database, leading to an error.

Response Format: JSON

Response:

The API response retrieves the history of completed seminars and training events, providing a list of events with their details.

Different response Screenshots

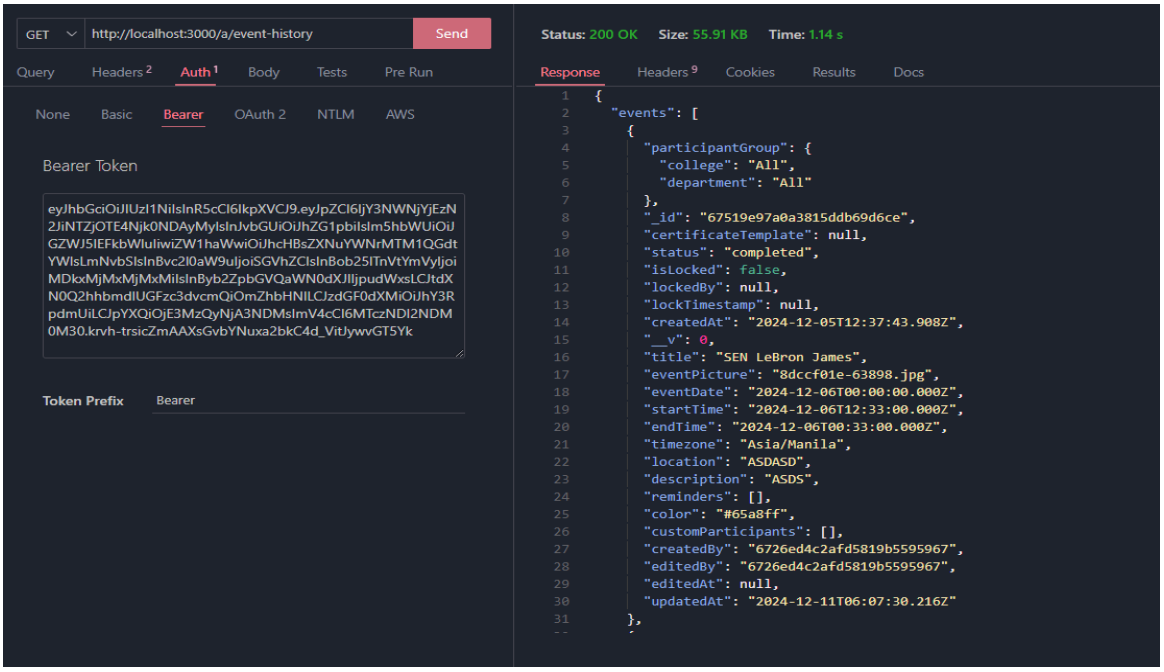


Figure 149. Successfully Retrieve the List of Completed Events Code 200

This figure shows a successful retrieval of the list of completed events, with a response code of 200, indicating that the request was processed successfully and the data was returned.

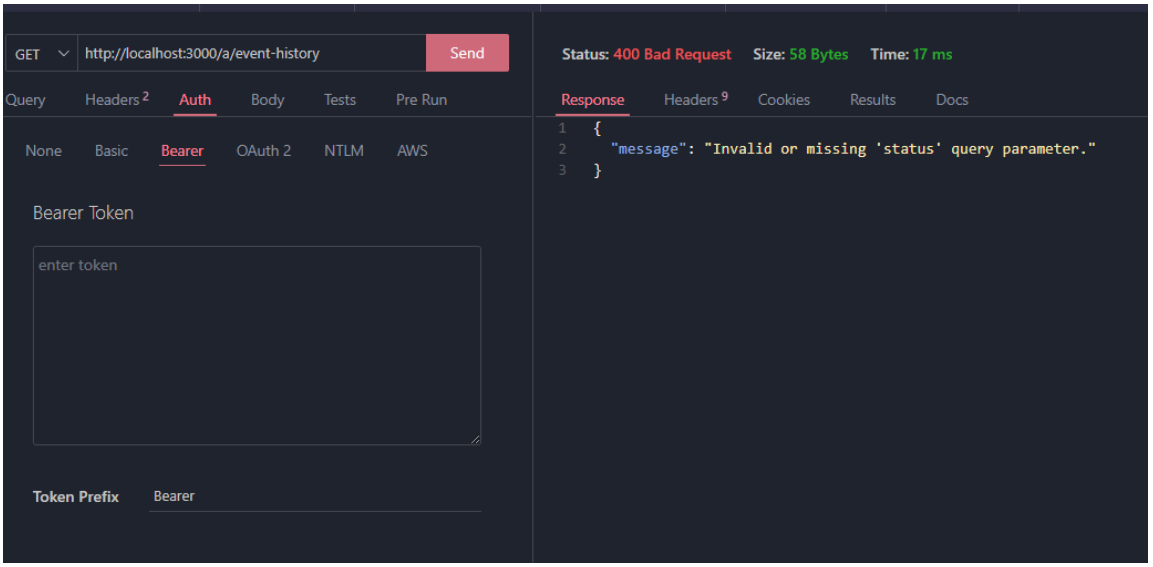


Figure 150. Failed to Retrieve the Completed Events Code 400

This figure shows a failed attempt to retrieve the completed events, with a response code of 400, meaning the request was invalid or there was an issue with the request parameters.

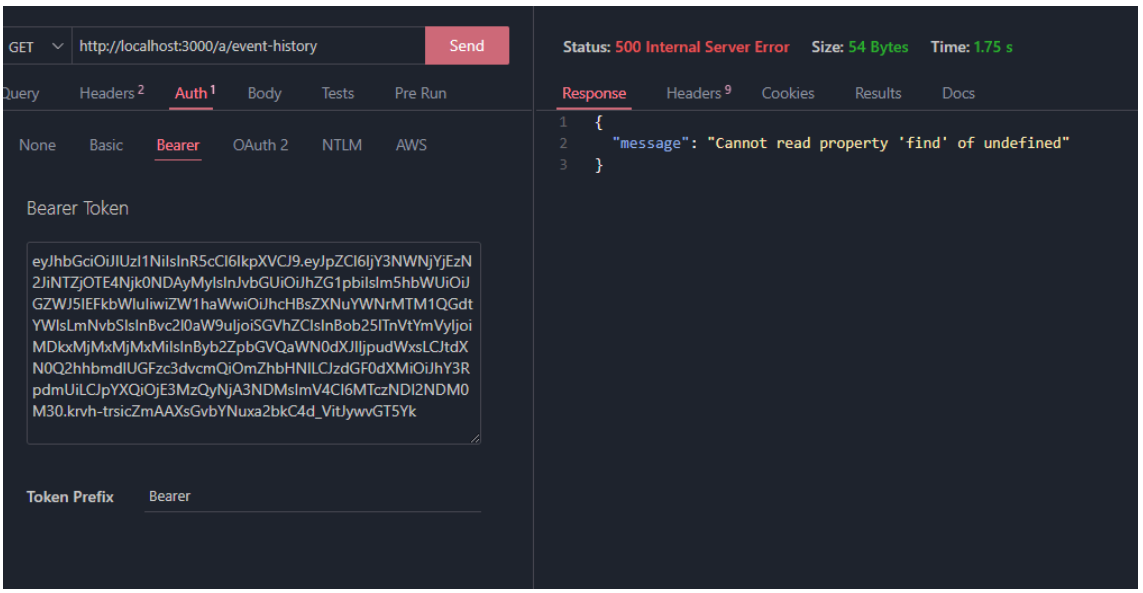


Figure 151. Server Error to Retrieve the Completed Events Code 500

This figure shows a server error while attempting to retrieve the completed events, with a response code of 500, indicating that an internal server error occurred during the request processing.

1.2.12 Retrieve User Registrations List

Version: 1.0

Date: November 25, 2024

Description: This API endpoint allows the retrieval of the list of user registrations for events. It fetches all user registrations from the database.

Endpoint: http://localhost:3000/a/event/registrations

Method: GET

Configurations: The API request should be validated using a token.

Parameters:

- **authToken**: A valid session token (JWT), included in the Authorization header, is required to authorize the request.

Response codes of this API endpoint:

Code	Message	Description
200	OK	The request was successful, and the list of registrations is returned.
403	Invalid Token	The request is invalid, possibly due to a missing or incorrect authToken.
500	Error retrieving events	An unexpected error occurred while retrieving the registrations.

Valid & Invalid Responses

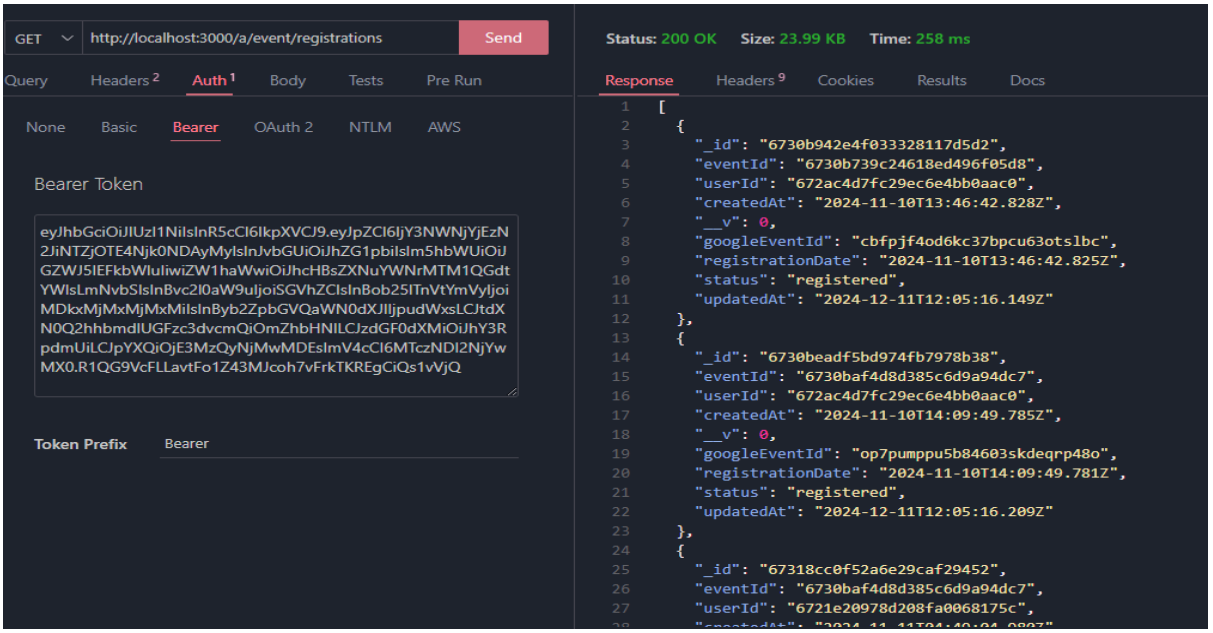


Figure 152. Valid GET Request for Retrieving All User Registrations

This figure illustrates a valid GET request made to retrieve all user registrations. The request is correctly formatted, allowing the system to successfully retrieve and return a list of all users who have registered for an event in the system.

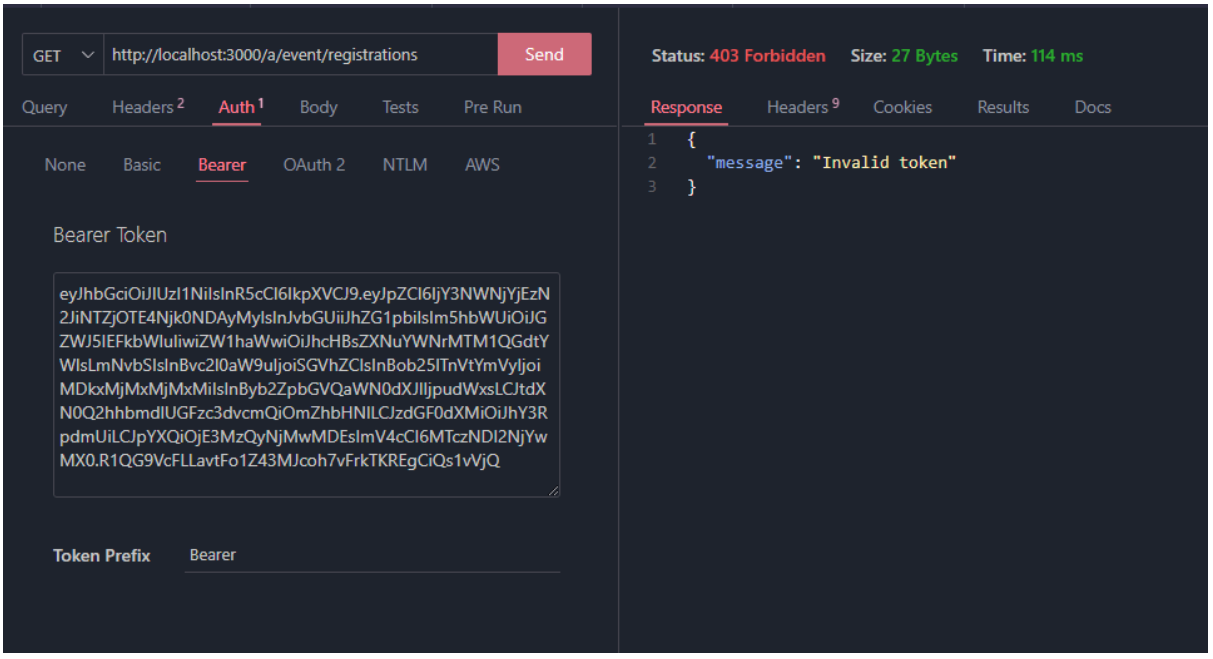


Figure 153. Invalid GET Request for Retrieving All User Registrations

This figure shows an invalid GET request for retrieving all user registrations. The request contains an invalid or missing authentication token, preventing the system from processing it and retrieving the list of user registrations.

Response Format: JSON

Response:

The API responds with a list of user registrations for events. On success, it includes user details like registration Id, event Id, user Id etc. Errors return an appropriate status code and message.

Different response Screenshots

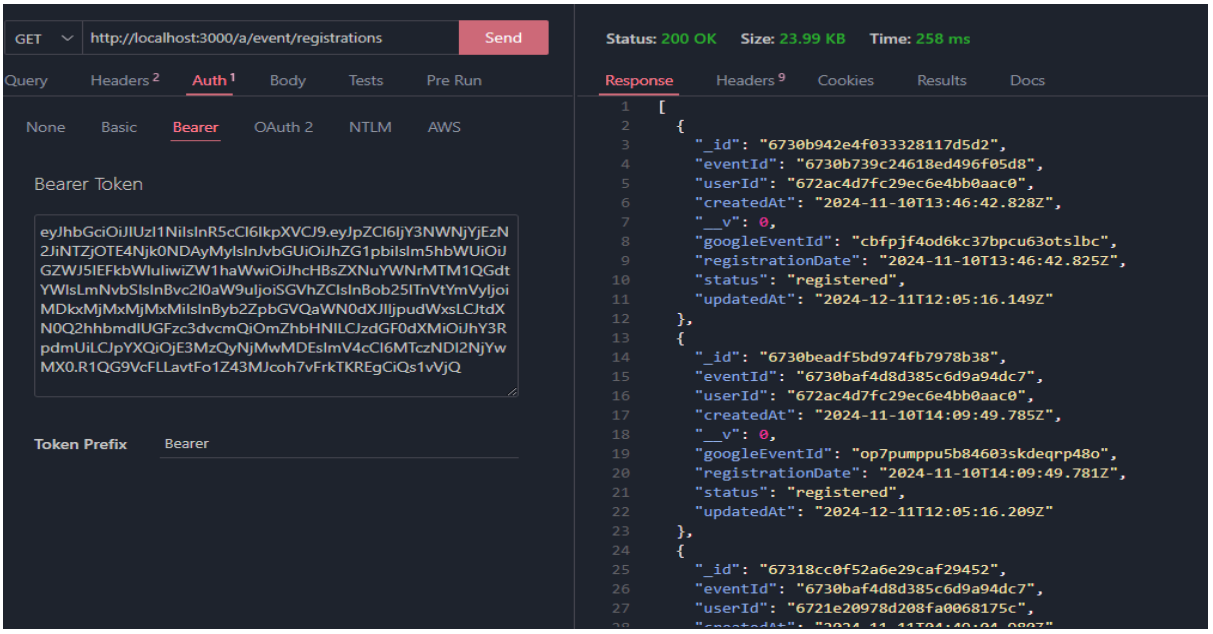


Figure 154. Successfully Retrieved All Users Registrations List Code 200

This figure shows a successful retrieval of the list of all user registrations, with a response code of 200. The request was processed successfully, and the system returned the list of all users who have registered for an event.

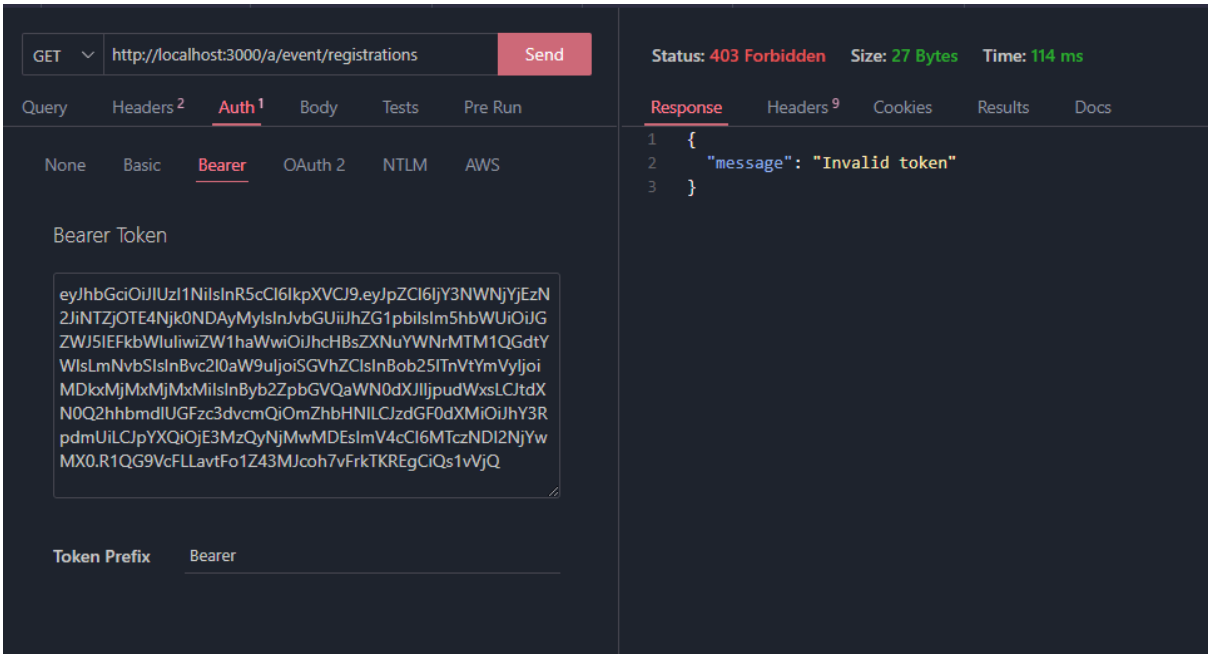


Figure 155. Failed to Retrieve All Users Registrations List Code 403

This figure shows a failed attempt to retrieve the list of all user registrations, with a response code of 403. This indicates that the request was forbidden, possibly due to insufficient permissions or an invalid token.

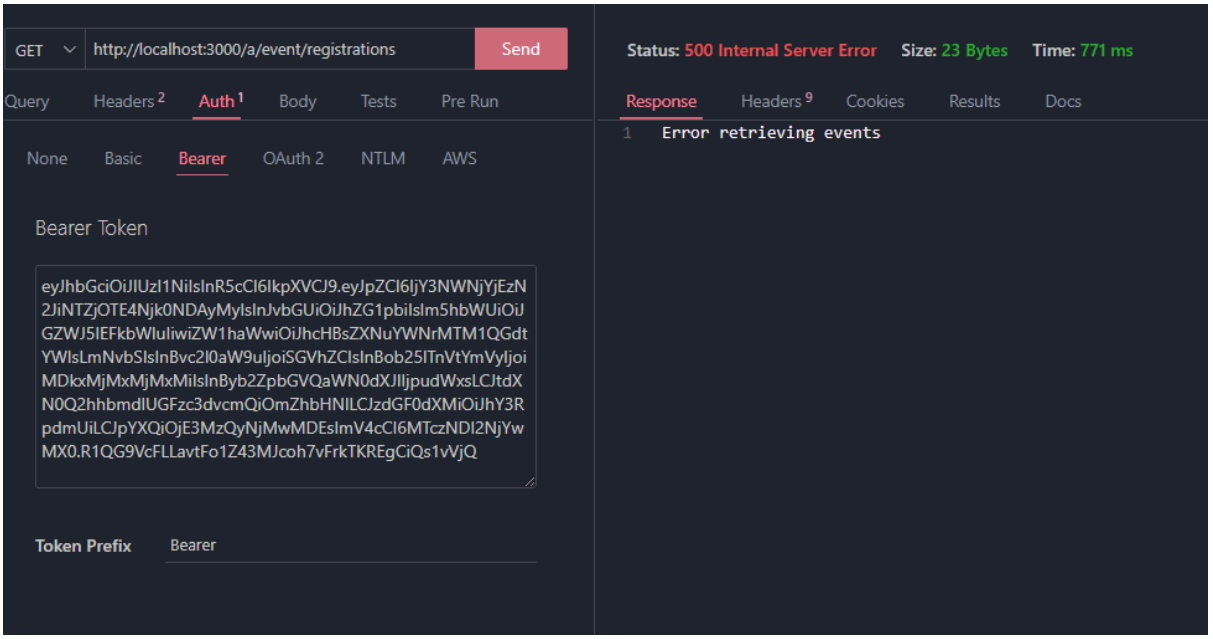


Figure 156. Server Error to Retrieve All Users Registrations List Code 500

This figure shows a server error while attempting to retrieve the list of all user registrations, with a response code of 500. An unexpected error occurred on the server while processing the request.

1.2.13 Retrieve Specific User Event Registrations

Version: 1.0

Date: November 25, 2024

Description: This API endpoint retrieves a list of users who have registered for a specific event. The request requires the event ID to identify the event and a valid session token for authorization.

Endpoint: http://localhost:3000/a/event/registrations/:id

Method: GET

Configurations: The API request should be validated using a token.

Parameters:

- **eventId:** The unique identifier of the event for which user registrations need to be retrieved.
- **authToken**: A valid session token (JWT), included in the Authorization header, is required to authorize the request.

Response codes of this API endpoint:

Code	Message	Description
200	OK	The request was successful, and the list of users who registered for the specific event is returned.
403	Invalid Token	The session token is missing or invalid, preventing access to the requested data.
404	No registrations found for this event.	No user registrations were found for the specific event.
500	Error fetching registered users	An unexpected error occurred while processing the request.

Valid & Invalid Responses

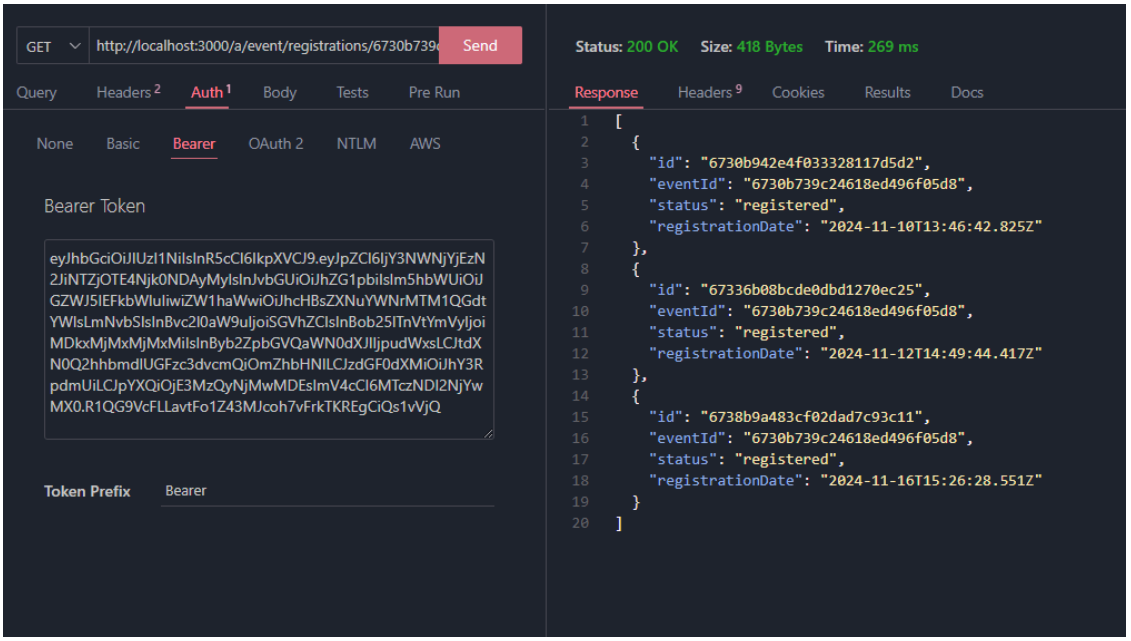


Figure 157. Valid GET Request for Retrieving User Registrations for the Event

This figure illustrates a valid GET request for retrieving the list of user registrations for a specific event. The request is properly formatted and successfully retrieves the registration data for the event from the system.

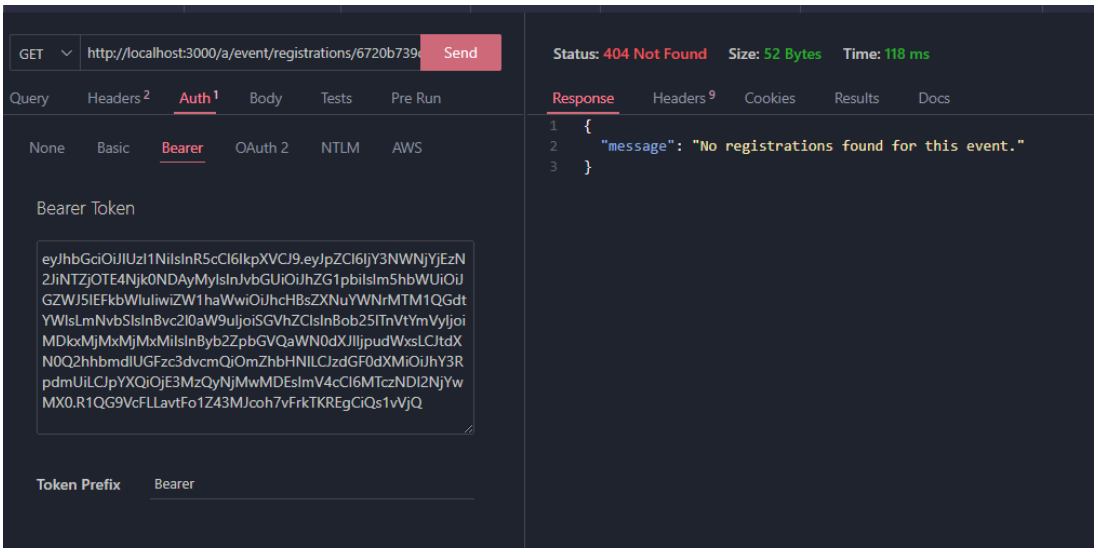


Figure 158. Invalid GET Request or Retrieving User Registrations for the Event

This figure shows an invalid GET request for retrieving user registrations for the event. The request fails because no registrations are found for the specified event. This can occur when there are no users registered for that event.

Response Format: JSON

Response:

The API responds with a list of users who have registered for the specific event. If successful, it includes details like registration id, user ID, event ID, registration status, and date. Errors return an appropriate status code and message.

Different response Screenshots

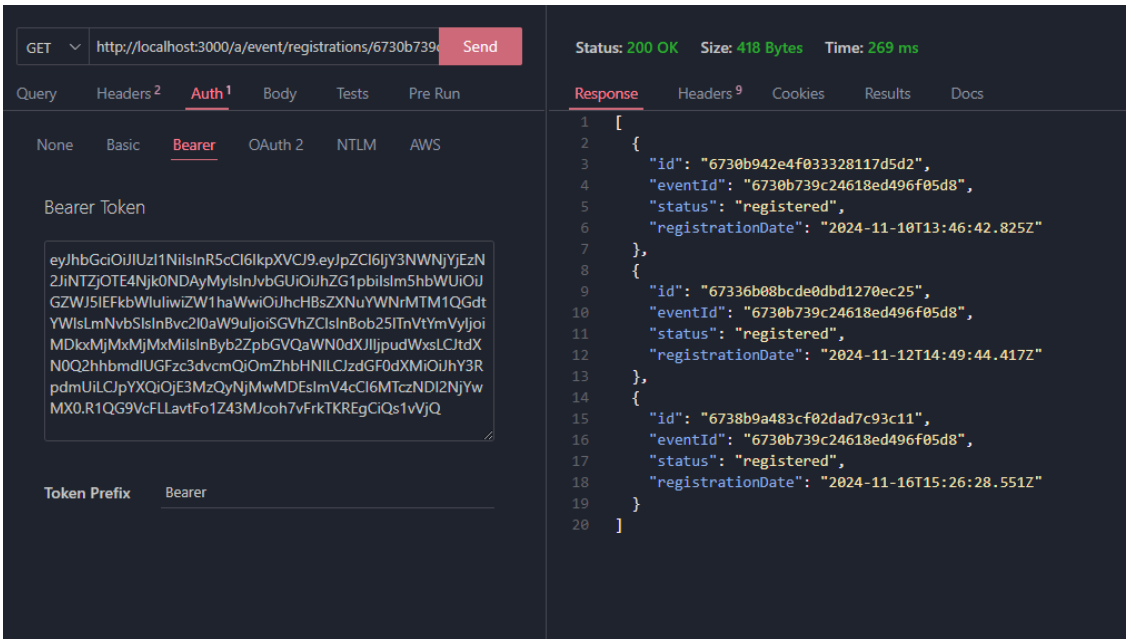


Figure 159. Successfully Retrieved the User Registrations List for that Event, Code 200

This figure shows a successful retrieval of the user registrations list for the event (Code 200), meaning the request worked correctly.

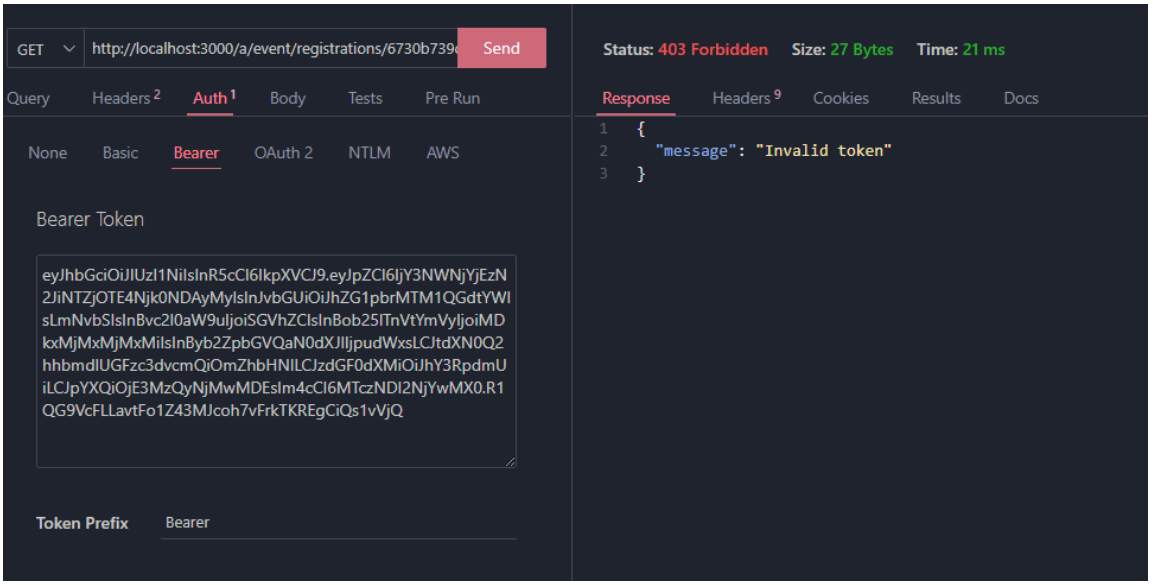


Figure 160. Failed to Retrieve the User Registrations List for that Event, Code 403

This figure shows a failure to retrieve the list due to insufficient permissions or an invalid token (Code 403).

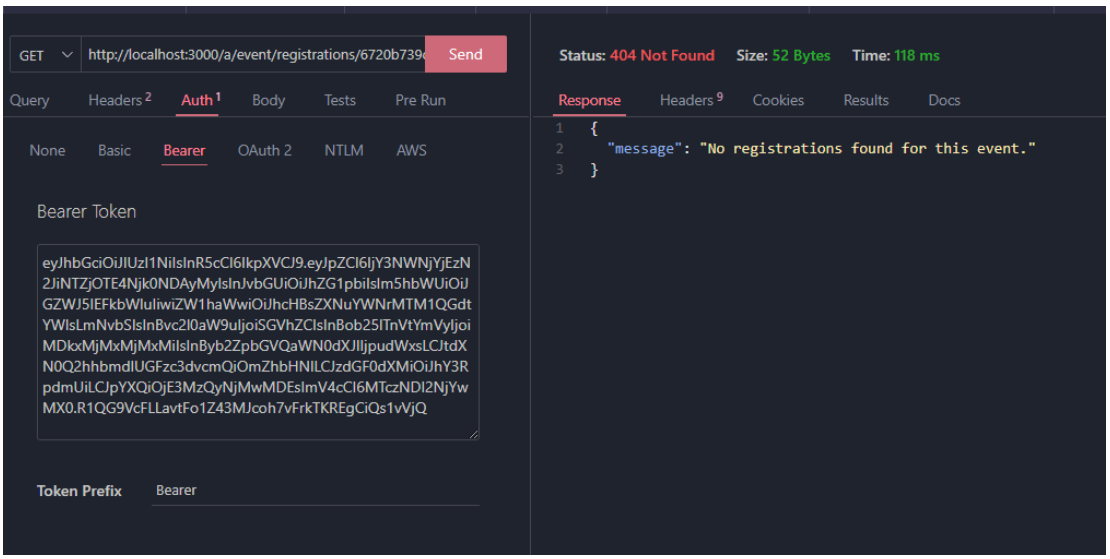


Figure 161. Failed to Retrieve the User Registrations List for that Event, Code 404

This figure shows a failure to retrieve the list because the event could not be found (Code 404).

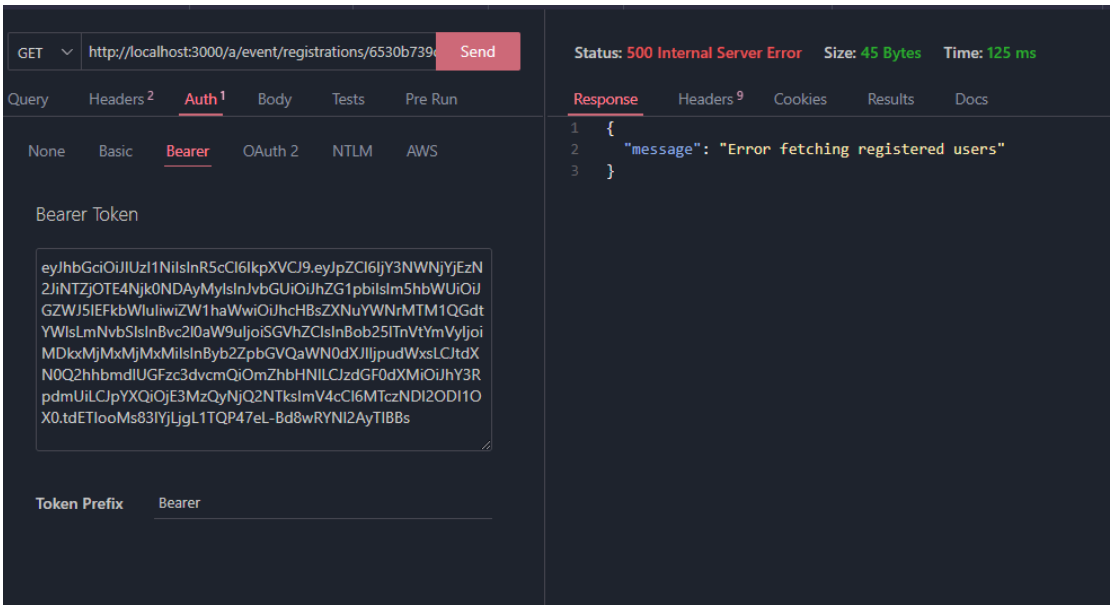


Figure 162. Server Error of Retrieving the User Registrations List for that Event, Code 500

This figure shows a server error while retrieving the list (Code 500), indicating an issue with the server.

1.2.14 Retrieve All Submitted Forms

Version: 1.0

Date: November 25, 2024

Description: This API endpoint retrieves all the form submissions from the database. It fetches the submitted forms for events.

Endpoint: http://localhost:3000/a/event/form-submissions

Method: GET

Configurations: The API request should be validated using a token.

Parameters:

- **authToken**: A valid session token (JWT), included in the Authorization header, is required to authorize the request.

Response codes of this API endpoint:

Code	Message	Description
200	OK	The request was successful, and the forms are returned.
403	Invalid Token	The request was denied due to an invalid session token.
500	Error Retrieving Events	An unexpected error occurred while retrieving the forms.

Valid & Invalid Responses

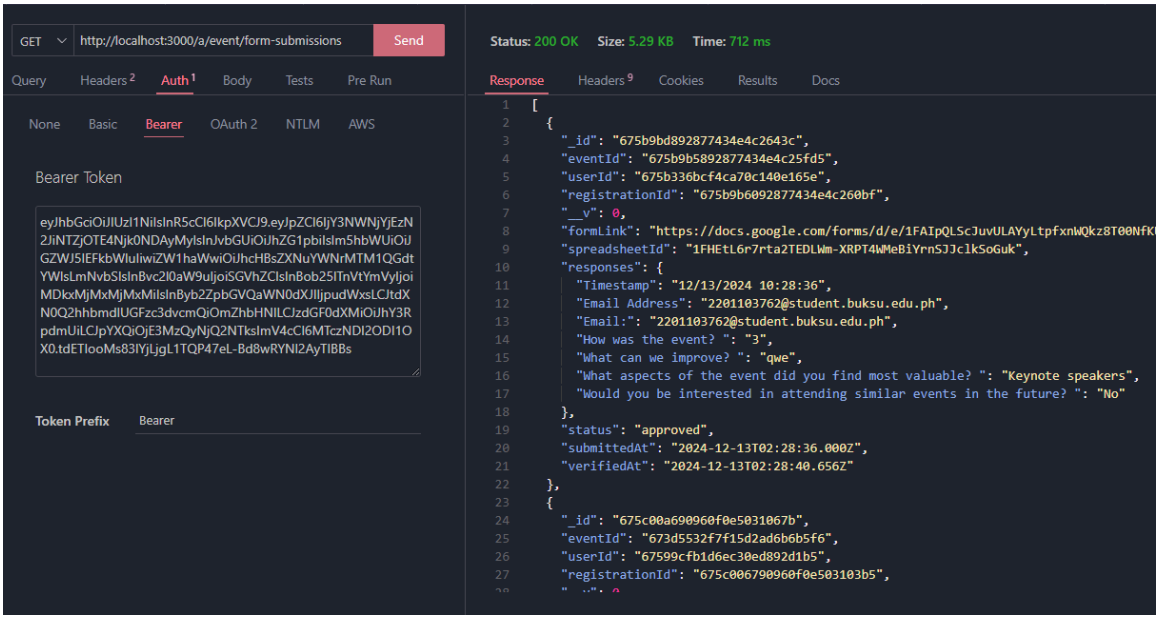


Figure 163. Valid GET Request for Retrieving All Event Submission List

This figure shows a valid GET request to retrieve the list of all event submissions. The request is successfully processed, and the list of submissions is returned.

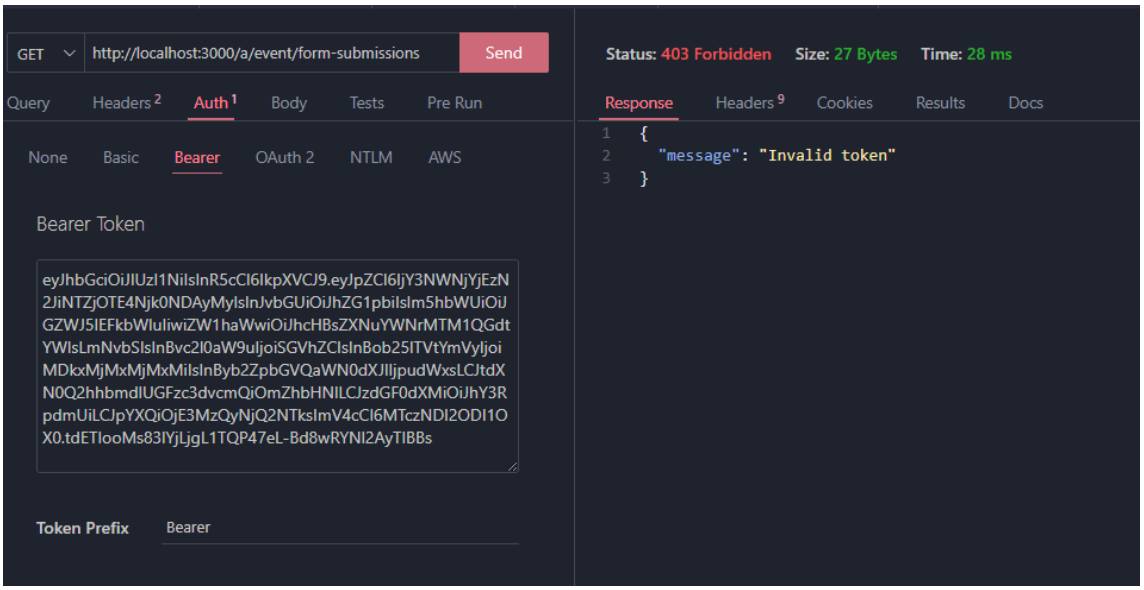


Figure 164. Invalid GET Request for Retrieving All Event Submission List

This figure shows an invalid GET request due to an invalid token. The request is rejected because the token used for authorization is not valid.

Response Format: JSON

Response:

The API returns a list of all submitted forms. If there's an issue with the token or server, an error message is displayed.

Different response Screenshots

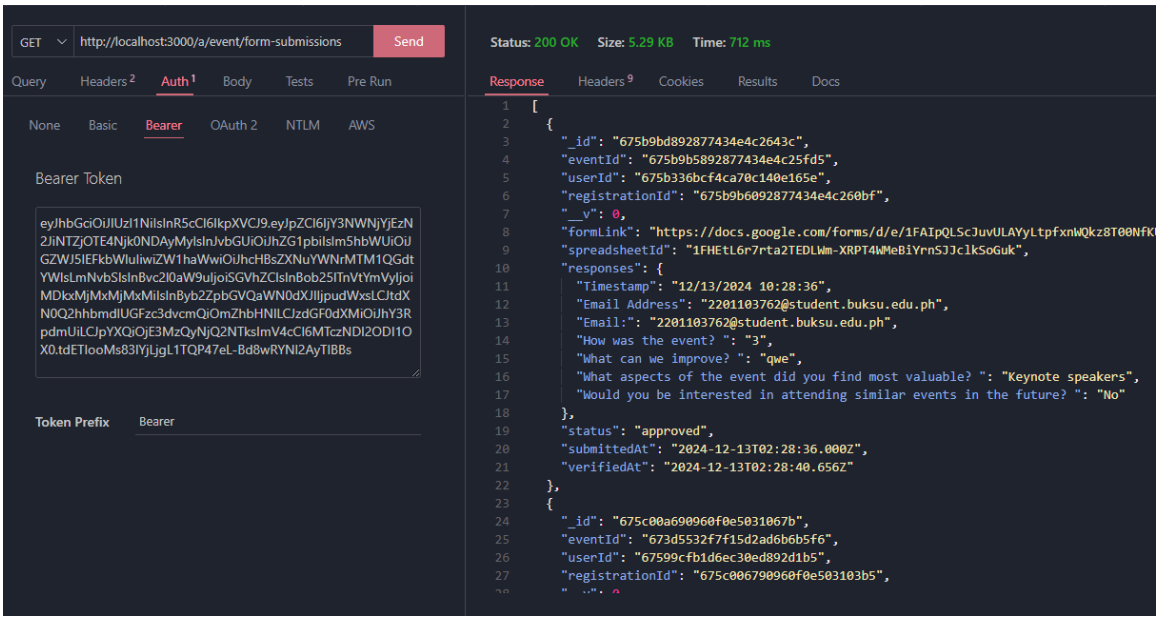


Figure 165. Successfully Retrieved All Event Submission List, Code 200

This figure shows that the request to retrieve the event submission list was successful, and the system returns the list with a status code of 200.

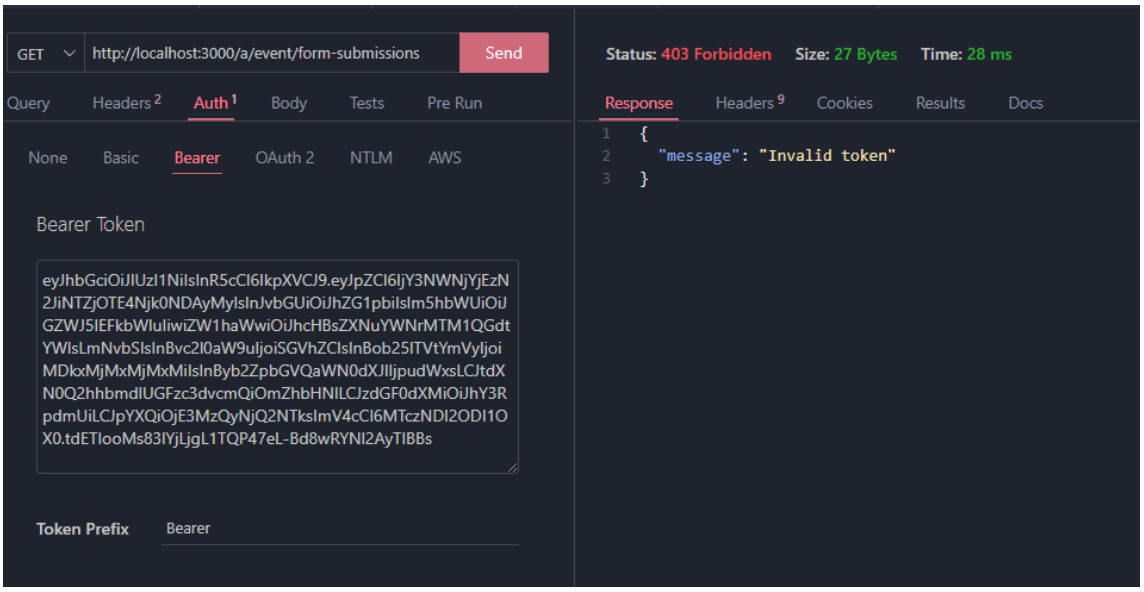


Figure 166. Failure Retrieval of All Event Submission List, Code 403

This figure shows that the request to retrieve the event submission list failed because the user did not have valid permissions or the session token was invalid, resulting in a status code of 403.

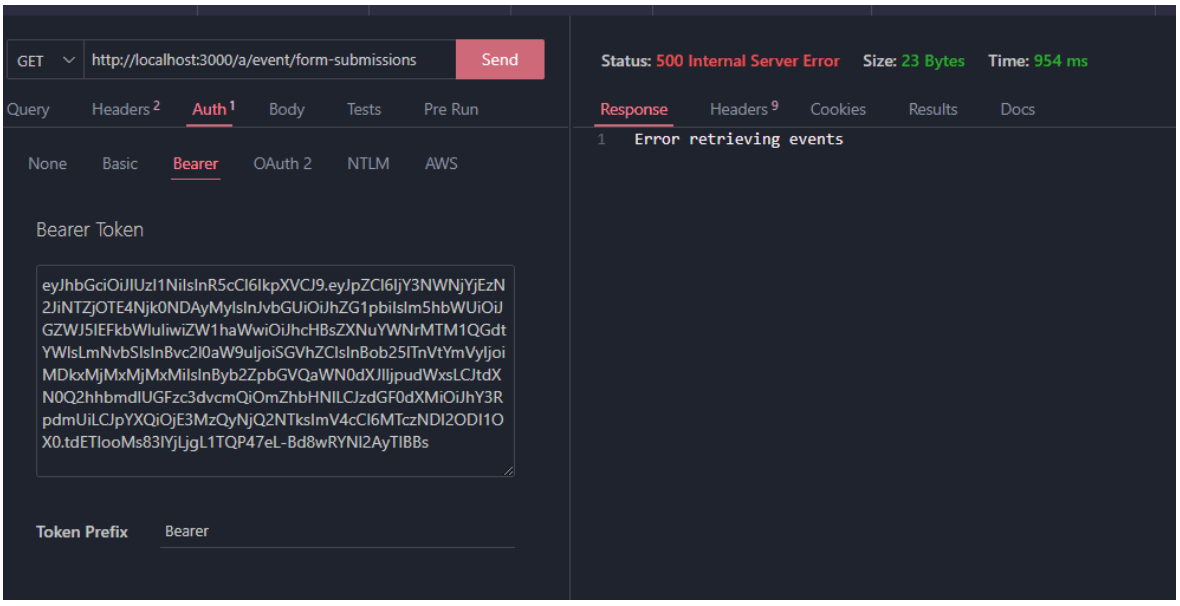


Figure 167. Server Error Retrieval of All Event Submission List, Code 500

This figure shows that there was a server error while trying to retrieve the event submission list, resulting in a status code of 500.

1.2.15 Retrieve All Submissions in a Specific Event

Version: 1.0

Date: November 25, 2024

Description: This API endpoint retrieves all submissions for a specific event. It requires the event ID to identify the event and a valid session token for authorization.

Endpoint: http://localhost:3000/a/event/form-submissions/:id

Method: GET

Configurations: The API request should be validated using a token.

Parameters:

- **eventId:** The unique identifier of the event whose submissions need to be retrieved.
- **authToken**: A valid session token (JWT), included in the Authorization header, is required to authorize the request.

Response codes of this API endpoint:

Code	Message	Description
200	OK	The list of submissions for the specific event is successfully retrieved.
200	No forms found in this event []	Successful request, but no submissions found for that event.
403	Invalid Token	The provided token is invalid or missing.
500	An error occurred while fetching forms.	An error occurred on the server while retrieving the submissions.

Valid & Invalid Responses

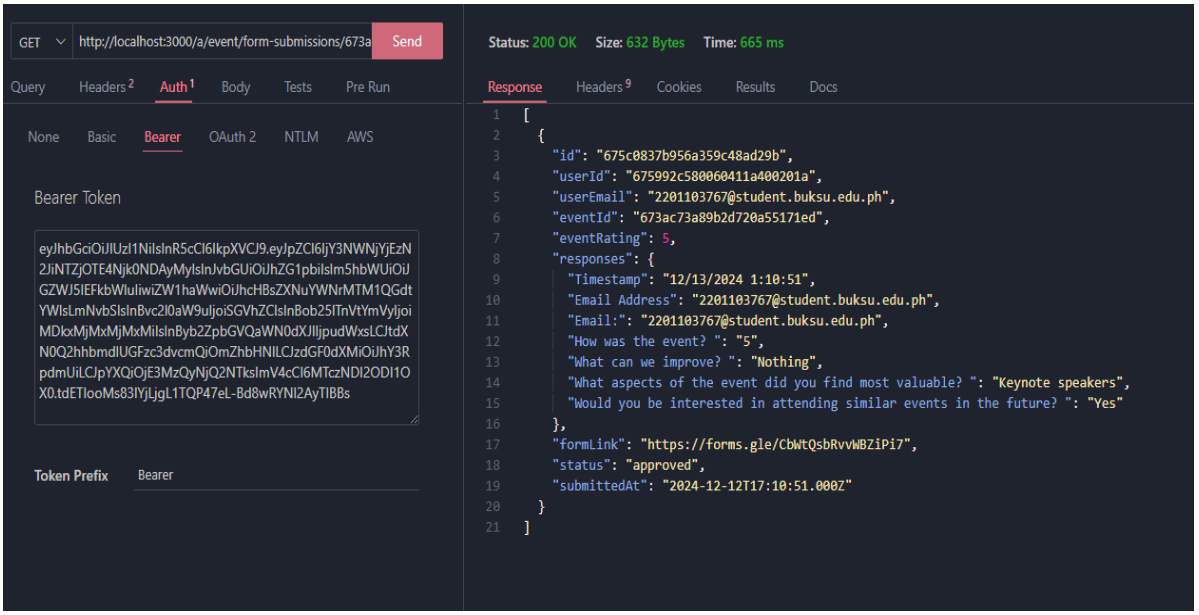


Figure 168. Valid GET Request for Retrieving User Submission Responses for the Event

This figure shows a successful GET request that retrieves the submission responses from users for a specific event. The request is properly formatted, and the system returns the list of responses.

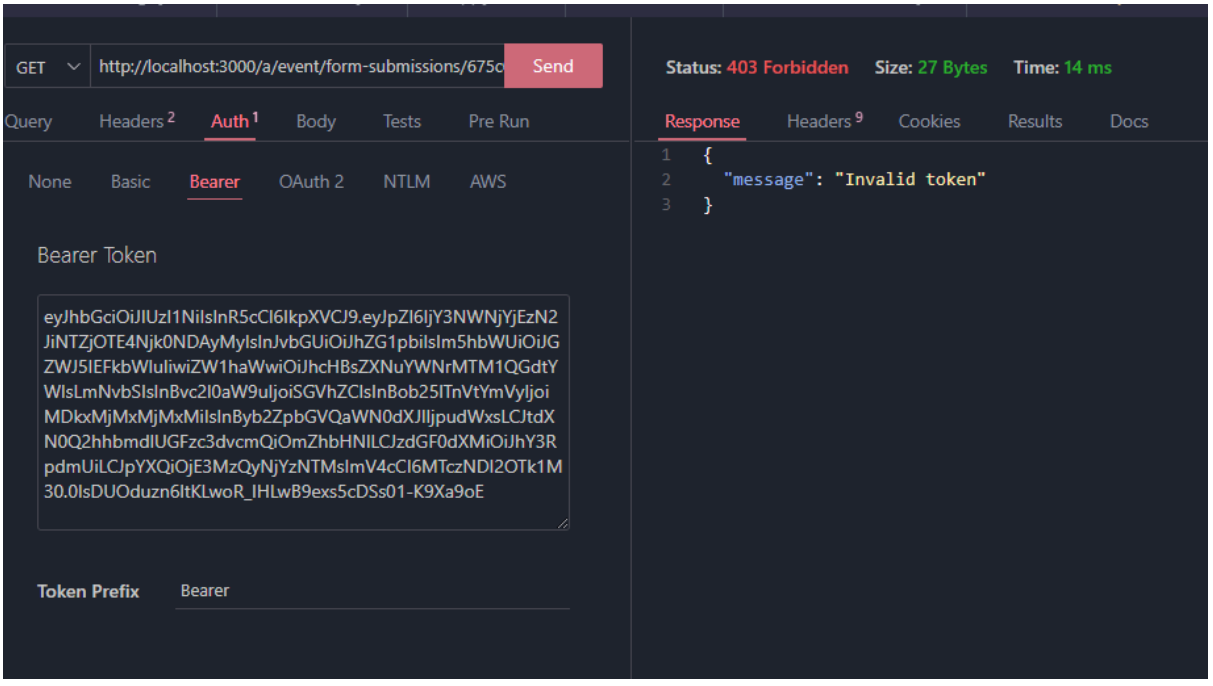


Figure 169. Invalid GET Request for Retrieving User Submission Responses for the Event

This figure shows an invalid GET request for the same purpose, which fails due to an error, such as an incorrect token or missing data.

Response Format: JSON

Response:

The API returns all form submissions for a specific event. If no submissions are found, it returns a 404 status. If successful, it includes details like the user's name, email, event rating, and submission date.

Different response Screenshots

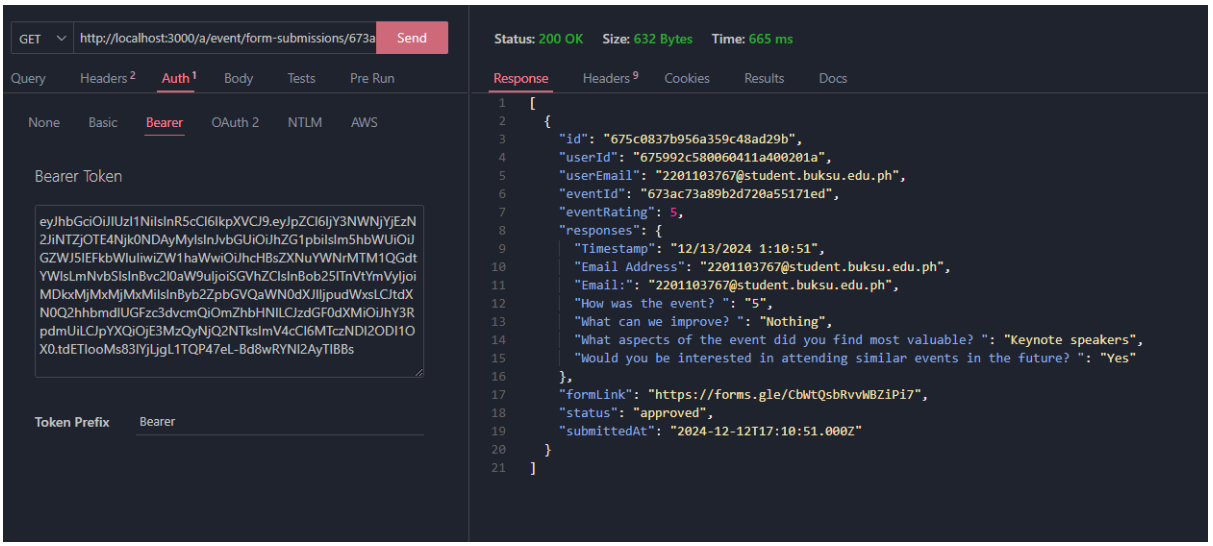


Figure 170. Successful Retrieval of Response Submissions with a List of Responses, Code 200.

This figure shows a successful request where all response submissions for an event are retrieved and returned in a list.

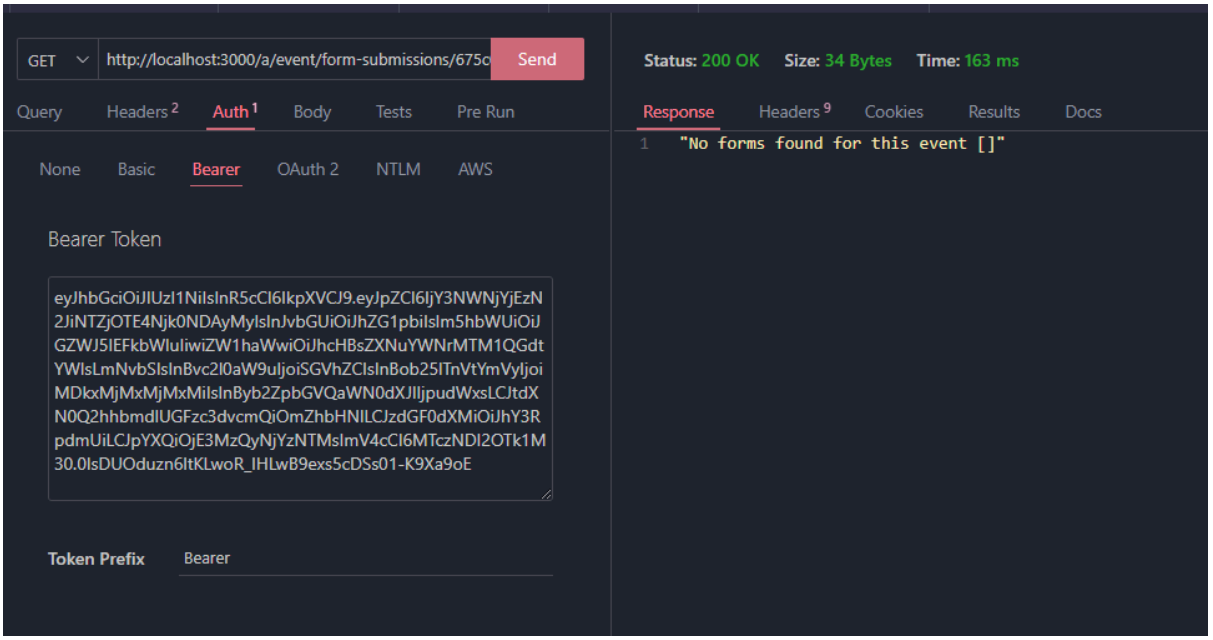


Figure 171. Successful Request, but No Submission Responses Found (Code 200).

This figure shows a successful request, but no submission responses are found for the event. The response is an empty list.

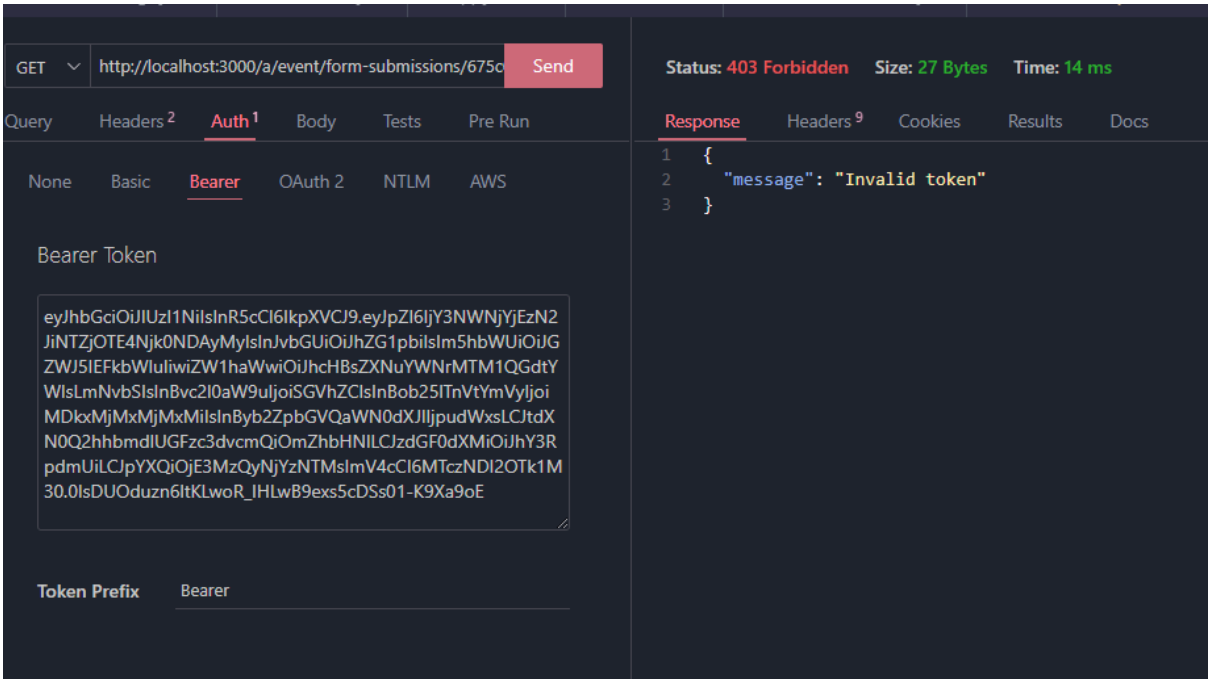


Figure 172. Failure to Retrieve Response Submissions, Code 403

This figure shows a failed request due to an invalid or missing token, which prevents access to the response submissions.

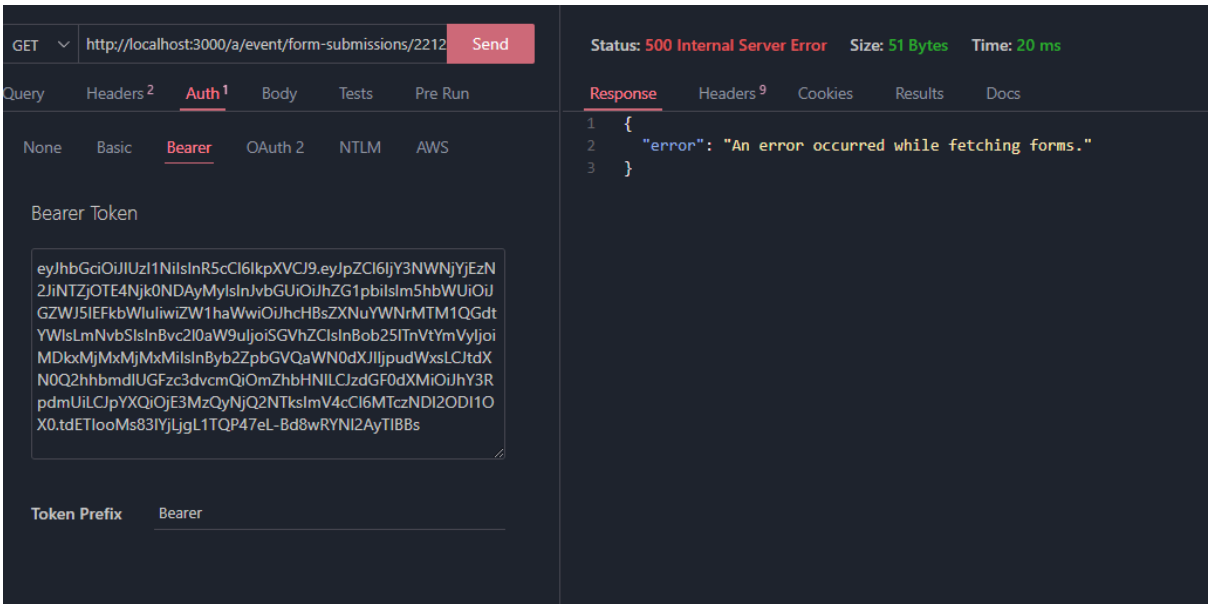


Figure 173. Server Error in Retrieving Response Submissions, Code 500

This figure shows a server error when attempting to retrieve response submissions, indicating an issue with the server.

1.2.16 Retrieving Admin Dashboard Data

Version: 1.0

Date: November 25, 2024

Description: This API endpoint retrieves the data required for the admin dashboard. It is designed to provide administrators with an overview of the system's data and statistics. The request requires a valid session token for authorization.

Endpoint: http://localhost:3000/a/dashboard

Method: GET

Configurations: The API request should be validated using a token.

Parameters:

- **authToken**: A valid session token (JWT), included in the Authorization header, is required to authorize the request.

Response codes of this API endpoint:

Code	Message	Description
200	OK	Successfully retrieves the admin dashboard data.
403	Invalid Token	The session token is invalid or missing. Access is denied.
500	message: error.message	A server error occurred. The error message will be provided.

Valid & Invalid Responses

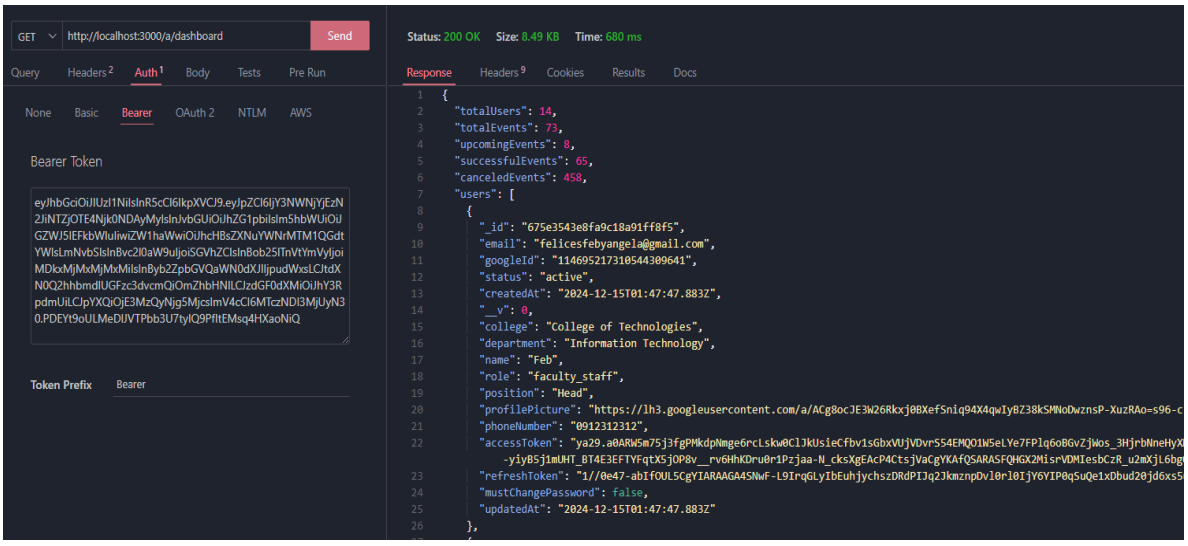


Figure 174. Valid GET Request for Retrieving Dashboard Statistics Data

This shows a valid GET request made to retrieve the dashboard statistics data. The request is correctly formatted, and the server responds with the appropriate data.

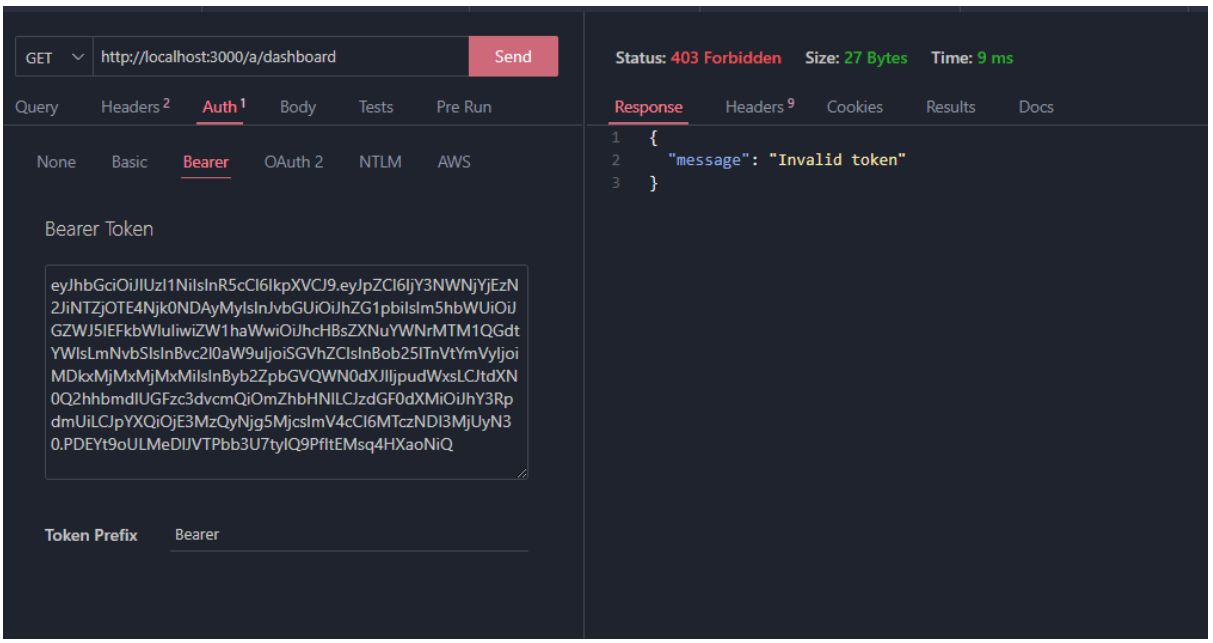


Figure 175. Invalid GET Request for Retrieving Dashboard Statistics Data

This shows an invalid GET request for retrieving dashboard data. The request is either missing a valid token or is incorrectly formatted, so the server doesn't return the data.

Response Format: JSON

Response:

The API response for retrieving admin dashboard data depends on the outcome of the request.

Different response Screenshots

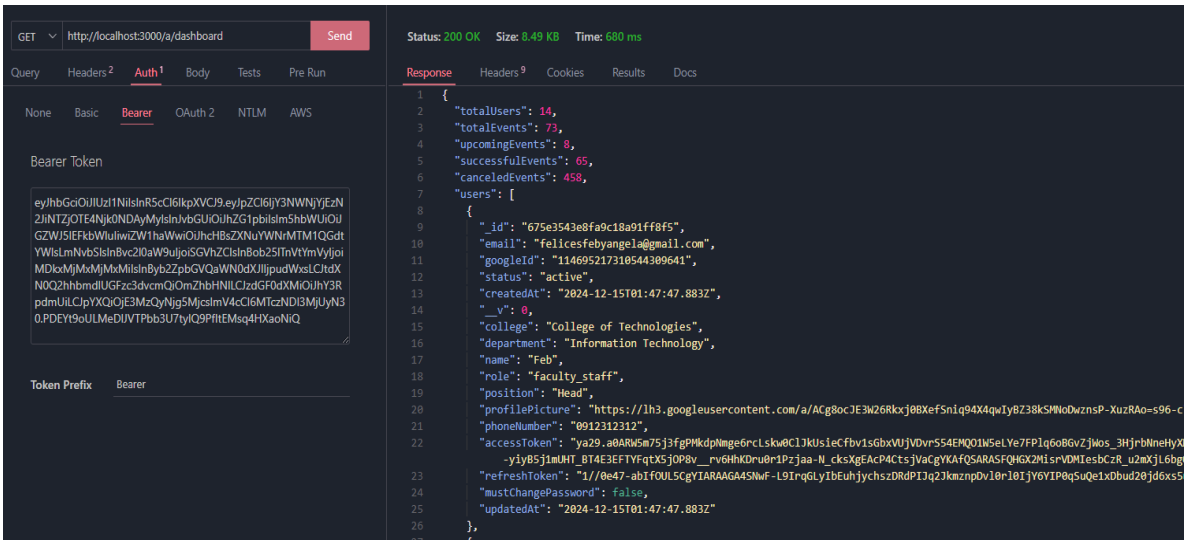


Figure 176. Successful Retrieval of Dashboard Data (Code 200)

This figure shows a successful request to retrieve dashboard data. The response code 200 indicates that the data was fetched successfully without any issues.

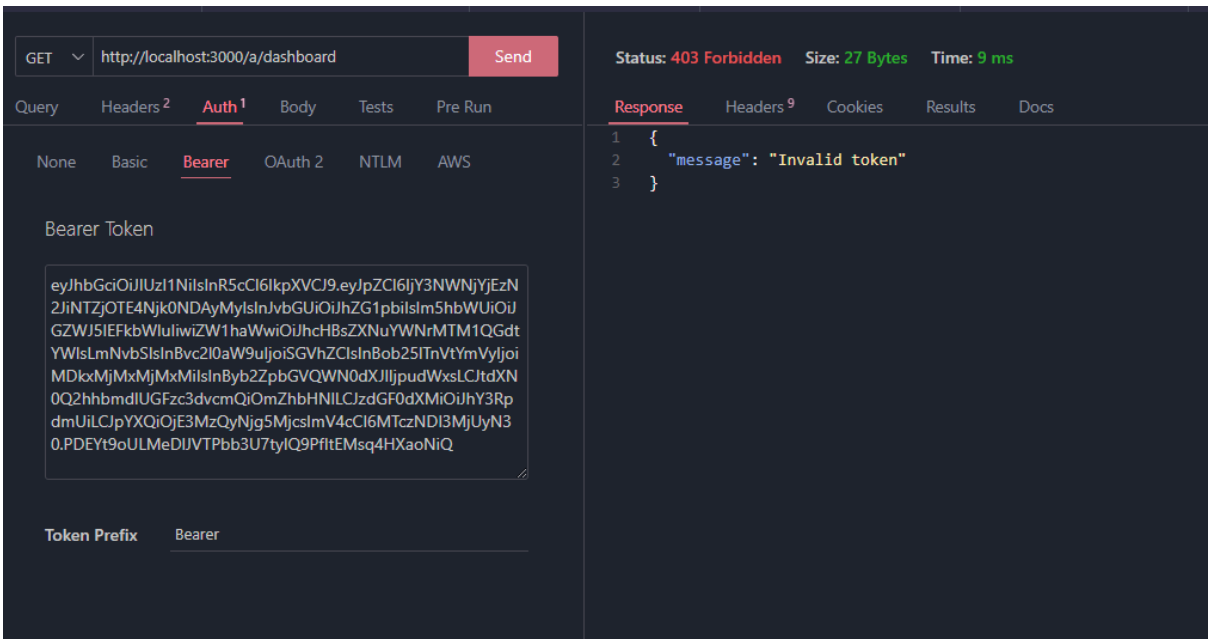


Figure 177. Failure Retrieval of Dashboard Data (Code 403)

This figure represents a failed request due to an authorization issue. The response code 403 means that the user does not have permission to access the requested dashboard data.

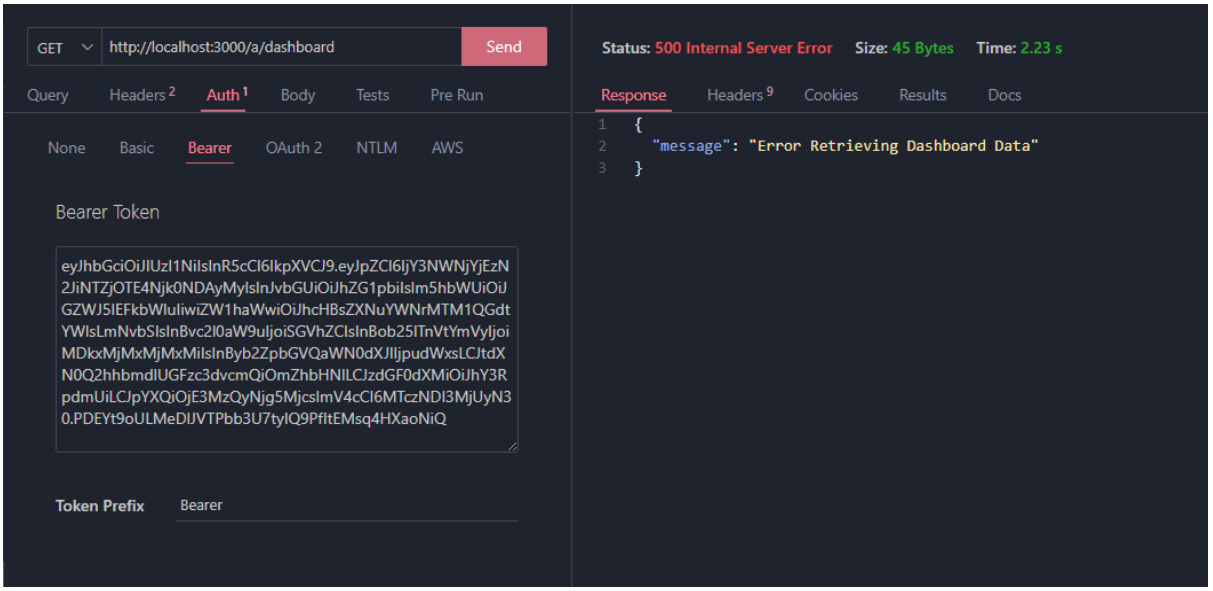


Figure 178. Server Error in Retrieving Dashboard Data (Code 500)

This figure shows a server-side error while trying to retrieve the dashboard data. The response code 500 indicates that something went wrong on the server, preventing the data from being fetched.

1.2.17 Retrieving Activity Logs

Version: 1.0

Date: November 25, 2024

Description: This API endpoint retrieves a list of activity logs from the system. The logs include user actions, such as login, form submissions, and other significant system activities. Each log entry includes the timestamp and associated user information, which is populated with user details like name, email, and profile picture. The request requires a valid session token for authorization to ensure secure access.

Endpoint: http://localhost:3000/a/activity-logs

Method: GET

Configurations: The API request should be validated using a token.

Parameters:

- **authToken**: A valid session token (JWT), included in the Authorization header, is required to authorize the request.

Response codes of this API endpoint:

Code	Message	Description
200	OK	The request was successful, and activity logs have been retrieved.
403	Invalid Token	The user does not have the required permissions to access the activity logs.
500	message: error.message	An unexpected error occurred while processing the request.

Valid & Invalid Responses

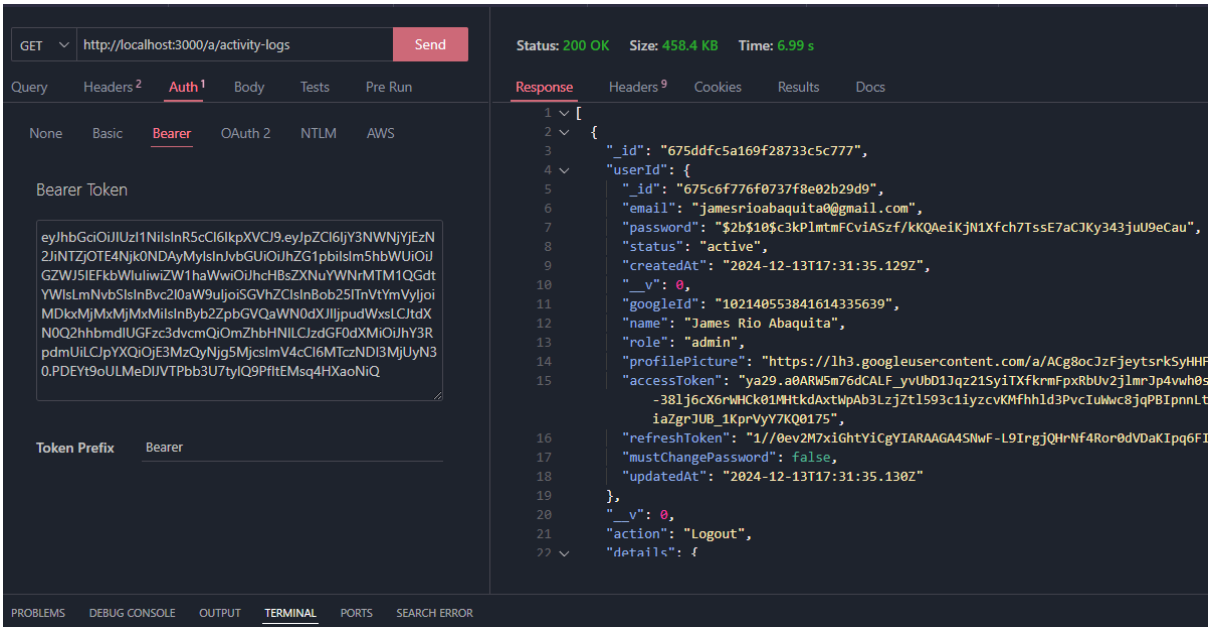


Figure 179. Valid GET Request for Retrieving Activity Logs

A valid GET request is made to retrieve the activity logs, and the server successfully returns the requested data.

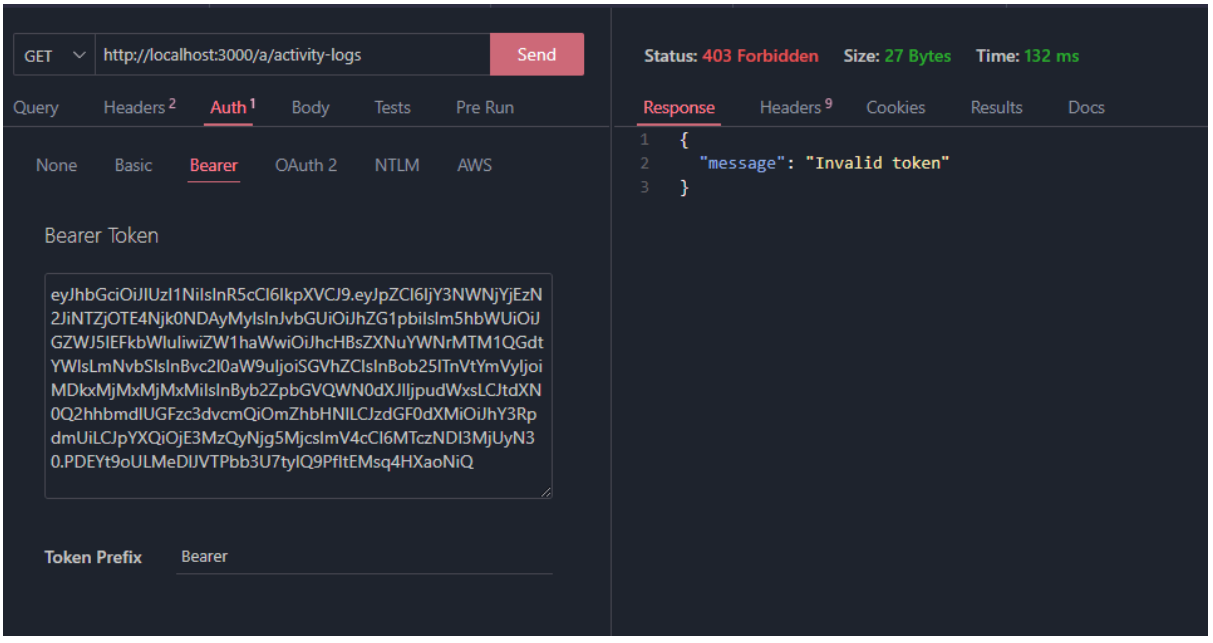


Figure 180. Invalid GET Request for Retrieving Activity Logs

An invalid GET request is made to retrieve the activity logs due to an invalid token, and the server responds with an error, denying access.

Response Format: JSON

Response:

The API response returns the requested activity logs or an error message, depending on the request's success or failure.

Different response Screenshots

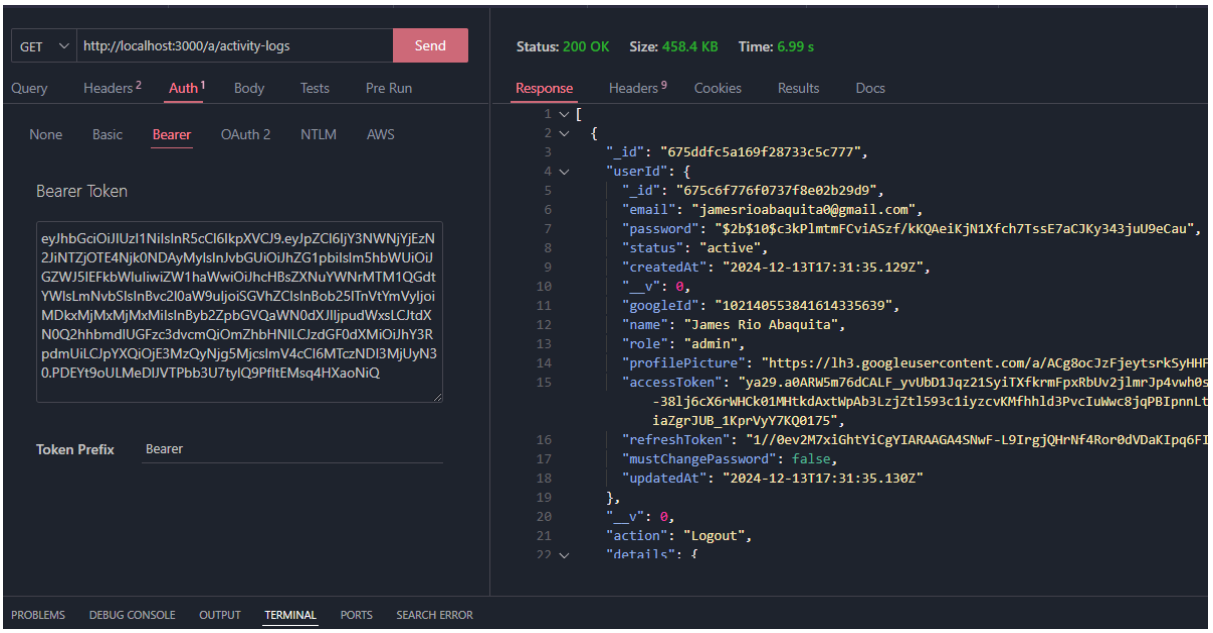


Figure 181. Successful Retrieval for Retrieving Activity Logs (Code 200)

The activity logs are successfully retrieved, and the server responds with the requested data.

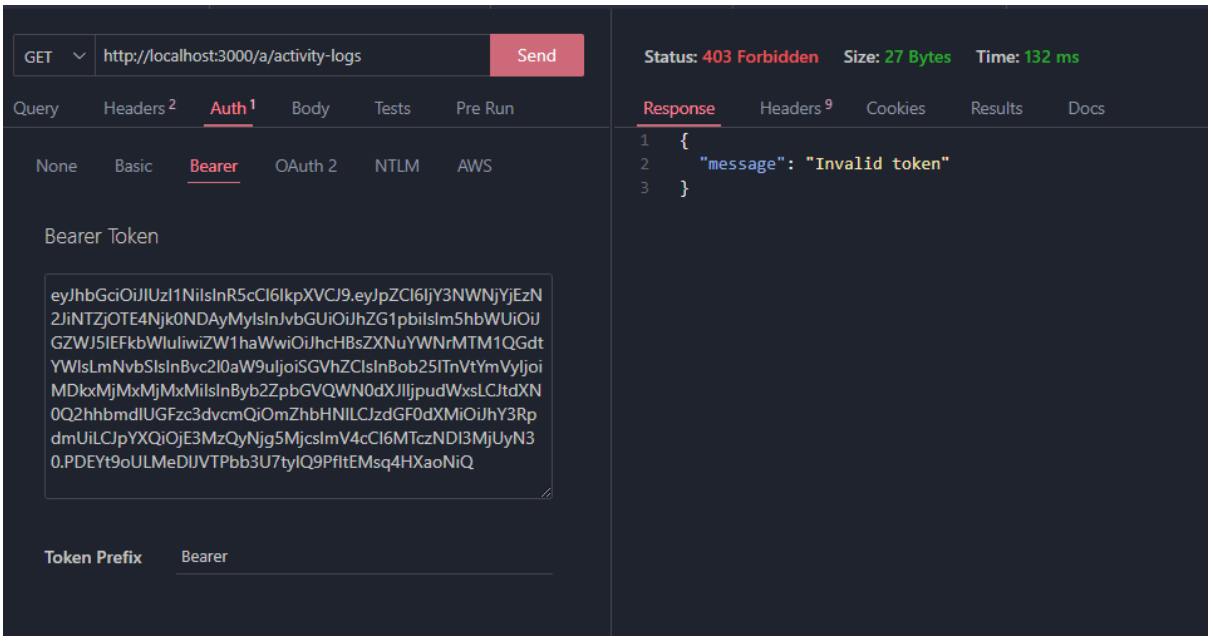


Figure 182. Failure Retrieval for Retrieving Activity Logs(Code 403)

The retrieval of activity logs fails due to insufficient permissions or invalid authorization.

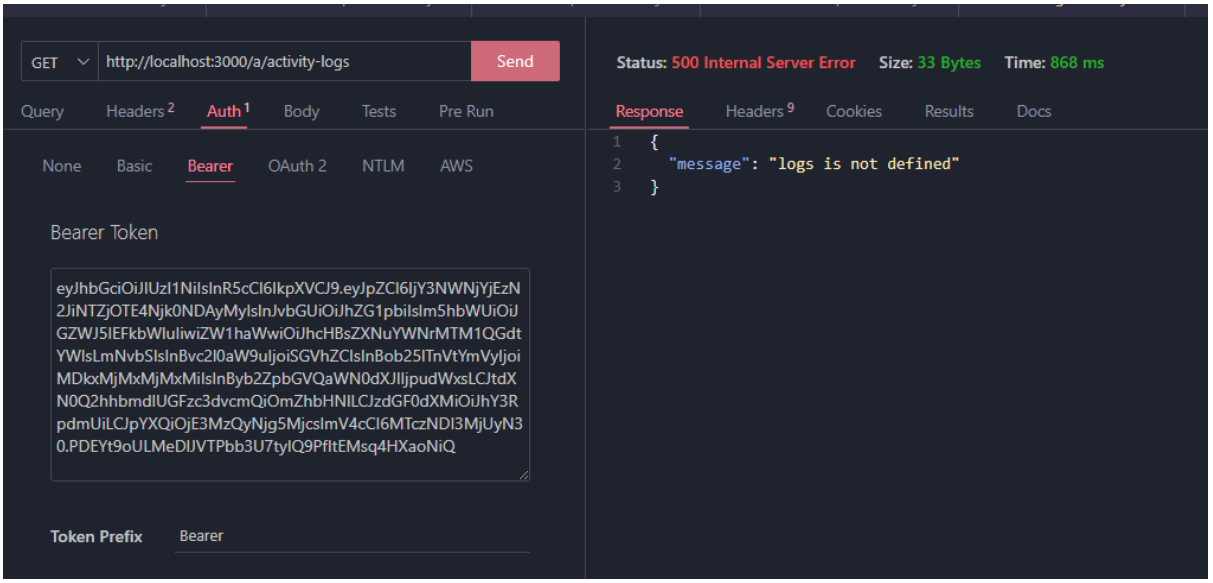


Figure 183. Server Error in Retrieving Activity Logs (Code 500)

A server error occurs while attempting to retrieve activity logs, preventing the request from being completed.

1.3 AUTHENTICATION MODULE API

1.3.1 Module Description

This section provides a comprehensive exploration of the routes and APIs associated with the authentication of the accounts within the University Training and Seminar Management System. It takes to define the functionalities, access points, and operational workflows that allows the users to efficiently access the system.

The API have the following endpoints:

Google Login Endpoints

http://localhost:3000/auth/google
http://localhost:3000/auth/google/callback

Manual Login Endpoints

http://localhost:3000/auth/login
http://localhost:3000/auth/logout
http://localhost:3000/auth/change-password

1.3.2 Retrieve User Google Account

Version: 1.0

Date: November 1, 2024

Description: This API endpoint will get the users google account using Oauth if they wish to login into the system using their Google Account.

Endpoint: http://localhost:3000/auth/google

Method: GET

Configurations: The API request should be validated using a token.

Parameters:

- **authToken -** A valid session token (JWT), included in the Authorization header, is required for secure access.

Response codes of this API:

Code	Message	Description
200	Request Successful (OK)	The request was valid and successfully retrieved users Google Account
404	User Not found	The request was invalid, or User's Google Account was invalid
500	Authentication Failed	The server encountered an error

Valid and Invalid Request Responses

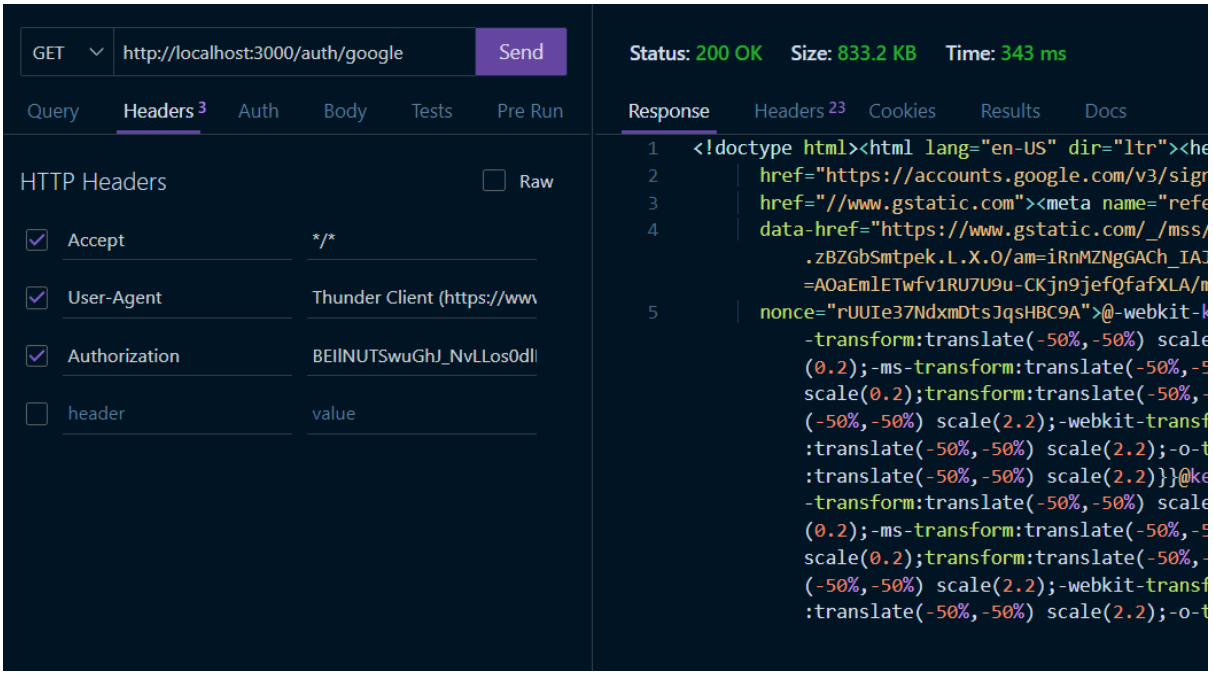


Figure 184. Valid GET Request for Retrieving User Google Account

This figure shows a valid GET request code 200 in retrieving the User's Google Account in the system.

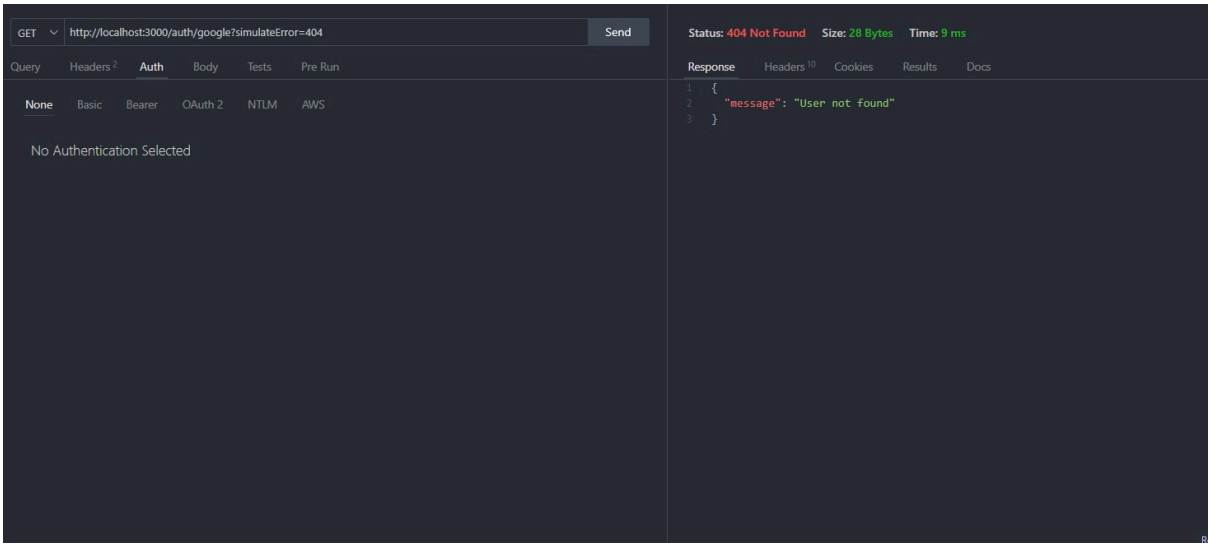


Figure 185. Invalid GET Request for Retrieving User Account

This figure displays an invalid GET Request with a request code of 404. This indicates that the requested resource was not found in the system database.

Different response Screenshots

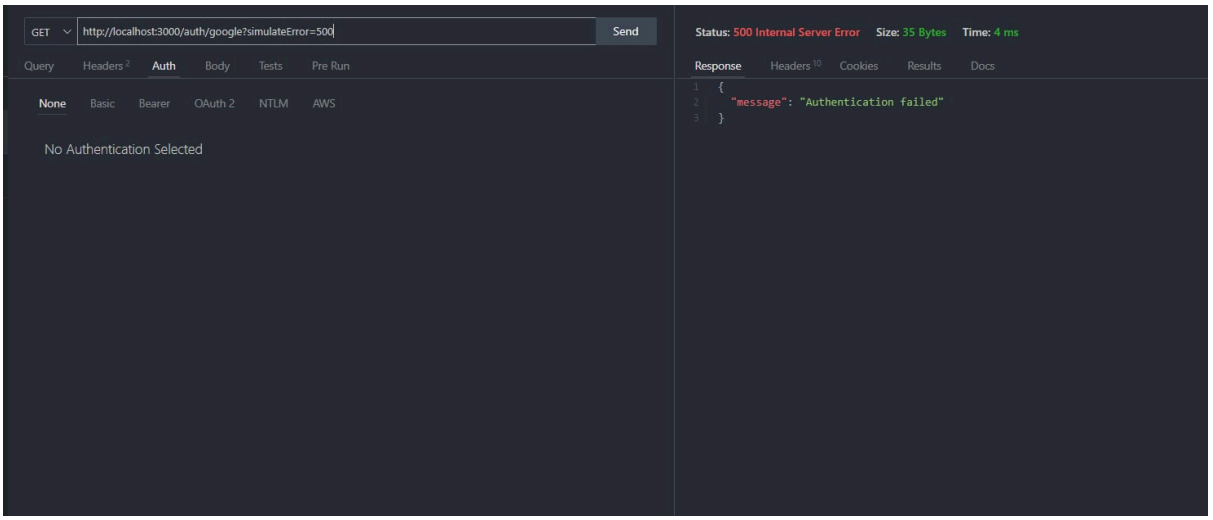


Figure 186. Server Failure to Authenticate Google Account (Status Code 500)

This figure displays a server-side error that shows there was a server error in logging in the user's Google Account. Status Code 500 is returned due to Internal Server Error.

1.3.3 Verify User Google Account

Version: 1.0

Date: November 1, 2024

Description: This API endpoint will validate whether the Google account being retrieved is valid or not.

Endpoint: http://localhost:3000/auth/google/callback

Method: GET

Configurations: The API request should

Parameters: No authentication or session token is required, and no query parameters are needed to retrieve all events.

Response codes of this API:

Code	Message	Description
200	Request Successful (OK)	The request was valid and successfully retrieved users Google Account
404	User Not found	The request was invalid. The User's Google Account was invalid
500	Authentication Failed	The server encountered an error

Valid and Invalid Request Responses

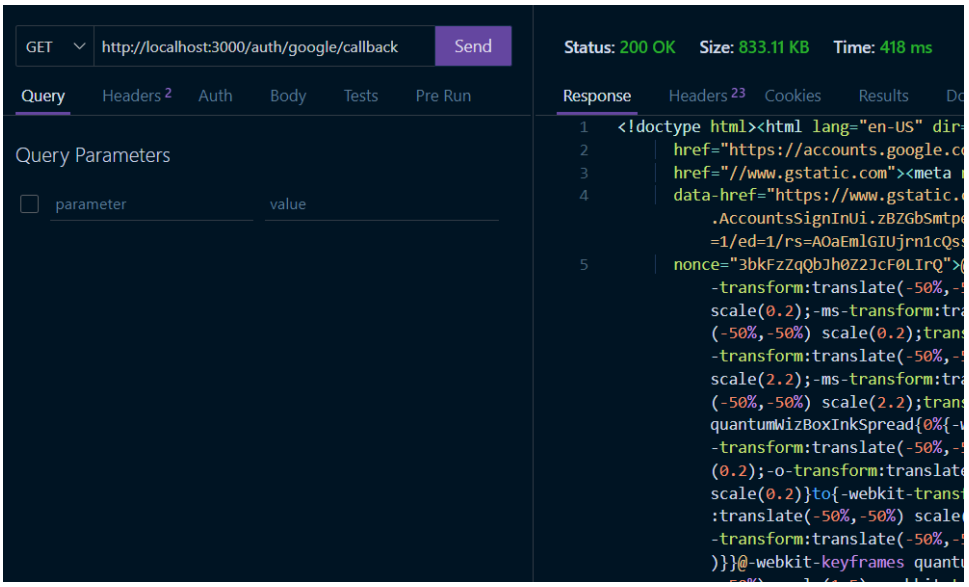


Figure 187. Valid GET Request for Retrieving User Google Account

This figure shows a valid GET request code 200 in retrieving the User's Google Account in the system.

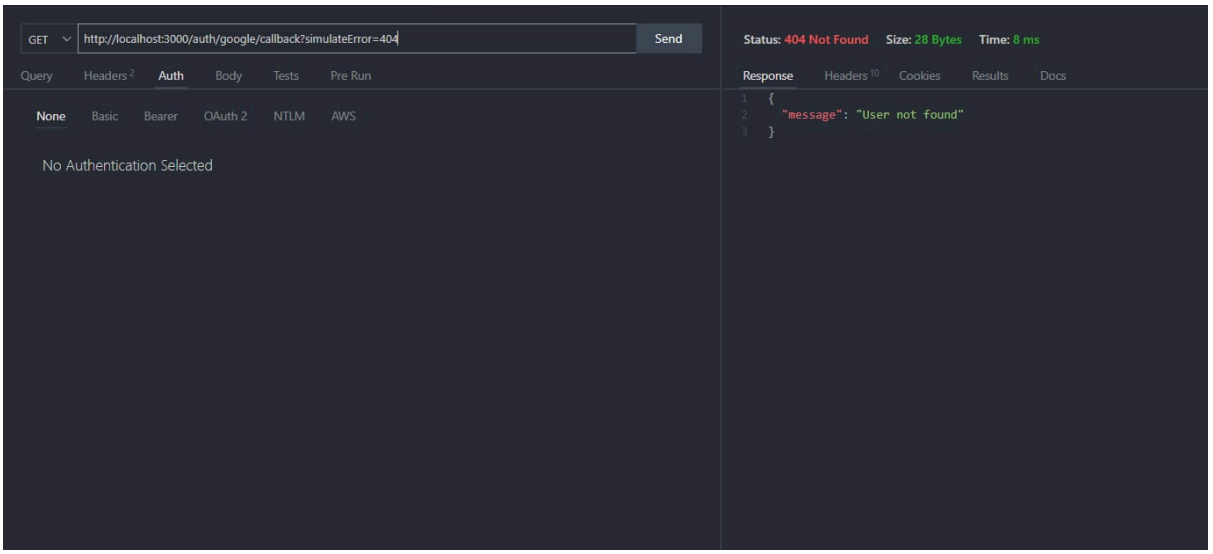


Figure 188. Invalid GET Request for Retrieving User Google Account

This figure displays an invalid GET Request with a request code of 404. This indicates that the requested resource was not found in the system database.

Different response Screenshots

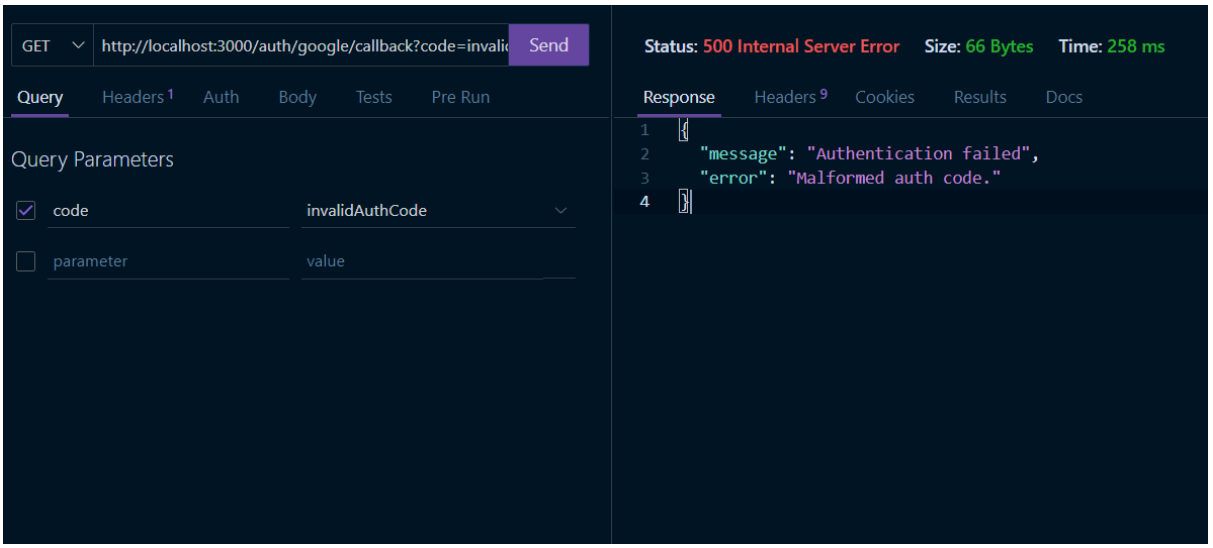


Figure 189. Server Failure to Authenticate Google Account (Status Code 500)

This figure displays a server-side error that shows there was a server error in logging in the user's Google Account. Status Code 500 is returned due to Internal Server Error.

1.3.4 Manual login and authenticate User Credentials

Version: 1.0

Date: November 3, 2024

Description: This API endpoint will allow the user to manually log in their accounts and will then validate it.

Endpoint: http://localhost:3000/auth/login

Method: POST

Configurations: This API endpoint enables users to manually log in and access the University Training and Seminar Management system using their login credentials

Parameters:

- **reCAPTCHA token:**
- **Body:** (JSON Object) Contains the following fields:
  - **email: (Required, String)** This is the user's email address to determine whose profile is being accessed.
  - **password: (Required, String)** This is the user's password to ensure security when accessing their account

Response codes of this API:

Code	Message	Description
200	Request Successful (OK)	This indicates a successful submission of the user's login credentials. The submission is processed and the user proceeds to the Homepage of the System
400	Invalid reCAPTCHA	This indicates that there is no reCAPTCHA token detected
401	User's email or password is invalid	This indicates a failed submission, typically due to missing characters or typos in the users' login credentials.
404	User Credentials not Found	This indicates a request resource was not found in the systems database
500	Login Error	An unexpected error-normally server-side, that occurred while attempting to login the user's credentials

Valid and Invalid Request Responses

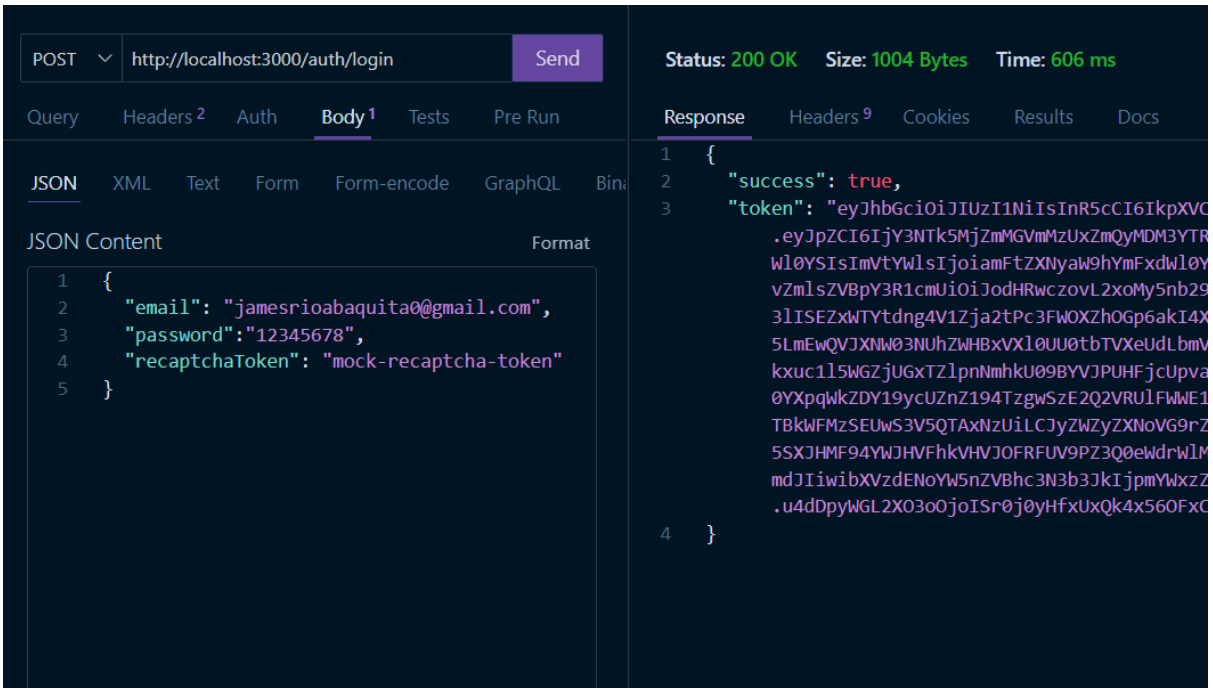


Figure 190. Valid POST Request for Manual Login from the database

This figure shows a valid POST request code 200 from the user's Manual Login and Authentication from their login credentials.

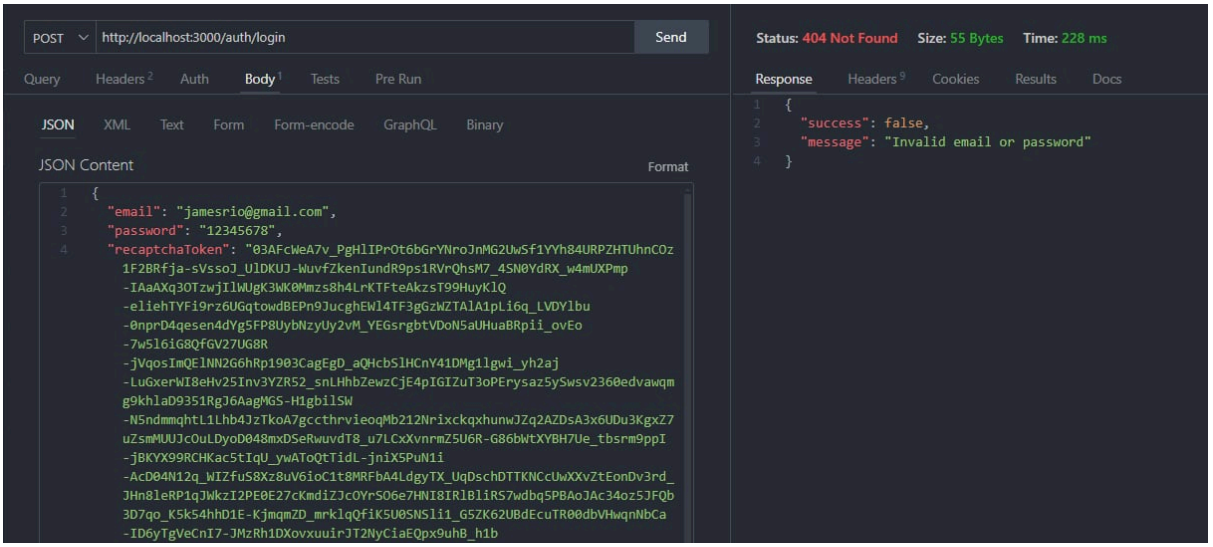


Figure 191. Invalid POST Request for Manual Login and Authentication

This figure displays an invalid POST Request with a request code of 404. This indicates that the requested resource was not found in the system database.

Different response Screenshots

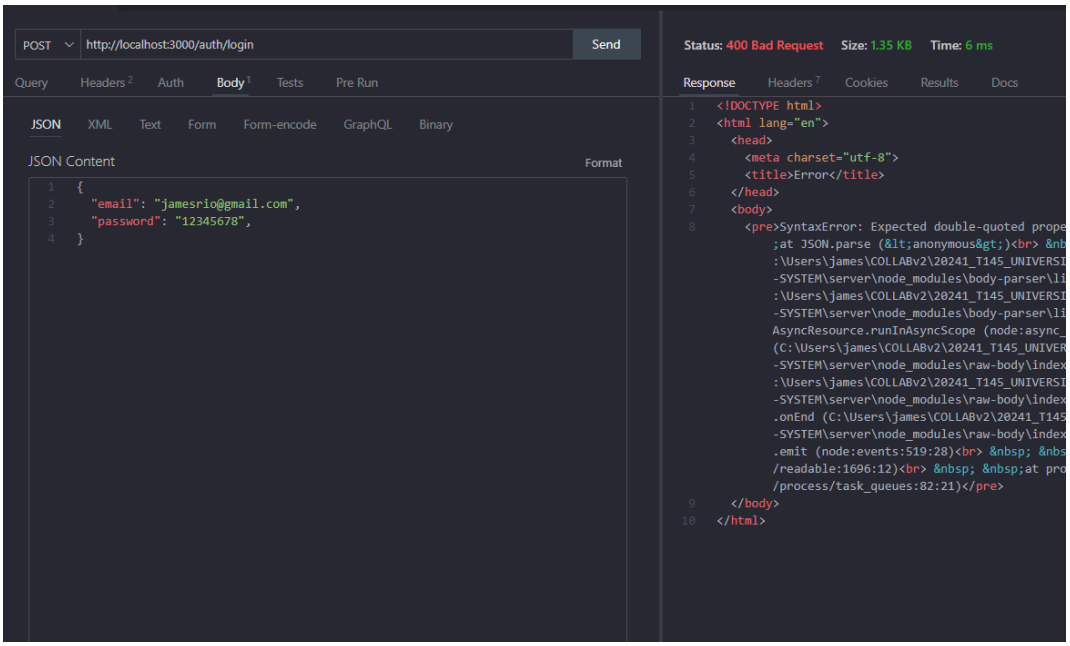


Figure 192. Invalid reCAPTCHA (Status Code 400)

This figure shows that there was an invalid reCAPTCHA token with a status code of 400. This failure occurs when there was no reCAPTCHA token detected as the user tried to login in the system

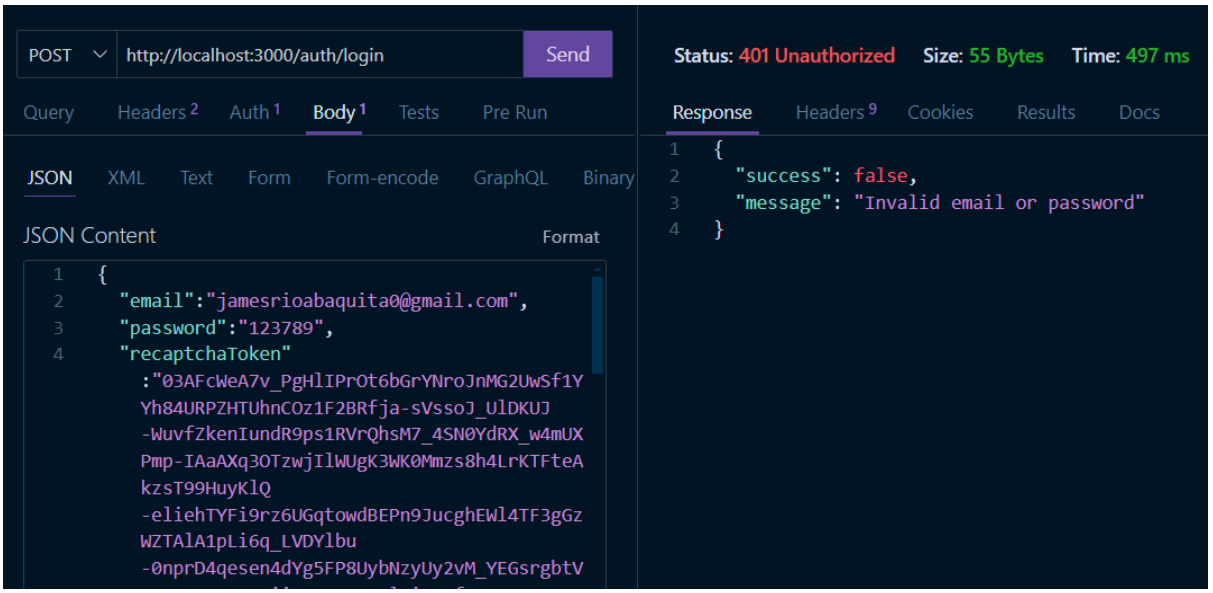


Figure 193. User Credentials not Authenticated (Status Code 401)

This figure shows that the system was not able to retrieve the user's account with a status code of 401. This failure occurs when there are missing characters or typos in the users login credentials.

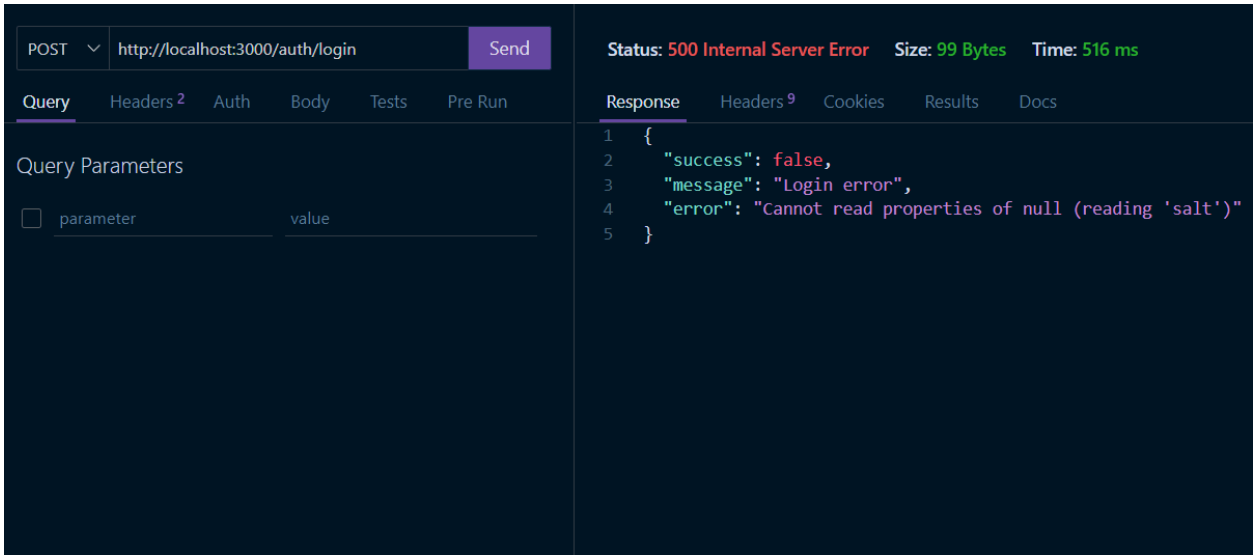


Figure 194. Server Failure to Login User (Status Code 500)

This figure displays a server-side error that shows there was a server error in logging in the user's login credentials. Status Code 500 is returned due to Internal Server Error

1.3.5 Logout User and Terminate Session

Version: 1.0

Date: November 6, 2024

Description: This API endpoint will securely logout the user from the system and this will terminate the user's session.

Endpoint: http://localhost:3000/auth/logout

Method: POST

Configurations: The API request should be validated using a token.

Parameters:

- **authToken -** A valid session token (JWT), included in the Authorization header, is required for secure access.

Response codes of this API:

Code	Message	Description
200	Request Successful (OK)	This indicates the request was successfully submitted. The request was processed, and the user's account was successfully logged out from the system, terminating their session
401	Access Denied. No token provided	This indicate the request failed because there was no token detected/retrieved by the system
403	Invalid Token	This indicates the token used to make the request is invalid
500	Logout Failed	This indicates that the server encountered unexpected errors that failed to proceed or prevented the request to continue

Valid and Invalid Request Responses

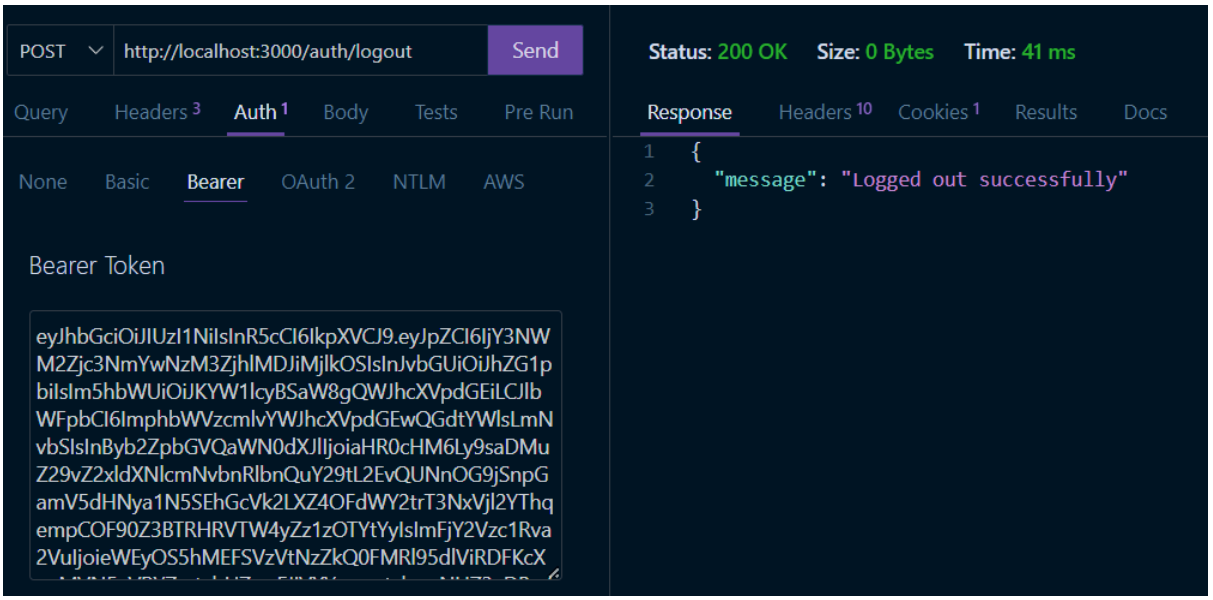


Figure 195. Valid POST Request for Retrieving User Account

This figure shows a valid POST request code 200 in logging out and terminating the session of the user.

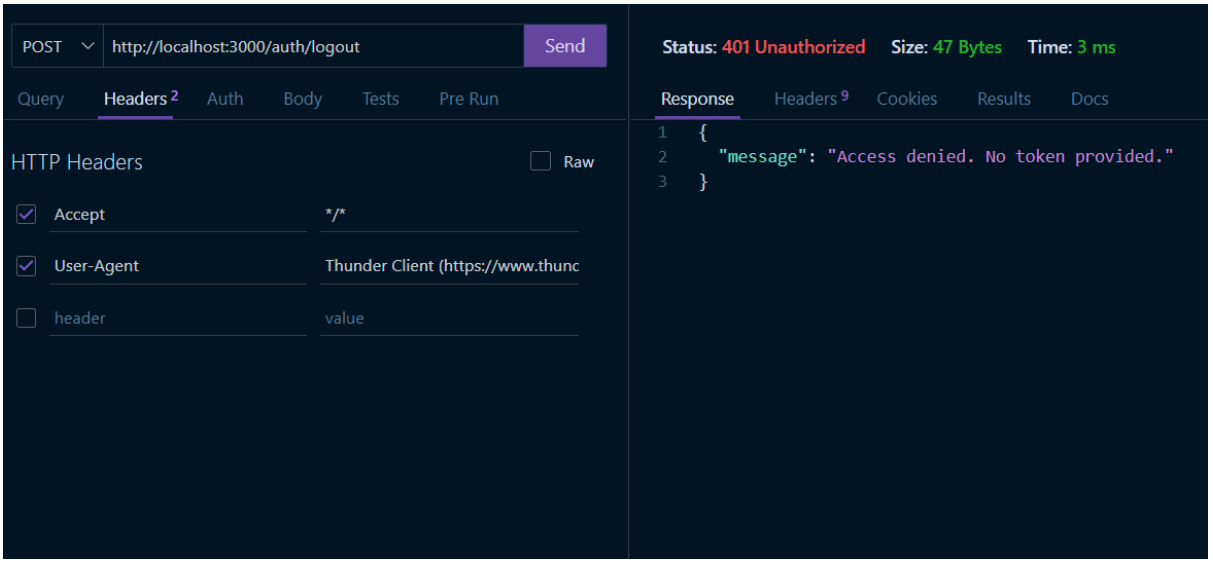


Figure 196. Invalid POST Request for Retrieving User Account

This figure displays an invalid POST Request with a request code of 401. This indicates that the request was denied due to no token being detected by the system.

Different response Screenshots

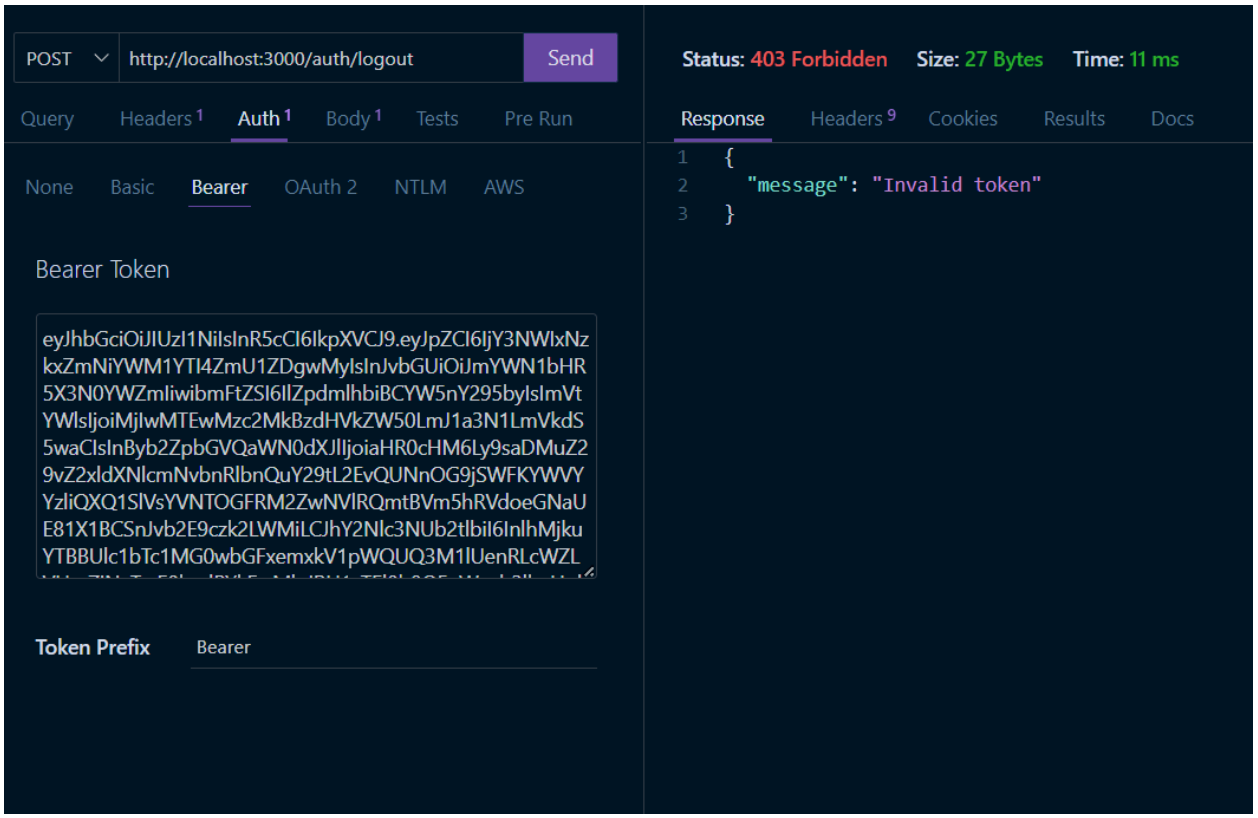


Figure 197. Invalid Token used in POST Request (Status Code 403)

This figure displays an invalid POST Request with a request code of 403. This indicates that the request was denied because the token used was invalid.

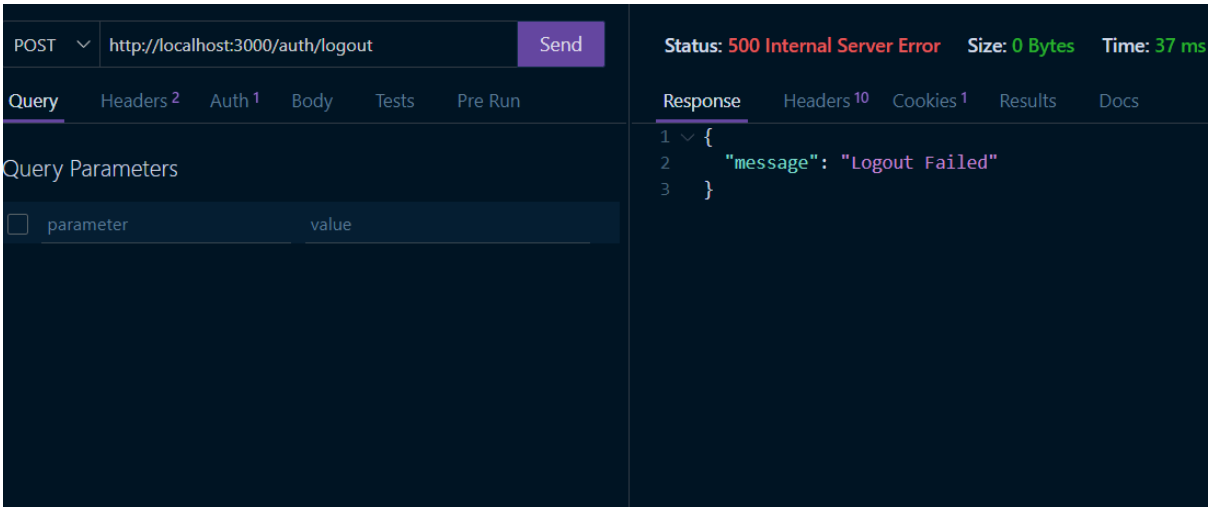


Figure 198. Server Failure to Authenticate Google Account (Status Code 500)

This figure displays a server-side error that shows there was a server error in logging out the users' account. Status Code 500 is returned due to Internal Server Error.

2.3.6 New User Password Set Up

Version: 1.0

Date: November 23,2024

Description: This API endpoint will enable new users to set up their password for their login credentials.

Endpoint: http://localhost:3000/auth/change-password

Method: POST

Configurations: The API request should be validated using a token.

Parameters:

- **userId**: The unique identifier of the user whose data is being modified.
- **newPassword:** The new password of the user's account

Response codes of this API:

Code	Message	Description
200	Request Successful (OK)	This indicates the request was successfully submitted. The request was processed the users successfully changed their password.
401	Access Denied. No token provided	This indicates that the request was not successful. The user credentials or token is not detected or found in the systems database
403	Invalid Token	This indicates the token used to make the request is invalid
500	Error Changing Password	This indicates that the server encountered unexpected errors that failed to proceed or prevented the request to continue

Valid and Invalid Request Responses

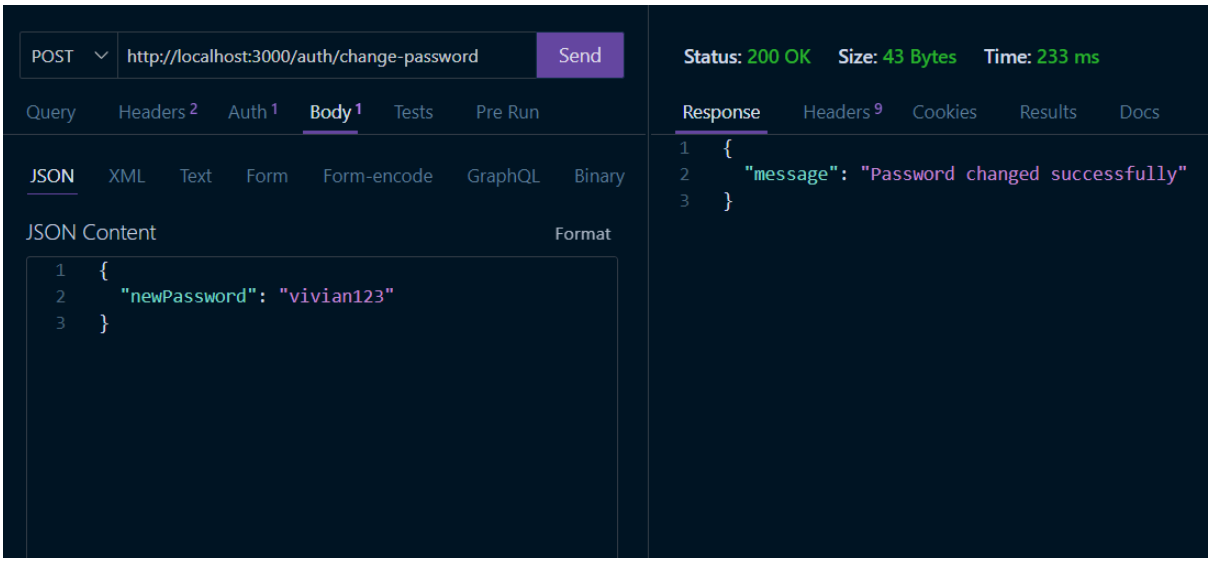


Figure 199. Valid POST Request for Setting up Users New Password

This figure shows a valid POST request code 200 in setting up the User's New Password.

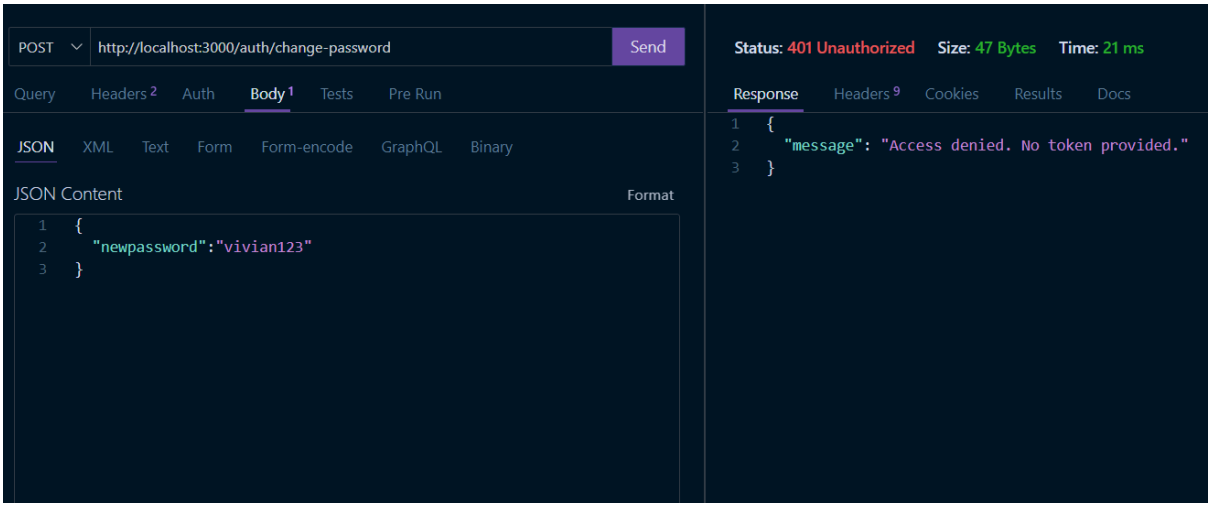


Figure 200. Invalid POST Request for Retrieving User Account

This figure displays an invalid POST Request with a request code of 401. This indicates that the request was denied due to no token being detected by the system.

Different response Screenshots

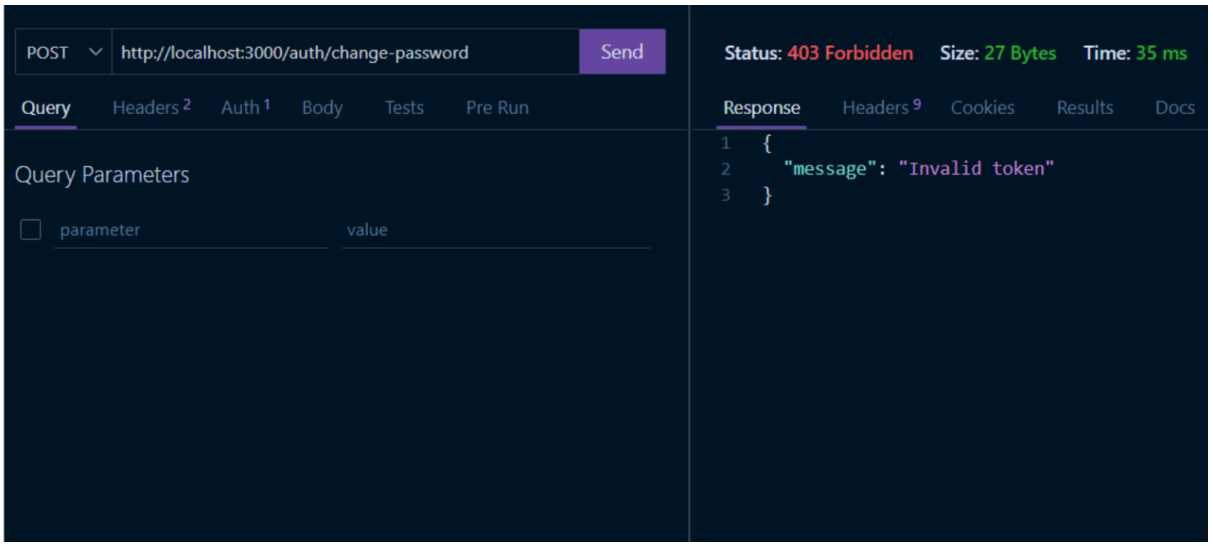


Figure 201. Invalid Token used in POST Request (Status Code 403)

This figure displays an invalid POST Request with a request code of 403. This indicates that the request was denied because the token used was invalid.

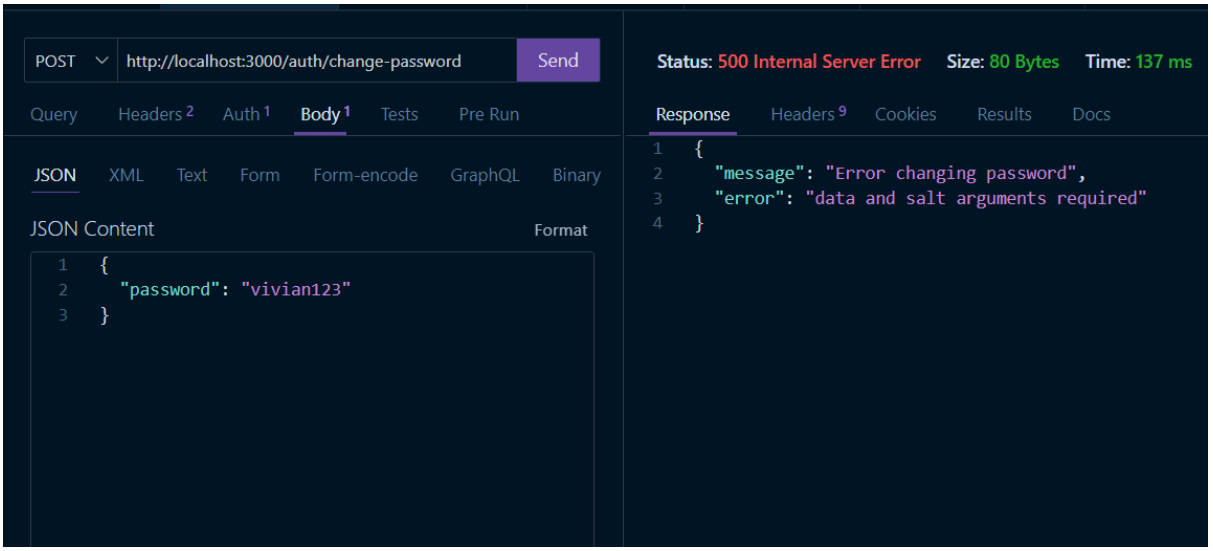


Figure 202. Server Failure in changing the Users' Password (Status Code 500)

This figure displays a server-side error that shows there was a server error in changing and saving the user's new password. Status Code 500 is returned due to Internal Server Error.

External API Documentation

1. **Google Auth 2.0 API**
    a. Description: Provides user authentication and secure access using OAuth 2.0.
    b. Version: 13.5.6.
    c. Documentation: https://developers.google.com/identity/protocols/oauth2#scope-response
2. **reCaptcha v3**
    a. Description: Protects websites from spam and abuse without user interaction, using risk analysis.
    b. Version: Latest version available as per Google documentation.
    c. Documentation: https://developers.google.com/recaptcha/docs/v3
3. **Gmail API**
    a. Description: Access and manage Gmail mailbox data programmatically.
    b. Version: Latest stable version.
    c. Documentation: https://developers.google.com/gmail/api/guides
4. **Google Sheets API**
    a. Description: Allows developers to read, write, and manage Google Sheets programmatically.
    b. Version: v4 (latest).
    c. Documentation:https://developers.google.com/sheets/api/guides/concepts
5. **Google Calendar API**
    a. Description: Enables integration with Google Calendar for event management and scheduling.
    b. Version: Latest stable version.
    c. Documentation:https://developers.google.com/calendar/api/guides/overview
6. **Google Form API**
    a. Description: Manage Google Forms programmatically, including creating and updating forms.
    b. Version: Latest.
    c. Documentation: https://developers.google.com/forms/api/reference/rest

Sample Code Snippets

Login Management Code Sample

- App Router - AppRouter.jsx

```
createRoot(document.getElementById("root")).render(
    <StrictMode>
        <BrowserRouter>
          <Routes>
            <Route path="/" element={<GoogleReCaptchaProvider reCaptchaKey={import.meta.env.VITE_RECAPTCHA_SITE_KEY}> <Login /></GoogleReCaptchaProvider>}> </Route>
            <Route path="/login/success" element={<LoginSuccess />}> </Route>
            <Route
              path="/a/*"
              element={
                <PrivateRoute>
                  <Admin />
                </PrivateRoute>
              }
            />
            <Route
              path="/u/*"
              element={
                <PrivateRoute>
                  <User />
                </PrivateRoute>
              }
            />
            <Route path="/set-password" element={<ChangePassword />} />
          </Routes>
        </BrowserRouter>
    </StrictMode>
```

Figure 1. *App Router to User and Admin*

- Login.jsx

```
function Login() {
  const navigate = useNavigate();
  const { executeRecaptcha } = useGoogleReCaptcha();

  const [formData, setFormData] = useState({ email: '', password: '' });
  const [errorMessage, setErrorMessage] = useState(''); // State for error message

  const handleInputChange = (event) => {
    setFormData({
      ...formData,
      [event.target.id]: event.target.value,
    });
  };

  const handleKeyDown = (event) => {
    if (event.key === 'Enter') {
      handleManualLogin();
    }
  };

  const handleGoogleLogin = () => {
    window.location.href = 'http://localhost:3000/auth/google';
  };

  const handleManualLogin = async () => {
    setErrorMessage(''); // Clear error message before attempting login
    try {
      const recaptchaToken = await executeRecaptcha('login');
```

Figure 2. *Login Client*

- Route - authRoutes.js

```
const express = require('express');
const passport = require('passport');
const authenticateJWT = require('../middleware/auth');
const authRoutes = express.Router();
const authService = require('../services/authServices');

//Google Login
authRoutes.get('/auth/google', passport.authenticate('google', { accessType: 'offline', prompt: 'consent', callbackURL: 'http://localhost:3000/auth/google/callback' }));
authRoutes.get('/auth/google/callback', authService.googleOAuthCallback);

//Manual Login
authRoutes.post('/auth/login', authService.manualLogin);

authRoutes.post('/auth/logout', authenticateJWT, authService.logout);

authRoutes.post('/auth/change-password', authenticateJWT, authService.changePassword);


module.exports = authRoutes;
```

Figure 3. *Authentication Routes*

- Service - authServices

```
const googleOAuthCallback = (req, res, next) => {
  passport.authenticate('google', async (error, user, info) => {
    if (error) {
      return res.status(500).json({ message: 'Authentication failed', error: error.message });
    }
    if (!user) {
      return res.status(401).json({ message: 'User not found' });
    }

    // Generate JWT token for authenticated user
    const token = jwt.sign(
      {
        id: user._id,
        role: user.role,
        name: user.name,
        email: user.email,
        position: user.position,
        department: user.department,
        phoneNumber: user.phoneNumber,
        profilePicture: user.profilePicture,
        accessToken: user.accessToken,
        refreshToken: user.refreshToken,
        mustChangePassword: user.mustChangePassword
      },
      JWT_SECRET,
      { expiresIn: JWT_EXPIRES_IN }
    );
```

Figure 4*. Google Auth Login Service*

```
// Manual login with reCAPTCHA
const manualLogin = async (req, res) => {
  const { email, password, recaptchaToken } = req.body;

  try {
    const isRecaptchaValid = await verifyRecaptcha(recaptchaToken);
    console.log("reCAPTCHA Response:", isRecaptchaValid);
    if (!isRecaptchaValid) {
      return res.status(400).json({ success: false, message: 'Invalid reCAPTCHA' });
    }

    const user = await User.findOne({ email });

    console.log('Login attempt with:', {
      email,
      providedPassword: password,
      storedSalt: user.salt
    });

    if (!user) {
      return res.status(401).json({ success: false, message: 'Invalid email or password' });
    }
```

Figure 5. *Manual Login to the System Service*

- LoginSuccess

```
import { jwtDecode } from "jwt-decode";

function LoginSuccess() {
  const navigate = useNavigate();
  const [searchParams] = useSearchParams();

  useEffect(() => {
    const token = searchParams.get("token");

    if (token) {
      sessionStorage.setItem("authToken", token);

      const decoded = jwtDecode(token);
      const userRole = decoded.role;
      const mustChangePassword = decoded.mustChangePassword;

      sessionStorage.setItem("userRole", userRole);

      console.log(decoded);

      if (mustChangePassword) {
        navigate("/set-password");
        return;
      }
```

Figure 6. *Login Success*

```
      sessionStorage.setItem("userRole", userRole);

      console.log(decoded);

      if (mustChangePassword) {
        navigate("/set-password");
        return;
      }

      if (userRole === "faculty_staff") {
        navigate("/u");
      } else if (
        userRole === "admin" ||
        userRole === "departmental_admin"
      ) {
        navigate("/a");
      } else {
        console.error("Unknown user role:", userRole);
        navigate("/"); // Redirect to login for unknown roles
      }
    } else {
      console.error("Token not found in query parameters");
      navigate("/login"); // Redirect to login if no token
    }
  }, [navigate, searchParams]);

  return <div>Redirecting...</div>;
}
```

Figure 7*. Login Navigation*

```
const changePassword = async (req, res) => {
  try {
    const { newPassword } = req.body;
    console.log("New password:", newPassword);

    const userId = req.user.id;
    const user = await User.findById(userId);

    if (!user) {
      return res.status(401).json({ message: 'User not found' });
    }

    const initialHash = await bcrypt.hash(newPassword, 10);

    const salt = await bcrypt.genSalt(10);
    const saltedPassword = initialHash + salt;

    const finalHashedPassword = await bcrypt.hash(saltedPassword, 10);
    console.log(finalHashedPassword);

    user.password = finalHashedPassword;
    user.salt = salt;
    user.mustChangePassword = false;
    await user.save();
    await emitNewActivity(userId, 'Password Changed', {userName: user.name})
    res.status(200).json({ message: 'Password changed successfully' });
  } catch (error) {
```

Figure 8. *Change Password Login Service*

```
const logout = async (req, res) => {

  req.session.destroy((err) => {
    if (err) {
      return res.status(500).json({ message: 'Logout failed', error: err });
    }

    // Clear the session cookie
    res.clearCookie('connect.sid');

    res.status(200).json({ message: 'Logged out successfully' });
  });
  await emitNewActivity(req.user.id, 'Logout', {userName: req.user.name})
};
```

Figure 9. *Logout Service*

Event Management Code Sample

- Client - CreateEvents.jsx & EventManagement.jsx

```
  // Prepare event data
  const eventData = {
    title,
    date,
    eventPicture,
    startTime: isoStartTime,
    endTime: isoEndTime,
    location,
    description,
    reminders,
    participants,
    formLink,
    formId,
    customParticipants: cleanedParticipants, // Include selected participants in the saved event
    color: eventColor,
  };
  try {
    // Save the event
    await onSave(eventData);
  } catch (error) {
    console.error('Error saving event:', error);
  }

  onClose();
};
```

Figure 10. *Handle Creation of Event (Client)*

```
//When edit button was clicked
const handleEdit = async (event) => {
  const token = sessionStorage.getItem('authToken');

  if (!token) {
    showToast('User is not authenticated. Please log in.','error');
    return;
  }

  const formattedSelectedEvent = event ? {
    ...event,
    eventDate: (event.eventDate instanceof Date ? event.eventDate : new Date(event.eventDate)).toISOString().split("T")[0], // format as YYYY-MM-DD
    startTime: (event.startTime instanceof Date ? event.startTime : new Date(event.startTime)).toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', hour12: false }),
    endTime: (event.endTime instanceof Date ? event.endTime : new Date(event.endTime)).toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', hour12: false }),
  } : null;

  setSelectedEvent(formattedSelectedEvent);
  setIsModalOpen(true);
```

Figure 11. *Handle Update Event (Client)*

```
const handleDelete = (eventId) => {
  setEventToDelete(eventId);
  setIsDeleteModalOpen(true);
};

const confirmDelete = async () => {
  const token = sessionStorage.getItem('authToken');
  try {
    await axios.delete(`http://localhost:3000/a/events/${eventToDelete}`, {
      headers: { Authorization: `Bearer ${token}` }
    });
    setEvents(events.filter(event => event._id !== eventToDelete));
    setEventToDelete(null);
    setIsDeleteModalOpen(false); // Close modal after deletion
  } catch (error) {
    console.error('Error deleting event:', error.message);
  }
```

Figure 12. *Handle Delete Event(Client)*

- Model - event.js & deletedEvents.js

```
// eventSchema.js
const mongoose = require('mongoose');
const encrypt = require('mongoose-encryption');
require('dotenv').config({path: '../.env'});

const EventSchema = new mongoose.Schema({
    title: {
        type: String,
        required: [true, 'Event title is required'],
        trim: true,
        maxlength: 100
    },
    eventPicture: {
        type: String
    },
    eventDate: {
        type: Date,
        required: [true, 'Event date is required']
    },
    startTime: {
        type: Date, // Use Date type for compatibility with date-time storage
        required: [true, 'Event start time is required']
    },
```

Figure 13. *Event Schema*

```
const mongoose = require('mongoose');
const encrypt = require('mongoose-encryption');
require('dotenv').config({path: '../.env'});

const DeletedEventSchema = new mongoose.Schema({

  title: String,
  description: String,
  eventDate: Date,
  startTime: Date,
  endTime: Date,
  location: String,
  deletedByName: String,
  deletedByRole: String,
}, {timestamps: true});

DeletedEventSchema.plugin(encrypt, {
    encryptionKey: process.env.MONGODB_ENCRYPTION_KEY,
    signingKey: process.env.MONGODB_SIGNING_KEY,
    excludeFromEncryption: ['deletedByName', 'deletedByRole'],
});

const DeletedEvent = mongoose.model('DeletedEvent', DeletedEventSchema);

module.exports = DeletedEvent;
```

Figure 14. *Deleted Event Schema*

- Route - adminEventRoutes.js

```
// Add a new event
eventRoutes.post('/a/events', authenticateJWT, adminService.upload.single('eventPicture'), adminService.addEvent);

// Update an existing event by ID (optional)
eventRoutes.put('/a/events/:id', authenticateJWT,  adminService.upload.single('eventPicture'), adminService.updateEvent);

// Delete an event by ID (optional)
eventRoutes.delete('/a/events/:id', authenticateJWT,  adminService.deleteEvent);

// Lock an event
eventRoutes.put('/a/events/:id/lock', authenticateJWT, concurrencyLock, lockingService.lockEvent);

// Unlock an event
eventRoutes.put('/a/events/:id/unlock', authenticateJWT, clearConcurrencyLock, lockingService.unlockEvent);

module.exports = eventRoutes;
```

Figure 15. *Event Routes*

- Services - adminEventServices.js

```
const addEvent = async (req, res) => {
  try {
    const {title, eventDate, startTime, endTime, location, description, participantGroup, color, customParticipants, reminders, formLink, formId} = req.body;
    console.log(req.files)
    const eventPicture = req.file ? req.file.filename : null // Set eventPicture

    const user = req.user;
    const newEvent = new Event({
      title,
      eventDate,
      eventPicture,
      startTime,
      endTime,
      location,
      description,
      participantGroup,
      color,
      customParticipants,
      reminders,
      formLink,
      formId,
      createdBy: user.id,
      isLocked: false,
      lockedBy: null,
    });
    console.log(newEvent);
    await newEvent.save();
```

Figure 16. *Add Event Service*

```
const updateEvent = async (req, res) => {
  try {
    const eventId = req.params.id;
    const userId = req.user.id;
    const updates = req.body;

    console.log('Starting event update for eventId:', eventId);
    console.log('Update data:', updates);

    // Fetch the existing event
    const existingEvent = await Event.findById(eventId);
    if (!existingEvent) {
        return res.status(404).json({ message: 'Event not found' });
```

Figure 17. *Updating an Event Service*

```
const deleteEvent = async (req, res) => {
  try {
    const eventId = req.params.id;
    const { name, role } = req.user;

    // Find the event to be deleted
    const event = await Event.findById(eventId);
    if (!event) {
      return res.status(404).json({ message: 'Event not found' });
    }

     // Delete the associated notifications for this event
     await Notification.deleteOne({ eventId });
```

Figure 18. *Deleting an Event Service*

Notification Code Sample

- Client - Topbar.jsx

```
    // Fetch notifications
    axios
        .get('http://localhost:3000/u/notification/items', {
            headers: { Authorization: `Bearer ${token}` },
        })
        .then((response) => {
            const filteredNotifications = response.data.map((notification) => {
                const relevantUserNotifications = notification.userNotifications?.filter(
                    (userNotif) => {
                        const isForAllUsers = notification.participantGroup === 'all';
                        return (isForAllUsers || userNotif.userId === decoded.id) && !userNotif.removedStatus;
                    }
                );

                return relevantUserNotifications.length > 0
                    ? { ...notification, userNotifications: relevantUserNotifications }
                    : null;
            }).filter(Boolean);

            // Map events to notifications
            const notificationsWithImages = filteredNotifications.map((notification) => {
                const event = events.find((e) => e._id === notification.eventId);
                return {
                    ...notification,
                    eventPicture: event ? event.eventPicture : null,
                };
            });

            setNotifications(notificationsWithImages);
```

Figure 19. *Handle Notification List (Client)*

```
    const handleViewEvent = async (notificationId, eventId) => {
        const token = sessionStorage.getItem('authToken');
        const decoded = jwtDecode(token);

        if (token) {
            try {
                const response = await axios.patch(
                    `http://localhost:3000/u/notification/update/${notificationId}`,
                    { userId: decoded.id },
                    {
                        headers: {
                            Authorization: `Bearer ${token}`,
                        },
                    }
                );

                if (response.status === 200) {
```

Figure 20. *Handle Updating Notification Read Status (Client)*

```
const handleRemoveNotification = async (notificationId) => {
    const token = sessionStorage.getItem('authToken');
    const decoded = jwtDecode(token);

    if (token) {
        try {
            const response = await axios.patch(
                `http://localhost:3000/u/notification/remove/${notificationId}`,
                { userId: decoded.id }, // Send userId as part of the request
                {
                    headers: { Authorization: `Bearer ${token}` },
                }
            );
```

Figure 21. *Handle Updating Notification Removed Status (Client)*

- Model - notification.js

```
const mongoose = require('mongoose');
const encrypt = require('mongoose-encryption');
require('dotenv').config({path: '../.env'});

const registrationSchema = new mongoose.Schema({
  eventId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Event',
    required: true
  },
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  googleEventId: {
    type: String,  // Store the Google Calendar event ID here
    required: true
  },
  registrationDate: {
    type: Date,
    default: Date.now
  },
  status: {
    type: String
```

Figure 22. *Notification Schema*

- Route - userNotificationRoutes.js & adminUserRoutes.js

```
// List all notification
userNotification.get('/u/notification/items',authenticateJWT, userNewFeatureService.listItems);

// Update exisiting notification item details
userNotification.patch('/u/notification/update/:id', authenticateJWT, userNewFeatureService. updateNotificationStatus);

// Update exisiting notification item removestatus
userNotification.patch('/u/notification/remove/:id',authenticateJWT,  userNewFeatureService. removeNotification);


module.exports = userNotification;
```

Figure 23. *User Notification Routes*

- Services - userNotificationService & adminUserServices

```
// List all items
const listItems = async (req, res) => {
    try {
        // Fetch notifications sorted by createdAt in descending order
        const items = await userNotification.find().sort({ createdAt: -1 });
        res.json(items);
    } catch (error) {
        console.error('Error listing items:', error);
        res.status(500).send('Error retrieving items');
    }
};
```

Figure 24. *List the Notification*

```
const updateNotificationStatus = async (req, res) => {
    try {
        const { id } = req.params; // Notification ID
        const { userId } = req.body; // User ID from request body

        // Update the notification for the specific user
        const updatedNotification = await userNotification.findOneAndUpdate(
            { _id: id, "userNotifications.userId": userId },
            {
                $set: {
                    "userNotifications.$.status": "read",
                    "userNotifications.$.readAt": new Date()
                }
            },
            { new: true }
        );

        if (!updatedNotification) return res.status(404).send("Notification not found");
        res.json({ message: "Notification marked as read", updatedNotification });
    } catch (error) {
        console.error("Error updating notification status:", error);
        res.status(500).send("Error updating notification status");
    }
};
```

Figure 25. *Update Read Status in Notification*

```
const removeNotification = async (req, res) => {
    try {
        const { id } = req.params;
        const { userId } = req.body; // Get userId from the request body

        // Find the notification and update the user-specific removedStatus and removedAt field
        const updatedNotification = await userNotification.findOneAndUpdate(
            { _id: id, "userNotifications.userId": userId },
            {
                $set: {
                    "userNotifications.$.removedStatus": true,
                    "userNotifications.$.removedAt": new Date(), // Set removedAt to current timestamp
                }
            },
            { new: true }
        );

        if (!updatedNotification) {
            console.log("Notification or user not found with ID:", id);
            return res.status(404).send('Notification or user not found');
        }
```

Figure 26. *Update Removed Status in Notification*

User Accounts Code Sample

- Client - addPersonnel.jsx & usersTable.jsx

```jsx
const AddPersonnelModal = ({ show, onClose }) => {
  const [formData, setFormData] = useState({
    profilePicture: null,
    name: "",
    email: "",
    phoneNumber: "",
    department: "",
    gender: "",
    position: "",
    role: "",
  });
  const [error, setError] = useState("");
  const navigate = useNavigate();

  useEffect(() => {
    const token = sessionStorage.getItem("authToken");

    // Check if user has access to this page
    const checkAccess = async () => {
      try {
        const response = await fetch("http://localhost:3000/a/personnel", {
          method: "GET",
          headers: {
```

Figure 27. *Adding a New Account (Client)*

```jsx
    // Update Data
    const handleUpdate = async (userId, formData) => {
      try {
        await axios.put(`http://localhost:3000/a/users/${userId}`, formData);
        setFilteredUsers(filteredUsers.map(user => (user._id === userId ? { ...user, ...formData } : user)));

        setStats(prevStats => ({
          ...prevStats,
          users: prevStats.users.map(user =>
            user._id === userId ? { ...user, ...formData } : user
          )
        }));
      } catch (err) {
        console.log(err);
      }
    };
```

Figure 28. *Updating an Account (Client)*

- Model - user.js

```
const encrypt = require('mongoose-encryption');
const bcrypt = require('bcrypt');
require('dotenv').config({ path: '../.env' });

const UserSchema = new mongoose.Schema({
    name: {
        type: String,
        required: true
    },
    email: {
        type: String,
        required: true,
        unique: true
    },
    password: {
        type: String,
        required: function() { return !this.googleId; } // Password is required only if googleId is not present
    },
    salt: {
        type: String,
    },
    googleId: {
        type: String,
        unique: true,
        sparse: true // Allows it to be unique but not required
    },
```

Figure 29. *User Schema Sample*

- Route - adminUserRoutes

```
userRoutes.get('/a/personnel', authenticateJWT, verifyGeneralAdmin, adminService.renderPersonnelPage);
userRoutes.post('/a/personnel', authenticateJWT, verifyGeneralAdmin, adminService.upload.single('profilePicture'), adminService.addPersonnelAccount);
userRoutes.get('/a/users', authenticateJWT, verifyGeneralAdmin, adminService.renderUserTable);
userRoutes.put('/a/users/:id', adminService.editUser);
```

Figure 30. *User Account Routes*

- Services - adminUserServices

```
46  const addPersonnelAccount = async (req, res) => {
47    try {
48      const { name, email, role, phoneNumber, department, position } = req.body;
49      const profilePicture = req.file ? req.file.filename : null;
50
51      const existingUser = await User.findOne({ email });
52      if (existingUser) {
53        throw new Error('User already exists');
54      }
55
56
57      const tempPassword = crypto.randomBytes(8).toString('hex');
58      console.log("Temporary password:", tempPassword);
59
60      const initialHash = await bcrypt.hash(tempPassword, 10);
61
62      const salt = await bcrypt.genSalt(10);
63      const saltedPassword = initialHash + salt;
64
65      const finalHashedPassword = await bcrypt.hash(saltedPassword, 10);
66      console.log(finalHashedPassword);
```

Figure 31. *Add User Account Service*

```
    // Send the temporary password email
    const emailContent = `
    Dear ${name},

    Welcome to BukSU Engage!

    Your account has been successfully created in the BukSU Training and Seminar Management System.

    Your Login Credentials:
    Email: ${email}
    Temporary Password: ${tempPassword}

    IMPORTANT SECURITY NOTICE:
    * For security reasons, you will be required to change your password upon your first login.
    * Please keep your credentials confidential.
    * If you didn't request this account, please contact the administrator immediately.

    To access the system:
    1. Visit the BukSU Engage Login page
    2. Enter your email and temporary password
    3. Follow the prompts to set your new password

    If you have any questions or need assistance, please contact our support team.

    Best regards,
    BukSU Engage Admin Team
    `;

  await sendEmail(newUser.email, 'Welcome to Your Account', emailContent, { accessToken: req.user.accessToken, refreshToken: req.user.refreshToken });
```

Figure 32. *Send Temporary Pass to User Service*

```
const renderUserTable = async (req, res) => {
  try {
    const users = await User.find().select('-password'); // Exclude password field
    console.log(users);
    res.status(200).json({ users });

  } catch (error) {
    res.status(500).json({ message: error.message });
    console.log(error);
  }
};
```

Figure 33. *List Users Account Table Service*

```
// Function to edit user details
const editUser = async (req, res) => {
  const userId = req.params.id; // Get user ID from route params
  const updatedData = req.body; // Data to update

  try {
    // Step 1: Find the user by ID
    const user = await User.findById(userId);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }

    // Step 2: Update the user's fields (unencrypted in memory)
    Object.keys(updatedData).forEach((field) => {
      user[field] = updatedData[field];
    });

    // Step 3: Save the user (triggers encryption)
    const updatedUser = await user.save();

    // Step 4: Respond with the updated user
    res.status(200).json({ updatedUser });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Figure 34. *Edit User Account Service*

Activity Logs Code Sample

- Client - activityLog.jsx

```
const ActivityLog = () => {
    const [logs, setLogs] = useState([]);
    const navigate = useNavigate();
    const [searchQuery, setSearchQuery] = useState('');
    const [currentPage, setCurrentPage] = useState(1);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);
    const logsPerPage = 8;


    useEffect(() => {
        // Check if user has access to this page
        const checkAccess = async () => {
            try {
                const token = sessionStorage.getItem("authToken");
                const response = await fetch("http://localhost:3000/a/activity-logs", {
                    method: "GET",
                        headers: {
                          Authorization: `Bearer ${token}`,
                        },
                    });

                    if (response.status === 403) {
                        // Redirect to a different page if access is denied
                        navigate("/a/dashboard"); // Change this to your desired redirect path
                    }
                    } catch (error) {
                        console.error("Access check failed:", error);
                    }
                };
        checkAccess();
    }, []);
```

Figure 35. *Retrieving ActivityLogs Client*

- Model - activityLog.js

```
const mongoose = require('mongoose');
const encrypt = require('mongoose-encryption');
require('dotenv').config({ path: '../.env' });


// Schema definition
const activityLogSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  action: {
    type: String,
    required: true
  },
  details: {
    type: Object
  },
}, { timestamps: true });


module.exports = mongoose.model('ActivityLog', activityLogSchema);
```

Figure 36. *ActivityLogs Schema*

- Route - adminPageRoutes.js

```
//Activity Logs
pageRoutes.get('/a/activity-logs', authenticateJWT, verifyGeneralAdmin, adminService.getLogs);
```

Figure 37. *ActivityLogs Route*

- Services - adminPageServices

```
const getLogs = async (req, res) => {
    try {
        const logs = await ActivityLog.find()
            .populate('userId', 'name email profilePicture')
            .sort({ timestamp: -1 });

        console.log(logs);

        const decryptedLogs = await Promise.all(
            logs.map(async log => {
                if (log.userId) {
                    try {
                        const user = await User.findById(log.userId._id);
                        log.userId = user; // Replace the populated user with decrypted user
                        console.log(user);
                    } catch (error) {
                        console.error(`Error fetching/decrypting userId for log ${log._id}:`, error.message);
                    }
                }
                return log;
            })
        );

        res.status(200).json(decryptedLogs);
    } catch (error) {
        res.status(500).json({ message: error.message });
        console.log(error);
    }
};
```

Figure 38. *Listing the Activity Logs Service*

Generate Certificate Code Sample

- Client - eventDetails.jsx

```
        const response = await axios.get(`http://localhost:3000/u/events/${id}/form-status`, {
            headers: { Authorization: `Bearer ${token}` }
        });

        setFormStatus(response.data);
        setHasSubmittedForm(response.data.hasSubmittedForm);
        if (response.data.hasSubmittedForm) {
            setCertificateStatus(response.data.certificateGenerated ? 'generated' : 'pending');
        }
    } catch (error) {
        if (error.response?.status === 404) {
            console.log('No form submission found');
            setHasSubmittedForm(false);
        } else {
            console.error('Error checking form status:', error);
        }
    }
};
```

Figure 39. *Handle Generating Certificates (Client)*

- Model - certificate.js

```
const encrypt = require('mongoose-encryption');
require('dotenv').config({ path: '../.env' });

const certificateSchema = new mongoose.Schema({
    userId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true
    },
    eventId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Event',
        required: true
    },
    submissionId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'FormSubmission',
        required: true
    },
    fileName: {
        type: String,
        required: true
    },
    issuedDate: {
        type: Date,
        required: true
    },
    status: {
        type: String,
        enum: ['issued', 'revoked'],
        default: 'issued'
    }
```

Figure 40. *Certificate Schema*

- Route - formSubmissionRoutes.js

```
const certificateService = require('../../services/user/certificateService');

formSubmissionRoutes.post('/u/events/:id/form-submission', authenticateJWT, formSubmissionService.recordFormSubmission);
formSubmissionRoutes.get('/u/events/:id/form-status', authenticateJWT, formSubmissionService.getFormSubmissionStatus);
formSubmissionRoutes.get('/u/events/:id/form-analytics', authenticateJWT, formSubmissionService.getFormAnalytics);
formSubmissionRoutes.get('/u/events/:id/certificate-status', authenticateJWT, certificateService.checkCertificateStatus);

module.exports = formSubmissionRoutes;
```

Figure 41. *Certificate Routes*

- Services - certificateService.js

```
const generateAndEmailCertificate = async (submissionId) => {
    let outputPath = null;
    try {
        const submission = await FormSubmission.findById(submissionId)
            .populate('userId')
            .populate('eventId');

        if (!submission) {
            throw new Error('Form submission not found');
        }

        const { userId: user, eventId: event } = submission;

        const certificatesDir = path.join(__dirname, '../../uploads/certificates');
        if (!fs.existsSync(certificatesDir)) {
            fs.mkdirSync(certificatesDir, { recursive: true });
        }

        // Format date to "Month DD, YYYY"
        const formatDate = (date) => {
            return date.toLocaleDateString('en-US', {
                month: 'long',
                day: 'numeric',
                year: 'numeric'
            });
        };
```

Figure 42. *Generate Certificate and Send to Email Services*

```
// Get user's certificates
const getUserCertificates = async (req, res) => {
    try {
        const userId = req.user.id;

        const certificates = await Certificate.find({ userId })
            .populate('eventId', 'title startTime')
            .sort({ issuedDate: -1 });

        res.json(certificates);
    } catch (error) {
        console.error('Error fe                    (property) message: string
        res.status(500).json({ message: 'Error fetching certificates' });
    }
};
```

Figure 43. *Get Certificate Status Service*

```
// Download specific certificate
const downloadCertificate = async (req, res) => {
    try {
        const certificateId = req.params.id;
        const userId = req.user.id;

        const certificate = await Certificate.findOne({
            _id: certificateId,
            userId
        });

        if (!certificate) {
            return res.status(404).json({ message: 'Certificate not found' });
        }

        res.download(certificate.filePath);
    } catch (error) {
        console.error('Error downloading certificate:', error);
        res.status(500).json({ message: 'Error downloading certificate' });
    }
};
```

Figure 44. *Download Certificate Service*

Event Registration Management Code Sample

- Client - eventDetails.jsx

```
const handleRegistration = async () => {
    const token = sessionStorage.getItem('authToken');
    try {
        const response = await axios.post(
            'http://localhost:3000/u/events/',
            { eventId: id },
            { headers: { Authorization: `Bearer ${token}` } }
        );
        setIsRegistered(true); // User is now registered
        setRegistrationStatus("Cancel Registration"); // Change the button text to "Cancel Registration"
        showToast('Event added to your Google Calendar with reminders', 'success');
    } catch (err) {
        showToast('Failed to register for the event', 'error');
    }
};
```

Figure 45. *Handle User Registers an Event (Client)*

- Model - registration.js

```
const mongoose = require('mongoose');
const encrypt = require('mongoose-encryption');
require('dotenv').config({path: '../.env'});

const registrationSchema = new mongoose.Schema({
  eventId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Event',
    required: true
  },
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  googleEventId: {
    type: String,  // Store the Google Calendar event ID here
    required: true
  },
  registrationDate: {
    type: Date,
    default: Date.now
  },
  status: {
    type: String,
```

Figure 46. *Register Event Schema*

- Route - userEventRoutes.js

```
// POST request for registration
userEventRoutes.post('/u/events', authenticateJWT, userEventService.registration);

userEventRoutes.get('/u/events/:id/check-registration', authenticateJWT, userEventService.checkRegistration);

// Add this new route for canceling registration
userEventRoutes.delete('/u/events/:id/cancellation', authenticateJWT, userEventService.cancelRegistration);
```

Figure 47. *Registration Management Event Route*

- Services - userEventService.js

```
// Handle registration and attendance
const registration = async (req, res) => {
    // Logic for event registration and attendance
    const eventId = req.body.eventId;
    const userId = req.user.id; // Assuming you have user authentication in place
    const token = req.user.accessToken;

    console.log("eventId", eventId);
    console.log("userId", userId);
    console.log("token", token)

    const GOOGLE_CALENDAR_COLORS = {
        '1': '#7986cb',  // Lavender
        '2': '#33b679',  // Green
        '3': '#8e24aa',  // Purple
        '4': '#e67c73',  // Red
        '5': '#f6bf26',  // Yellow
        '6': '#f4511e',  // Orange
        '7': '#039be5',  // Turquoise
        '8': '#616161',  // Gray
        '9': '#3f51b5',  // Bold Blue
        '10': '#0b8043', // Bold Green
        '11': '#d50000'  // Bold Red
      };
```

Figure 48. *Register an Event Service*

```
const checkRegistration = async (req, res) => {
    const eventId = req.params.id;
    const userId = req.user.id;

    try {
        const registration = await Registration.findOne({ eventId, userId });
        if (registration) {
            res.status(200).json({
                isRegistered: true,
                status: registration.status,
                registrationDate: registration.registrationDate,
                googleEventId: registration.googleEventId
            });
        } else {
            res.status(200).json({
                isRegistered: false,
                status: null
            });
        }
    } catch (error) {
        console.error('Registration check error:', error);
        res.status(500).json({
            error: 'Error checking registration status',
            details: error.message
        });
    }
};
```

Figure 49. *Check Registration Service*

```
// Add this new service method
const cancelRegistration = async (req, res) => {
    const eventId = req.params.id;
    const userId = req.user.id;
    const token = req.user.accessToken;
    const event = await Event.findById(eventId);
    const oauth2Client = new google.auth.OAuth2();
    oauth2Client.setCredentials({ access_token: token });

    const calendar = google.calendar({ version: 'v3', auth: oauth2Client }); // Re-initialize calendar here

    try {
        const registration = await Registration.findOne({ eventId, userId });
        if (!registration) {
            return res.status(404).json({ message: 'Registration not found' });
        }

        // If you're using Google Calendar, remove the event
        if (registration.googleEventId) {
            try {
                await calendar.events.delete({
                    calendarId: 'primary',
                    eventId: registration.googleEventId
                });
                console.log(`Event with ID ${registration.googleEventId} deleted from Google Calendar`);
```

Figure 50. *Cancel Registration Service*

Form Submission Code Sample

- Client - eventDetails.jsx

```
// Handle form submission
const handleFormSubmission = async () => {
    try {
        setSubmissionStatus('checking');
        setSubmissionMessage('Verifying your form submission...');
        setIsLoading(true);

        const token = sessionStorage.getItem('authToken');
        if (!token) {
            throw new Error('Please log in to submit the form');
        }

        if (!event.formLink) {
            throw new Error('Invalid form link. Please contact the event organizer.');
        }

        // Updated form link validation to handle more Google Form URL patterns
        const formId = event.formLink.match(/\/forms\/d\/e\/([^/]+)\//) ||  // Pattern 1: .../forms/d/e/[formId]/...
            event.formLink.match(/\/forms\/d\/([^/]+)\//) ||      // Pattern 2: .../forms/d/[formId]/...
            event.formLink.match(/\/forms\/([^/]+)/) ||           // Pattern 3: .../forms/[formId]
            event.formLink.match(/forms.gle\/(\w+)/);             // Pattern 4: forms.gle/[shortId]

        if (!formId || !formId[1]) {
            console.error('Form link:', event.formLink); // For debugging
```

Figure 51. *Form Submission Client*

- Model - formSubmission.js

```
const mongoose = require('mongoose');
const encrypt = require('mongoose-encryption');
require('dotenv').config({ path: '../.env' });

// Form Submission Schema
const formSubmissionSchema = new mongoose.Schema({
    eventId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Event',
        required: true
    },
    userId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true
    },
    registrationId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Registration',
        required: true
    },
    formLink: {
        type: String,
        required: true
    },
    spreadsheetId: {
        type: String,
        required: false
    },
    responses: {
```

Figure 52. *Form Submission Schema*

- Route - formSubmissionRoutes.js

```
const express = require('express');
const formSubmissionRoutes = express.Router();
const authenticateJWT = require('../../middleware/auth');
const formSubmissionService = require('../../services/user/formSubmissionService');
const certificateService = require('../../services/user/certificateService');

formSubmissionRoutes.post('/u/events/:id/form-submission', authenticateJWT, formSubmissionService.recordFormSubmission);
formSubmissionRoutes.get('/u/events/:id/form-status', authenticateJWT, formSubmissionService.getFormSubmissionStatus);
formSubmissionRoutes.get('/u/events/:id/form-analytics', authenticateJWT, formSubmissionService.getFormAnalytics);


module.exports = formSubmissionRoutes;
```

Figure 53. *Form Submission Routes*

- Services - formSubmissionService.js

```
// Helper function to get or create linked spreadsheet
const getOrCreateSpreadsheet = async (formId, formTitle) => {
    try {
        const { forms, sheets } = await initialize();

        // First, try to get existing linked spreadsheet
        const formDetails = await forms.forms.get({
            formId,
            fields: 'linkedSheetId'
        });

        if (formDetails.data.linkedSheetId) {
            console.log('Found linked spreadsheet:', formDetails.data.linkedSheetId);
            return formDetails.data.linkedSheetId;
        }

        // If no linked sheet exists, create one
        console.log('No linked spreadsheet found. Creating new one...');
        const spreadsheet = await sheets.spreadsheets.create({
            requestBody: {
                properties: {
                    title: `${formTitle} - Responses`
                }
```

Figure 54. *Helper function to get or create linked spreadsheet Service*

```
// Function to record form submission
const recordFormSubmission = async (req, res) => {
    try {
        const eventId = req.params.id;
        const userId = req.user.id;

        console.log('Processing form submission for:', { eventId, userId });

        // Get event and form details
        const event = await Event.findById(eventId);
        if (!event?.formId) {
            return res.status(400).json({ message: 'Event form not found' });
        }
        console.log('Event:', event);
        // Check registration
        const registration = await Registration.findOne({ eventId, userId });
        if (!registration) {
            return res.status(400).json({ message: 'Not registered for this event' });
        }
```

Figure 55. *Record Form Submission Service*

```
// Function to export responses to CSV
const exportResponses = async (req, res) => {
    try {
        const eventId = req.params.id;

        const event = await Event.findById(eventId);
        if (!event?.formId) {
            return res.status(404).json({ message: 'Event form not found' });
        }

        const formId = extractFormId(event.formId);
        const spreadsheetId = await getOrCreateSpreadsheet(formId, event.title);

        const { sheets } = await initialize();

        // Get all responses
        const response = await sheets.spreadsheets.values.get({
            spreadsheetId: spreadsheetId,
            range: 'Form Responses 1'
        });

        const rows = response.data.values || [];
        if (rows.length <= 1) {
            return res.status(404).json({ message: 'No responses found' });
        }
```

Figure 56. *Extract Responses Service*

```
// Function to get form submission status
const getFormSubmissionStatus = async (req, res) => {
    try {
        const eventId = req.params.id;
        const userId = req.user.id;

        const submission = await FormSubmission.findOne({ eventId, userId })
            .select('status submittedAt responses');

        if (!submission) {
            return res.status(404).json({ message: 'No form submission found' });
        }

        res.json({
            status: submission.status,
            submittedAt: submission.submittedAt,
            hasSubmittedForm: true
        });
    } catch (error) {
        console.error('Error getting form submission status:', error);
        res.status(500).json({ message: 'Error getting form submission status' });
    }
};
```

Figure 57. *Form Status Service*

Calendar Code Sample

- Client - calendar.jsx

```
};
    const renderCalendar = () => {
        switch (currentView) {
            case 'day':
                return renderDayView();
            case 'week':
                return renderWeekView();
            case 'month':
                return renderMonthView();
            case 'year':
                return renderYearView();
            default:
                return renderMonthView();
        }
    };
```

Figure 58. *Render a Calendar Events (Client)*

- Route - adminPageRoutes.js & userEventRoutes.js

```
// Calendar
pageRoutes.get('/a/calendar', adminService.renderCalendarPage);
```

Figure 59. *Admin Calendar Route*

```
// Calendar Page
userEventRoutes.get('/u/calendar', userEventService.renderCalendar);
```

Figure 70. *User Calendar Route*

- Services - adminPageServices & userEventService.js

```
// Render Google Calendar integration page
const renderCalendar = async (req, res) => {
    try {
        const registrations = await Registration.find();
        res.status(200).json(registrations);
    } catch (error) {
        res.status(500).send('Error retrieving events');
    }
};
```

Figure 71. *Admin Calendar Service*

```
const renderCalendarPage = async (req, res) => {
    try {
        // Logic for rendering the calendar page with events
        const events = []; // Fetch events data
        res.send('Welcome to Calendar');
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
};
```

Figure 72. *User Calendar Service*

Coding Standards and Guidelines

1. **Code Formatting**
    - **Readability -** Emphasizes consistent indentation, spacing, and alignment to improve code readability. It suggests using spaces rather than tabs and adhering to a specific width (e.g., 2 or 4 spaces). (GeeksforGeeks Standard)
    - **Application to Buksu Enage or (UTMS)**
        - **For Node.js (Backend):** Ensure consistent indentation with 2 or 4 spaces for functions, objects, and routes. Use a tool like Prettier to auto-format your code.
        - **For React (Frontend):** Consistently structure your JSX, ensuring proper alignment of attributes and components, which enhances readability.
2. **Naming Conventions**
    - Recommends meaningful names for variables, functions, and classes using a consistent style like camelCase for variables and PascalCase for classes and React components. (GeeksforGeeks Standard)
3. **Commenting**
    - This system uses comments to explain complex code and provides documentation for functions, methods, and classes using both inline and block comments.
4. **Code Structure**
    - Advocates for well-structured and modular code, ensuring that the code is easy to maintain and extend. (GeeksforGeeks Standard)

    Buksu Enage Code Structure

    - **Backend (Node.js):** The backend is organized into well-defined folders, such as /services, /routes, and /models, to separate concerns and improve maintainability. User and admin routes and services are separated, as well as the authentication routes and services.
    - **Client (React):** The components for users and admins are separated, each with its own CSS. Additionally, the pages for admins and users are organized separately.
5. **Error Handling**
    - Encourages proper error handling with clear messages and logging to ensure robustness.(GeeksforGeeks Standard)
        - For the backend, try-catch blocks are used along with a centralized error handler.

- For the frontend, graceful error handling and user-friendly error messages are implemented.

Project Management Artifacts

Sprint Click Up

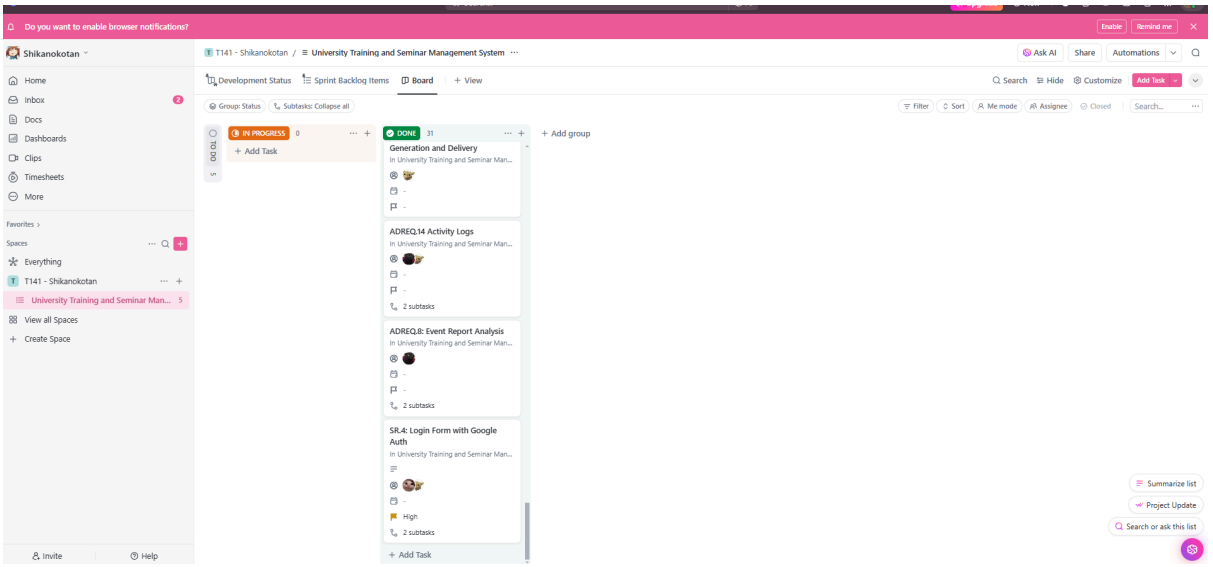


Figure 1 . *Click Up Sprint Board*

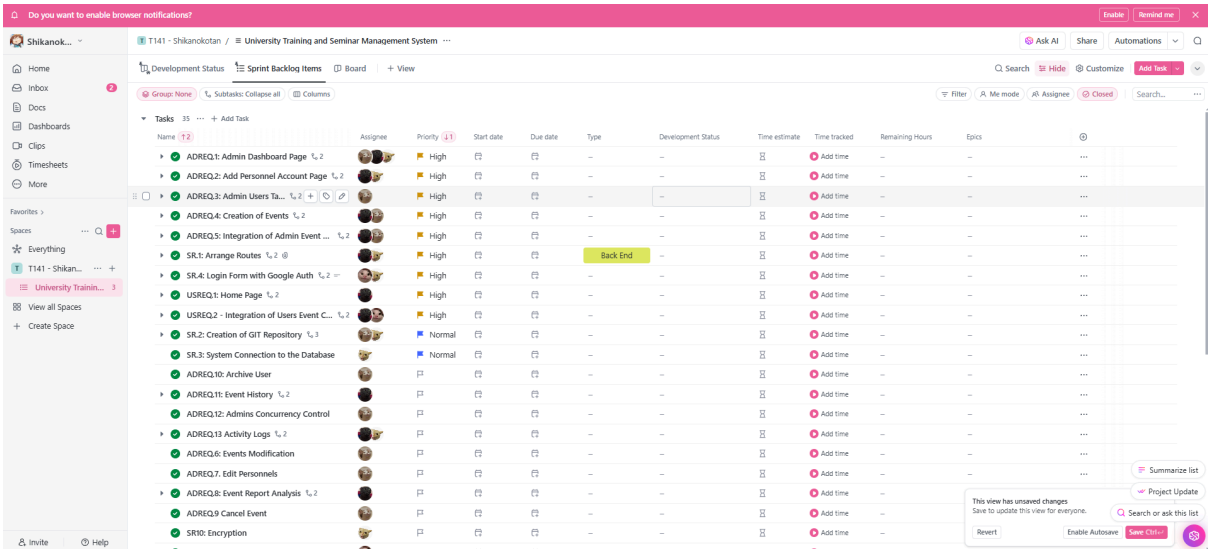


Figure 2 . *Click Up Backlog Items*

Screenshots of System

System Interface

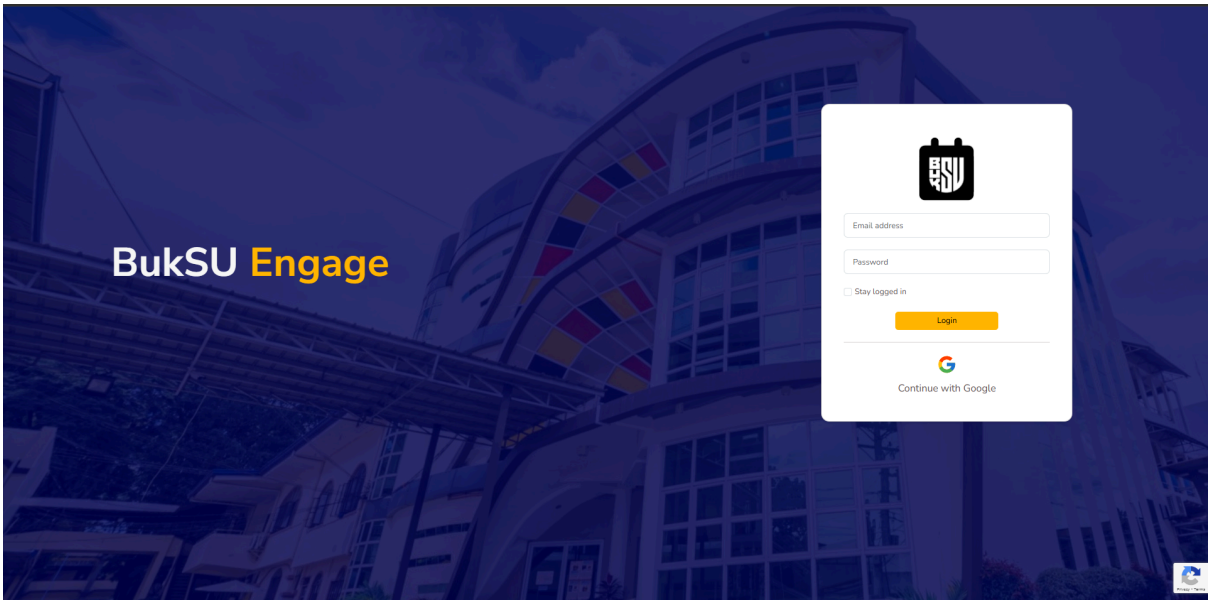


Figure 3 : *Log In*

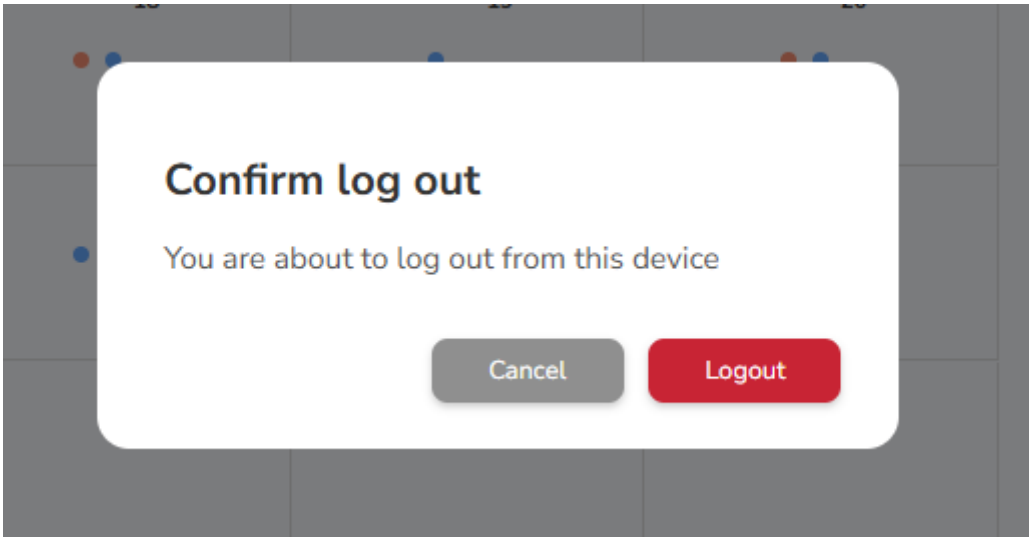


Figure 4 : *Log out*

Figure 5 : *Footer*

User Interface

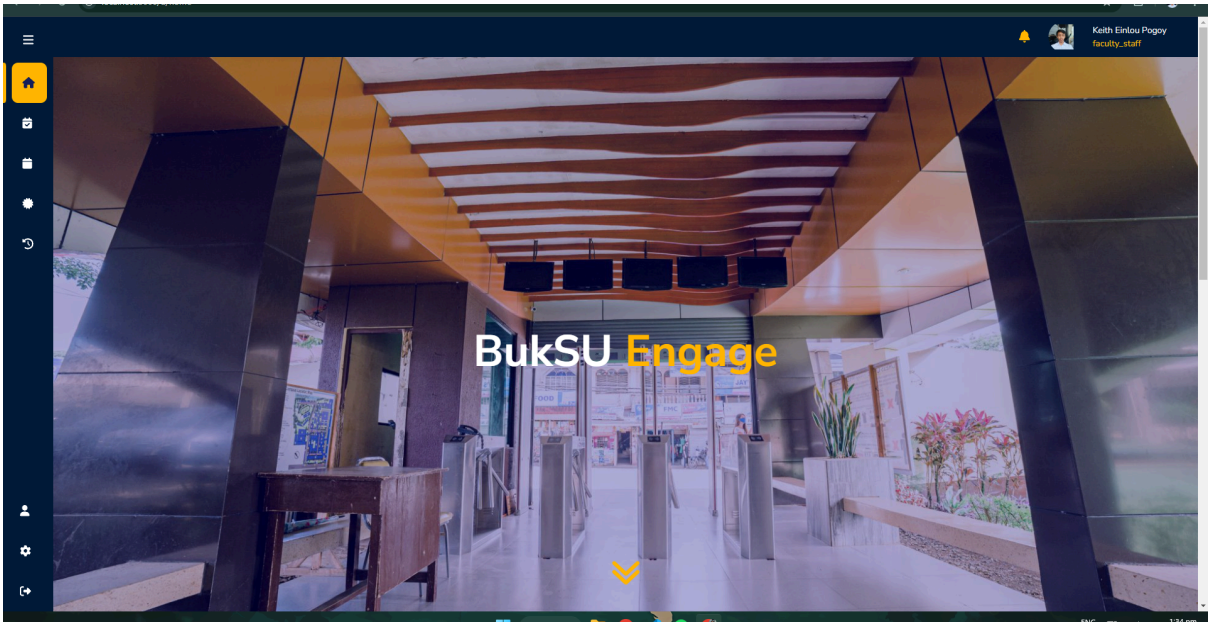


Figure 6 : *User Homepage*

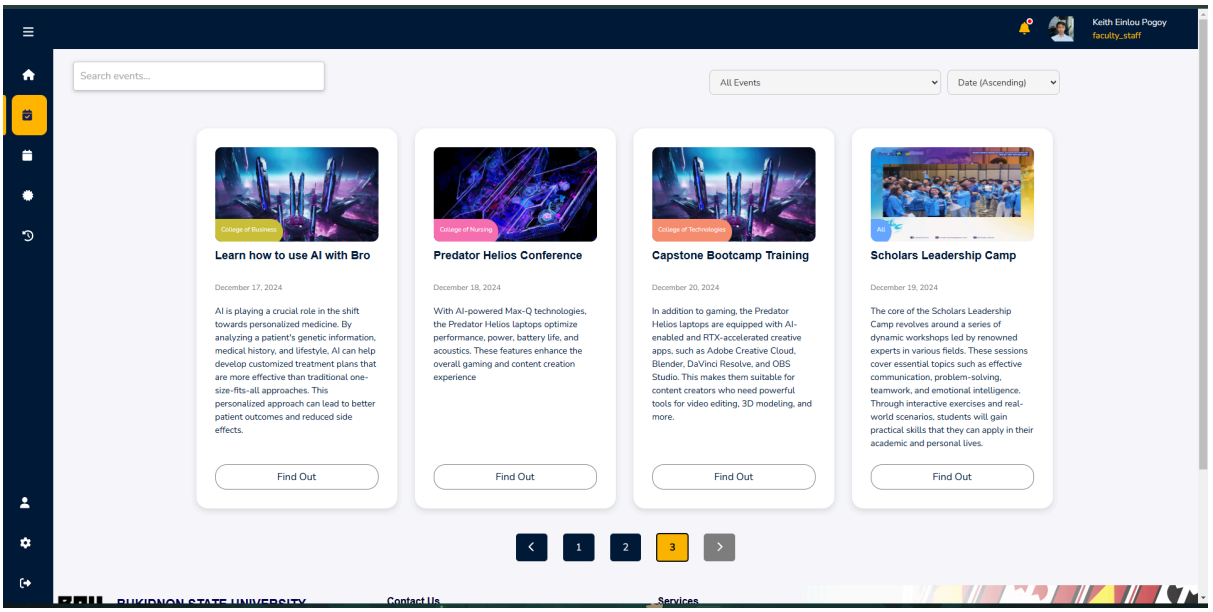


Figure 7 : *Events Page*

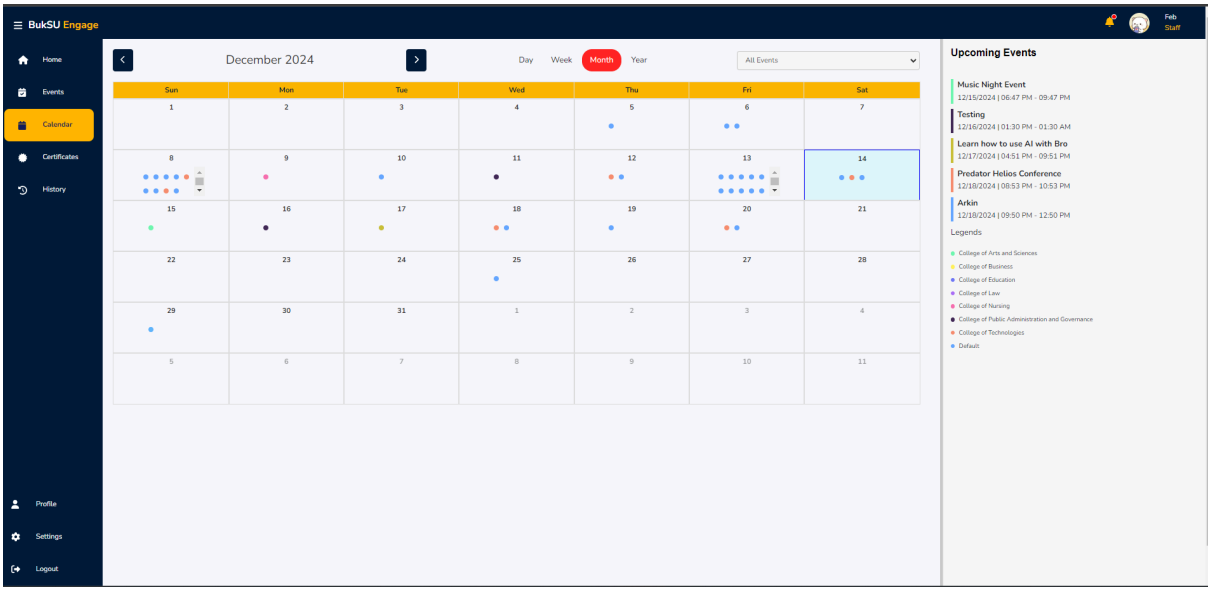


Figure 8 : *User Calendar*

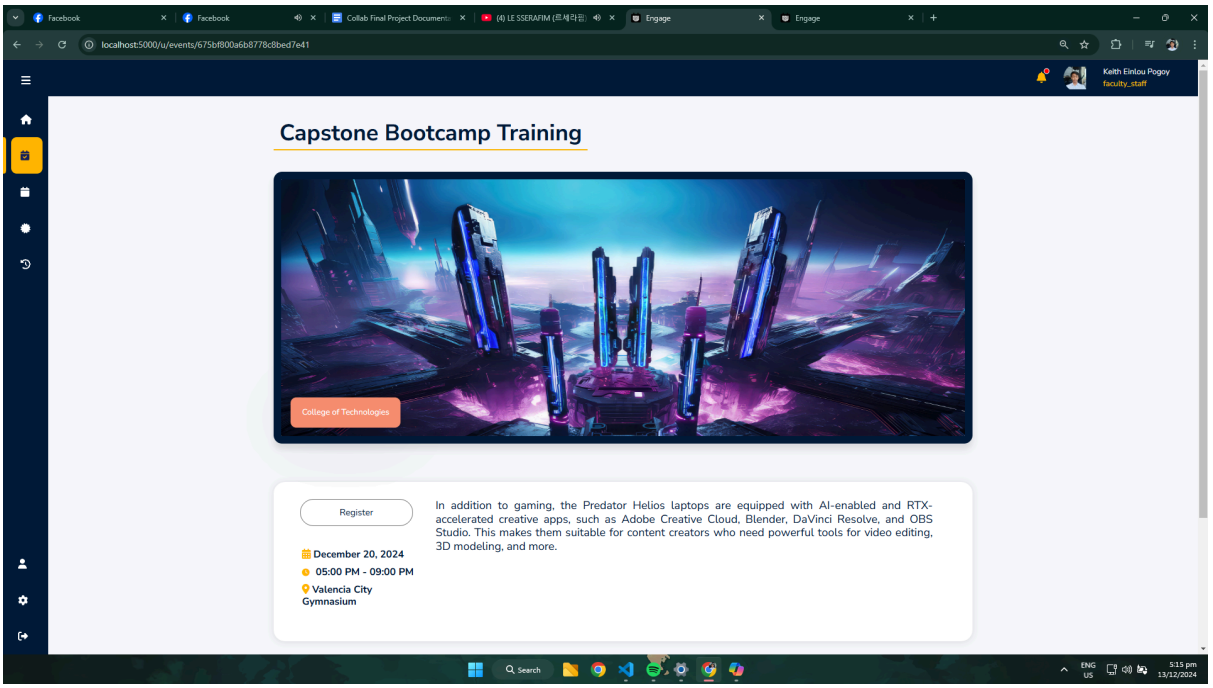


Figure 9 : *Event Registration*

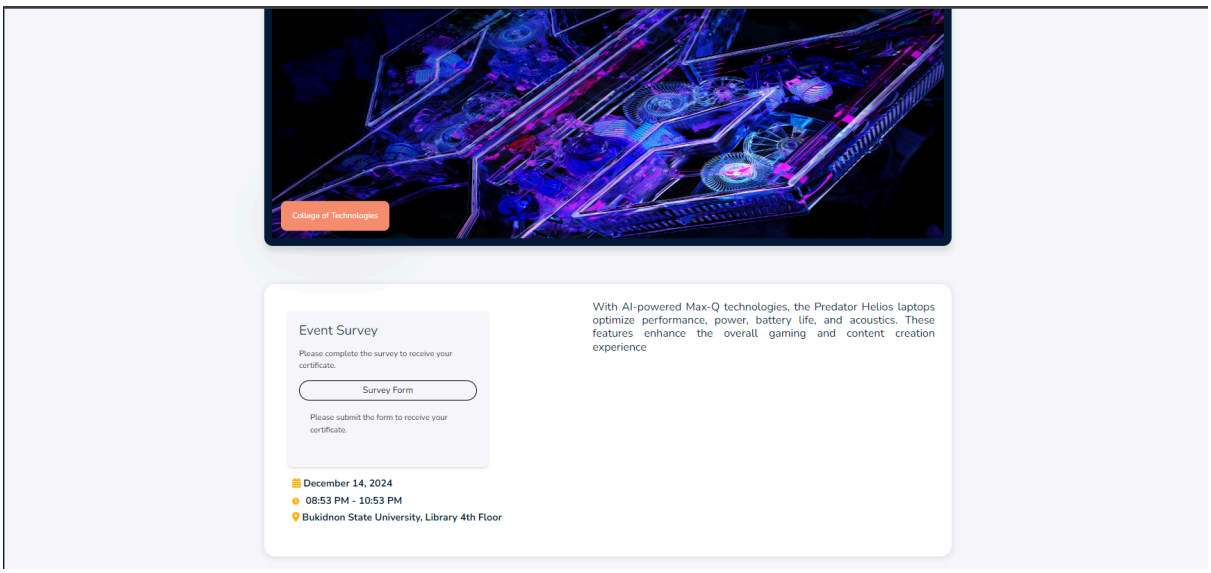


Figure 10 : *Event Survey Form*

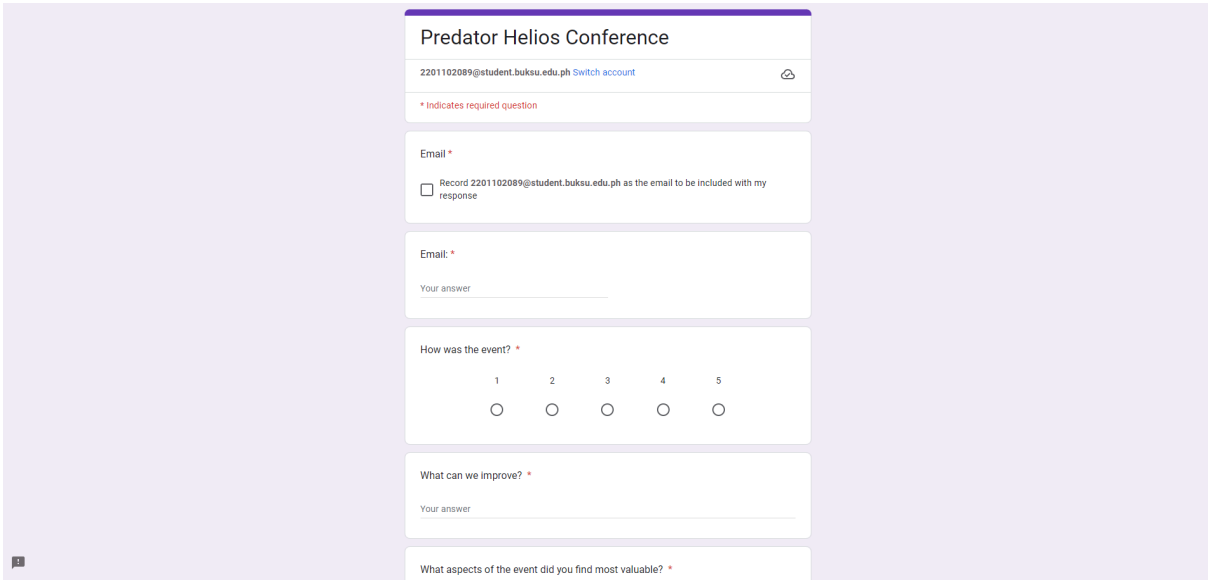


Figure 11 : *Event Survey Google Form*

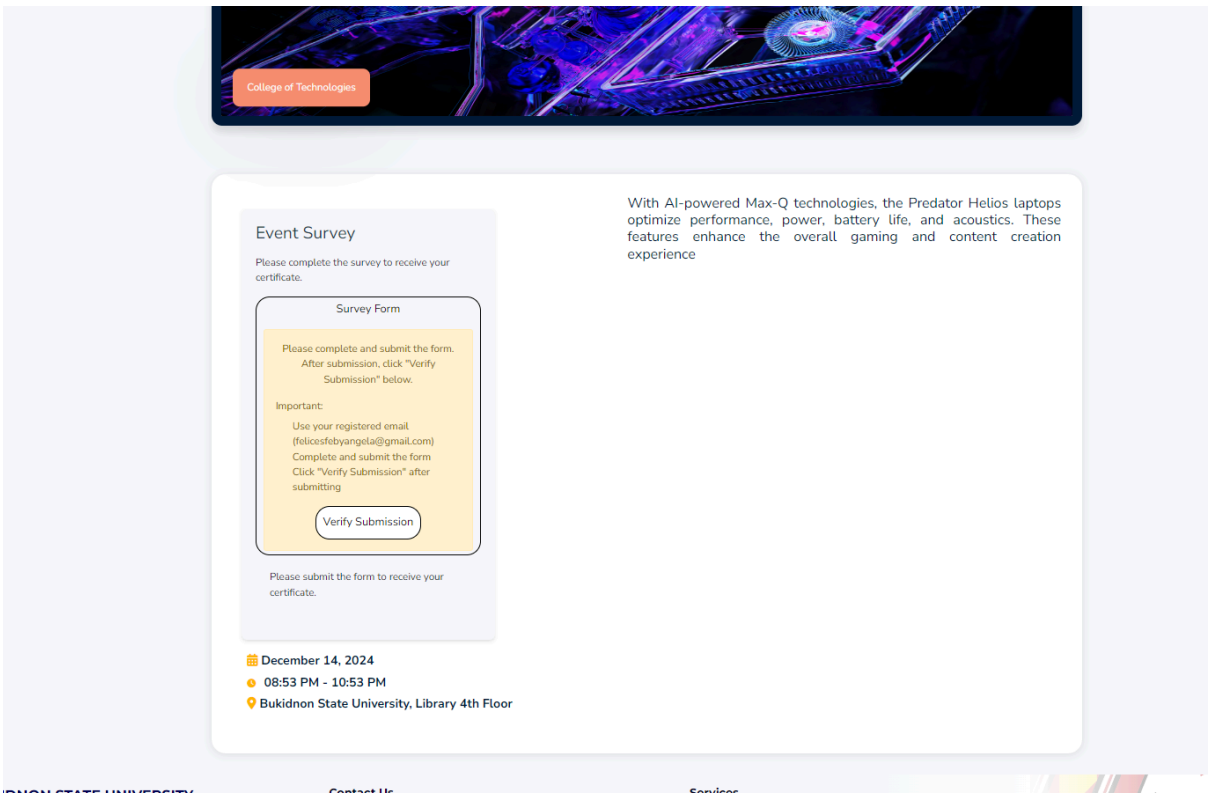


Figure 12 : *User Verifying the Submission*

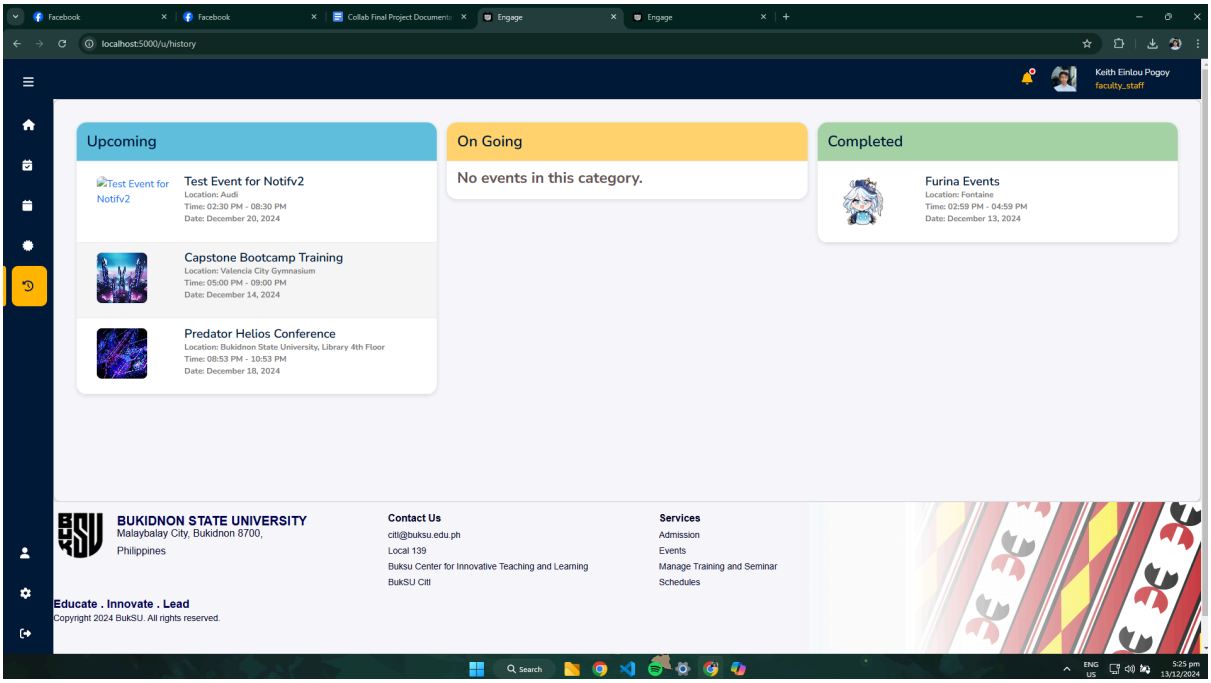


Figure 13 : *Users Kanban*

Figure 14 : *Users Certificate Gallery*

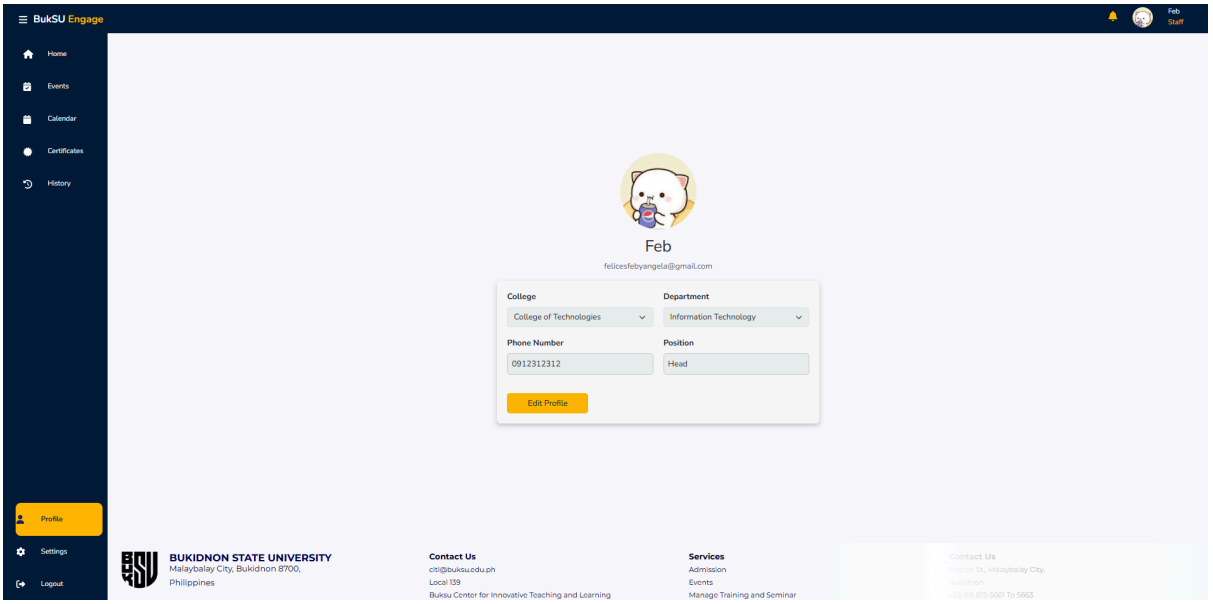


Figure 15 : *Users Profile Page*

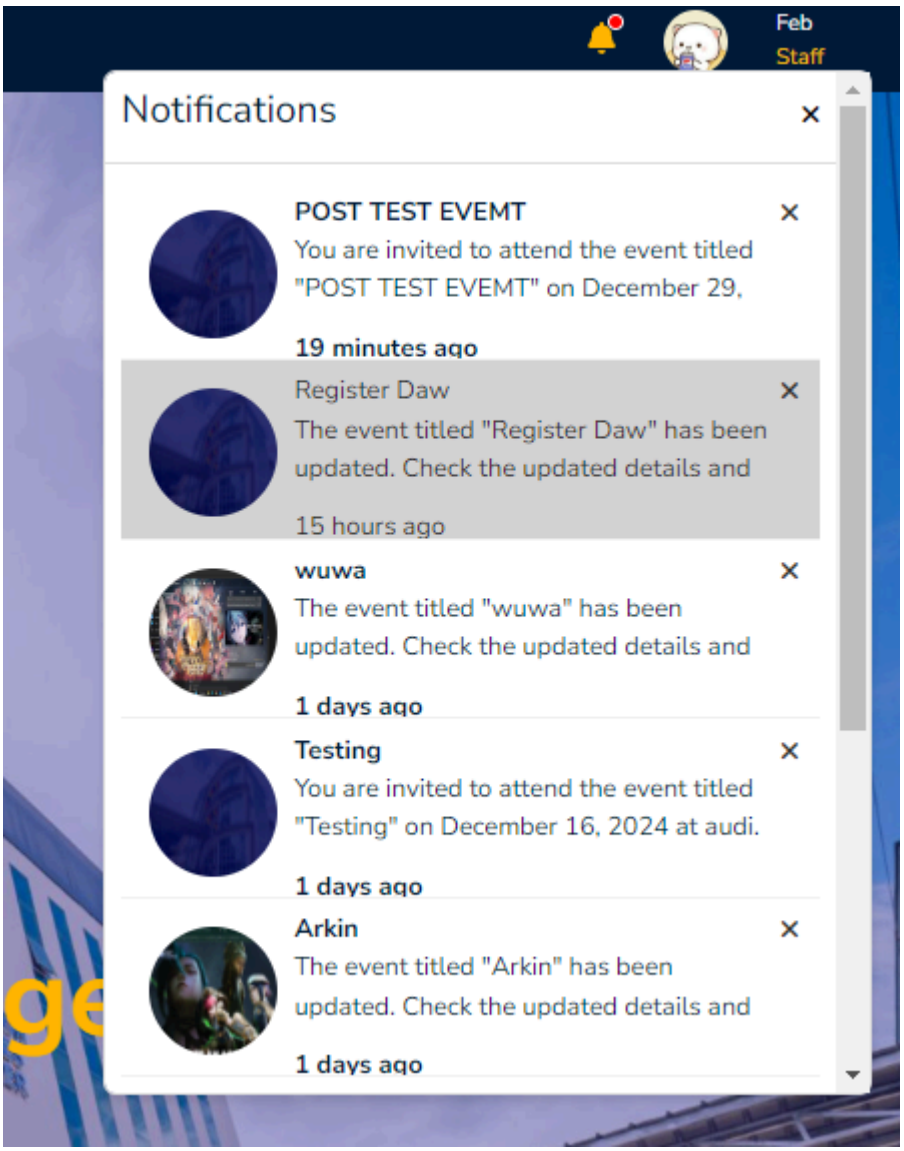


Figure 16 : *Users Notification List*

Admin & Departmental Admins Interface

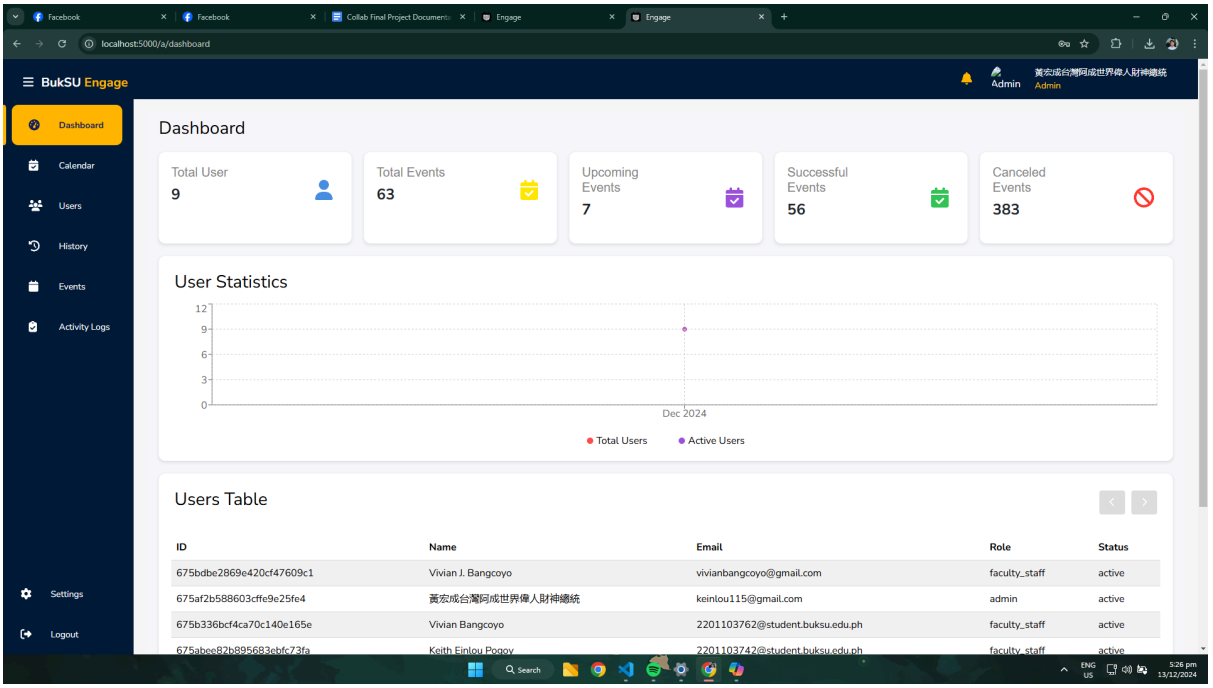


Figure 17 : *Admin & Departmental Admins Dashboard*

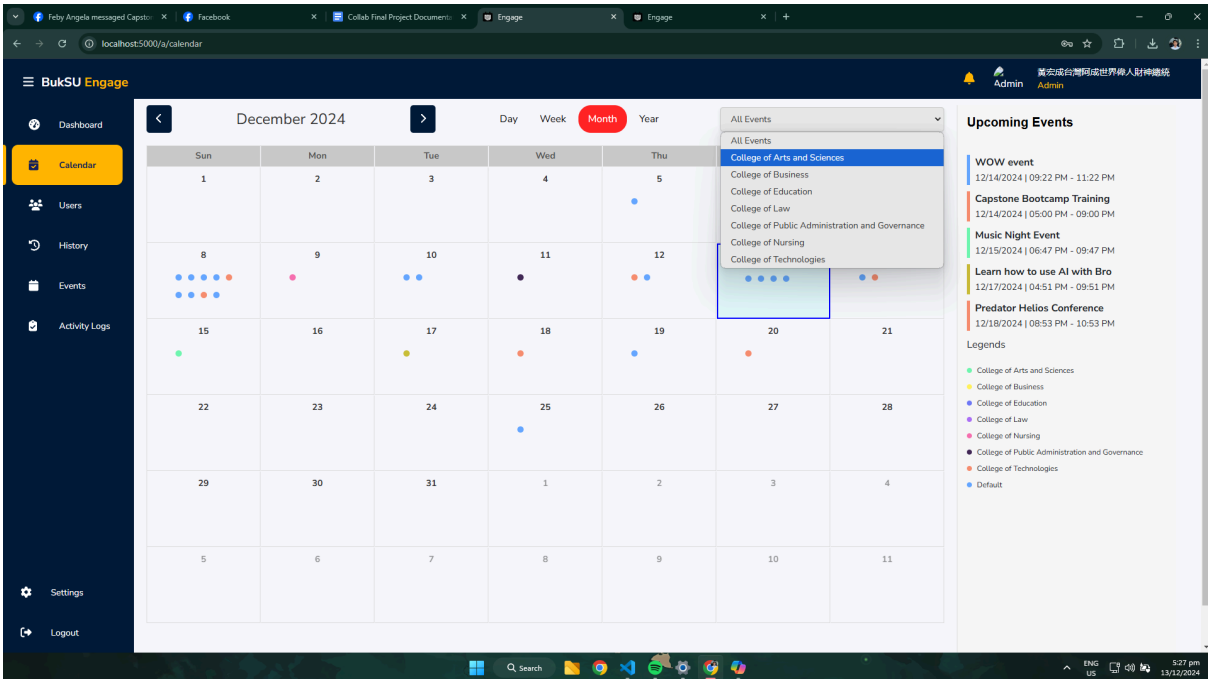


Figure 18: *Admin & Departmental Admins Calendar*

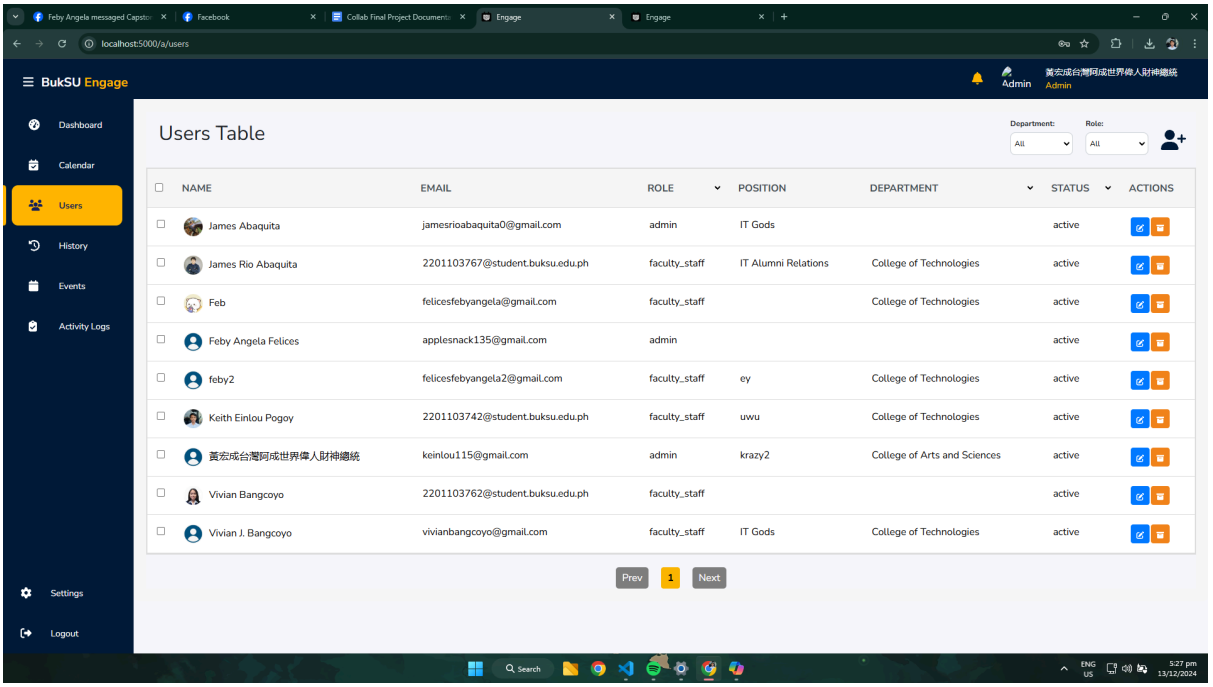


Figure 19 : *Admin User Table (Departmental Admin does not have this interface)*

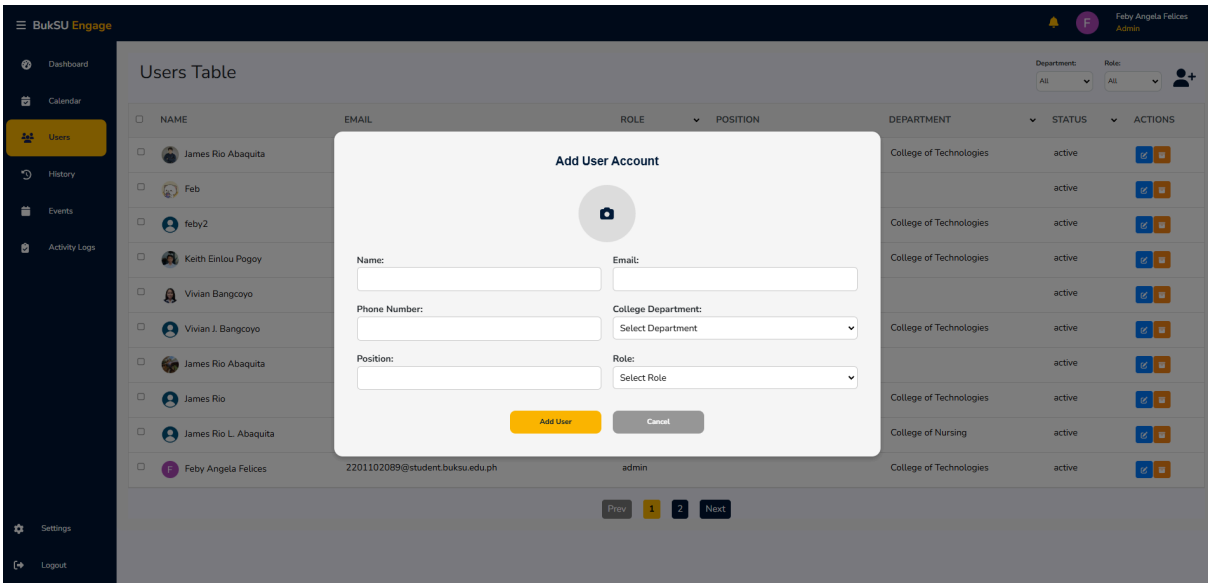


Figure 20 : *Admin Adding New User Account (Departmental Admin does not have this feature)*

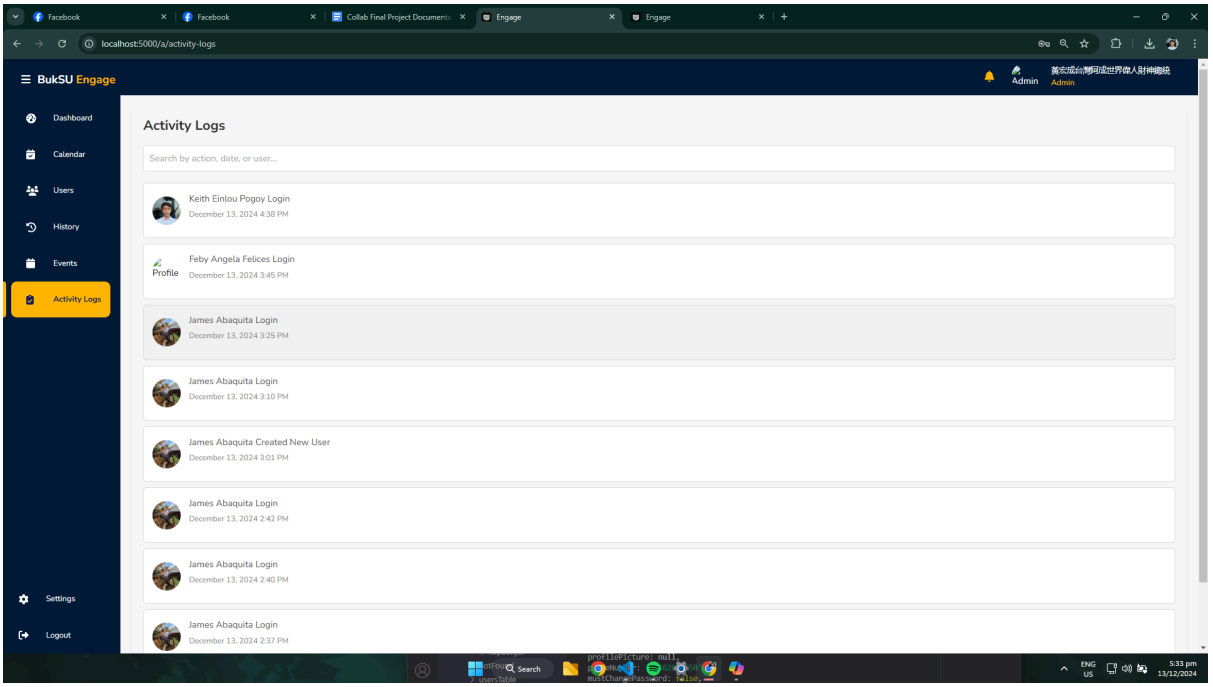


Figure 21 : *Admin Activity Logs (Departmental Admin does not have this interface)*

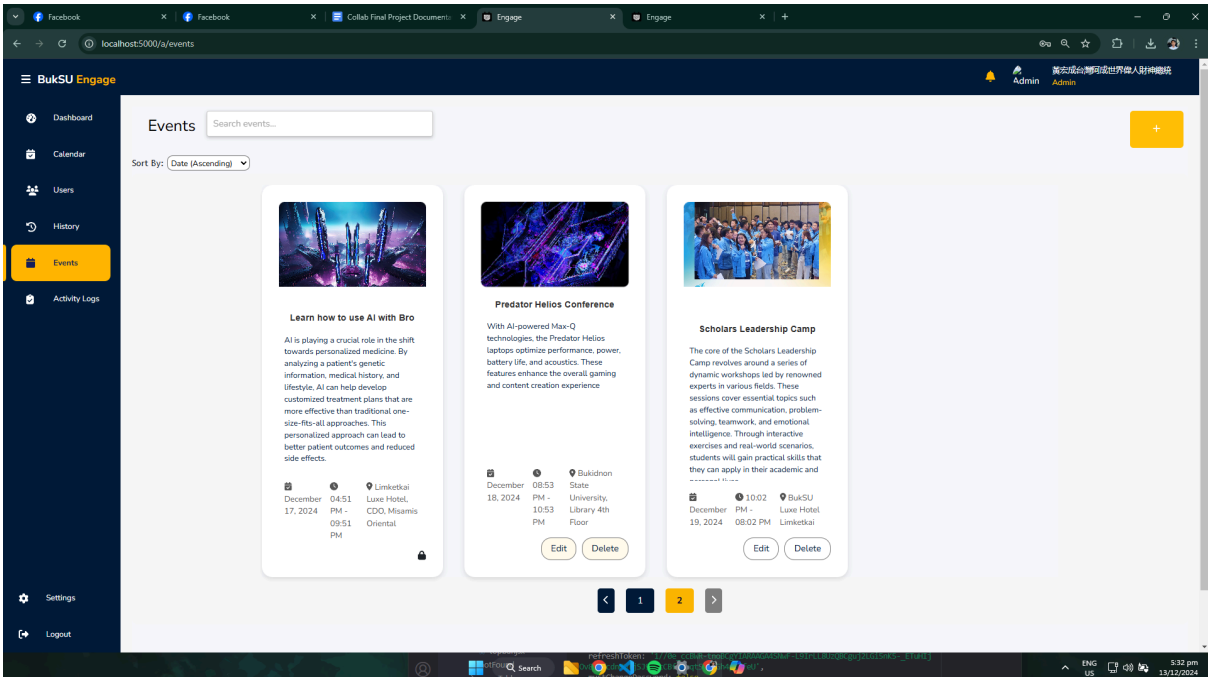


Figure 22: *Admin Event Page (Displays all events accessible to the admin.)*

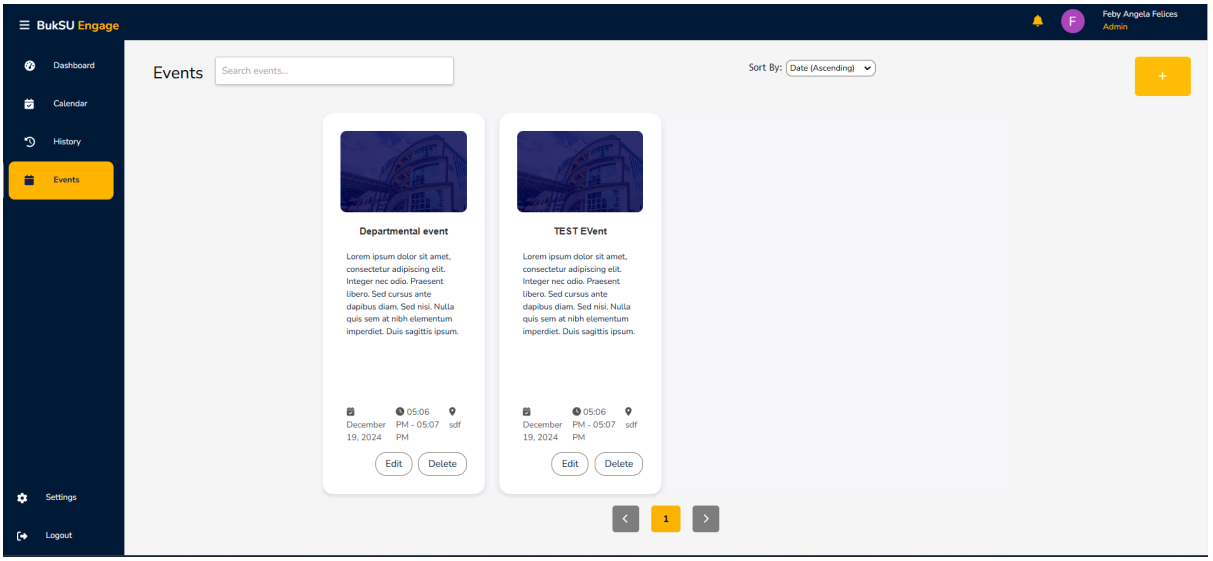


Figure 23. *Departmental Admins Event Page (Displays only their created events.)*

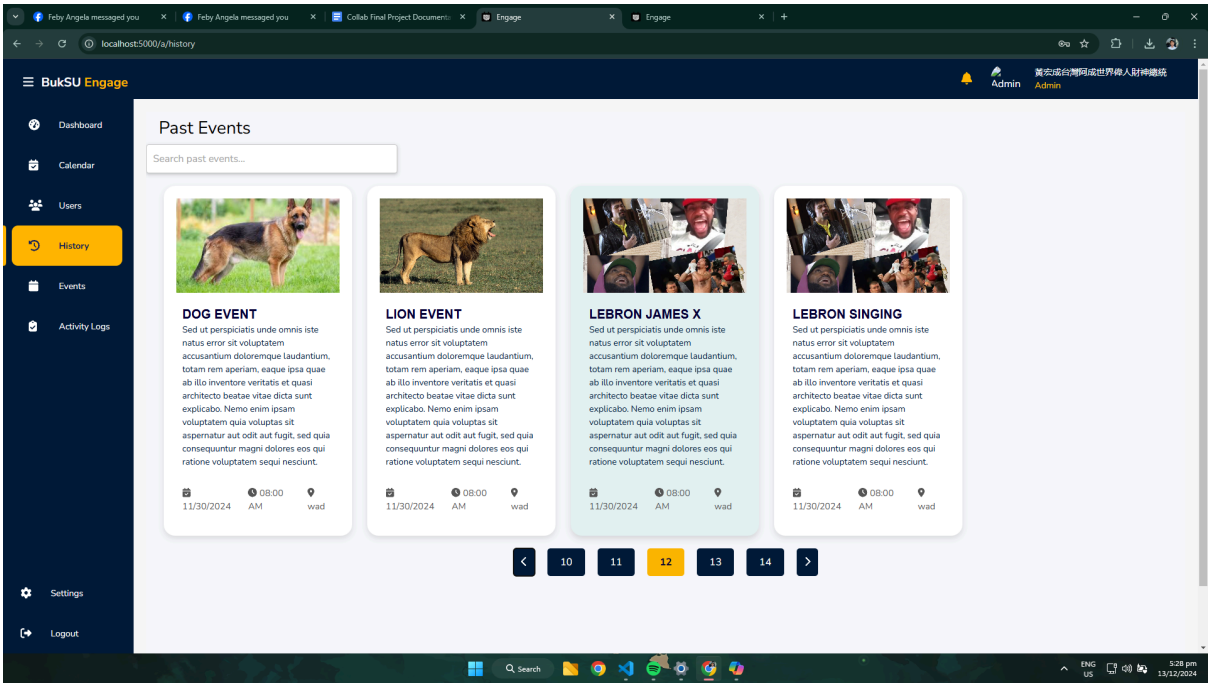


Figure 24 : *Admin & Departmental Admins Event History*

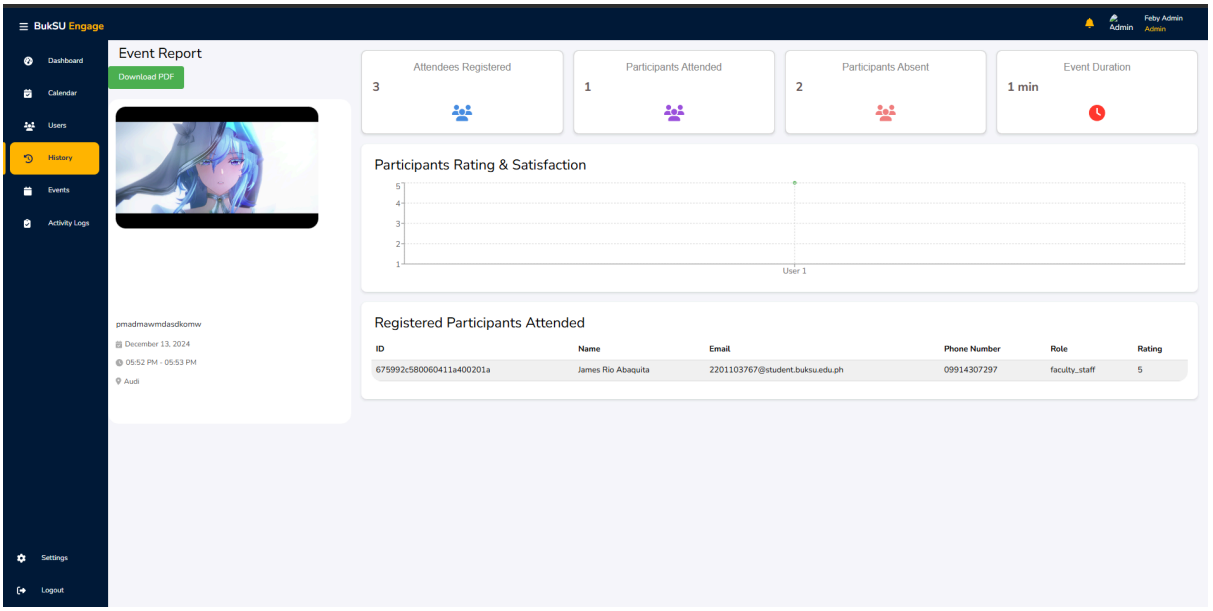


Figure 25 : *Admin & Departmental Admins Event Report Analytics*

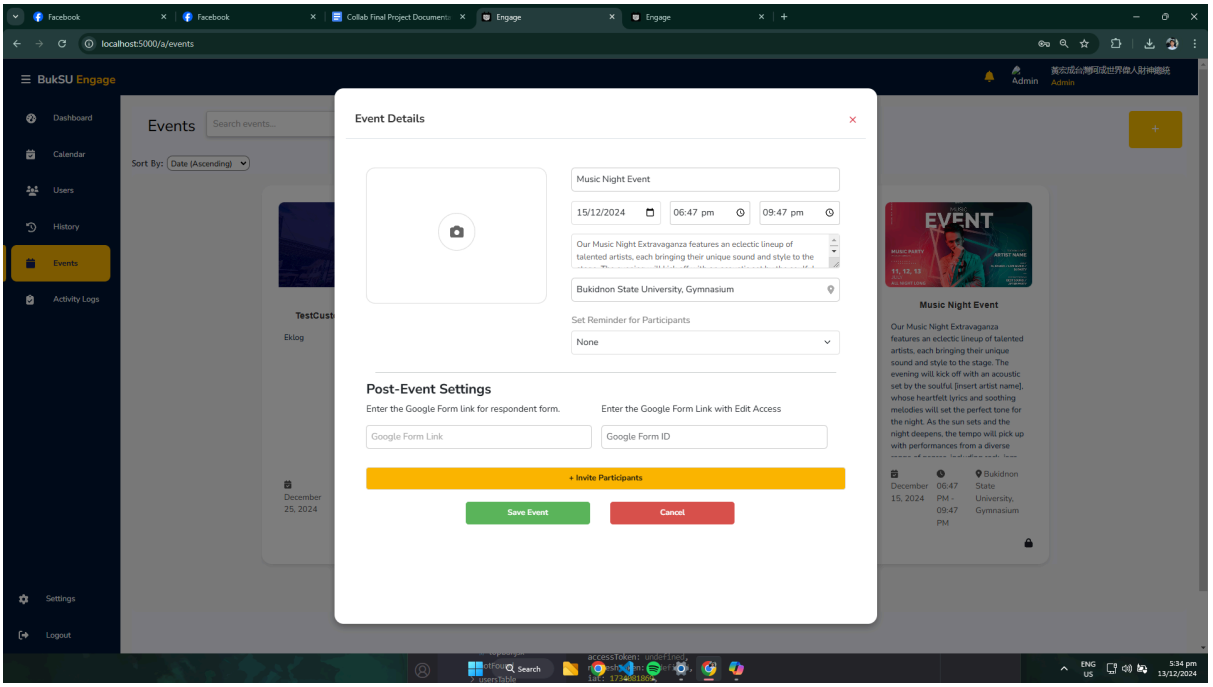


Figure 26 : *Add / Edit Event Modal*

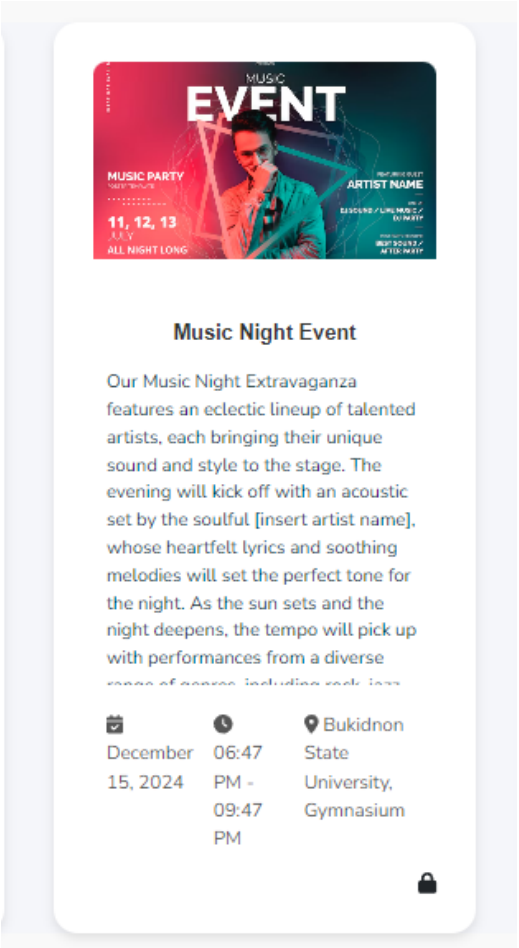


Figure 27 : *Event Being Edited by Another Admin or Department Admin*

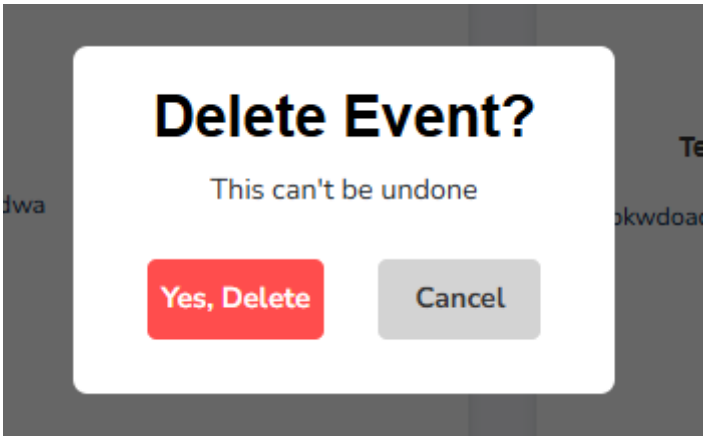
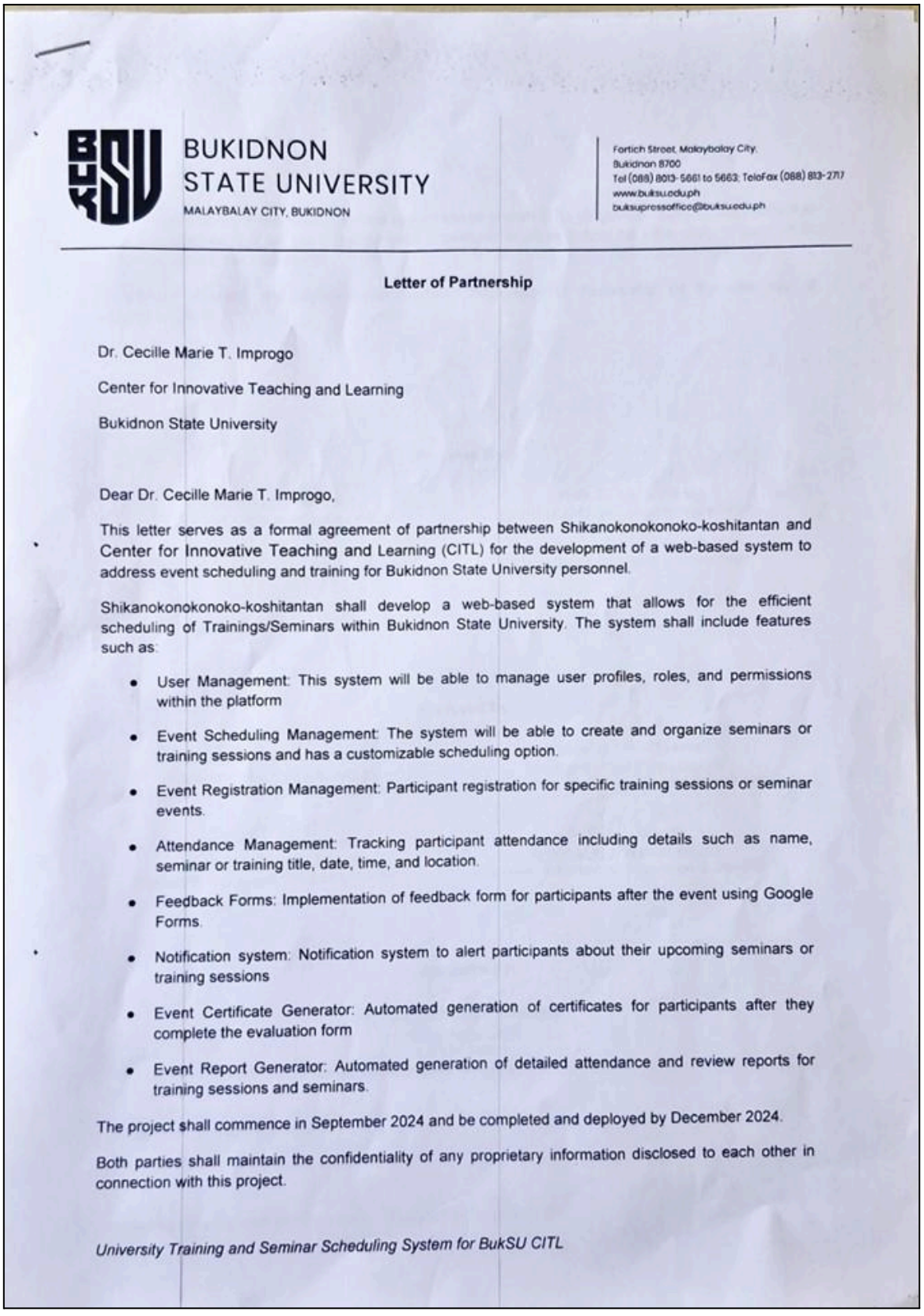


Figure 28 : *Delete Modal Confirmation*

Stakeholder Validation Documents



BUKIDNON
STATE UNIVERSITY
MALAYBALAY CITY, BUKIDNON

Fortich Street, Malaybalay City,
Bukidnon 8700
Tel (088) 8013-5661 to 5663; TeleFax (088) 813-2717
www.buksu.edu.ph
buksupressoffice@buksu.edu.ph

Letter of Partnership

Dr. Cecille Marie T. Improgo

Center for Innovative Teaching and Learning

Bukidnon State University

Dear Dr. Cecille Marie T. Improgo,

This letter serves as a formal agreement of partnership between Shikanokonokonoko-koshitantan and Center for Innovative Teaching and Learning (CITL) for the development of a web-based system to address event scheduling and training for Bukidnon State University personnel.

Shikanokonokonoko-koshitantan shall develop a web-based system that allows for the efficient scheduling of Trainings/Seminars within Bukidnon State University. The system shall include features such as:

- User Management: This system will be able to manage user profiles, roles, and permissions within the platform
- Event Scheduling Management: The system will be able to create and organize seminars or training sessions and has a customizable scheduling option.
- Event Registration Management: Participant registration for specific training sessions or seminar events.
- Attendance Management: Tracking participant attendance including details such as name, seminar or training title, date, time, and location.
- Feedback Forms: Implementation of feedback form for participants after the event using Google Forms.
- Notification system: Notification system to alert participants about their upcoming seminars or training sessions
- Event Certificate Generator: Automated generation of certificates for participants after they complete the evaluation form
- Event Report Generator: Automated generation of detailed attendance and review reports for training sessions and seminars.

The project shall commence in September 2024 and be completed and deployed by December 2024.

Both parties shall maintain the confidentiality of any proprietary information disclosed to each other in connection with this project.

University Training and Seminar Scheduling System for BukSU CITL

Figure 29. *Letter of Partnership Page 1*

Either party may terminate this agreement upon material breach of its obligations under this agreement or upon the occurrence of an event that makes it impossible or impracticable for either party to perform its obligations under this agreement.

In witness whereof, the parties have executed this Letter of Partnership on this 9th day of September2024.

Prepared By:

James Rio U. Abaquita
2201103767@student.buksu.edu.ph

Keith Emlou J. Pogoy
2201103742@student.buksu.edu.ph

Vivian J. Bangcoyo
2201103762@student.buksu.edu.ph

Feby Angela H. Felices
2201102089@student.buksu.edu.ph

Approved By:

Czarissa Louise A. Navidad
czarissanavidad@buksu.edu.ph
Instructor, Systems Integration & Architecture 1

Klevie Jun R. Caseres
kleviecaseres@buksu.edu.ph
Instructor, Integrative Programming & Technologies

Gil Nicholas T. Cagande
gilcagande@buksu.edu.ph
Instructor, Advanced Database Systems

Joseph L. Abella
josephabella@buksu.edu.ph
Instructor, Information Assurance & Security 1

Accepted By:

Dr. Cecille Marie T. Improgo
citl@buksu.edu.ph
CITL Director
Center for Innovative Teaching and Learning

Figure 30. *Letter of Partnership Page 2*

BUKIDNON
STATE UNIVERSITY
MALAYBALAY CITY, BUKIDNON

Fortich Street, Malaybalay City,
Bukidnon 8700
Tel (088) 8013-5661 to 5663; TeleFax (088) 813-2717
www.buksu.edu.ph
buksupressoffice@buksu.edu.ph

Project Charter

Project Title: University Training and Seminar Management

Project Manager: Feby Angela H. Felices

Date: Oct 18, 2024

Project Client: Dr. Cecille Marie T. Improgo, Director of Center for Innovative Teaching and Learning (CITL)

Project Purpose

This collaborative project aims to develop a web-based system that addresses a specific need or problem identified within the university community. Students will gain hands-on experience in designing, developing, and securing a real-world application by integrating the knowledge and skills from four core Information Technology subjects, namely: Systems Integration & Architecture 1, Integrative Programming and Technologies 2, Advanced Database Systems, and Information Assurance & Security 1.

This project aims to develop a web-based system known as the University Training and Seminar Management System that is designed to improve the organization and management of training sessions and events conducted by CITL. Its main objective is to simplify administrative tasks related to these events, enhancing overall efficiency and ensuring a smoother user experience.

Project Objectives

1. **Analysis**
   - **Specific**
     - Identify users requirements for each objective: organized scheduling, user management integration, registration and attendance tracking, certificate generation, and report analysis.
     - Analyze different user roles (general admin, departmental admins, faculty/staff) to determine their specific functionalities and access levels.
   - Measurable
     - Number of events scheduled (daily, weekly, monthly).
     - User registration numbers and attendance rates.
     - Data collected for post-event analysis reports.
   - Achievable
     - The analysis will provide insights to improve scheduling efficiency, user management, and participant engagement.

Figure 31. *Letter of Project Charter Approval Page 1*

- Relevant
  - The analysis will provide insights to improve scheduling efficiency, user management, and participant engagement.
- Time-bound
  - Complete the analysis phase within two weeks.

2. **Design**
   - Specific
     - Design a user-friendly interface for event scheduling, registration, and attendance tracking, with role-based customization (admin, department, users).
   - Measurable
     - Develop interface layouts and reporting tools that meet user needs and system requirements.
   - Achievable
     - Leverage user-friendly design principles for seamless integration and customization.
   - Relevant
     - Ensure the design enhances system usability and operational efficiency.
   - Time-bound
     - Complete the design phase within four weeks.
3. **Development**
   - Specific
     - Develop features including event creation, calendar interface, user management, registration, attendance tracking, certificate generation, and report analysis.
   - Measurable
     - Measure success by tracking the completion and integration of each feature.
   - Achievable
     - Ensure that all functionalities are fully operational and user-friendly within the system, with regular testing and feedback.
   - Relevant
     - The system will enhance event management efficiency, improve user experience, and support operational goals.
   - Time-bound
     - It will be fully developed two months from now.

Figure 32. *Letter of Project Charter Approval Page 2*

Project Scope:

The project scope includes the following:

- User Management:
  - Creation of account for the admin, faculty and participants
  - Profiling feature that includes a history of past seminars, training sessions, and certifications for each participant
- Scheduling Management:
  - Creation of new seminars or training sessions
  - Modification of schedules
  - Management of event details, including title, description, date, time, location, and participants
  - Event Calendar
- Registration Management:
  - Participant registration for specific training sessions or seminar events
- Attendance Management:
  - Tracking participant attendance, including details such as name, seminar or training title, date, time, and location
- Feedback Forms:
  - Implementation of a feedback form for participants after the event using Google Forms
- Notifications:
  - Notification system to alert participants about their upcoming seminars or training sessions
- Certificate management:
  - Automated generation of certificates for participants after they complete the evaluation form

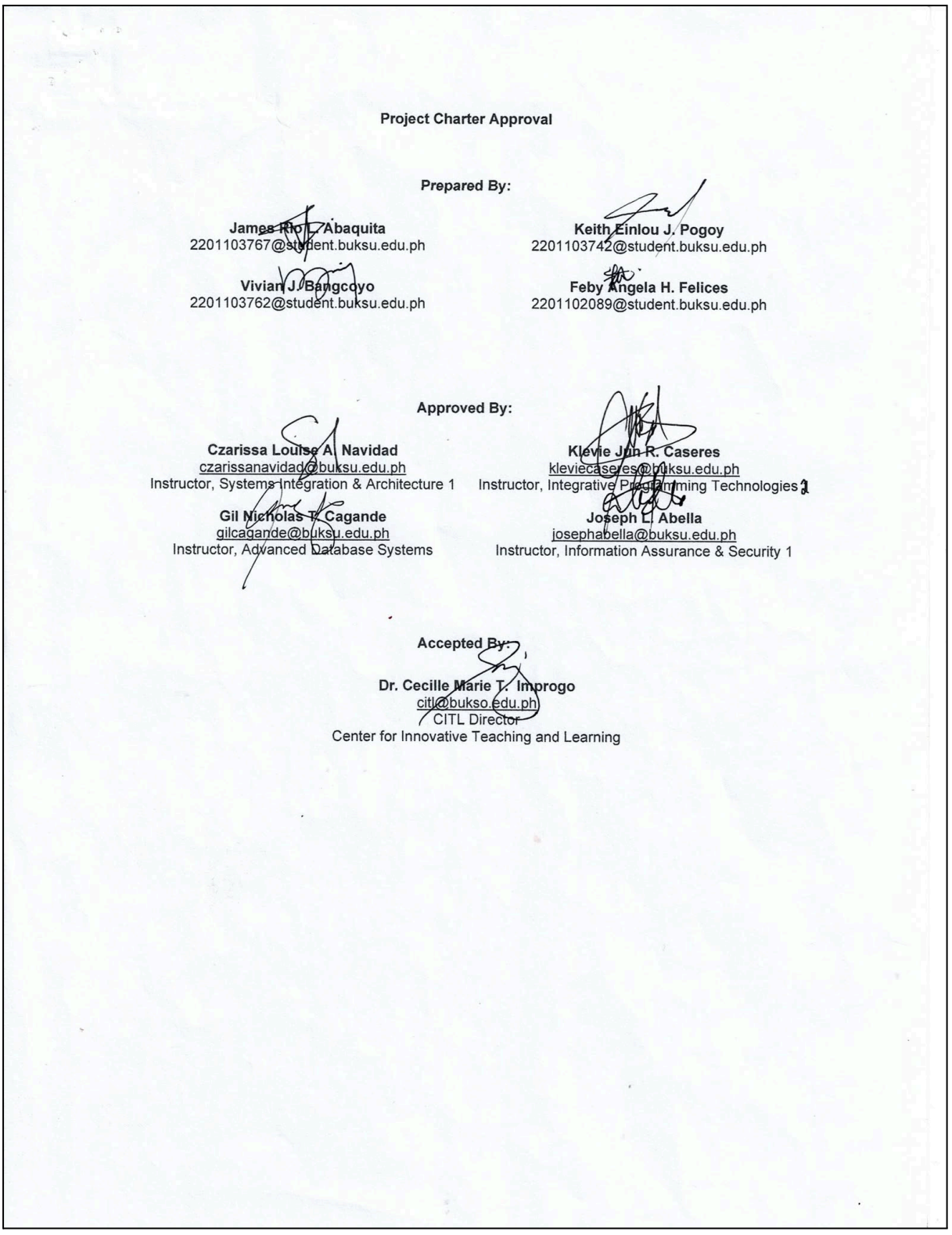
The project excludes:

- Communication or social networking features (e.g Messaging, Chatting)
- AI Implementation

Key Stakeholders:

- Faculty/Staff of Bukidnon State University
- Departmental Representative
- Center for Innovative Teaching and Learning (CITL)

Figure 33. *Letter of Project Charter Approval Page 3*



Project Charter Approval

Prepared By:

James Rio L. Abaquita
2201103767@student.buksu.edu.ph

Keith Einlou J. Pogoy
2201103742@student.buksu.edu.ph

Vivian J. Bangcoyo
2201103762@student.buksu.edu.ph

Feby Angela H. Felices
2201102089@student.buksu.edu.ph

Approved By:

Czarissa Louise A. Navidad
czarissanavidad@buksu.edu.ph
Instructor, Systems Integration & Architecture 1

Klevie Jun R. Caseres
kleviecaseres@buksu.edu.ph
Instructor, Integrative Programming Technologies 1

Gil Nicholas T. Cagande
gilcagande@buksu.edu.ph
Instructor, Advanced Database Systems

Joseph L. Abella
josephabella@buksu.edu.ph
Instructor, Information Assurance & Security 1

Accepted By:

Dr. Cecille Marie T. Improgo
citl@bukso.edu.ph
CITL Director
Center for Innovative Teaching and Learning

Figure 34. *Letter of Project Charter Approval Page 4*